

Structure and Interpretation of Computer Programs

Python Edition (SICPy)

Harold Abelson and **Gerald Jay Sussman** with **Julie Sussman**

Adapted to Python by **Martin Henz**

School of Computing, National University of Singapore (2026)

Foreword to Structure and Interpretation of Computer Programs, 1984

Educators, generals, dieticians, psychologists, and parents program. Armies, students, and some societies are programmed. An assault on large problems employs a succession of programs, most of which spring into existence en route. These programs are rife with issues that appear to be particular to the problem at hand. To appreciate programming as an intellectual activity in its own right you must turn to computer programming; you must read and write computer programs—many of them. It doesn't matter much what the programs are about or what applications they serve. What does matter is how well they perform and how smoothly they fit with other programs in the creation of still greater programs. The programmer must seek both perfection of part and adequacy of collection. In this book the use of "program" is focused on the creation, execution, and study of programs written in a dialect of Lisp for execution on a digital computer. Using Lisp we restrict or limit not what we may program, but only the notation for our program descriptions.

Our traffic with the subject matter of this book involves us with three foci of phenomena: the human mind, collections of computer programs, and the computer. Every computer program is a model, hatched in the mind, of a real or mental process. These processes, arising from human experience and thought, are huge in number, intricate in detail, and at any time only partially understood. They are modeled to our permanent satisfaction rarely by our computer programs. Thus even though our programs are carefully handcrafted discrete collections of symbols, mosaics of interlocking functions, they continually evolve: we change them as our perception of the model deepens, enlarges, generalizes until the model ultimately attains a metastable place within still another model with which we struggle. The source of the exhilaration associated with computer programming is the continual unfolding within the mind and on the computer of mechanisms expressed as programs and the explosion of perception they generate. If art interprets our dreams, the computer executes them in the guise of programs!

For all its power, the computer is a harsh taskmaster. Its programs must be correct, and what we wish to say must be said accurately in every detail. As in every other symbolic activity, we become convinced of program truth through argument. Lisp itself can be assigned a semantics (another model, by the way), and if a program's function can be specified, say, in the predicate calculus, the proof methods of logic can be used to make an acceptable correctness argument. Unfortunately, as programs get large and complicated, as they almost always do, the adequacy, consistency, and correctness of the specifications themselves become open to doubt, so that complete formal arguments of correctness seldom accompany large programs. Since large programs grow from small ones, it is crucial that we develop an arsenal of standard program structures of whose

correctness we have become sure—we call them idioms—and learn to combine them into larger structures using organizational techniques of proven value. These techniques are treated at length in this book, and understanding them is essential to participation in the Promethean enterprise called programming. More than anything else, the uncovering and mastery of powerful organizational techniques accelerates our ability to create large, significant programs. Conversely, since writing large programs is very taxing, we are stimulated to invent new methods of reducing the mass of function and detail to be fitted into large programs.

Unlike programs, computers must obey the laws of physics. If they wish to perform rapidly—a few nanoseconds per state change—they must transmit electrons only small distances (at most $\frac{1}{2}$ feet). The heat generated by the huge number of devices so concentrated in space has to be removed. An exquisite engineering art has been developed balancing between multiplicity of function and density of devices. In any event, hardware always operates at a level more primitive than that at which we care to program. The processes that transform our Lisp programs to "machine" programs are themselves abstract models which we program. Their study and creation give a great deal of insight into the organizational programs associated with programming arbitrary models. Of course the computer itself can be so modeled. Think of it: the behavior of the smallest physical switching element is modeled by quantum mechanics described by differential equations whose detailed behavior is captured by numerical approximations represented in computer programs executing on computers composed of ...!

It is not merely a matter of tactical convenience to separately identify the three foci. Even though, as they say, it's all in the head, this logical separation induces an acceleration of symbolic traffic between these foci whose richness, vitality, and potential is exceeded in human experience only by the evolution of life itself. At best, relationships between the foci are metastable. The computers are never large enough or fast enough. Each breakthrough in hardware technology leads to more massive programming enterprises, new organizational principles, and an enrichment of abstract models. Every reader should ask himself periodically "Toward what end, toward what end?"—but do not ask it too often lest you pass up the fun of programming for the constipation of bittersweet philosophy.

Among the programs we write, some (but never enough) perform a precise mathematical function such as sorting or finding the maximum of a sequence of numbers, determining primality, or finding the square root. We call such programs algorithms, and a great deal is known of their optimal behavior, particularly with respect to the two important parameters of execution time and data storage requirements. A programmer should acquire good algorithms and idioms. Even though some programs resist precise specifications, it is the responsibility of the programmer to estimate, and always to attempt to improve, their performance.

Lisp is a survivor, having been in use for about a quarter of a century. Among the active programming languages only Fortran has had a longer life. Both languages have supported the programming needs of important areas of application, Fortran for scientific and engineering

computation and Lisp for artificial intelligence. These two areas continue to be important, and their programmers are so devoted to these two languages that Lisp and Fortran may well continue in active use for at least another quarter-century.

Lisp changes. The Scheme dialect used in this text has evolved from the original Lisp and differs from the latter in several important ways, including static scoping for variable binding and permitting functions to yield functions as values. In its semantic structure Scheme is as closely akin to Algol 60 as to early Lisps. Algol 60, never to be an active language again, lives on in the genes of Scheme and Pascal. It would be difficult to find two languages that are the communicating coin of two more different cultures than those gathered around these two languages. Pascal is for building pyramids—imposing, breathtaking, static structures built by armies pushing heavy blocks into place. Lisp is for building organisms—imposing, breathtaking, dynamic structures built by squads fitting fluctuating myriads of simpler organisms into place. The organizing principles used are the same in both cases, except for one extraordinarily important difference: The discretionary exportable functionality entrusted to the individual Lisp programmer is more than an order of magnitude greater than that to be found within Pascal enterprises. Lisp programs inflate libraries with functions whose utility transcends the application that produced them. The list, Lisp's native data structure, is largely responsible for such growth of utility. The simple structure and natural applicability of lists are reflected in functions that are amazingly nonidiosyncratic. In Pascal the plethora of declarable data structures induces a specialization within functions that inhibits and penalizes casual cooperation. It is better to have 100 functions operate on one data structure than to have 10 functions operate on 10 data structures. As a result the pyramid must stand unchanged for a millennium; the organism must evolve or perish.

To illustrate this difference, compare the treatment of material and exercises within this book with that in any first-course text using Pascal. Do not labor under the illusion that this is a text digestible at MIT only, peculiar to the breed found there. It is precisely what a serious book on programming Lisp must be, no matter who the student is or where it is used.

Note that this is a text about programming, unlike most Lisp books, which are used as a preparation for work in artificial intelligence. After all, the critical programming concerns of software engineering and artificial intelligence tend to coalesce as the systems under investigation become larger. This explains why there is such growing interest in Lisp outside of artificial intelligence.

As one would expect from its goals, artificial intelligence research generates many significant programming problems. In other programming cultures this spate of problems spawns new languages. Indeed, in any very large programming task a useful organizing principle is to control and isolate traffic within the task modules via the invention of language. These languages tend to become less primitive as one approaches the boundaries of the system where we humans interact most often. As a result, such systems contain complex language-processing functions replicated many times. Lisp has such a simple syntax and semantics that parsing can be treated as an

elementary task. Thus parsing technology plays almost no role in Lisp programs, and the construction of language processors is rarely an impediment to the rate of growth and change of large Lisp systems. Finally, it is this very simplicity of syntax and semantics that is responsible for the burden and freedom borne by all Lisp programmers. No Lisp program of any size beyond a few lines can be written without being saturated with discretionary functions. Invent and fit; have fits and reinvent! We toast the Lisp programmer who pens his thoughts within nests of parentheses.

— Alan J. Perlis, New Haven, Connecticut

Prefaces to Structure and Interpretation of Computer Programs, 1996 & 1984

Preface to the Second Edition of SICP, 1996

Is it possible that software is not like anything else, that it is meant to be discarded: that the whole point is to always see it as a soap bubble? — Alan J. Perlis

The material in this book has been the basis of MIT's entry-level computer science subject since 1980. We had been teaching this material for four years when the first edition was published, and twelve more years have elapsed until the appearance of this second edition. We are pleased that our work has been widely adopted and incorporated into other texts. We have seen our students take the ideas and programs in this book and build them in as the core of new computer systems and languages. In literal realization of an ancient Talmudic pun, our students have become our builders. We are lucky to have such capable students and such accomplished builders.

In preparing this edition, we have incorporated hundreds of clarifications suggested by our own teaching experience and the comments of colleagues at MIT and elsewhere. We have redesigned most of the major programming systems in the book, including the generic-arithmetic system, the interpreters, the register-machine simulator, and the compiler; and we have rewritten all the program examples to ensure that any Scheme implementation conforming to the IEEE Scheme standard (IEEE 1990) will be able to run the code.

This edition emphasizes several new themes. The most important of these is the central role played by different approaches to dealing with time in computational models: objects with state, concurrent programming, functional programming, lazy evaluation, and nondeterministic programming. We have included new sections on concurrency and nondeterminism, and we have tried to integrate this theme throughout the book.

The first edition of the book closely followed the syllabus of our MIT one-semester subject. With all the new material in the second edition, it will not be possible to cover everything in a single semester, so the instructor will have to pick and choose. In our own teaching, we sometimes skip the section on logic programming (section), we have students use the register-machine simulator but we do not cover its implementation (section), and we give only a cursory overview of the compiler (section). Even so, this is still an intense course. Some instructors may wish to cover only the first three or four chapters, leaving the other material for subsequent courses.

The World Wide Web site of MIT Press provides support for users of this book. This includes programs from the book, sample programming assignments, supplementary materials, and downloadable implementations of the Scheme dialect of Lisp.

— Harold Abelson and Gerald Jay Sussman

Preface to the First Edition of SICP, 1984

A computer is like a violin. You can imagine a novice trying first a phonograph and then a violin. The latter, he says, sounds terrible. That is the argument we have heard from our humanists and most of our computer scientists. Computer programs are good, they say, for particular purposes, but they aren't flexible. Neither is a violin, or a typewriter, until you learn how to use it. — Marvin Minsky, "Why Programming Is a Good Medium for Expressing Poorly-Understood and Sloppily-Formulated Ideas"

"The Structure and Interpretation of Computer Programs" is the entry-level subject in computer science at the Massachusetts Institute of Technology. It is required of all students at MIT who major in electrical engineering or in computer science, as one-fourth of the "common core curriculum," which also includes two subjects on circuits and linear systems and a subject on the design of digital systems. We have been involved in the development of this subject since 1978, and we have taught this material in its present form since the fall of 1980 to between 600 and 700 students each year. Most of these students have had little or no prior formal training in computation, although many have played with computers a bit and a few have had extensive programming or hardware-design experience.

Our design of this introductory computer-science subject reflects two major concerns. First, we want to establish the idea that a computer language is not just a way of getting a computer to perform operations but rather that it is a novel formal medium for expressing ideas about methodology. Thus, programs must be written for people to read, and only incidentally for machines to execute. Second, we believe that the essential material to be addressed by a subject at this level is not the syntax of particular programming-language constructs, nor clever algorithms for computing particular functions efficiently, nor even the mathematical analysis of algorithms and the foundations of computing, but rather the techniques used to control the intellectual complexity of large software systems.

Our goal is that students who complete this subject should have a good feel for the elements of style and the aesthetics of programming. They should have command of the major techniques for controlling complexity in a large system. They should be capable of reading a 50-page-long program, if it is written in an exemplary style. They should know what not to read, and what they need not understand at any moment. They should feel secure about modifying a program, retaining the spirit and style of the original author.

These skills are by no means unique to computer programming. The techniques we teach and draw upon are common to all of engineering design. We control complexity by building abstractions that hide details when appropriate. We control complexity by establishing conventional interfaces that enable us to construct systems by combining standard, well-understood pieces in a "mix and match" way. We control complexity by establishing new languages for describing a design, each of which emphasizes particular aspects of the design and deemphasizes others.

Underlying our approach to this subject is our conviction that "computer science" is not a science and that its significance has little to do with computers. The computer revolution is a revolution in the way we think and in the way we express what we think. The essence of this change is the emergence of what might best be called *procedural epistemology*—the study of the structure of knowledge from an imperative point of view, as opposed to the more declarative point of view taken by classical mathematical subjects. Mathematics provides a framework for dealing precisely with notions of "what is." Computation provides a framework for dealing precisely with notions of "how to."

In teaching our material we use a dialect of the programming language Lisp. We never formally teach the language, because we don't have to. We just use it, and students pick it up in a few days. This is one great advantage of Lisp-like languages: They have very few ways of forming compound expressions, and almost no syntactic structure. All of the formal properties can be covered in an hour, like the rules of chess. After a short time we forget about syntactic details of the language (because there are none) and get on with the real issues—figuring out what we want to compute, how we will decompose problems into manageable parts, and how we will work on the parts. Another advantage of Lisp is that it supports (but does not enforce) more of the large-scale strategies for modular decomposition of programs than any other language we know. We can make procedural and data abstractions, we can use higher-order functions to capture common patterns of usage, we can model local state using assignment and data mutation, we can link parts of a program with streams and delayed evaluation, and we can easily implement embedded languages. All of this is embedded in an interactive environment with excellent support for incremental program design, construction, testing, and debugging. We thank all the generations of Lisp wizards, starting with John McCarthy, who have fashioned a fine tool of unprecedented power and elegance.

Scheme, the dialect of Lisp that we use, is an attempt to bring together the power and elegance of Lisp and Algol. From Lisp we take the metalinguistic power that derives from the simple syntax, the uniform representation of programs as data objects, and the garbage-collected heap-allocated data. From Algol we take lexical scoping and block structure, which are gifts from the pioneers of programming-language design who were on the Algol committee. We wish to cite John Reynolds and Peter Landin for their insights into the relationship of Church's lambda calculus to the structure of programming languages. We also recognize our debt to the mathematicians who scouted out this territory decades before computers appeared on the scene. These pioneers include Alonzo Church, Barkley Rosser, Stephen Kleene, and Haskell Curry.

— Harold Abelson and Gerald Jay Sussman

1 Building Abstractions with Functions

*The acts of the mind, wherein it exerts its power over simple ideas, are chiefly these three: 1. Combining several simple ideas into one compound one, and thus all complex ideas are made. 2. The second is bringing two ideas, whether simple or complex, together, and setting them by one another so as to take a view of them at once, without uniting them into one, by which it gets all its ideas of relations. 3. The third is separating them from all other ideas that accompany them in their real existence: this is called abstraction, and thus all its general ideas are made. — John Locke, *An Essay Concerning Human Understanding*, 1690*

We are about to study the idea of a *computational process*. Computational processes are abstract beings that inhabit computers. As they evolve, processes manipulate other abstract things called *data*. The evolution of a process is directed by a pattern of rules called a *program*. People create programs to direct processes. In effect, we conjure the spirits of the computer with our spells.

A computational process is indeed much like a sorcerer's idea of a spirit. It cannot be seen or touched. It is not composed of matter at all. However, it is very real. It can perform intellectual work. It can answer questions. It can affect the world by disbursing money at a bank or by controlling a robot arm in a factory. The programs we use to conjure processes are like a sorcerer's spells. They are carefully composed from symbolic expressions in arcane and esoteric *programming languages* that prescribe the tasks we want our processes to perform.

A computational process, in a correctly working computer, executes programs precisely and accurately. Thus, like the sorcerer's apprentice, novice programmers must learn to understand and to anticipate the consequences of their conjuring. Even small errors (usually called *bugs*) in programs can have complex and unanticipated consequences.

Fortunately, learning to program is considerably less dangerous than learning sorcery, because the spirits we deal with are conveniently contained in a secure way. Real-world programming, however, requires care, expertise, and wisdom. A small bug in a computer-aided design program, for example, can lead to the catastrophic collapse of an airplane or a dam or the self-destruction of an industrial robot.

Master software engineers have the ability to organize programs so that they can be reasonably sure that the resulting processes will perform the tasks intended. They can visualize the behavior of their systems in advance. They know how to structure programs so that unanticipated problems do not lead to catastrophic consequences, and when problems do arise, they can *debug* their programs. Well-designed computational systems, like well-designed automobiles or nuclear

reactors, are designed in a modular manner, so that the parts can be constructed, replaced, and debugged separately.

Programming in Python

We need an appropriate language for describing processes, and we will use for this purpose the programming language Python. Just as our everyday thoughts are usually expressed in our natural language (such as English, German, or Chinese), and descriptions of quantitative phenomena are expressed with mathematical notations, our procedural thoughts will be expressed in Python. Python was conceived in late 1989 by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in Amsterdam, and first released in 1991. Van Rossum designed it as a high-level scripting language for the Amoeba distributed operating system—a language for the kind of administrative tasks that were too intricate for shell scripts yet not large enough to be worth writing out as a full, compiled program. From the outset Python was meant to cooperate with software written in other languages, and it became especially well known for its ability to serve as a glue that binds together components written in C. The name "Python" is not a reference to the snake but to the British comedy troupe Monty Python's Flying Circus, and is a registered trademark of the Python Software Foundation.

Despite its inception as a language for scripting and gluing together software components, Python is a general-purpose programming language. A Python interpreter is a machine that carries out processes described in the Python language. The first Python interpreter was implemented by van Rossum at CWI; written in C, it remains the reference implementation of the language and is known today as CPython. Python's most direct ancestor is ABC, a teaching language that van Rossum had helped develop at CWI, from which Python took such traits as the use of indentation to express block structure and a set of rich built-in data types; further influences came from Modula-3 and, for much of its syntax and its implementation, from C. Unlike the original version of this book, which used the Lisp dialect Scheme, Python does not descend from the Lisp tradition. It nonetheless provides the features on which this book most depends—first-class functions, lexical scoping, and dynamic typing.

Unlike Java and C, which are normally compiled to a lower-level language before they run, a Python program is first translated into an intermediate form called *bytecode*, which is then *interpreted* by a virtual machine; CPython works in exactly this way, and alternative implementations such as PyPy exist as well. Python is not overseen by a formal standards body. Instead the language is defined by its reference implementation together with the *Python Language Reference*, and it evolves through a community process organized around design documents called *Python Enhancement Proposals*, or PEPs. The language has had two major lines: Python 2, which appeared in 2000, and Python 3, which appeared in 2008 and deliberately broke backward compatibility with its predecessor; the two coexisted for more than a decade, until support for Python 2 ended in January 2020 (PEP 373). This book uses Python 3.

The growing use of Python for tasks that demand high performance—above all in scientific computing and data analysis—encouraged work on executing Python programs efficiently. Interpreting bytecode, as CPython does, is rarely the fastest way to carry out a computation. Two responses have proved important. The first, and in practice the more consequential, is to keep the program itself in Python while delegating its numerically intensive parts to libraries written in lower-level languages such as C and Fortran; the widely used NumPy library is the canonical example, and this strategy relies on exactly the cooperation with C described earlier. The second is Just-In-Time (JIT) compilation, in which the frequently executed portions of a program are translated into native machine code as the program runs. PyPy has long applied this technique to good effect, and as of this writing (2026) CPython has begun to incorporate a JIT compiler of its own, though it remains experimental and its performance and future are still being worked out. Python is today among the most widely used programming languages, prominent in fields ranging from web development and automation to scientific computing, data science, machine learning, and the teaching of programming.

Python possesses a set of features that make it an excellent medium for studying important programming constructs and data structures and for relating them to the linguistic features that support them. Its lexically scoped first-class functions give direct access to functional abstraction; that abstraction is expressed chiefly through the function-definition statement `def`, with the more restricted `lambda` expression available for short anonymous functions—a division of labor that echoes the distinction between `define` and `lambda` in Scheme itself. Python's dynamic typing allows the adaptation to remain close to the Scheme original throughout the book. Above and beyond these considerations, programming in Python is great fun.

1.1 The Elements of Programming

A powerful programming language is more than just a means for instructing a computer to perform tasks. The language also serves as a framework within which we organize our ideas about processes. Thus, when we describe a language, we should pay particular attention to the means that the language provides for combining simple ideas to form more complex ideas. Every powerful language has three mechanisms for accomplishing this:

- **primitive expressions**, which represent the simplest entities the language is concerned with,
- **means of combination**, by which compound elements are built from simpler ones, and
- **means of abstraction**, by which compound elements can be named and manipulated as units.

In programming, we deal with two kinds of elements: functions and data. (Later we will discover that they are really not so distinct.) Informally, data is "stuff" that we want to manipulate, and functions are descriptions of the rules for manipulating the data. Thus, any powerful programming

language should be able to describe primitive data and primitive functions and should have methods for combining and abstracting functions and data.

In this chapter we will deal only with simple numerical data so that we can focus on the rules for building functions. [^1: The characterization of numbers as "simple data" is a barefaced bluff. In fact, the treatment of numbers is one of the trickiest and most confusing aspects of any programming language. Some typical issues involved are these: Some computer systems distinguish *integers*, such as 2, from *real numbers*, such as 2.71. Is the real number 2.00 different from the integer 2? Are the arithmetic operations used for integers the same as the operations used for real numbers? Does 6 divided by 2 produce 3, or 3.0? How large a number can we represent? How many decimal places of accuracy can we represent? Is the range of integers the same as the range of real numbers? Above and beyond these questions, of course, lies a collection of issues concerning roundoff and truncation errors—the entire science of numerical analysis. Since our focus in this book is on large-scale program design rather than on numerical techniques, we are going to ignore these problems. The numerical examples in this chapter will exhibit the usual roundoff behavior that one observes when using arithmetic operations that preserve a limited number of decimal places of accuracy in noninteger operations.] In later chapters we will see that these same rules allow us to build functions to manipulate compound data as well.

1.1.1 Expressions

One easy way to get started at programming is to examine some typical interactions with an interpreter for the Python language. You type a *statement*, and pass it to the interpreter, which then *evaluates* that statement.

In this edition, the mouse click on Python statements with dark background is programmed in such a way that a Python interpreter is displayed, which can *evaluate* the statement and display the values it prints.

One kind of statement you might type is an *expression*. One kind of primitive expression is a number. (More precisely, the expression that you type consists of the numerals that represent the number in base 10.) If you ask our script to evaluate the expression statement

```
486
```

by clicking it, it will respond by displaying a Python interpreter with the option to evaluate the statement by pressing a "Run" button. Click on the primitive expression statement, and what happens is—nothing! To see the result of evaluating `486`, we need to apply the function `print` to the expression, using the usual mathematical notation of function application

```
print(486)
```

resulting in the display of `[^2`: Throughout this book, we distinguish between the input typed by the user and any text printed by the interpreter by showing the latter in slanted characters.]

```
486
```

in a separate part in the browser window.

Expressions representing numbers may be combined with operators (such as `+` or `*`) to form a compound expression that represents the application of a corresponding built-in function to those numbers. For example, evaluate any of the following expression statements by clicking on it:

```
print(137 + 349)
```

```
486
```

```
print(1000 - 334)
```

```
666
```

```
print(5 * 99)
```

```
495
```

```
print(10 / 4)
```

```
2.5
```

```
print(2.7 + 10)
```

```
12.7
```

Expressions such as `137 + 349`, which contain other expressions as components, are called *combinations*. Combinations that are formed by an *operator* symbol in the middle, and *operand* expressions to the left and right of it, are called *operator combinations*. The value of an operator combination is obtained by applying the function specified by the operator to the arguments that are the values of the operands.

The convention of placing the operator between the operands is known as *infix notation*. It follows the mathematical notation that you are most likely familiar with from school and everyday life. As in mathematics, operator combinations can be *nested*, that is, they can have operands that themselves are operator combinations:

```
print((3 * 5) + (10 - 6))
```

```
19
```

As usual, parentheses are used to group operator combinations in order to avoid ambiguities. Python also follows the usual conventions when parentheses are omitted: multiplication and division bind more strongly than addition and subtraction. For example,

```
3 * 5 + 10 / 2
```

stands for

```
(3 * 5) + (10 / 2)
```

We say that `*` and `/` have *higher precedence* than `+` and `-`. Sequences of additions and subtractions are read from left to right, as are sequences of multiplications and divisions. Thus,

```
1 - 5 / 2 * 4 + 3
```

stands for

```
(1 - ((5 / 2) * 4)) + 3
```

We say that the operators `+`, `-`, `*` and `/` are *left-associative*.

There is no limit (in principle) to the depth of such nesting and to the overall complexity of the expressions that the Python interpreter can evaluate. It is we humans who might get confused by still relatively simple expressions such as

```
3 * (2 * 4 + (3 + 5)) + ((10 - 7) + 6)
```

which the interpreter would readily evaluate to be 57. We can help ourselves by writing such an expression in the form

```
(3 * (2 * 4 + (3 + 5))
+
((10 - 7) + 6))
```

to visually separate the major components of the expression. [^3: The additional parentheses are necessary, if we want to spread an expression over multiple lines and there are no surrounding parentheses already.]

Even with complex expressions, the interpreter always operates in the same basic cycle: It reads a statement typed by the user, evaluates the statement, and prints the result of any applications of `print`. This mode of operation is often expressed by saying that the interpreter runs in a *read-evaluate-print loop*. Observe, however, that it is necessary to explicitly instruct the interpreter to print the value of the expression.

1.1.2 Naming and the Environment

A critical aspect of a programming language is the means it provides for using names to refer to computational objects. We say that the name identifies a *variable* whose *value* is the object.

In Python, we name things with *declaration assignments*.

```
size = 2
```

causes the interpreter to associate the value 2 with the name `size`. [^4: Python uses the same syntax `=` to reassign the value of a `` even when the name has already been assigned previously with a declaration assignment. In this and the next chapter, we do not use this option. Chapter 3 discusses reassignment.] Once the name `size` has been associated with the number 2, we can refer to the value 2 by name:

```
print(size)
```

```
2
```

```
print(5 * size)
```

```
10
```

The Python interpreter needs to execute the declaration assignment for `size` before the name `size` can be used in an expression. In this online book, the statements that need to be evaluated before a new statement are omitted for brevity. However, to see and play with the program, you

can click on it. The program then appears inline with all dependencies included. Thus, as a result of clicking on

```
print(5 * size)
```

you see:

```
size = 2  
print(5 * size)
```

Here are further examples of the use of declaration assignments:

```
pi = 3.14159
```

```
radius = 10
```

```
print(pi * radius * radius)
```

```
314.159
```

```
circumference = 2 * pi * radius
```

```
print(circumference)
```

```
62.8318
```

Declaration assignment is our language's simplest means of abstraction, for it allows us to use simple names to refer to the results of compound operations, such as the `circumference` computed above. In general, computational objects may have very complex structures, and it would be extremely inconvenient to have to remember and repeat their details each time we want to use them. Indeed, complex programs are constructed by building, step by step, computational objects of increasing complexity. The interpreter makes this step-by-step program construction particularly convenient because name-object associations can be created incrementally in successive interactions. This feature encourages the incremental development and testing of programs and is largely responsible for the fact that a Python program usually consists of a large number of relatively simple functions.

It should be clear that the possibility of associating values with names and later retrieving them means that the interpreter must maintain some sort of memory that keeps track of the name-

object pairs. This memory is called the *environment* (more precisely the *global environment*, since we will see later that a computation may involve a number of different environments). [^5: Chapter 3 will show that this notion of environment is crucial for understanding how the interpreter works. Chapter will use environments for implementing interpreters.]

1.1.3 Evaluating Operator Combinations

One of our goals in this chapter is to isolate issues about thinking procedurally. As a case in point, let us consider that, in evaluating operator combinations, the interpreter is itself following a procedure.

- To evaluate an operator combination, do the following:
- Evaluate the operand expressions of the combination.
- Apply the function that is denoted by the operator to the arguments that are the values of the operands.

Even this simple rule illustrates some important points about processes in general. First, observe that the first step dictates that in order to accomplish the evaluation process for a combination we must first perform the evaluation process on each operand of the combination. Thus, the evaluation rule is *recursive* in nature; that is, it includes, as one of its steps, the need to invoke the rule itself.

Notice how succinctly the idea of recursion can be used to express what, in the case of a deeply nested combination, would otherwise be viewed as a rather complicated process. For example, evaluating

```
(2 + 4 * 6) * (3 + 12)
```

requires that the evaluation rule be applied to four different combinations. We can obtain a picture of this process by representing the combination in the form of a tree, as shown in figure 1.1. Each combination is represented by a node with branches corresponding to the operator and the operands of the combination stemming from it. The terminal nodes (that is, nodes with no branches stemming from them) represent either operators or numbers. Viewing evaluation in terms of the tree, we can imagine that the values of the operands percolate upward, starting from the terminal nodes and then combining at higher and higher levels. In general, we shall see that recursion is a very powerful technique for dealing with hierarchical, treelike objects. In fact, the "percolate values upward" form of the evaluation rule is an example of a general kind of process known as *tree accumulation*.

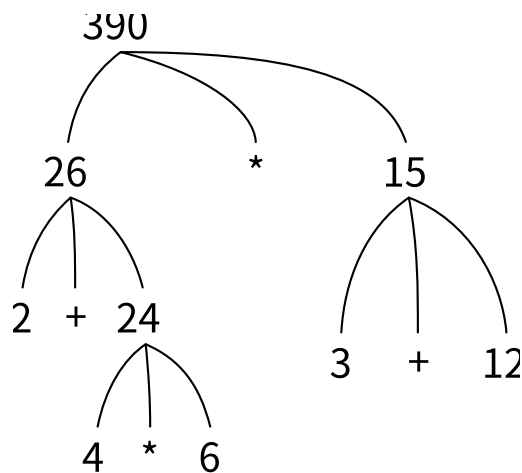


Figure 1.1. Tree representation, showing the value of each subexpression.

Next, observe that the repeated application of the first step brings us to the point where we need to evaluate, not combinations, but primitive expressions such as numerals or names. We take care of the primitive cases by stipulating that

- the values of numerals are the numbers that they name, and
- the values of names are the objects associated with those names in the environment. The key point to notice is the role of the environment in determining the meaning of the names in expressions. In an interactive language such as Python, it is meaningless to speak of the value of an expression such as `x + 1` without specifying any information about the environment that would provide a meaning for the name `x`. As we shall see in chapter 3, the general notion of the environment as providing a context in which evaluation takes place will play an important role in our understanding of program execution.

Notice that the evaluation rule given above does not handle declaration assignments. For instance, evaluating `x = 3` does not apply an operator `=` to two arguments, one of which is the value of the name `x` and the other of which is 3, since the purpose of the declaration assignment is precisely to associate `x` with a value. (That is, `x = 3` is not a combination.)

The shape of the declaration assignment instructs the Python interpreter to treat the statement in a special way. Each such *syntactic form* has its own evaluation rule. The various kinds of statements and expressions (each with its associated evaluation rule) constitute the syntax of the programming language.

1.1.4 Compound Functions

We have identified in Python some of the elements that must appear in any powerful programming language:

- Numbers and arithmetic operations are primitive data and functions.
- Nesting of combinations provides a means of combining operations.

- Declaration assignments that associate names with values provide a limited means of abstraction. Now we will learn about *function definitions*, a much more powerful abstraction technique by which a compound operation can be given a name and then referred to as a unit.

We begin by examining how to express the idea of "squaring." We might say, "To square something, take it times itself." This is expressed in our language as

```
def square(x): return x * x
```

We can understand this in the following way:

```
def    square(    x    ): return x    *    x
#      ^      ^      ^      ^      ^      ^      ^
# To    square something, take it times itself.
```

We have here a *compound function*, which has been given the name `square`. The function represents the operation of multiplying something by itself. The thing to be multiplied is given a local name, `x`, which plays the same role that a pronoun plays in natural language. Evaluating the definition creates this compound function and associates it with the name `square`. [^6: Observe that there are two different operations being combined here: we are creating the function, and we are giving it the name `square`. It is possible, indeed important, to be able to separate these two notions—to create functions without naming them, and to give names to functions that have already been created. We will see how to do this in section 1.3.2.]

The simplest form of a function definition is

```
def *name*(*parameters*): return *expression*
```

The *name* is a symbol to be associated with the function definition in the environment. [^7: Throughout this book, we will describe the general syntax of expressions by using italic symbols—e.g., *name*—to denote the "slots" in the expression to be filled in when such an expression is actually used.] The *parameters* are the names used within the body of the function to refer to the corresponding arguments of the function. The word `def` is a *keyword* in Python. Keywords carry a particular meaning, and thus cannot be used as names. A keyword in a program component instructs the Python interpreter to treat the component as a syntactic form with its own evaluation rule. The *parameters* are grouped within parentheses and separated by commas, as they will be in an application of the function being defined. In the simplest form, the *body* of a function definition is a single *return statement*, [^8: More generally, the body of the function can be a sequence of statements. In this case, the interpreter evaluates each statement in the sequence in turn until a return statement determines the value of the function application.] which consists of the keyword

`return` followed by the *return expression* that will yield the value of the function application, when the parameters are replaced by the actual arguments to which the function is applied.

Having defined `square`, we can now use it in a *function application* expression:

```
square(21)
```

Function applications are—after operator combinations—the second kind of combination of expressions into larger expressions that we encounter. The general form of a function application is

```
*function-expression>(*argument-expressions*)
```

where the *function-expression* of the application specifies the function to be applied to the comma-separated *argument-expressions*. To evaluate a function application, the interpreter follows a procedure quite similar to the procedure for operator combinations described in section 1.1.3.

- To evaluate a function application, do the following:
- Evaluate the subexpressions of the application, namely the function expression and the argument expressions.
- Apply the function that is the value of the function expression to the values of the argument expressions.

The same procedure applies to applications of the built-in function `print` that we encountered already in section 1.1.1.

```
print(square(2 + 5))
```

```
49
```

Here, the argument expression of `print` is a compound expression, the application expression `square(2 + 5)`, whose argument expression is itself a compound expression, the operator combination `2 + 5`.

```
print(square(square(3)))
```

```
81
```

Of course function application expressions can be further nested.

We can also use `square` as a building block in defining other functions. For example, $x^2 + y^2$ can be expressed as

```
square(x) + square(y)
```

We can easily define a function `sum_of_squares` that, given any two numbers as arguments, produces the sum of their squares:

```
def sum_of_squares(x, y):  
    return square(x) + square(y)
```

For readability, we can start a new line after the colon, in which case the function body needs to be indented. [^9: The number of characters used in the indentation is flexible but needs to be consistent throughout the function body. In this book, we mostly use indentation by four characters.]

```
print(sum_of_squares(3, 4))
```

```
25
```

Now we can use `sum_of_squares` as a building block in constructing further functions:

```
def f(a):  
    return sum_of_squares(a + 1, a * 2)
```

```
print(f(5))
```

```
136
```

In addition to compound functions, any Python environment provides built-in functions that come with the interpreter or are loaded from libraries. Besides the built-in functions provided by the operators and the built-in function `print`, the Python environment used in this book includes additional built-in functions such as the function `math_log`, which computes the natural logarithm of its argument. These additional built-in functions are used in exactly the same way as compound functions; evaluating the application `print(math_log(1))` results in the display of the number 0.0. Indeed, one could not tell by looking at the definition of `sum_of_squares` given above whether `square` was built into the interpreter, loaded from a library, or defined as a compound function.

1.1.5 The Substitution Model for Function Application

To evaluate a function application, the interpreter follows the process described in section 1.1.4. That is, the interpreter evaluates the elements of the application and applies the function (which is

the value of the function expression of the application) to the arguments (which are the values of the argument expressions of the application).

We can assume that the application of built-in functions is handled by the interpreter or libraries. For compound functions, the application process is as follows:

- To apply a compound function to arguments, evaluate the return expression of the function with each parameter replaced by the corresponding argument. [¹⁰: If the body of the function is a sequence of statements, the body is evaluated with the parameters replaced, and the value of the application is the value of the return expression of the first return statement encountered.] To illustrate this process, let's evaluate the application

```
f(5)
```

where `f` is the function defined in section 1.1.4. We begin by retrieving the return expression of `f`:

```
sum_of_squares(a + 1, a * 2)
```

Then we replace the parameter `a` by the argument 5:

```
sum_of_squares(5 + 1, 5 * 2)
```

Thus the problem reduces to the evaluation of an application with two arguments and a function expression `sum_of_squares`. Evaluating this application involves three subproblems. We must evaluate the function expression to get the function to be applied, and we must evaluate the argument expressions to get the arguments. Now `5 + 1` produces 6 and `5 * 2` produces 10, so we must apply the `sum_of_squares` function to 6 and 10. These values are substituted for the parameters `x` and `y` in the body of `sum_of_squares`, reducing the expression to

```
square(6) + square(10)
```

If we use the definition of `square`, this reduces to

```
(6 * 6) + (10 * 10)
```

which reduces by multiplication to

```
36 + 100
```

and finally to

The process we have just described is called the *substitution model* for function application. It can be taken as a model that determines the "meaning" of function application, insofar as the functions in this chapter are concerned. However, there are two points that should be stressed:

- The purpose of the substitution is to help us think about function application, not to provide a description of how the interpreter really works. Typical interpreters do not evaluate function applications by manipulating the text of a function to substitute values for the parameters. In practice, the "substitution" is accomplished by using a local environment for the parameters. We will discuss this more fully in chapters 3 and 4 when we examine the implementation of an interpreter in detail.
- Over the course of this book, we will present a sequence of increasingly elaborate models of how interpreters work, culminating with a complete implementation of an interpreter and compiler in chapter . The substitution model is only the first of these models—a way to get started thinking formally about the evaluation process. In general, when modeling phenomena in science and engineering, we begin with simplified, incomplete models. As we examine things in greater detail, these simple models become inadequate and must be replaced by more refined models. The substitution model is no exception. In particular, when we address in chapter 3 the use of functions with "mutable data," we will see that the substitution model breaks down and must be replaced by a more complicated model of function application. [^11: Despite the simplicity of the substitution idea, it turns out to be surprisingly complicated to give a rigorous mathematical definition of the substitution process. The problem arises from the possibility of confusion between the names used for the parameters of a function and the (possibly identical) names used in the expressions to which the function may be applied. Indeed, there is a long history of erroneous definitions of *substitution* in the literature of logic and programming semantics. See Stoy 1977 for a careful discussion of substitution.]

Applicative order versus normal order

According to the description of evaluation given in section 1.1.4, the interpreter first evaluates the function and argument expressions and then applies the resulting function to the resulting arguments. This is not the only way to perform evaluation. An alternative evaluation model would not evaluate the arguments until their values were needed. Instead it would first substitute argument expressions for parameters until it obtained an expression involving only operators and built-in functions, and would then perform the evaluation. If we used this method, the evaluation of

`f(5)`

would proceed according to the sequence of expansions

```
sum_of_squares(5 + 1, 5 * 2)

square(5 + 1)      + square(5 * 2)

(5 + 1) * (5 + 1) + (5 * 2) * (5 * 2)
```

followed by the reductions

```
6   *   6   +   10   *   10

36           +       100

136
```

This gives the same answer as our previous evaluation model, but the process is different. In particular, the evaluations of `5 + 1` and `5 * 2` are each performed twice here, corresponding to the reduction of the expression

```
x * x
```

with `x` replaced respectively by `5 + 1` and `5 * 2`.

This alternative "fully expand and then reduce" evaluation method is known as *normal-order evaluation*, in contrast to the "evaluate the arguments and then apply" method that the interpreter actually uses, which is called *applicative-order evaluation*. It can be shown that, for function applications that can be modeled using substitution (including all the functions in the first two chapters of this book) and that yield legitimate values, normal-order and applicative-order evaluation produce the same value. (See exercise 1.5 for an instance of an "illegitimate" value where normal-order and applicative-order evaluation do not give the same result.)

Python uses applicative-order evaluation, partly because of the additional efficiency obtained from avoiding multiple evaluations of expressions such as those illustrated with `5 + 1` and `5 * 2` above and, more significantly, because normal-order evaluation becomes much more complicated to deal with when we leave the realm of functions that can be modeled by substitution. On the other hand, normal-order evaluation can be an extremely valuable tool, and we will investigate some of its implications in chapters 3 and 4. [¹²: In chapter 3 we will introduce *stream processing*, which is a way of handling apparently "infinite" data structures by incorporating a limited form of normal-order evaluation. In section we will modify the Python interpreter to produce a normal-order variant of Python.]

1.1.6 Conditional Expressions and Predicates

The expressive power of the class of functions that we can define at this point is very limited, because we have no way to make tests and to perform different operations depending on the result of a test. For instance, we cannot define a function that computes the absolute value of a number by testing whether the number is nonnegative and taking different actions in each case according to the rule

$$|x| = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{otherwise} \end{cases}$$

This construct is a *case analysis* and can be written in Python using a *conditional expression* as

```
def abs(x):  
    return x if x >= 0 else -x
```

which could be expressed in English as "Return x , if x is greater than or equal to zero; otherwise return $-x$." The general form of a conditional expression is

```
*consequent-expression* if *predicate* else *alternative-expression*
```

The keyword `if` in conditional expressions is followed by a *predicate*—that is, an expression whose value is interpreted as either true or false. [¹³: "Interpreted as either true or false" means this: In Python, there are two distinguished values that are denoted by the constants `True` and `False`. When the interpreter checks a predicate's value, it interprets `False` as false and `True` as true. Python considers any value to be either true or false, but in this book we only use these two.] The keyword `if` is preceded by the *consequent-expression*, and followed by the `else` keyword and finally by the *alternative-expression*.

To evaluate a conditional expression, the interpreter starts by evaluating the *predicate* of the expression. If the *predicate* evaluates to true, the interpreter evaluates the *consequent-expression* and returns its value as the value of the conditional. If the *predicate* evaluates to false, it evaluates the *alternative-expression* and returns its value as the value of the conditional. [¹⁴: Conditionals in full Python accept any value, not just the boolean values 1 and 0, as the result of evaluating the *predicate* expression (see footnote in section for details). The programs in this book use only boolean values as predicates of conditionals.]

The word *predicate* is used for operators and functions that return true or false, as well as for expressions that evaluate to true or false. The absolute-value function `abs` makes use of the built-in predicate `>=`, an operator that takes two numbers as arguments and tests whether the first number is greater than or equal to the second number, returning true or false accordingly.

If we prefer to handle the zero case separately, we can specify the function that computes the absolute value of a number by writing

$$|x| = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -x & \text{otherwise} \end{cases}$$

In Python, we express a case analysis with multiple cases by nesting conditional expressions as alternative expressions inside other conditional expressions:

```
def abs(x):
    return x if x > 0 else 0 if x == 0 else -x
```

Parentheses are not needed around the alternative expression `0 if x == 0 else -x`, because the conditional-expression syntactic form is right-associative. The general form of a case analysis is

$$e_1 \text{ if } p_1 \text{ else } e_2 \text{ if } p_2 \text{ } \cdots \text{ } e_n \text{ if } p_n \text{ else } \textit{final-alternative-expression}$$

We call a consequent expression e_i together with its predicate p_i a *clause*. A case analysis can be seen as a sequence of clauses, followed by a final alternative expression. According to the evaluation of conditional expressions, a case analysis is evaluated by first evaluating the predicate p_1 . If its value is false, then p_2 is evaluated. If p_2 's value is also false, then p_3 is evaluated. This process continues until a predicate is found whose value is true, in which case the interpreter returns the value of the corresponding consequent expression e of the clause as the value of the case analysis. If none of the p 's is found to be true, the value of the case analysis is the value of the final alternative expression.

In addition to built-in predicates such as `>=`, `>`, `<`, `<=`, `==`, and `!=` that are applied to numbers, [^15: For now, we restrict these operators to number arguments. In sections 3.3.1, we shall generalize the equality and inequality predicates `==` and `!=`.] there are logical composition operations, which enable us to construct compound predicates. The three most frequently used are these:

- $expression_1$ and $expression_2$ This operation expresses *logical conjunction*, meaning roughly the same as the English word "and." We assume [^16: This assumption is justified by the restriction mentioned in footnote . Full Python needs to consider the case where the result of evaluating $expression_1$ is neither true nor false.] this syntactic form to be syntactic sugar [^17: Syntactic forms that are simply convenient alternative surface structures for things that can be written in more uniform ways are sometimes called *syntactic sugar*, to use a phrase coined by Peter Landin.] for `expression_2 if expression_1 else False`.
- $expression_1$ or $expression_2$ This operation expresses *logical disjunction*, meaning roughly the same as the English word "or." We assume this syntactic form to be syntactic sugar for `True if expression_1 else expression_2`.

- `not expression` This operation expresses *logical negation*, meaning roughly the same as the English word "not." The value of the expression is true when *expression* evaluates to false, and false when *expression* evaluates to true. Notice that `and` and `or` are syntactic forms, not operators; their right-hand expression is not always evaluated. The operator `not`, on the other hand, follows the evaluation rule of section 1.1.3. It is a *unary* operator, which means that it takes only one argument, whereas the arithmetic operators and built-in predicates discussed so far are *binary*, taking two arguments. The operator `not` precedes its argument; we call it a *prefix operator*. Another prefix operator is the numeric negation operator, an example of which is the expression `- x` in the `abs` functions above.

As an example of how these predicates are used, the condition that a number x be in the range $5 < x < 10$ may be expressed as

```
x > 5 and x < 10
```

The syntactic form `and` has lower precedence than the comparison operators `>` and `<`, and the conditional-expression syntactic form `$\cdots$ if $\cdots$ else $\cdots$` has lower precedence than any other operator we have encountered so far, a property we used in the `abs` functions above.

As another example, we can define a predicate to test whether one number is greater than or equal to another as

```
def greater_or_equal(x, y):
    return x > y or x == y
```

or alternatively as

```
def greater_or_equal(x, y):
    return not (x < y)
```

The function `greater_or_equal`, when applied to two numbers, behaves the same as the operator `>=`. Unary operators have higher precedence than binary operators, which makes the parentheses in this example necessary.

Exercise 1.1. Below is a sequence of statements. What is the result printed by the interpreter in response to each statement? Assume that the sequence is to be evaluated in the order in which it is presented.

```
print(10)
```

```
print(5 + 3 + 4)
```

```
print(9 - 1)
```

```
print(6 / 2)
```

```
print(2 * 4 + (4 - 6))
```

```
a = 3
```

```
b = a + 1
```

```
print(a + b + a * b)
```

```
print(a == b)
```

```
print(b if b > a and b < a * b else a)
```

```
print(6 if a == 4 else 6 + 7 + a if b == 4 else 25)
```

```
print(2 + (b if b < a else a))
```

```
print((a if a > b else b if a < b else -1) * (a + 1))
```

The parentheses around the conditional expressions in the last two statements are necessary because the conditional-expression syntactic form has lower precedence than the arithmetic operators `+` and `*`.

Exercise 1.2. Translate the following expression into Python
$$\frac{5+4+\left(2-\left(3-\left(6+\frac{4}{5}\right)\right)\right)}{(6-2)(2-7)}$$

Exercise 1.3. Define a function that takes three numbers as arguments and returns the sum of the squares of the two larger numbers.

Exercise 1.4. Observe that our model of evaluation allows for applications whose function expressions are compound expressions. Use this observation to describe the behavior of `a_plus_abs_b`:

```
def plus(a, b): return a + b

def minus(a, b): return a - b

def a_plus_abs_b(a, b):
    return (plus if b >= 0 else minus)(a, b)
```

Exercise 1.5. Ben Bitdiddle has invented a test to determine whether the interpreter he is faced with is using applicative-order evaluation or normal-order evaluation. He defines the following two functions:

```
def p(): return p()

def test(x, y):
    return 0 if x == 0 else y
```

Then he evaluates the statement

```
test(0, p())
```

What behavior will Ben observe with an interpreter that uses applicative-order evaluation? What behavior will he observe with an interpreter that uses normal-order evaluation? Explain your answer. (Assume that the evaluation rule for conditional expressions is the same whether the interpreter is using normal or applicative order: The predicate expression is evaluated first, and the result determines whether to evaluate the consequent or the alternative expression.)

1.1.7 Example: Square Roots by Newton's Method

Functions, as introduced above, are much like ordinary mathematical functions. They specify a value that is determined by one or more parameters. But there is an important difference between mathematical functions and computer functions. Computer functions must be effective.

As a case in point, consider the problem of computing square roots. We can define the square-root function as

$\sqrt{x}$ = the y such that $y \geq 0$ and $y^2 = x$

This describes a perfectly legitimate mathematical function. We could use it to recognize whether one number is the square root of another, or to derive facts about square roots in general. On the other hand, the definition does not describe a computer function. Indeed, it tells us almost nothing about how to actually find the square root of a given number. It will not help matters to rephrase this definition in pseudo-Python:

```
def sqrt(x):
    return the y with y >= 0 and square(y) == x
```

This only begs the question.

The contrast between mathematical function and computer function is a reflection of the general distinction between describing properties of things and describing how to do things, or, as it is sometimes referred to, the distinction between declarative knowledge and imperative knowledge. In mathematics we are usually concerned with declarative (what is) descriptions, whereas in computer science we are usually concerned with imperative (how to) descriptions. [^18: Declarative and imperative descriptions are intimately related, as indeed are mathematics and computer science. For instance, to say that the answer produced by a program is "correct" is to make a declarative statement about the program. There is a large amount of research aimed at establishing techniques for proving that programs are correct, and much of the technical difficulty of this subject has to do with negotiating the transition between imperative statements (from which programs are constructed) and declarative statements (which can be used to deduce things). In a related vein, programming language designers have explored so-called very high-level languages, in which one actually programs in terms of declarative statements. The idea is to make interpreters sophisticated enough so that, given "what is" knowledge specified by the programmer, they can generate "how to" knowledge automatically. This cannot be done in general, but there are important areas where progress has been made. We shall revisit this idea in chapter .]

How does one compute square roots? The most common way is to use Newton's method of successive approximations, which says that whenever we have a guess y for the value of the square root of a number x , we can perform a simple manipulation to get a better guess (one closer to the actual square root) by averaging y with x/y . [^19: This square-root algorithm is actually a special case of Newton's method, which is a general technique for finding roots of equations. The square-root algorithm itself was developed by Heron of Alexandria in the first century CE. We will see how to express the general Newton's method as a Python function in section 1.3.4.] For example, we can compute the square root of 2 as follows. Suppose our initial guess is 1:

$$\begin{array}{llll} \text{Guess} & \& \text{Quotient} & \& \text{Average} \\ \frac{2}{1} = 2 & \& \frac{(2+1)}{2} = 1.5 & \& 1.5 \\ \frac{2}{1.3333} & \& \frac{(1.3333+1.5)}{2} = 1.4167 & \& 1.4167 \\ \frac{2}{1.4167} & \& \frac{(1.4167+1.4118)}{2} = 1.4142 & \& 1.4142 & \& \text{Idots} \end{array}$$

Continuing this process, we obtain better and better approximations to the square root.

Now let's formalize the process in terms of functions. We start with a value for the radicand (the number whose square root we are trying to compute) and a value for the guess. If the guess is

good enough for our purposes, we are done; if not, we must repeat the process with an improved guess. We write this basic strategy as a function:

```
def sqrt_iter(guess, x):  
    return guess  
        if is_good_enough(guess, x)  
        else sqrt_iter(improve(guess, x), x)
```

A guess is improved by averaging it with the quotient of the radicand and the old guess:

```
def improve(guess, x):  
    return average(guess, x / guess)
```

where

```
def average(x, y):  
    return (x + y) / 2
```

We also have to say what we mean by "good enough." The following will do for illustration, but it is not really a very good test. (See exercise 1.7.) The idea is to improve the answer until it is close enough so that its square differs from the radicand by less than a predetermined tolerance (here 0.001): [^20: We will usually give predicates names starting with `is_`, to help us remember that they are predicates.]

```
def is_good_enough(guess, x):  
    return abs(square(guess) - x) < 0.001
```

Finally, we need a way to get started. For instance, we can always guess that the square root of any number is 1:

```
def sqrt(x):  
    return sqrt_iter(1, x)
```

If we type these definitions to the interpreter, we can use `sqrt` just as we can use any function:

```
print(sqrt(9))
```

```
3.00009155413138
```

```
print(sqrt(100 + 37))
```



```
11.704699917758145
```

```
print(sqrt(sqrt(2) + sqrt(3)))
```

```
1.7739279023207892
```

```
print(square(sqrt(1000)))
```

```
1000.000369924366
```

The `sqrt` program also illustrates that the simple functional language we have introduced so far is sufficient for writing any purely numerical program that one could write in, say, C or Pascal. This might seem surprising, since we have not included in our language any iterative (looping) constructs that direct the computer to do something over and over again. The function `sqrt_iter`, on the other hand, demonstrates how iteration can be accomplished using no special construct other than the ordinary ability to call a function. [^21: Readers who are worried about the efficiency issues involved in using function calls to implement iteration should note the remarks on "tail recursion" in section 1.2.1.]

Exercise 1.6. Alyssa P. Hacker doesn't see why `if` needs to be provided as a syntactic form. "Why can't I just define it as an ordinary conditional function whose application works just like conditional expressions?" she asks. [^22: As a Lisp hacker from the original *Structure and Interpretation of Computer Programs*, Alyssa prefers a simpler, more uniform syntax.] Alyssa's friend Eva Lu Ator claims this can indeed be done, and she defines a `conditional` function as follows:

```
def conditional(predicate, then_clause, else_clause):  
    return then_clause if predicate else else_clause
```

Eva demonstrates the program for Alyssa:

```
print(conditional(2 == 3, 0, 5))
```

```
5
```

```
print(conditional(1 == 1, 0, 5))
```

```
0
```

Delighted, Alyssa uses `conditional` to rewrite the square-root program:

```
def sqrt_iter(guess, x):  
    return conditional(is_good_enough(guess, x),  
                      guess,  
                      sqrt_iter(improve(guess, x),  
                                x))
```

What happens when Alyssa attempts to use this to compute square roots? Explain.

Exercise 1.7. The `is_good_enough` test used in computing square roots will not be very effective for finding the square roots of very small numbers. Also, in real computers, arithmetic operations are almost always performed with limited precision. This makes our test inadequate for very large numbers. Explain these statements, with examples showing how the test fails for small and large numbers. An alternative strategy for implementing `is_good_enough` is to watch how `guess` changes from one iteration to the next and to stop when the change is a very small fraction of the guess. Design a square-root function that uses this kind of end test. Does this work better for small and large numbers?

Exercise 1.8. Newton's method for cube roots is based on the fact that if y is an approximation to the cube root of x , then a better approximation is given by the value

$$\frac{x}{y^2} + \frac{2y}{3}$$

Use this formula to implement a cube-root function analogous to the square-root function. (In section 1.3.4 we will see how to implement Newton's method in general as an abstraction of these square-root and cube-root functions.)

1.1.8 Functions as Black-Box Abstractions

The function `sqrt` is our first example of a process defined by a set of mutually defined functions. Notice that the definition of `sqrt_iter` is *recursive*; that is, the function is defined in terms of itself. The idea of being able to define a function in terms of itself may be disturbing; it may seem unclear how such a "circular" definition could make sense at all, much less specify a well-defined process to be carried out by a computer. This will be addressed more carefully in section 1.2. But first let's consider some other important points illustrated by the `sqrt` example.

Observe that the problem of computing square roots breaks up naturally into a number of subproblems: how to tell whether a guess is good enough, how to improve a guess, and so on. Each of these tasks is accomplished by a separate function. The entire `sqrt` program can be viewed as a cluster of functions (shown in figure 1.2) that mirrors the decomposition of the problem into subproblems.

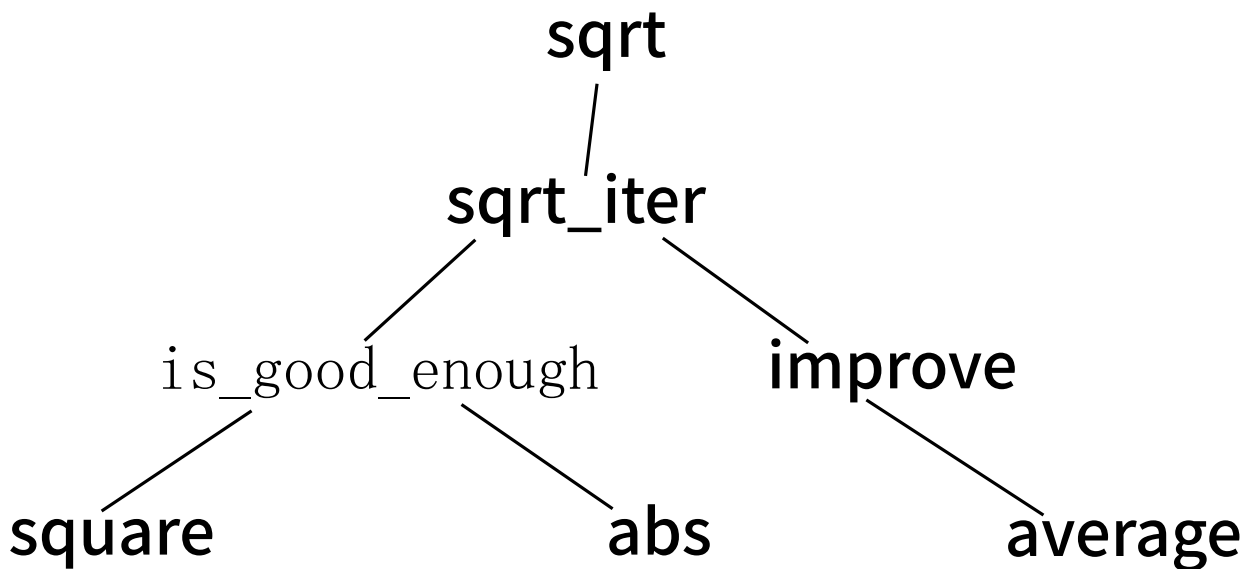


Figure 1.2. Functional decomposition of the `sqrt` program.

The importance of this decomposition strategy is not simply that one is dividing the program into parts. After all, we could take any large program and divide it into parts—the first ten lines, the next ten lines, the next ten lines, and so on. Rather, it is crucial that each function accomplishes an identifiable task that can be used as a module in defining other functions. For example, when we define the `is_good_enough` function in terms of `square`, we are able to regard the `square` function as a "black box." We are not at that moment concerned with *how* the function computes its result, only with the fact *that* it computes the square. The details of how the square is computed can be suppressed, to be considered at a later time. Indeed, as far as the `is_good_enough` function is concerned, `square` is not quite a function but rather an abstraction of a function, a so-called *functional abstraction*. At this level of abstraction, any function that computes the square is equally good.

Thus, considering only the values they return, the following two functions squaring a number should be indistinguishable. Each takes a numerical argument and produces the square of that number as the value. [^23: It is not even clear which of these functions is a more efficient implementation. This depends upon the hardware available. There are machines for which the "obvious" implementation is the less efficient one. Consider a machine that has extensive tables of logarithms and antilogarithms stored in a very efficient manner.]

```
def square(x): return x * x
```

```
def square(x):  
    return math_exp(double(math_log(x)))  
  
def double(x): return x + x
```

So a function should be able to suppress detail. The users of the function may not have written the function themselves, but may have obtained it from another programmer as a black box. A user should not need to know how the function is implemented in order to use it.

Local names

One detail of a function's implementation that should not matter to the user of the function is the implementer's choice of names for the function's parameters. Thus, the following functions should not be distinguishable:

```
def square(x): return x * x
```

```
def square(y): return y * y
```

This principle—that the meaning of a function should be independent of the parameter names used by its author—seems on the surface to be self-evident, but its consequences are profound. The simplest consequence is that the parameter names of a function must be local to the body of the function. For example, we used `square` in the definition of `is_good_enough` in our square-root function:

```
def is_good_enough(guess, x):  
    return abs(square(guess) - x) < 0.001
```

The intention of the author of `is_good_enough` is to determine if the square of the first argument is within a given tolerance of the second argument. We see that the author of `is_good_enough` used the name `guess` to refer to the first argument and `x` to refer to the second argument. The argument of `square` is `guess`. If the author of `square` used `x` (as above) to refer to that argument, we see that the `x` in `is_good_enough` must be a different `x` than the one in `square`. Running the function `square` must not affect the value of `x` that is used by `is_good_enough`, because that value of `x` may be needed by `is_good_enough` after `square` is done computing.

If the parameters were not local to the bodies of their respective functions, then the parameter `x` in `square` could be confused with the parameter `x` in `is_good_enough`, and the behavior of `is_good_enough` would depend upon which version of `square` we used. Thus, `square` would not be the black box we desired.

A parameter of a function has a very special role in the function definition, in that it doesn't matter what name the parameter has. Such a name is called *bound*, and we say that the function definition *binds* its parameters. The meaning of a function definition is unchanged if a bound name is consistently renamed throughout the definition. [^24: The concept of consistent renaming is actually subtle and difficult to define formally. Famous logicians have made embarrassing errors here.] If a name is not bound, we say that it is *free*. The set of statements for which a binding

defines a name is called the *scope* of that name. In a function definition, the bound names declared as the parameters of the function have the body of the function as their scope.

In the definition of `is_good_enough` above, `guess` and `x` are bound variables but `abs` and `square` are free. The meaning of `is_good_enough` should be independent of the names we choose for `guess` and `x` so long as they are distinct and different from `abs` and `square`. (If we renamed `guess` to `abs` we would have introduced a bug by *capturing* the variable `abs`. It would have changed from free to bound.) The meaning of `is_good_enough` is not independent of the names of its free variables, however. It surely depends upon the fact (external to this definition) that the name `abs` refers to a function for computing the absolute value of a number. The function `is_good_enough` will compute a different function if we substitute `math.cos` (the primitive cosine function) for `abs` in its definition.

Internal definitions and block structure

We have one kind of name isolation available to us so far: The parameters of a function are local to the body of the function. The square-root program illustrates another way in which we would like to control the use of names. The existing program consists of separate functions:

```
def sqrt(x):
    return sqrt_iter(1, x)

def sqrt_iter(guess, x):
    return (guess
            if is_good_enough(guess, x)
            else sqrt_iter(improve(guess, x), x))

def is_good_enough(guess, x):
    return abs(square(guess) - x) < 0.001

def improve(guess, x):
    return average(guess, x / guess)
```

The problem with this program is that the only function that is important to users of `sqrt` is `sqrt`. The other functions (`sqrt_iter`, `is_good_enough`, and `improve`) only clutter up their minds. They may not define any other function called `is_good_enough` as part of another program to work together with the square-root program, because `sqrt` needs it. The problem is especially severe in the construction of large systems by many separate programmers. For example, in the construction of a large library of numerical functions, many numerical functions are computed as successive approximations and thus might have functions named `is_good_enough` and `improve` as auxiliary functions. We would like to localize the subfunctions, hiding them inside `sqrt` so that `sqrt` could coexist with other successive approximations, each having its own private `is_good_enough` function. To make this possible, we allow a function to have internal definitions that are local to that function. For example, in the square-root problem we can write

```

def sqrt(x):
    def is_good_enough(guess, x):
        return abs(square(guess) - x) < 0.001
    def improve(guess, x):
        return average(guess, x / guess)
    def sqrt_iter(guess, x):
        return (guess
                if is_good_enough(guess, x)
                else sqrt_iter(improve(guess, x), x))
    return sqrt_iter(1, x)

```

The body of a function definition forms a *block*; definitions inside it are local to the function. Such nesting of definitions, called *block structure*, is basically the right solution to the simplest name-packaging problem. But there is a better idea lurking here. In addition to internalizing the definitions of the auxiliary functions, we can simplify them. Since `x` is bound in the definition of `sqrt`, the functions `is_good_enough`, `improve`, and `sqrt_iter`, which are defined internally to `sqrt`, are in the scope of `x`. Thus, it is not necessary to pass `x` explicitly to each of these functions. Instead, we allow `x` to be a free name in the internal definitions, as shown below. Then `x` gets its value from the argument with which the enclosing function `sqrt` is called. This discipline is called *lexical scoping*. [²⁵: Lexical scoping dictates that free names in a function are taken to refer to bindings made by enclosing function definitions; that is, they are looked up in the environment in which the function was defined. We will see how this works in detail in chapter 3 when we study environments and the detailed behavior of the interpreter.]

```

def sqrt(x):
    def is_good_enough(guess):
        return abs(square(guess) - x) < 0.001
    def improve(guess):
        return average(guess, x / guess)
    def sqrt_iter(guess):
        return (guess
                if is_good_enough(guess)
                else sqrt_iter(improve(guess)))
    return sqrt_iter(1)

```

We will use block structure extensively to help us break up large programs into tractable pieces. [²⁶: Embedded definitions must come first in a function body. The management is not responsible for the consequences of running programs that intertwine definition and use; see also footnote in section 1.3.2.] The idea of block structure originated with the programming language Algol 60. It appears in most advanced programming languages and is an important tool for helping to organize the construction of large programs.

1.2 Functions and the Processes They Generate

We have now considered the elements of programming: We have used primitive arithmetic operations, we have combined these operations, and we have abstracted these composite operations by defining them as compound functions. But that is not enough to enable us to say that we know how to program. Our situation is analogous to that of someone who has learned the rules for how the pieces move in chess but knows nothing of typical openings, tactics, or strategy. Like the novice chess player, we don't yet know the common patterns of usage in the domain. We lack the knowledge of which moves are worth making (which functions are worth defining). We lack the experience to predict the consequences of making a move (executing a function).

The ability to visualize the consequences of the actions under consideration is crucial to becoming an expert programmer, just as it is in any synthetic, creative activity. In becoming an expert photographer, for example, one must learn how to look at a scene and know how dark each region will appear on a print for each possible choice of exposure and processing options. Only then can one reason backward, planning framing, lighting, exposure, and processing to obtain the desired effects. So it is with programming, where we are planning the course of action to be taken by a process and where we control the process by means of a program. To become experts, we must learn to visualize the processes generated by various types of functions. Only after we have developed such a skill can we learn to reliably construct programs that exhibit the desired behavior.

A function is a pattern for the *local evolution* of a computational process. It specifies how each stage of the process is built upon the previous stage. We would like to be able to make statements about the overall, or *global*, behavior of a process whose local evolution has been specified by a function. This is very difficult to do in general, but we can at least try to describe some typical patterns of process evolution.

In this section we will examine some common "shapes" for processes generated by simple functions. We will also investigate the rates at which these processes consume the important computational resources of time and space. The functions we will consider are very simple. Their role is like that played by test patterns in photography: as oversimplified prototypical patterns, rather than practical examples in their own right.

1.2.1 Linear Recursion and Iteration

We begin by considering the factorial function, defined by

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1$$

There are many ways to compute factorials. One way is to make use of the observation that $n!$ is equal to $n \times (n-1)!$ for any positive integer n :

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 3 \cdot 2 \cdot 1 \quad = \quad n \cdot (n-1)!$$

Thus, we can compute $n!$ by computing $(n-1)!$ and multiplying the result by n . If we add the stipulation that $1!$ is equal to 1, this observation translates directly into a computer function:

```
def factorial(n):
    return 1 if n == 1 else n * factorial(n - 1)
```

We can use the substitution model of section 1.1.5 to watch this function in action computing $6!$, as shown in figure 1.3.

```
factorial(6)
6 * factorial(5)
6 * (5 * factorial(4))
6 * (5 * (4 * factorial(3)))
6 * (5 * (4 * (3 * factorial(2))))
6 * (5 * (4 * (3 * (2 * factorial(1)))))
6 * (5 * (4 * (3 * (2 * 1))))
6 * (5 * (4 * (3 * 2)))
6 * (5 * (4 * 6))
6 * (5 * 24)
6 * 120
720
```

Figure 1.3. A linear recursive process for computing $6!$.

Now let's take a different perspective on computing factorials. We could describe a rule for computing $n!$ by specifying that we first multiply 1 by 2, then multiply the result by 3, then by 4, and so on until we reach n . More formally, we maintain a running product, together with a counter that counts from 1 up to n . We can describe the computation by saying that the counter and the product simultaneously change from one step to the next according to the rule

$$\begin{array}{l} \text{product} \rightarrow \text{product} \cdot \text{counter} \\ \text{counter} \rightarrow \text{counter} + 1 \end{array}$$

and stipulating that $n!$ is the value of the product when the counter exceeds n .

Once again, we can recast our description as a function for computing factorials: [^27: In a real program we would probably use the block structure introduced in the last section to hide the

definition of `fact_iter` : We avoided doing this here so as to minimize the number of things to think about at once.]

```
def factorial(n):
    def iterate(product, counter):
        return (product
                if counter > n
                else iterate(counter * product, counter + 1))
    return iterate(1, 1)
```

```
def factorial(n):
    return fact_iter(1, 1, n)

def fact_iter(product, counter, max_count):
    return (product
            if counter > max_count
            else fact_iter(counter * product, counter + 1, max_count))
```

As before, we can use the substitution model to visualize the process

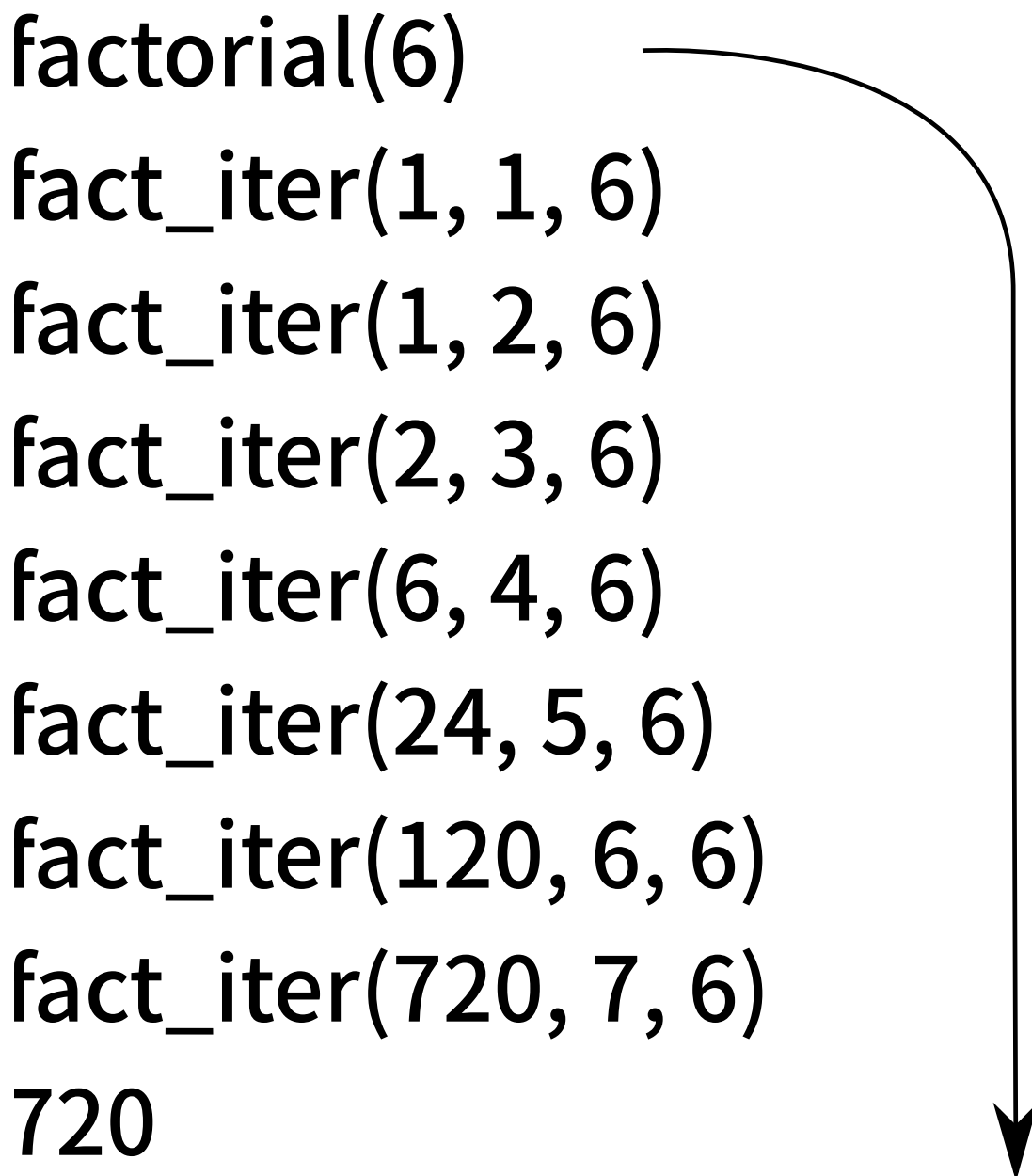


Figure 1.4. A linear iterative process for computing $6!$.

of computing $6!$, as shown in figure 1.4.

Compare the two processes. From one point of view, they seem hardly different at all. Both compute the same mathematical function on the same domain, and each requires a number of steps proportional to n to compute $n!$. Indeed, both processes even carry out the same sequence of multiplications, obtaining the same sequence of partial products. On the other hand, when we consider the "shapes" of the two processes, we find that they evolve quite differently.

Consider the first process. The substitution model reveals a shape of expansion followed by contraction, indicated by the arrow in figure 1.3. The expansion occurs as the process builds up a chain of *deferred operations* (in this case, a chain of multiplications). The contraction occurs as the operations are actually performed. This type of process, characterized by a chain of deferred

operations, is called a *recursive process*. Carrying out this process requires that the interpreter keep track of the operations to be performed later on. In the computation of $n!$, the length of the chain of deferred multiplications, and hence the amount of information needed to keep track of it, grows linearly with n (is proportional to n), just like the number of steps. Such a process is called a *linear recursive process*.

By contrast, the second process does not grow and shrink. At each step, all we need to keep track of, for any n , are the current values of the names `product`, `counter`, and `max_count`. We call this an *iterative process*. In general, an iterative process is one whose state can be summarized by a fixed number of *state variables*, together with a fixed rule that describes how the state variables should be updated as the process moves from state to state and an (optional) end test that specifies conditions under which the process should terminate. In computing $n!$, the number of steps required grows linearly with n . Such a process is called a *linear iterative process*.

The contrast between the two processes can be seen in another way. In the iterative case, the state variables provide a complete description of the state of the process at any point. If we stopped the computation between steps, all we would need to do to resume the computation is to supply the interpreter with the values of the three state variables. Not so with the recursive process. In this case there is some additional "hidden" information, maintained by the interpreter and not contained in the state variables, which indicates "where the process is" in negotiating the chain of deferred operations. The longer the chain, the more information must be maintained. [28: When we discuss the implementation of functions on register machines in chapter 4, we will see that any iterative process can be realized "in hardware" as a machine that has a fixed set of registers and no auxiliary memory. In contrast, realizing a recursive process requires a machine that uses an auxiliary data structure known as a *stack*.]

In contrasting iteration and recursion, we must be careful not to confuse the notion of a recursive *process* with the notion of a recursive *function*. When we describe a function as recursive, we are referring to the syntactic fact that the function definition refers (either directly or indirectly) to the function itself. But when we describe a process as following a pattern that is, say, linearly recursive, we are speaking about how the process evolves, not about the syntax of how a function is written. It may seem disturbing that we refer to a recursive function such as `fact_iter` as generating an iterative process. However, the process really is iterative: Its state is captured completely by its three state variables, and an interpreter need keep track of only three names in order to execute the process.

One reason that the distinction between process and function may be confusing is that most implementations of common languages (including C, Java, and Python) are designed in such a way that the interpretation of any recursive function consumes an amount of memory that grows with the number of function calls, even when the process described is, in principle, iterative. As a consequence, these languages can describe iterative processes only by resorting to special-purpose "looping constructs" such as `do`, `repeat`, `until`, `for`,

and `while`. The implementation of Python we shall consider in chapter does not share this defect. It will execute an iterative process in constant space, even if the iterative process is described by a recursive function. An implementation with this property is called *tail-recursive*. [^29: Tail recursion has long been known as a compiler optimization trick. A coherent semantic basis for tail recursion was provided by Carl Hewitt (1977), who explained it in terms of the "message-passing" model of computation that we shall discuss in chapter 3. Inspired by this, Gerald Jay Sussman and Guy Lewis Steele Jr. (see Steele 1975) constructed a tail-recursive interpreter for Scheme. Steele later showed how tail recursion is a consequence of the natural way to compile function calls (Steele 1977). The IEEE standard for Scheme requires that Scheme implementations be tail-recursive. The Python Language Reference does not specify tail recursion, one way or the other. While most implementations of Python are not tail-recursive, we assume in this book an implementation that is.] With a tail-recursive implementation, iteration can be expressed using the ordinary function call mechanism, so that special iteration constructs are useful only as syntactic sugar. [^30: Exercise explores Python's while loops as syntactic sugar for functions that give rise to iterative processes. The full language Python, like other conventional languages, features a plethora of syntactic forms, all of which can be expressed more uniformly in the language Lisp.]

Exercise 1.9. Each of the following two functions defines a method for adding two positive integers in terms of the functions `inc`, which increments its argument by 1, and `dec`, which decrements its argument by 1.

```
def plus(a, b):
    return b if a == 0 else inc(plus(dec(a), b))
```

```
def plus(a, b):
    return b if a == 0 else plus(dec(a), inc(b))
```

Using the substitution model, illustrate the process generated by each function in evaluating `plus(4, 5)`. Are these processes iterative or recursive?

Exercise 1.10. The following function computes a mathematical function called Ackermann's function.

```
def A(x, y):
    return (0 if y == 0
            else 2 * y if x == 0
            else 2 if y == 1
            else A(x - 1, A(x, y - 1)))
```

What is printed by the following statements?

```
print(A(1, 10))
```

```
print(A(2, 4))
```

```
print(A(3, 3))
```

Consider the following functions, where `A` is the function defined above:

```
def f(n):  
    return A(0, n)  
  
def g(n):  
    return A(1, n)  
  
def h(n):  
    return A(2, n)  
  
def k(n):  
    return 5 * n * n
```

Give concise mathematical definitions for the functions computed by the functions `f`, `g`, and `h` for positive integer values of n . For example, $k(n)$ computes $5n^2$.

1.2.2 Tree Recursion

Another common pattern of computation is called *tree recursion*. As an example, consider computing the sequence of Fibonacci numbers, in which each number is the sum of the preceding two:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, \dots$$

In general, the Fibonacci numbers can be defined by the rule

$$\text{Fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{Fib}(n-1) + \text{Fib}(n-2) & \text{otherwise} \end{cases}$$

We can immediately translate this definition into a recursive function for computing Fibonacci numbers:

```
def fib(n):  
    return (0 if n == 0  
            else 1 if n == 1  
            else fib(n - 1) + fib(n - 2))
```

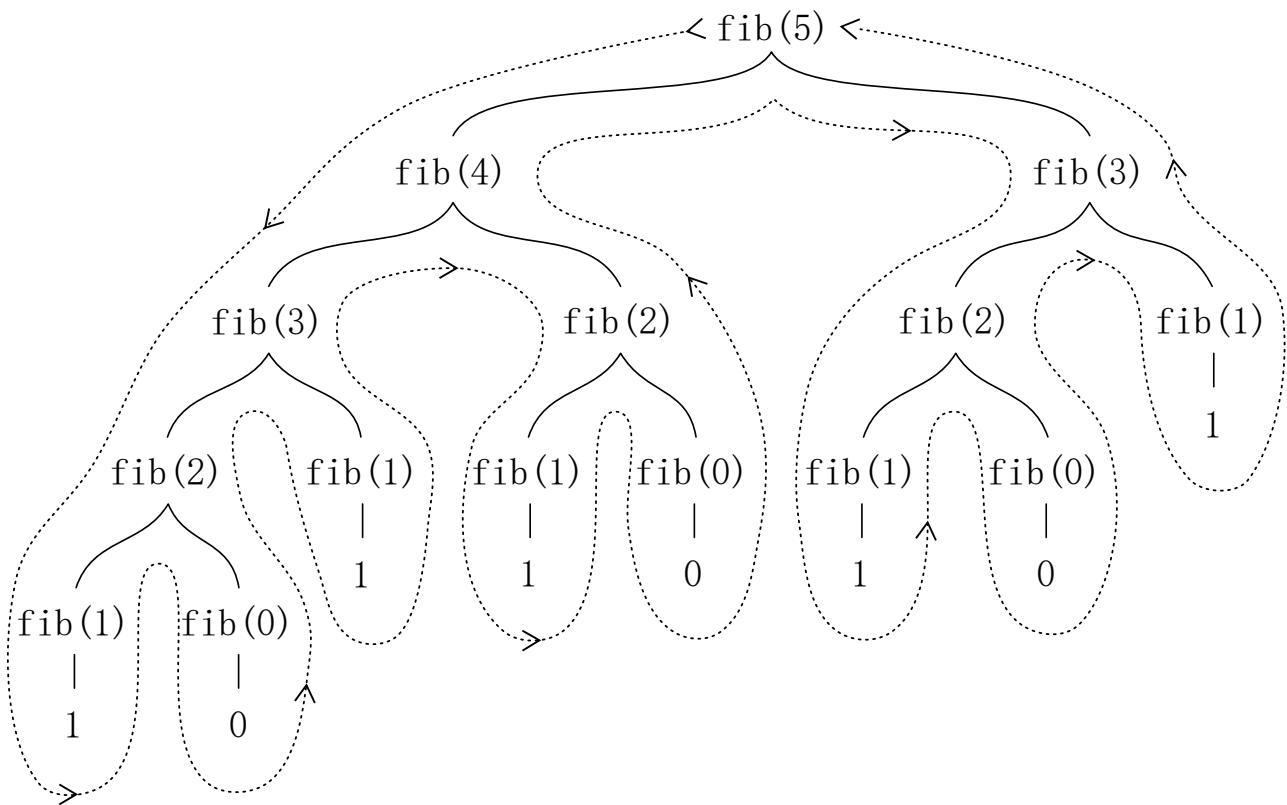


Figure 1.5. The tree-recursive process generated in computing `fib(5)` .

Consider the pattern of this computation. To compute `fib(5)` , we compute `fib(4)` and `fib(3)` . To compute `fib(4)` , we compute `fib(3)` and `fib(2)` . In general, the evolved process looks like a tree, as shown in figure 1.5. Notice that the branches split into two at each level (except at the bottom); this reflects the fact that the `fib` function calls itself twice each time it is invoked.

This function is instructive as a prototypical tree recursion, but it is a terrible way to compute Fibonacci numbers because it does so much redundant computation. Notice in figure 1.5 that the entire computation of `fib(3)` —almost half the work—is duplicated. In fact, it is not hard to show that the number of times the function will compute `fib(1)` or `fib(0)` (the number of leaves in the above tree, in general) is precisely $\text{Fib}(n+1)$. To get an idea of how bad this is, one can show that the value of $\text{Fib}(n)$ grows exponentially with n . More precisely (see exercise 1.13), $\text{Fib}(n)$ is the closest integer to $\frac{\phi^n}{\sqrt{5}}$, where

$$\phi = \frac{1 + \sqrt{5}}{2} \approx 1.6180$$

is the *golden ratio*, which satisfies the equation

$$\phi^2 = \phi + 1$$

Thus, the process uses a number of steps that grows exponentially with the input. On the other hand, the space required grows only linearly with the input, because we need keep track only of which nodes are above us in the tree at any point in the computation. In general, the number of steps required by a tree-recursive process will be proportional to the number of nodes in the tree, while the space required will be proportional to the maximum depth of the tree.

We can also formulate an iterative process for computing the Fibonacci numbers. The idea is to use a pair of integers a and b , initialized to $\text{Fib}(1)=1$ and $\text{Fib}(0)=0$, and to repeatedly apply the simultaneous transformations

$$\begin{array}{l} a \text{ and } b \\ \leftarrow a+b \quad b \leftarrow a \end{array}$$

It is not hard to show that, after applying this transformation n times, a and b will be equal, respectively, to $\text{Fib}(n+1)$ and $\text{Fib}(n)$. Thus, we can compute Fibonacci numbers iteratively using the function

```
def fib(n):
    return fib_iter(1, 0, n)

def fib_iter(a, b, count):
    return b if count == 0 else fib_iter(a + b, a, count - 1)
```

This second method for computing $\text{Fib}(n)$ is a linear iteration. The difference in number of steps required by the two methods—one linear in n , one growing as fast as $\text{Fib}(n)$ itself—is enormous, even for small inputs.

One should not conclude from this that tree-recursive processes are useless. When we consider processes that operate on hierarchically structured data rather than numbers, we will find that tree recursion is a natural and powerful tool. [31: An example of this was hinted at in section 1.1.3: The interpreter itself evaluates expressions using a tree-recursive process.] But even in numerical operations, tree-recursive processes can be useful in helping us to understand and design programs. For instance, although the first `fib` function is much less efficient than the second one, it is more straightforward, being little more than a translation into Python of the definition of the Fibonacci sequence. To formulate the iterative algorithm required noticing that the computation could be recast as an iteration with three state variables.

Example: Counting change

It takes only a bit of cleverness to come up with the iterative Fibonacci algorithm. In contrast, consider the following problem: How many different ways can we make change of \$1.00 (100 cents), given half-dollars, quarters, dimes, nickels, and pennies (50 cents, 25 cents, 10 cents, 5 cents, and 1 cent, respectively)? More generally, can we write a function to compute the number of ways to change any given amount of money?

This problem has a simple solution as a recursive function. Suppose we think of the types of coins available as arranged in some order. Then the following relation holds:

The number of ways to change amount a using n kinds of coins equals

- *the number of ways to change amount a using all but the first kind of coin, plus*

- the number of ways to change amount $a-d$ using all n kinds of coins, where d is the denomination of the first kind of coin.

To see why this is true, observe that the ways to make change can be divided into two groups: those that do not use any of the first kind of coin, and those that do. Therefore, the total number of ways to make change for some amount is equal to the number of ways to make change for the amount without using any of the first kind of coin, plus the number of ways to make change assuming that we do use the first kind of coin. But the latter number is equal to the number of ways to make change for the amount that remains after using a coin of the first kind.

Thus, we can recursively reduce the problem of changing a given amount to problems of changing smaller amounts or using fewer kinds of coins. Consider this reduction rule carefully, and convince yourself that we can use it to describe an algorithm if we specify the following degenerate cases: [^32: For example, work through in detail how the reduction rule applies to the problem of making change for 10 cents using pennies and nickels.]

- If a is exactly 0, we should count that as 1 way to make change.
- If a is less than 0, we should count that as 0 ways to make change.
- If n is 0, we should count that as 0 ways to make change. We can easily translate this description into a recursive function:

```
def count_change(amount):
    return cc(amount, 5)

def cc(amount, kinds_of_coins):
    return (1 if amount == 0
            else 0 if amount < 0 or kinds_of_coins == 0
            else cc(amount, kinds_of_coins - 1)
                + cc(amount - first_denomination(kinds_of_coins),
                    kinds_of_coins))

def first_denomination(kinds_of_coins):
    return (1 if kinds_of_coins == 1
            else 5 if kinds_of_coins == 2
            else 10 if kinds_of_coins == 3
            else 25 if kinds_of_coins == 4
            else 50 if kinds_of_coins == 5
            else 0)
```

(The `first_denomination` function takes as input the number of kinds of coins available and returns the denomination of the first kind. Here we are thinking of the coins as arranged in order from largest to smallest, but any order would do as well.) We can now answer our original question about changing a dollar:

```
print(count_change(100))
```


The function `count_change` generates a tree-recursive process with redundancies similar to those in our first implementation of `fib`. On the other hand, it is not obvious how to design a better algorithm for computing the result, and we leave this problem as a challenge. The observation that a tree-recursive process may be highly inefficient but often easy to specify and understand has led people to propose that one could get the best of both worlds by designing a "smart compiler" that could transform tree-recursive functions into more efficient functions that compute the same result. [^33: One approach to coping with redundant computations is to arrange matters so that we automatically construct a table of values as they are computed. Each time we are asked to apply the function to some argument, we first look to see if the value is already stored in the table, in which case we avoid performing the redundant computation. This strategy, known as *tabulation* or *memoization*, can be implemented in a straightforward way. Tabulation can sometimes be used to transform processes that require an exponential number of steps (such as `count_change`) into processes whose space and time requirements grow linearly with the input. See exercise 3.27.]

Exercise 1.11. A function f is defined by the rules $f(n)=n$ if $n < 3$ and $f(n)=f(n-1)+2f(n-2)+3f(n-3)$ if $n \geq 3$. Write a Python function that computes f by means of a recursive process. Write a function that computes f by means of an iterative process.

Exercise 1.12. The following pattern of numbers is called *Pascal's triangle*.

```

\begin{array}{rrrrrrrr}
& & & & 1 & & & \\
& & & \backslash & & / & & \\
& & 1 & & 1 & & & \\
& & \backslash & & / & & & \\
& 1 & & 2 & & 1 & & \\
& \backslash & & / & & \backslash & & / \\
1 & & 3 & & 3 & & 1 & \\
& \backslash & & / & & \backslash & & / \\
& 1 & & 4 & & 6 & & 4 & & 1 \\
& \backslash & & / & & \backslash & & / & & \backslash & & / \\
& & & & & & & & \dots & & & & 
\end{array}

```

The numbers at the edge of the triangle are all 1, and each number inside the triangle is the sum of the two numbers above it. [^34: The elements of Pascal's triangle are called the *binomial coefficients*, because the n th row consists of the coefficients of the terms in the expansion of $(x+y)^n$. This pattern for computing the coefficients appeared in Blaise Pascal's 1653 seminal work on probability theory, *Traité du triangle arithmétique*. According to Edwards (2019), the same pattern appears in the works of the eleventh-century Persian mathematician Al-Karaji, in the works of the twelfth-century Hindu mathematician Bhaskara, and in the works of the thirteenth-century Chinese mathematician Yang Hui.] Write a function that computes elements of Pascal's triangle by means of a recursive process.

Exercise 1.13. Prove that $\text{Fib}(n)$ is the closest integer to $\phi^n / \sqrt{5}$, where $\phi = (1 + \sqrt{5})/2$. Hint: Use induction and the definition of the Fibonacci numbers to prove that $\text{Fib}(n) = (\phi^n - \psi^n) / \sqrt{5}$, where $\psi = (1 - \sqrt{5})/2$.

1.2.3 Orders of Growth

The previous examples illustrate that processes can differ considerably in the rates at which they consume computational resources. One convenient way to describe this difference is to use the notion of *order of growth* to obtain a gross measure of the resources required by a process as the inputs become larger.

Let n be a parameter that measures the size of the problem, and let $R(n)$ be the amount of resources the process requires for a problem of size n . In our previous examples we took n to be the number for which a given function is to be computed, but there are other possibilities. For instance, if our goal is to compute an approximation to the square root of a number, we might take n to be the number of digits accuracy required. For matrix multiplication we might take n to be the number of rows in the matrices. In general there are a number of properties of the problem with respect to which it will be desirable to analyze a given process. Similarly, $R(n)$ might measure the number of internal storage registers used, the number of elementary machine operations performed, and so on. In computers that do only a fixed number of operations at a time, the time required will be proportional to the number of elementary machine operations performed.

We say that $R(n)$ has order of growth $\Theta(f(n))$, written $R(n) = \Theta(f(n))$ (pronounced "theta of $f(n)$ "), if there are positive constants k_1 and k_2 independent of n such that

$$\begin{array}{l} k_1 f(n) \leq R(n) \leq k_2 f(n) \end{array}$$

for any sufficiently large value of n . (In other words, for large n , the value $R(n)$ is sandwiched between $k_1 f(n)$ and $k_2 f(n)$.)

For instance, with the linear recursive process for computing factorial described in section 1.2.1 the number of steps grows proportionally to the input n . Thus, the steps required for this process grows as $\Theta(n)$. We also saw that the space required grows as $\Theta(1)$. For the iterative factorial, the number of steps is still $\Theta(n)$ but the space is $\Theta(1)$ —that is, constant. [^35: These statements mask a great deal of oversimplification. For instance, if we count process steps as "machine operations" we are making the assumption that the number of machine operations needed to perform, say, a multiplication is independent of the size of the numbers to be multiplied, which is false if the numbers are sufficiently large. Similar remarks hold for the estimates of space. Like the design and description of a process, the analysis of a process can be carried out at various levels of abstraction.] The tree-recursive Fibonacci computation requires $\Theta(\phi^n)$ steps and space $\Theta(n)$, where ϕ is the golden ratio described in section 1.2.2.

Orders of growth provide only a crude description of the behavior of a process. For example, a process requiring n^2 steps and a process requiring $1000n^2$ steps and a process requiring $3n^2 + 10n + 17$ steps all have $\Theta(n^2)$ order of growth. On the other hand, order of growth

provides a useful indication of how we may expect the behavior of the process to change as we change the size of the problem. For a $\Theta(n)$ (linear) process, doubling the size will roughly double the amount of resources used. For an exponential process, each increment in problem size will multiply the resource utilization by a constant factor. In the remainder of section 1.2 we will examine two algorithms whose order of growth is logarithmic, so that doubling the problem size increases the resource requirement by a constant amount.

Exercise 1.14. Draw the tree illustrating the process generated by the `count_change` function of section 1.2.2 in making change for 11 cents. What are the orders of growth of the space and number of steps used by this process as the amount to be changed increases?

Exercise 1.15. The sine of an angle (specified in radians) can be computed by making use of the approximation $\sin x \approx x$ if x is sufficiently small, and the trigonometric identity

$$\sin x = 3 \sin \left(\frac{x}{3} \right) - 4 \sin^3 \left(\frac{x}{3} \right)$$

to reduce the size of the argument of $\sin$. (For purposes of this exercise an angle is considered "sufficiently small" if its magnitude is not greater than 0.1 radians.) These ideas are incorporated in the following functions:

```
def cube(x):
    return x * x * x

def p(x):
    return 3 * x - 4 * cube(x)

def sine(angle):
    return angle if not (abs(angle) > 0.1) else p(sine(angle / 3))
```

a. How many times is the function `p` applied when `sine(12.15)` is evaluated? b. What is the order of growth in space and number of steps (as a function of a) used by the process generated by the `sine` function when `sine(a)` is evaluated?

1.2.4 Exponentiation

Consider the problem of computing the exponential of a given number. We would like a function that takes as arguments a base b and a positive integer exponent n and computes b^n . One way to do this is via the recursive definition

$$b^n = b \cdot b^{n-1} \quad b^0 = 1$$

which translates readily into the function

```
def expt(b, n):
    return 1 if n == 0 else b * expt(b, n - 1)
```

This is a linear recursive process, which requires $\Theta(n)$ steps and $\Theta(n)$ space. Just as with factorial, we can readily formulate an equivalent linear iteration:

```
def expt(b, n):
    return expt_iter(b, n, 1)

def expt_iter(b, counter, product):
    return (product
            if counter == 0
            else expt_iter(b, counter - 1, b * product))
```

This version requires $\Theta(n)$ steps and $\Theta(1)$ space.

We can compute exponentials in fewer steps by using successive squaring. For instance, rather than computing b^8 as

$$b \cdot b \cdot b \cdot b \cdot b \cdot b \cdot b \cdot b$$

we can compute it using three multiplications:

$$b^2 = b \cdot b \quad b^4 = b^2 \cdot b^2 \quad b^8 = b^4 \cdot b^4$$

This method works fine for exponents that are powers of 2. We can also take advantage of successive squaring in computing exponentials in general if we use the rule

$$b^n = (b^{n/2})^2 \quad \text{if } n \text{ is even} \quad b^n = b \cdot b^{n-1} \quad \text{if } n \text{ is odd}$$

We can express this method as a function, where the operator `//` denotes *integer division*, which discards any fractional part of the quotient:

```
def fast_expt(b, n):
    return (1 if n == 0
            else square(fast_expt(b, n // 2)) if is_even(n)
            else b * fast_expt(b, n - 1))
```

where the predicate to test whether an integer is even is defined in terms of the operator `%`, which computes the remainder after integer division, by

```
def is_even(n):
    return n % 2 == 0
```

The process evolved by `fast_expt` grows logarithmically with n in both space and number of steps. To see this, observe that computing b^{2n} using `fast_expt` requires only one more multiplication than computing b^n . The size of the exponent we can compute therefore doubles

(approximately) with every new multiplication we are allowed. Thus, the number of multiplications required for an exponent of n grows about as fast as the logarithm of n to the base 2. The process has $\Theta(\log n)$ growth. [^36: More precisely, the number of multiplications required is equal to 1 less than the log base 2 of n , plus the number of ones in the binary representation of n . This total is always less than twice the log base 2 of n . The arbitrary constants k_1 and k_2 in the definition of order notation imply that, for a logarithmic process, the base to which logarithms are taken does not matter, so all such processes are described as $\Theta(\log n)$.]

The difference between $\Theta(\log n)$ growth and $\Theta(n)$ growth becomes striking as n becomes large. For example, `fast_expt` for $n=1000$ requires only 14 multiplications. [^37: You may wonder why anyone would care about raising numbers to the 1000th power. See section 1.2.6.] It is also possible to use the idea of successive squaring to devise an iterative algorithm that computes exponentials with a logarithmic number of steps (see exercise 1.16), although, as is often the case with iterative algorithms, this is not written down so straightforwardly as the recursive algorithm. [^38: This iterative algorithm is ancient. It appears in the *Chandah-sutra* by Áchárya, written before 200 BCE. See Knuth 1997b, section 4.6.3, for a full discussion and analysis of this and other methods of exponentiation.]

Exercise 1.16. Design a function that evolves an iterative exponentiation process that uses successive squaring and uses a logarithmic number of steps, as does `fast_expt`. (Hint: Using the observation that $(b^{n/2})^2 = (b^2)^{n/2}$, keep, along with the exponent n and the base b , an additional state variable a , and define the state transformation in such a way that the product $a \cdot b^n$ is unchanged from state to state. At the beginning of the process a is taken to be 1, and the answer is given by the value of a at the end of the process. In general, the technique of defining an *invariant quantity* that remains unchanged from state to state is a powerful way to think about the design of iterative algorithms.)

Exercise 1.17. The exponentiation algorithms in this section are based on performing exponentiation by means of repeated multiplication. In a similar way, one can perform integer multiplication by means of repeated addition. The following multiplication function (in which it is assumed that our language can only add, not multiply) is analogous to the `expt` function:

```
def times(a, b):
    return 0 if b == 0 else a + times(a, b - 1)
```

This algorithm takes a number of steps that is linear in `b`. Now suppose we include, together with addition, the functions `double`, which doubles an integer, and `halve`, which divides an (even) integer by 2. Using these, design a multiplication function analogous to `fast_expt` that uses a logarithmic number of steps.

Exercise 1.18. Using the results of exercises 1.16 and 1.17, devise a function that generates an iterative process for multiplying two integers in terms of adding, doubling, and halving and uses a logarithmic number of steps. [^39: This algorithm, which is sometimes known as the "Russian

peasant method" of multiplication, is ancient. Examples of its use are found in the Rhind Papyrus, one of the two oldest mathematical documents in existence, written about 1700 BCE (and copied from an even older document) by an Egyptian scribe named A'h-mose.]

Exercise 1.19. There is a clever algorithm for computing the Fibonacci numbers in a logarithmic number of steps. Recall the transformation of the state variables a and b in the `fib_iter` process of section 1.2.2: $a \leftarrow a+b$ and $b \leftarrow a$. Call this transformation T , and observe that applying T over and over again n times, starting with 1 and 0, produces the pair $(\text{Fib}(n+1), \text{Fib}(n))$. In other words, the Fibonacci numbers are produced by applying T^n , the n th power of the transformation T , starting with the pair $(1,0)$. Now consider T to be the special case of $p=0$ and $q=1$ in a family of transformations T_{pq} , where T_{pq} transforms the pair (a,b) according to $a \leftarrow bq+aq+ap$ and $b \leftarrow bp+aq$. Show that if we apply such a transformation T_{pq} twice, the effect is the same as using a single transformation $T_{p'q'}$ of the same form, and compute p' and q' in terms of p and q . This gives us an explicit way to square these transformations, and thus we can compute T^n using successive squaring, as in the `fast_expt` function. Put this all together to complete the following function, which runs in a logarithmic number of steps: [^40: This exercise was suggested by Joe Stoy, based on an example in Kaldewaij 1990.]

```
def fib(n):
    return fib_iter(1, 0, 0, 1, n)

def fib_iter(a, b, p, q, count):
    return (b
            if count == 0
            else fib_iter(a,
                          b,
                          ??,          # compute p'
                          ??,          # compute q'
                          count // 2)
            if is_even(count)
            else fib_iter(b * q + a * q + a * p,
                          b * p + a * q,
                          p,
                          q,
                          count - 1))
```

1.2.5 Greatest Common Divisors

The greatest common divisor (GCD) of two integers a and b is defined to be the largest integer that divides both a and b with no remainder. For example, the GCD of 16 and 28 is 4. In chapter 2, when we investigate how to implement rational-number arithmetic, we will need to be able to compute GCDs in order to reduce rational numbers to lowest terms. (To reduce a rational number to lowest terms, we must divide both the numerator and the denominator by their

GCD. For example, 16/28 reduces to 4/7.) One way to find the GCD of two integers is to factor them and search for common factors, but there is a famous algorithm that is much more efficient.

The idea of the algorithm is based on the observation that, if r is the remainder when a is divided by b , then the common divisors of a and b are precisely the same as the common divisors of b and r . Thus, we can use the equation

$$\text{GCD}(a, b) = \text{GCD}(b, r)$$

to successively reduce the problem of computing a GCD to the problem of computing the GCD of smaller and smaller pairs of integers. For example,

$$\text{GCD}(206, 40) = \text{GCD}(40, 6) = \text{GCD}(6, 4) = \text{GCD}(4, 2) = \text{GCD}(2, 0) = 2$$

reduces $\text{GCD}(206, 40)$ to $\text{GCD}(2, 0)$, which is 2. It is possible to show that starting with any two positive integers and performing repeated reductions will always eventually produce a pair where the second number is 0. Then the GCD is the other number in the pair. This method for computing the GCD is known as *Euclid's Algorithm*. [41: Euclid's Algorithm is so called because it appears in Euclid's *Elements* (Book 7, ca. 300 BCE). According to Knuth (1997a), it can be considered the oldest known nontrivial algorithm. The ancient Egyptian method of multiplication (exercise 1.18) is surely older, but, as Knuth explains, Euclid's Algorithm is the oldest known to have been presented as a general algorithm, rather than as a set of illustrative examples.]

It is easy to express Euclid's Algorithm as a function:

```
def gcd(a, b):
    return a if b == 0 else gcd(b, a % b)
```

This generates an iterative process, whose number of steps grows as the logarithm of the numbers involved.

The fact that the number of steps required by Euclid's Algorithm has logarithmic growth bears an interesting relation to the Fibonacci numbers:

Lamé's Theorem: *If Euclid's Algorithm requires k steps to compute the GCD of some pair, then the smaller number in the pair must be greater than or equal to the k th Fibonacci number. [42: This theorem was proved in 1845 by Gabriel Lamé, a French mathematician and engineer known chiefly for his contributions to mathematical physics. To prove the theorem, we consider pairs (a_k, b_k) , where $a_k \geq b_k$, for which Euclid's Algorithm terminates in k steps. The proof is based on the claim that, if $(a_{k+1}, b_{k+1}) \rightarrow (a_k, b_k) \rightarrow (a_{k-1}, b_{k-1})$ are three successive pairs in the reduction process, then we must have $b_{k+1} \geq b_k + b_{k-1}$. To verify the claim, consider that a reduction step is defined by applying the transformation $a_{k-1} = b_k$, $b_{k-1} = \text{remainder of } a_k \text{ divided by } b_k$.*

$\text{divided by } b_k$. The second equation means that $a_k = qb_k + b_{k-1}$ for some positive integer q . And since q must be at least 1 we have $a_k = qb_k + b_{k-1} \geq b_k + b_{k-1}$. But in the previous reduction step we have $b_{k+1} = a_k$. Therefore, $b_{k+1} = a_k \geq b_k + b_{k-1}$. This verifies the claim. Now we can prove the theorem by induction on k , the number of steps that the algorithm requires to terminate. The result is true for $k=1$, since this merely requires that b be at least as large as $\text{Fib}(1)=1$. Now, assume that the result is true for all integers less than or equal to k and establish the result for $k+1$. Let $(a_{k+1}, b_{k+1}) \rightarrow (a_k, b_k) \rightarrow (a_{k-1}, b_{k-1})$ be successive pairs in the reduction process. By our induction hypotheses, we have $b_{k-1} \geq \text{Fib}(k-1)$ and $b_k \geq \text{Fib}(k)$. Thus, applying the claim we just proved together with the definition of the Fibonacci numbers gives $b_{k+1} \geq b_k + b_{k-1} \geq \text{Fib}(k) + \text{Fib}(k-1) = \text{Fib}(k+1)$, which completes the proof of Lamé's Theorem.]

We can use this theorem to get an order-of-growth estimate for Euclid's Algorithm. Let n be the smaller of the two inputs to the function. If the process takes k steps, then we must have $n \geq \text{Fib}(k) \approx \phi^k / \sqrt{5}$. Therefore the number of steps k grows as the logarithm (to the base ϕ) of n . Hence, the order of growth is $\Theta(\log n)$.

Exercise 1.20. The process that a function generates is of course dependent on the rules used by the interpreter. As an example, consider the iterative `gcd` function given above. Suppose we were to interpret this function using normal-order evaluation, as discussed in section 1.1.5. (The normal-order-evaluation rule for conditional expressions is described in exercise 1.5.) Using the substitution method (for normal order), illustrate the process generated in evaluating `gcd(206, 40)` and indicate the `remainder` operations that are actually performed. How many `remainder` operations are actually performed in the normal-order evaluation of `gcd(206, 40)`? In the applicative-order evaluation?

1.2.6 Example: Testing for Primality

This section describes two methods for checking the primality of an integer n , one with order of growth $\Theta(\sqrt{n})$, and a "probabilistic" algorithm with order of growth $\Theta(\log n)$. The exercises at the end of this section suggest programming projects based on these algorithms.

Searching for divisors

Since ancient times, mathematicians have been fascinated by problems concerning prime numbers, and many people have worked on the problem of determining ways to test if numbers are prime. One way to test if a number is prime is to find the number's divisors. The following program finds the smallest integral divisor (greater than 1) of a given number n . It does this in a straightforward way, by testing n for divisibility by successive integers starting with 2.


```
def smallest_divisor(n):
    return find_divisor(n, 2)

def find_divisor(n, test_divisor):
    return (n if square(test_divisor) > n
            else test_divisor if divides(test_divisor, n)
            else find_divisor(n, test_divisor + 1))

def divides(a, b):
    return b % a == 0
```

We can test whether a number is prime as follows: n is prime if and only if n is its own smallest divisor.

```
def is_prime(n):
    return n == smallest_divisor(n)
```

The end test for `find_divisor` is based on the fact that if n is not prime it must have a divisor less than or equal to $\sqrt{n}$. [⁴³ If d is a divisor of n , then so is n/d . But d and n/d cannot both be greater than $\sqrt{n}$.] This means that the algorithm need only test divisors between 1 and $\sqrt{n}$. Consequently, the number of steps required to identify n as prime will have order of growth $\Theta(\sqrt{n})$.

The Fermat test

The $\Theta(\log n)$ primality test is based on a result from number theory known as Fermat's Little Theorem. [⁴⁴ Pierre de Fermat (1601–1665) is considered to be the founder of modern number theory. He obtained many important number-theoretic results, but he usually announced just the results, without providing his proofs. Fermat's Little Theorem was stated in a letter he wrote in 1640. The first published proof was given by Euler in 1736 (and an earlier, identical proof was discovered in the unpublished manuscripts of Leibniz). The most famous of Fermat's results—known as Fermat's Last Theorem—was jotted down in 1637 in his copy of the book *Arithmetic* (by the third-century Greek mathematician Diophantus) with the remark "I have discovered a truly remarkable proof, but this margin is too small to contain it." Finding a proof of Fermat's Last Theorem became one of the most famous challenges in number theory. A complete solution was finally given in 1995 by Andrew Wiles of Princeton University.]

Fermat's Little Theorem: If n is a prime number and a is any positive integer less than n , then a raised to the n th power is congruent to a modulo n .

(Two numbers are said to be *congruent modulo* n if they both have the same remainder when divided by n . The remainder of a number a when divided by n is also referred to as the *remainder of* a *modulo* n , or simply as a *modulo* n .)

If n is not prime, then, in general, most of the numbers $a < n$ will not satisfy the above relation. This leads to the following algorithm for testing primality: Given a number n , pick a random number $a < n$ and compute the remainder of a^n modulo n . If the result is not equal to a , then n is certainly not prime. If it is a , then chances are good that n is prime. Now pick another random number a and test it with the same method. If it also satisfies the equation, then we can be even more confident that n is prime. By trying more and more values of a , we can increase our confidence in the result. This algorithm is known as the Fermat test.

To implement the Fermat test, we need a function that computes the exponential of a number modulo another number:

```
def expmod(base, exp, m):
    return (1 if exp == 0
            else square(expmod(base, exp // 2, m)) % m if is_even(exp)
            else (base * expmod(base, exp - 1, m)) % m)
```

This is very similar to the `fast_expt` function of section 1.2.4. It uses successive squaring, so that the number of steps grows logarithmically with the exponent. [45: The reduction steps in the cases where the exponent e is greater than 1 are based on the fact that, for any integers x , y , and m , we can find the remainder of x times y modulo m by computing separately the remainders of x modulo m and y modulo m , multiplying these, and then taking the remainder of the result modulo m . For instance, in the case where e is even, we compute the remainder of $b^{e/2}$ modulo m , square this, and take the remainder modulo m . This technique is useful because it means we can perform our computation without ever having to deal with numbers much larger than m . (Compare exercise 1.25.)]

The Fermat test is performed by choosing at random a number a between 1 and $n-1$ inclusive and checking whether the remainder modulo n of the n th power of a is equal to a . The random number a is chosen using the built-in function `random_random`, which returns a nonnegative number less than 1. Hence, to obtain a random number between 1 and $n-1$, we multiply the return value of `random_random` by $n-1$, round down the result with the built-in function `math_floor`, and add 1:

```
def fermat_test(n):
    def try_it(a):
        return expmod(a, n, n) == a
    return try_it(1 + math_floor(random_random() * (n - 1)))
```

The following function runs the test a given number of times, as specified by a parameter. Its value is true if the test succeeds every time, and false otherwise.

```
def fast_is_prime(n, times):
    return (True if times == 0
            else fast_is_prime(n, times - 1) if fermat_test(n)
            else False)
```

Probabilistic methods

The Fermat test differs in character from most familiar algorithms, in which one computes an answer that is guaranteed to be correct. Here, the answer obtained is only probably correct. More precisely, if n ever fails the Fermat test, we can be certain that n is not prime. But the fact that n passes the test, while an extremely strong indication, is still not a guarantee that n is prime. What we would like to say is that for any number n , if we perform the test enough times and find that n always passes the test, then the probability of error in our primality test can be made as small as we like.

Unfortunately, this assertion is not quite correct. There do exist numbers that fool the Fermat test: numbers n that are not prime and yet have the property that a^n is congruent to a modulo n for all integers $a < n$. Such numbers are extremely rare, so the Fermat test is quite reliable in practice. [^46: Numbers that fool the Fermat test are called *Carmichael numbers*, and little is known about them other than that they are extremely rare. There are 255 Carmichael numbers below 100,000,000. The smallest few are 561, 1105, 1729, 2465, 2821, and 6601. In testing primality of very large numbers chosen at random, the chance of stumbling upon a value that fools the Fermat test is less than the chance that cosmic radiation will cause the computer to make an error in carrying out a "correct" algorithm. Considering an algorithm to be inadequate for the first reason but not for the second illustrates the difference between mathematics and engineering.] There are variations of the Fermat test that cannot be fooled. In these tests, as with the Fermat method, one tests the primality of an integer n by choosing a random integer $a < n$ and checking some condition that depends upon n and a . (See exercise 1.28 for an example of such a test.) On the other hand, in contrast to the Fermat test, one can prove that, for any n , the condition does not hold for most of the integers $a < n$ unless n is prime. Thus, if n passes the test for some random choice of a , the chances are better than even that n is prime. If n passes the test for two random choices of a , the chances are better than 3 out of 4 that n is prime. By running the test with more and more randomly chosen values of a we can make the probability of error as small as we like.

The existence of tests for which one can prove that the chance of error becomes arbitrarily small has sparked interest in algorithms of this type, which have come to be known as *probabilistic algorithms*. There is a great deal of research activity in this area, and probabilistic algorithms have been fruitfully applied to many fields. [^47: One of the most striking applications of probabilistic prime testing has been to the field of cryptography. Although it is computationally infeasible to factor an arbitrary 300-digit number as of this writing (2021), the primality of such a number can be checked in a few seconds with the Fermat test. This fact forms the basis of a technique for

constructing "unbreakable codes" suggested by Rivest, Shamir, and Adleman (1977). The resulting *RSA algorithm* has become a widely used technique for enhancing the security of electronic communications. Because of this and related developments, the study of prime numbers, once considered the epitome of a topic in "pure" mathematics to be studied only for its own sake, now turns out to have important practical applications to cryptography, electronic funds transfer, and information retrieval.]

Exercise 1.21. Use the `smallest_divisor` function to find the smallest divisor of each of the following numbers: 199, 1999, 19999.

Exercise 1.22. Assume a built-in function `get_time` of no arguments that returns the number of milliseconds that have passed since 00:00:00 UTC on Thursday, 1 January, 1970. [^48: This date is called the *UNIX epoch* and is part of the specification of functions that deal with time in the UNIXTM operating system.] The following `timed_prime_test` function, when called with an integer `n`, prints `n` and checks to see if `n` is prime. If `n` is prime, the function prints three asterisks [^49: The built-in function `print` prints its argument. Here `" *** "` is a *string*, a sequence of characters that we pass as argument to the `print` function. Section introduces strings more thoroughly.] followed by the amount of time used in performing the test.

```
def timed_prime_test(n):
    print(n)
    return start_prime_test(n, get_time())

def start_prime_test(n, start_time):
    return (report_prime(get_time() - start_time) if is_prime(n)
            else False)

def report_prime(elapsed_time):
    print(" *** ")
    print(elapsed_time)
    return True
```

Using this function, write a function `search_for_primes` that checks the primality of consecutive odd integers in a specified range. Use your function to find the three smallest primes larger than 1000; larger than 10,000; larger than 100,000; larger than 1,000,000. Note the time needed to test each prime. Since the testing algorithm has order of growth of $\Theta(\sqrt{n})$, you should expect that testing for primes around 10,000 should take about $\sqrt{10}$ times as long as testing for primes around 1000. Do your timing data bear this out? How well do the data for 100,000 and 1,000,000 support the $\sqrt{n}$ prediction? Is your result compatible with the notion that programs on your machine run in time proportional to the number of steps required for the computation?

Exercise 1.23. The `smallest_divisor` function shown at the start of this section does lots of needless testing: After it checks to see if the number is divisible by 2 there is no point in checking to see if it is divisible by any larger even numbers. This suggests that the values used for

`test_divisor` should not be 2, 3, 4, 5, 6, ... but rather 2, 3, 5, 7, 9, To implement this change, define a function `next_int` that returns 3 if its input is equal to 2 and otherwise returns its input plus 2. Modify the `smallest_divisor` function to use `next_int(test_divisor)` instead of `test_divisor + 1`. With `timed_prime_test` incorporating this modified version of `smallest_divisor`, run the test for each of the 12 primes found in exercise 1.22. Since this modification halves the number of test steps, you should expect it to run about twice as fast. Is this expectation confirmed? If not, what is the observed ratio of the speeds of the two algorithms, and how do you explain the fact that it is different from 2?

Exercise 1.24. Modify the `timed_prime_test` function of exercise 1.22 to use `fast_is_prime` (the Fermat method), and test each of the 12 primes you found in that exercise. Since the Fermat test has $\Theta(\log n)$ growth, how would you expect the time to test primes near 1,000,000 to compare with the time needed to test primes near 1000? Do your data bear this out? Can you explain any discrepancy you find?

Exercise 1.25. Alyssa P. Hacker complains that we went to a lot of extra work in writing `expmod`. After all, she says, since we already know how to compute exponentials, we could have simply written

```
def expmod(base, exp, m):
    return fast_expt(base, exp) % m
```

Is she correct? Would this function serve as well for our fast prime tester? Explain.

Exercise 1.26. Louis Reasoner is having great difficulty doing exercise 1.24. His `fast_is_prime` test seems to run more slowly than his `is_prime` test. Louis calls his friend Eva Lu Ator over to help. When they examine Louis's code, they find that he has rewritten the `expmod` function to use an explicit multiplication, rather than calling `square`:

```
def expmod(base, exp, m):
    return (1 if exp == 0
           else (expmod(base, exp // 2, m)
                 * expmod(base, exp // 2, m)) % m if is_even(exp)
           else (base * expmod(base, exp - 1, m)) % m)
```

"I don't see what difference that could make," says Louis. "I do." says Eva. "By writing the function like that, you have transformed the $\Theta(\log n)$ process into a $\Theta(n)$ process." Explain.

Exercise 1.27. Demonstrate that the Carmichael numbers listed in footnote really do fool the Fermat test. That is, write a function that takes an integer n and tests whether a^n is congruent to a modulo n for every $a < n$, and try your function on the given Carmichael numbers.

Exercise 1.28. One variant of the Fermat test that cannot be fooled is called the *Miller–Rabin test* (Miller 1976; Rabin 1980). This starts from an alternate form of Fermat's Little Theorem, which states that if n is a prime number and a is any positive integer less than n , then a raised to the $(n-1)$ st power is congruent to 1 modulo n . To test the primality of a number n by the Miller–Rabin test, we pick a random number $a < n$ and raise a to the $(n-1)$ st power modulo n using the `expmod` function. However, whenever we perform the squaring step in `expmod`, we check to see if we have discovered a "nontrivial square root of 1 modulo n ," that is, a number not equal to 1 or $n-1$ whose square is equal to 1 modulo n . It is possible to prove that if such a nontrivial square root of 1 exists, then n is not prime. It is also possible to prove that if n is an odd number that is not prime, then, for at least half the numbers $a < n$, computing a^{n-1} in this way will reveal a nontrivial square root of 1 modulo n . (This is why the Miller–Rabin test cannot be fooled.) Modify the `expmod` function to signal if it discovers a nontrivial square root of 1, and use this to implement the Miller–Rabin test with a function analogous to `fermat_test`. Check your function by testing various known primes and non-primes. Hint: One convenient way to make `expmod` signal is to have it return 0.

1.3 Formulating Abstractions with Higher-Order Functions

We have seen that functions are, in effect, abstractions that describe compound operations on numbers independent of the particular numbers. For example, when we define

```
def cube(x):  
    return x * x * x
```

we are not talking about the cube of a particular number, but rather about a method for obtaining the cube of any number. Of course we could get along without ever defining this function, by always writing expressions such as

```
3 * 3 * 3  
x * x * x  
y * y * y
```

and never mentioning `cube` explicitly. This would place us at a serious disadvantage, forcing us to work always at the level of the particular operations that happen to be primitives in the language (multiplication, in this case) rather than in terms of higher-level operations. Our programs would be able to compute cubes, but our language would lack the ability to express the concept of cubing. One of the things we should demand from a powerful programming language is the ability to build abstractions by assigning names to common patterns and then to work in terms of the

abstractions directly. Functions provide this ability. This is why all but the most primitive programming languages include mechanisms for defining functions.

Yet even in numerical processing we will be severely limited in our ability to create abstractions if we are restricted to functions whose parameters must be numbers. Often the same programming pattern will be used with a number of different functions. To express such patterns as concepts, we will need to construct functions that can accept functions as arguments or return functions as values. Functions that manipulate functions are called *higher-order functions*. This section shows how higher-order functions can serve as powerful abstraction mechanisms, vastly increasing the expressive power of our language.

1.3.1 Functions as Arguments

Consider the following three functions. The first computes the sum of the integers from `a` through `b`:

```
def sum_integers(a, b):  
    return 0 if a > b else a + sum_integers(a + 1, b)
```

The second computes the sum of the cubes of the integers in the given range:

```
def sum_cubes(a, b):  
    return 0 if a > b else cube(a) + sum_cubes(a + 1, b)
```

The third computes the sum of a sequence of terms in the series

$$\frac{1}{1 \cdot 3} + \frac{1}{5 \cdot 7} + \frac{1}{9 \cdot 11} + \cdots$$

which converges to $\pi/8$ (very slowly): [^50: This series, usually written in the equivalent form $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$, is due to Leibniz. We'll see how to use this as the basis for some fancy numerical tricks in section 3.5.3.]

```
def pi_sum(a, b):  
    return 0 if a > b else 1 / (a * (a + 2)) + pi_sum(a + 4, b)
```

These three functions clearly share a common underlying pattern. They are for the most part identical, differing only in the name of the function, the function of `a` used to compute the term to be added, and the function that provides the next value of `a`. We could generate each of the functions by filling in slots in the same template:

```
def *name*(a, b):
    return (0 if a > b
           else *term*(a) + *name*(*next*(a), b))
```

The presence of such a common pattern is strong evidence that there is a useful abstraction waiting to be brought to the surface. Indeed, mathematicians long ago identified the abstraction of *summation of a series* and invented "sigma notation," for example

$$\sum_{n=a}^b f(n) = f(a) + \dots + f(b)$$

to express this concept. The power of sigma notation is that it allows mathematicians to deal with the concept of summation itself rather than only with particular sums—for example, to formulate general results about sums that are independent of the particular series being summed.

Similarly, as program designers, we would like our language to be powerful enough so that we can write a function that expresses the concept of summation itself rather than only functions that compute particular sums. We can do so readily in our functional language by taking the common template shown above and transforming the "slots" into parameters:

```
def sum(term, a, next, b):
    return 0 if a > b else term(a) + sum(term, next(a), next, b)
```

Notice that `sum` takes as its arguments the lower and upper bounds `a` and `b` together with the functions `term` and `next`. We can use `sum` just as we would any function. For example, we can use it (along with a function `inc` that increments its argument by 1) to define `sum_cubes`:

```
def inc(n):
    return n + 1
def sum_cubes(a, b):
    return sum(cube, a, inc, b)
```

Using this, we can compute the sum of the cubes of the integers from 1 to 10:

```
print(sum_cubes(1, 10))
```

```
3025
```

With the aid of an identity function to compute the term, we can define `sum_integers` in terms of `sum`:

```
def identity(x):
    return x
```



```
def sum_integers(a, b):
    return sum(identity, a, inc, b)
```

Then we can add up the integers from 1 to 10:

```
print(sum_integers(1, 10))
```

```
55
```

We can also define `pi_sum` in the same way: [^51: Notice that we have used block structure (section 1.1.8) to embed the definitions of `pi_next` and `pi_term` within `pi_sum`, since these functions are unlikely to be useful for any other purpose. We will see how to get rid of them altogether in section 1.3.2.]

```
def pi_sum(a, b):
    def pi_term(x):
        return 1 / (x * (x + 2))
    def pi_next(x):
        return x + 4
    return sum(pi_term, a, pi_next, b)
```

Using these functions, we can compute an approximation to π :

```
print(8 * pi_sum(1, 1000))
```

```
3.139592655589783
```

Once we have `sum`, we can use it as a building block in formulating further concepts. For instance, the definite integral of a function f between the limits a and b can be approximated numerically using the formula

$$\int_a^b f(x) \, dx \approx \left[f\left(a + \frac{dx}{2}\right) + f\left(a + dx + \frac{dx}{2}\right) + f\left(a + 2dx + \frac{dx}{2}\right) + \cdots \right] dx$$

for small values of dx . We can express this directly as a function:

```
def integral(f, a, b, dx):
    def add_dx(x):
        return x + dx
    return sum(f, a + dx / 2, add_dx, b) * dx
```

```
print(integral(cube, 0, 1, 0.01))
```

```
0.24998750000000042
```

```
print(integral(cube, 0, 1, 0.001))
```

```
0.249998750000001
```

(The exact value of the integral of `cube` between 0 and 1 is 1/4.)

Exercise 1.29. Simpson's Rule is a more accurate method of numerical integration than the method illustrated above. Using Simpson's Rule, the integral of a function f between a and b is approximated as

$$\frac{h}{3} [y_0 + 4y_1 + 2y_2 + 4y_3 + 2y_4 + \cdots + 2y_{n-2} + 4y_{n-1} + y_n]$$

where $h = (b-a)/n$, for some even integer n , and $y_k = f(a+kh)$. (Increasing n increases the accuracy of the approximation.) Define a function that takes as arguments f , a , b , and n and returns the value of the integral, computed using Simpson's Rule. Use your function to integrate `cube` between 0 and 1 (with $n=100$ and $n=1000$), and compare the results to those of the `integral` function shown above.

Exercise 1.30. The `sum` function above generates a linear recursion. The function can be rewritten so that the sum is performed iteratively. Show how to do this by filling in the missing expressions in the following definition:

```
def sum(term, a, next, b):
    def iterate(a, result):
        return (??
                if ??
                else iterate(??, ??))
    return iterate(??, ??)
```

Exercise 1.31. a. The `sum` function is only the simplest of a vast number of similar abstractions that can be captured as higher-order functions. [^52: The intent of exercises 1.31–1.33 is to demonstrate the expressive power that is attained by using an appropriate abstraction to consolidate many seemingly disparate operations. However, though accumulation and filtering are elegant ideas, our hands are somewhat tied in using them at this point since we do not yet have data structures to provide suitable means of combination for these abstractions. We will return to these ideas in section 2.2.3 when we show how to use *sequences* as interfaces for combining filters and accumulators to build even more powerful abstractions. We will see there how these methods really come into their own as a powerful and elegant approach to designing programs.] Write an

analogous function called `product` that returns the product of the values of a function at points over a given range. Show how to define `factorial` in terms of `product`. Also use `product` to compute approximations to π using the formula [53: This formula was discovered by the seventeenth-century English mathematician John Wallis.]

$$\frac{\pi}{4} = \frac{2 \cdot 4 \cdot 4 \cdot 6 \cdot 6 \cdot 8 \cdots}{3 \cdot 3 \cdot 5 \cdot 5 \cdot 7 \cdot 7 \cdots}$$

b. If your `product` function generates a recursive process, write one that generates an iterative process. If it generates an iterative process, write one that generates a recursive process.

Exercise 1.32. a. Show that `sum` and `product` (exercise 1.31) are both special cases of a still more general notion called `accumulate` that combines a collection of terms, using some general accumulation function:

```
accumulate(combiner, null_value, term, a, next, b)
```

The function `accumulate` takes as arguments the same term and range specifications as `sum` and `product`, together with a `combiner` function (of two arguments) that specifies how the current term is to be combined with the accumulation of the preceding terms and a `null_value` that specifies what base value to use when the terms run out. Write `accumulate` and show how `sum` and `product` can both be defined as simple calls to `accumulate`. b. If your `accumulate` function generates a recursive process, write one that generates an iterative process. If it generates an iterative process, write one that generates a recursive process.

Exercise 1.33. You can obtain an even more general version of `accumulate` (exercise 1.32) by introducing the notion of a *filter* on the terms to be combined. That is, combine only those terms derived from values in the range that satisfy a specified condition. The resulting `filtered_accumulate` abstraction takes the same arguments as `accumulate`, together with an additional predicate of one argument that specifies the filter. Write `filtered_accumulate` as a function. Show how to express the following using `filtered_accumulate`: a. the sum of the squares of the prime numbers in the interval a to b (assuming that you have an `is_prime` predicate already written) b. the product of all the positive integers less than n that are relatively prime to n (i.e., all positive integers $i < n$ such that $\text{GCD}(i, n) = 1$).

1.3.2 Constructing Functions using Lambda Expressions

In using `sum` as in section 1.3.1, it seems terribly awkward to have to define trivial functions such as `pi_term` and `pi_next` just so we can use them as arguments to our higher-order function. Rather than define `pi_next` and `pi_term`, it would be more convenient to have a way to directly specify "the function that returns its input incremented by 4" and "the function that returns the reciprocal of its input times its input plus 2." We can do this by introducing the *lambda expression*

as a syntactic form for creating functions. Using lambda expressions, we can describe what we want as

```
lambda x: x + 4
```

and

```
lambda x: 1 / (x * (x + 2))
```

Then we can express our `pi_sum` function without defining any auxiliary functions:

```
def pi_sum(a, b):  
    return sum(lambda x: 1 / (x * (x + 2)),  
               a,  
               lambda x: x + 4,  
               b)
```

Again using a lambda expression, we can write the `integral` function without having to define the auxiliary function `add_dx`:

```
def integral(f, a, b, dx):  
    return sum(f,  
               a + dx / 2,  
               lambda x: x + dx,  
               b) * dx
```

In general, lambda expressions are used to create functions in the same way as function definitions, except that no name is specified for the function and the parentheses around the parameters and the `return` keyword are omitted.

```
lambda *parameters*: *expression*
```

The resulting function is just as much a function as one that is created using a function definition statement. The only difference is that it has not been associated with any name in the environment. In fact,

```
plus4 = lambda x: x + 4
```

is equivalent to

```
def plus4(x):  
    return x + 4
```

We can read a lambda expression as follows:

```
lambda      x      :      x  +  4
#           ^      ^
# the function of an argument x that results in the value plus 4
```

Like any expression that has a function as its value, a lambda expression can be used as the function expression in an application such as

```
print((lambda x, y, z: x + y + square(z))(1, 2, 3))
```

```
12
```

or, more generally, in any context where we would normally use a function name. [^54: It would be clearer and less intimidating to people learning Python if a term more obvious than *lambda expression*, such as *function definition expression*, were used. But the convention is very firmly entrenched, not just for Lisp and Scheme but also for Python, Java and other languages, no doubt partly due to the influence of the Scheme editions of this book. The notation is adopted from the λ calculus, a mathematical formalism introduced by the mathematical logician Alonzo Church (1941). Church developed the λ calculus to provide a rigorous foundation for studying the notions of function and function application. The λ calculus has become a basic tool for mathematical investigations of the semantics of programming languages.]Note that a `lambda` expression has lower precedence than function application and thus the parentheses around the lambda expression are necessary here.

Using declaration assignment to create local variables

Another use of `lambda` is in creating local variables. We often need local variables in our functions other than those that have been bound as parameters. For example, suppose we wish to compute the function

$$f(x, y) = x(1 + xy)^2 + y(1 - y) + (1 + xy)(1 - y)$$

which we could also express as

$$a = 1 + xy \quad b = 1 - y \quad f(x, y) = x a^2 + y b + a b$$

In writing a function to compute f , we would like to include as local variables not only x and y but also the names of intermediate quantities like a and b . One way to accomplish this is to use an auxiliary function to bind the local variables:

```
def f(x, y):
    def f_helper(a, b):
        return x * square(a) + y * b + a * b
    return f_helper(1 + x * y, 1 - y)
```

Of course, we could use a `lambda` expression to specify an anonymous function for binding our local variables. The body of `f` then becomes a single call to that function:

```
def f_2(x, y):
    return (lambda a, b:
            x * square(a) + y * b + a * b)(1 + x * y, 1 - y)
```

A more convenient way to declare local variables is by using declaration assignments within the body of the function. Using a declaration assignment, the function can be written as

```
def f_3(x, y):
    a = 1 + x * y
    b = 1 - y
    return x * square(a) + y * b + a * b
```

Variables that are declared with declaration assignments inside a function have the body of the immediately surrounding function as their scope. [^55: Note that a name declared in a function cannot be used before the declaration is fully evaluated, regardless of whether the same name is declared outside the function. Thus in the program below, the attempt to use the `a` declared at the top level to provide a value for the calculation of the `b` declared in `f` cannot work. The program leads to an error, because the `a` in `a + x` is used before its declaration is evaluated. We will return to this program in section (exercise), after we learn more about evaluation.]

```
a = 1
def f(x):
    b = a + x
    a = 5
    return a + b
f(10)
```

\$^,\$ [^56: The substitution model can be expanded to say that for a declaration assignment, the value of the expression after `=` is substituted for the name before `=` in the rest of the function body (after the declaration), similar to the substitution of arguments for parameters in the evaluation of a function application.]

Conditional statements

We have seen that it is often useful to declare variables that are local to function definitions. When functions become big, we should keep the computation associated with the variables as restricted

as possible. Consider for example `expmod` in exercise 1.26.

```
def expmod(base, exp, m):
    return (1 if exp == 0
            else (expmod(base, exp // 2, m)
                  * expmod(base, exp // 2, m)) % m if is_even(exp)
            else (base * expmod(base, exp - 1, m)) % m)
```

This function is unnecessarily inefficient, because it contains two identical calls:

```
expmod(base, exp // 2, m)
```

While this can be easily fixed in this example using the `square` function, this is not so easy in general. Without using `square`, we would be tempted to introduce a local name for the expression as follows:

```
def expmod(base, exp, m):
    half_exp = expmod(base, exp // 2, m)
    return (1 if exp == 0
            else (half_exp * half_exp) % m if is_even(exp)
            else (base * expmod(base, exp - 1, m)) % m)
```

This would make the function not just inefficient, but actually nonterminating! The problem is that the declaration assignment appears outside the conditional expression, which means that it is executed even when the base case `exp == 0` is met. To avoid this situation, we provide for *conditional statements*, and allow return statements to appear in the branches of the statement. Using a conditional statement, we can write the function `expmod` as follows:

```
def expmod(base, exp, m):
    if exp == 0:
        return 1
    else:
        if is_even(exp):
            half_exp = expmod(base, exp // 2, m)
            return (half_exp * half_exp) % m
        else:
            return (base * expmod(base, exp - 1, m)) % m
```

The simplest form of a conditional statement is

```
if *predicate*:
    *consequent-statements*
else:
    *alternative-statements*
```

As for a conditional expression, the interpreter first evaluates the *predicate*. If it evaluates to true, the interpreter evaluates the *consequent-statements* in sequence, and if it evaluates to false, the interpreter evaluates the *alternative-statements* in sequence. Evaluation of a return statement returns from the surrounding function, ignoring any statements in the sequence after the return statement and any statements after the conditional statement.

Python provides the keyword `elif` to avoid deeply nested `else: if` statements. Using `elif`, we can write the `expmod` function as follows:

```
def expmod(base, exp, m):
    if exp == 0:
        return 1
    elif is_even(exp):
        half_exp = expmod(base, exp // 2, m)
        return (half_exp * half_exp) % m
    else:
        return (base * expmod(base, exp - 1, m)) % m
```

Exercise 1.34. Suppose we define the function

```
def f(g):
    return g(2)
```

Then we have

```
print(f(square))
```

4

```
print(f(lambda z: z * (z + 1)))
```

6

What happens if we (perversely) ask the interpreter to evaluate the application `f(f)`? Explain.

1.3.3 Functions as General Methods

We introduced compound functions in section 1.1.4 as a mechanism for abstracting patterns of numerical operations so as to make them independent of the particular numbers involved. With higher-order functions, such as the `integral` function of section 1.3.1, we began to see a more powerful kind of abstraction: functions used to express general methods of computation, independent of the particular functions involved. In this section we discuss two more elaborate

examples—general methods for finding zeros and fixed points of functions—and show how these methods can be expressed directly as functions.

Finding roots of equations by the half-interval method

The *half-interval method* is a simple but powerful technique for finding roots of an equation $f(x)=0$, where f is a continuous function. The idea is that, if we are given points a and b such that $f(a) < 0 < f(b)$, then f must have at least one zero between a and b . To locate a zero, let x be the average of a and b and compute $f(x)$. If $f(x) > 0$, then f must have a zero between a and x . If $f(x) < 0$, then f must have a zero between x and b . Continuing in this way, we can identify smaller and smaller intervals on which f must have a zero. When we reach a point where the interval is small enough, the process stops. Since the interval of uncertainty is reduced by half at each step of the process, the maximal number of steps required grows as $\Theta(\log(L/T))$, where L is the length of the original interval and T is the error tolerance (that is, the size of the interval we will consider "small enough"). Here is a function that implements this strategy:

```
def search(f, neg_point, pos_point):
    midpoint = average(neg_point, pos_point)
    if close_enough(neg_point, pos_point):
        return midpoint
    else:
        test_value = f(midpoint)
        if positive(test_value):
            return search(f, neg_point, midpoint)
        elif negative(test_value):
            return search(f, midpoint, pos_point)
        else: return midpoint
```

We assume that we are initially given the function f together with points at which its values are negative and positive. We first compute the midpoint of the two given points. Next we check to see if the given interval is small enough, and if so we simply return the midpoint as our answer. Otherwise, we compute as a test value the value of f at the midpoint. If the test value is positive, then we continue the process with a new interval running from the original negative point to the midpoint. If the test value is negative, we continue with the interval from the midpoint to the positive point. Finally, there is the possibility that the test value is 0, in which case the midpoint is itself the root we are searching for. To test whether the endpoints are "close enough" we can use a function similar to the one used in section 1.1.7 for computing square roots: [^57: We have used 0.001 as a representative "small" number to indicate a tolerance for the acceptable error in a calculation. The appropriate tolerance for a real calculation depends upon the problem to be solved and the limitations of the computer and the algorithm. This is often a very subtle consideration, requiring help from a numerical analyst or some other kind of magician.]

```
def close_enough(x, y):
    return abs(x - y) < 0.001
```

The function `search` is awkward to use directly, because we can accidentally give it points at which f 's values do not have the required sign, in which case we get a wrong answer. Instead we will use `search` via the following function, which checks to see which of the endpoints has a negative function value and which has a positive value, and calls the `search` function accordingly. If the function has the same sign on the two given points, the half-interval method cannot be used, in which case the function signals an error. [^58: This can be accomplished using `error`, which takes as argument a string that is printed as error message along with the number of the program line that gave rise to the call of `error`.]

```
def half_interval_method(f, a, b):
    a_value = f(a)
    b_value = f(b)
    if negative(a_value) and positive(b_value):
        return search(f, a, b)
    elif negative(b_value) and positive(a_value):
        return search(f, b, a)
    else: error("values are not of opposite sign")
```

The following example uses the half-interval method to approximate π as the root between 2 and 4 of $\sin, x = 0$:

```
print(half_interval_method(math.sin, 2, 4))
```

```
3.14111328125
```

Here is another example, using the half-interval method to search for a root of the equation $x^3 - 2x - 3 = 0$ between 1 and 2:

```
print(half_interval_method(lambda x: x * x * x - 2 * x - 3, 1, 2))
```

```
1.89306640625
```

Finding fixed points of functions

A number x is called a *fixed point* of a function f if x satisfies the equation $f(x)=x$. For some functions f we can locate a fixed point by beginning with an initial guess and applying f repeatedly,

$$\begin{array}{l} f(x), \ f(f(x)), \ f(f(f(x))), \ \ldots \end{array}$$

until the value does not change very much. Using this idea, we can devise a function `fixed_point` that takes as inputs a function and an initial guess and produces an approximation to a fixed point of the function. We apply the function repeatedly until we find two successive values whose difference is less than some prescribed tolerance:

```
tolerance = 0.00001
def fixed_point(f, first_guess):
    def close_enough(x, y):
        return abs(x - y) < tolerance
    def try_with(guess):
        next = f(guess)
        return next if close_enough(guess, next) else try_with(next)
    return try_with(first_guess)
```

For example, we can use this method to approximate the fixed point of the cosine function, starting with 1 as an initial approximation: [^59: To obtain a fixed point of cosine on a calculator, set it to radians mode and then repeatedly press the $\cos$ button until the value does not change any longer.]

```
print(fixed_point(math.cos, 1))
```

```
0.7390822985224023
```

Similarly, we can find a solution to the equation $y = \sin y + \cos y$:

```
print(fixed_point(lambda y: math.sin(y) + math.cos(y), 1))
```

```
1.2587315962971173
```

The fixed-point process is reminiscent of the process we used for finding square roots in section 1.1.7. Both are based on the idea of repeatedly improving a guess until the result satisfies some criterion. In fact, we can readily formulate the square-root computation as a fixed-point search. Computing the square root of some number x requires finding a y such that $y^2 = x$. Putting this equation into the equivalent form $y = x/y$, we recognize that we are looking for a fixed point of the function [^60: $\text{\textit{\textit{mapsto}}}$ (pronounced "maps to") is the mathematician's way of writing lambda expressions. $y \text{\textit{\textit{mapsto}}} x/y$ means `lambda y: x / y`, that is, the function whose value at y is x/y .] $y \text{\textit{\textit{mapsto}}} x/y$, and we can therefore try to compute square roots as

```
def sqrt(x):
    return fixed_point(lambda y: x / y, 1)
```

Unfortunately, this fixed-point search does not converge. Consider an initial guess y_1 . The next guess is $y_2 = x/y_1$ and the next guess is $y_3 = x/y_2 = x/(x/y_1) = y_1$. This results in an infinite loop in which the two guesses y_1 and y_2 repeat over and over, oscillating about the answer.

One way to control such oscillations is to prevent the guesses from changing so much. Since the answer is always between our guess y and x/y , we can make a new guess that is not as far from y as x/y by averaging y with x/y , so that the next guess after y is $\frac{1}{2}(y+x/y)$ instead of x/y . The process of making such a sequence of guesses is simply the process of looking for a fixed point of $y \mapsto \frac{1}{2}(y+x/y)$:

```
def sqrt(x):
    return fixed_point(lambda y: average(y, x / y), 1)
```

(Note that $y = \frac{1}{2}(y+x/y)$ is a simple transformation of the equation $y = x/y$; to derive it, add y to both sides of the equation and divide by 2.)

With this modification, the square-root function works. In fact, if we unravel the definitions, we can see that the sequence of approximations to the square root generated here is precisely the same as the one generated by our original square-root function of section 1.1.7. This approach of averaging successive approximations to a solution, a technique we call *average damping*, often aids the convergence of fixed-point searches.

Exercise 1.35. Show that the golden ratio ϕ (section 1.2.2) is a fixed point of the transformation $x \mapsto 1 + 1/x$, and use this fact to compute ϕ by means of the `fixed_point` function.

Exercise 1.36. Modify `fixed_point` so that it prints the sequence of approximations it generates, using the built-in function `print`. Then find a solution to $x^x = 1000$ by finding a fixed point of $x \mapsto \log(1000)/\log(x)$. (Use the built-in function `math_log`, which computes natural logarithms.) Compare the number of steps this takes with and without average damping. (Note that you cannot start `fixed_point` with a guess of 1, as this would cause division by $\log(1)=0$.)

Exercise 1.37. An infinite *continued fraction* is an expression of the form

$$\frac{f}{g + \frac{N_1}{D_1 + \frac{N_2}{D_2 + \frac{N_3}{D_3 + \cdots}}}}$$

As an example, one can show that the infinite continued fraction expansion with the N_i and the D_i all equal to 1 produces $1/\phi$, where ϕ is the golden ratio (described in section 1.2.2). One way to approximate an infinite continued fraction is to truncate the expansion after a given number of terms. Such a truncation—a so-called *k-term finite continued fraction*—has the form

$$\frac{1}{1 + \frac{1}{1 + \frac{1}{\ddots + \frac{1}{1 + \frac{1}{1 + \frac{1}{D_K}}}}}}$$

a. Suppose that `n` and `d` are functions of one argument (the term index i) that return the N_i and D_i of the terms of the continued fraction. Define a function `cont_frac` such that evaluating `cont_frac(n, d, k)` computes the value of the k -term finite continued fraction. Check your function by approximating $1/\phi$ using

```
print(cont_frac(lambda i: 1, lambda i: 1, k))
```

for successive values of `k`. How large must you make `k` in order to get an approximation that is accurate to 4 decimal places? b. If your `cont_frac` function generates a recursive process, write one that generates an iterative process. If it generates an iterative process, write one that generates a recursive process.

Exercise 1.38. In 1737, the Swiss mathematician Leonhard Euler published a memoir *De Fractionibus Continuis*, which included a continued fraction expansion for e^{-2} , where e is the base of the natural logarithms. In this fraction, the N_i are all 1, and the D_i are successively 1, 2, 1, 1, 4, 1, 1, 6, 1, 1, 8, Write a program that uses your `cont_frac` function from exercise 1.37 to approximate e , based on Euler's expansion.

Exercise 1.39. A continued fraction representation of the tangent function was published in 1770 by the German mathematician J.H. Lambert:

$$\tan x = \cfrac{x}{1 - \cfrac{x^2}{3 - \cfrac{x^2}{5 - \cfrac{x^2}{\ddots}}}}$$

where x is in radians. Define a function `tan_cf(x, k)` that computes an approximation to the tangent function based on Lambert's formula. As in exercise 1.37, `k` specifies the number of terms to compute.

1.3.4 Functions as Returned Values

The above examples demonstrate how the ability to pass functions as arguments significantly enhances the expressive power of our programming language. We can achieve even more expressive power by creating functions whose returned values are themselves functions.

We can illustrate this idea by looking again at the fixed-point example described at the end of section 1.3.3. We formulated a new version of the square-root function as a fixed-point search, starting with the observation that $\sqrt{x}$ is a fixed-point of the function $y \mapsto x/y$. Then we used average damping to make the approximations converge. Average damping is a useful general technique in itself. Namely, given a function f , we consider the function whose value at x is equal to the average of x and $f(x)$.

We can express the idea of average damping by means of the following function:

```
def average_damp(f):  
    return lambda x: average(x, f(x))
```

The function `average_damp` takes as its argument a function `f` and returns as its value a function (produced by the lambda expression) that, when applied to a number `x`, produces the average of `x` and `f(x)`. For example, applying `average_damp` to the `square` function produces a function whose value at some number x is the average of x and x^2 . Applying this resulting function to 10 returns the average of 10 and 100, or 55: [^61: Observe that this is an application whose function expression is itself an application. Exercise 1.4 already demonstrated the ability to form such applications, but that was only a toy example. Here we begin to see the real need for such applications—when applying a function that is obtained as the value returned by a higher-order function.]

```
print(average_damp(square)(10))
```

```
55.0
```

Using `average_damp`, we can reformulate the square-root function as follows:

```
def sqrt(x):  
    return fixed_point(average_damp(lambda y: x / y), 1)
```

Notice how this formulation makes explicit the three ideas in the method: fixed-point search, average damping, and the function $y \mapsto x/y$. It is instructive to compare this formulation of the square-root method with the original version given in section 1.1.7. Bear in mind that these functions express the same process, and notice how much clearer the idea becomes when we express the process in terms of these abstractions. In general, there are many ways to formulate a process as a function. Experienced programmers know how to choose process formulations that are particularly perspicuous, and where useful elements of the process are exposed as separate entities that can be reused in other applications. As a simple example of reuse, notice that the cube root of x is a fixed point of the function $y \mapsto x/y^2$, so we can immediately generalize our square-root function to one that extracts cube roots: [^62: See exercise 1.45 for a further generalization.]

```
def cube_root(x):  
    return fixed_point(average_damp(lambda y: x / square(y)), 1)
```

Newton's method

When we first introduced the square-root function, in section 1.1.7, we mentioned that this was a special case of *Newton's method*. If $x \mapsto g(x)$ is a differentiable function, then a solution of

the equation $g(x)=0$ is a fixed point of the function $x \mapsto f(x)$ where

$$f(x) = x - \frac{g(x)}{Dg(x)}$$

and $Dg(x)$ is the derivative of g evaluated at x . Newton's method is the use of the fixed-point method we saw above to approximate a solution of the equation by finding a fixed point of the function f . [^63: Elementary calculus books usually describe Newton's method in terms of the sequence of approximations $x_{n+1}=x_n-g(x_n)/Dg(x_n)$. Having language for talking about processes and using the idea of fixed points simplifies the description of the method.] For many functions g and for sufficiently good initial guesses for x , Newton's method converges very rapidly to a solution of $g(x)=0$. [^64: Newton's method does not always converge to an answer, but it can be shown that in favorable cases each iteration doubles the number-of-digits accuracy of the approximation to the solution. In such cases, Newton's method will converge much more rapidly than the half-interval method.]

In order to implement Newton's method as a function, we must first express the idea of derivative. Note that "derivative," like average damping, is something that transforms a function into another function. For instance, the derivative of the function $x \mapsto x^3$ is the function $x \mapsto 3x^2$. In general, if g is a function and dx is a small number, then the derivative Dg of g is the function whose value at any number x is given (in the limit of small dx) by

$$Dg(x) = \frac{g(x+dx) - g(x)}{dx}$$

Thus, we can express the idea of derivative (taking dx to be, say, 0.00001) as the function

```
def deriv(g):  
    return lambda x: (g(x + dx) - g(x)) / dx
```

along with the definition

```
dx = 0.00001
```

Like `average_damp`, `deriv` is a function that takes a function as argument and returns a function as value. For example, to approximate the derivative of $x \mapsto x^3$ at 5 (whose exact value is 75) we can evaluate

```
def cube(x):  
    return x * x * x  
  
print(deriv(cube)(5))
```

```
75.00014999664018
```

With the aid of `deriv`, we can express Newton's method as a fixed-point process:

```
def newton_transform(g):
    return lambda x: x - g(x) / deriv(g)(x)
def newtons_method(g, guess):
    return fixed_point(newton_transform(g), guess)
```

The `newton_transform` function expresses the formula at the beginning of this section, and `newtons_method` is readily defined in terms of this. It takes as arguments a function that computes the function for which we want to find a zero, together with an initial guess. For instance, to find the square root of x , we can use Newton's method to find a zero of the function $y \mapsto y^2 - x$ starting with an initial guess of 1. [^65: For finding square roots, Newton's method converges rapidly to the correct solution from any starting point.] This provides yet another form of the square-root function:

```
def sqrt(x):
    return newtons_method(lambda y: square(y) - x, 1)
```

Abstractions and first-class functions

We've seen two ways to express the square-root computation as an instance of a more general method, once as a fixed-point search and once using Newton's method. Since Newton's method was itself expressed as a fixed-point process, we actually saw two ways to compute square roots as fixed points. Each method begins with a function and finds a fixed point of some transformation of the function. We can express this general idea itself as a function:

```
def fixed_point_of_transform(g, transform, guess):
    return fixed_point(transform(g), guess)
```

This very general function takes as its arguments a function `g` that computes some function, a function that transforms `g`, and an initial guess. The returned result is a fixed point of the transformed function.

Using this abstraction, we can recast the first square-root computation from this section (where we look for a fixed point of the average-damped version of $y \mapsto x/y$) as an instance of this general method:

```
def sqrt(x):
    return fixed_point_of_transform(lambda y: x / y,
                                    average_damp,
                                    1)
```


Similarly, we can express the second square-root computation from this section (an instance of Newton's method that finds a fixed point of the Newton transform of $y \mapsto y^2 - x$) as

```
def sqrt(x):
    return fixed_point_of_transform(lambda y: square(y) - x,
                                    newton_transform,
                                    1)
```

We began section 1.3 with the observation that compound functions are a crucial abstraction mechanism, because they permit us to express general methods of computing as explicit elements in our programming language. Now we've seen how higher-order functions permit us to manipulate these general methods to create further abstractions.

As programmers, we should be alert to opportunities to identify the underlying abstractions in our programs and to build upon them and generalize them to create more powerful abstractions. This is not to say that one should always write programs in the most abstract way possible; expert programmers know how to choose the level of abstraction appropriate to their task. But it is important to be able to think in terms of these abstractions, so that we can be ready to apply them in new contexts. The significance of higher-order functions is that they enable us to represent these abstractions explicitly as elements in our programming language, so that they can be handled just like other computational elements.

In general, programming languages impose restrictions on the ways in which computational elements can be manipulated. Elements with the fewest restrictions are said to have *first-class* status. Some of the "rights and privileges" of first-class elements are: [^66: The notion of first-class status of programming-language elements is due to the British computer scientist Christopher Strachey (1916–1975).]

- They may be referred to using names.
- They may be passed as arguments to functions.
- They may be returned as the results of functions.
- They may be included in data structures. [^67: We'll see examples of this after we introduce data structures in chapter 2.] Python, like other high-level programming languages, awards functions full first-class status. This poses challenges for efficient implementation, but the resulting gain in expressive power is enormous. [^68: The major implementation cost of first-class functions is that allowing functions to be returned as values requires reserving storage for a function's free names even while the function is not executing. In the Python implementation we will study in section , these names are stored in the function's environment.]

Exercise 1.40. Define a function `cubic` that can be used together with the `newtons_method` function in expressions of the form

```
newtons_method(cubic(a, b, c), 1)
```

to approximate zeros of the cubic $x^3 + ax^2 + bx + c$.

Exercise 1.41. Define a function `double` that takes a function of one argument as argument and returns a function that applies the original function twice. For example, if `inc` is a function that adds 1 to its argument, then `double(inc)` should be a function that adds 2. What value is printed by

```
print(double(double(double))(inc)(5))
```

Exercise 1.42. Let f and g be two one-argument functions. The *composition* $f \circ g$ is defined to be the function $x \mapsto f(g(x))$. Define a function `compose` that implements composition. For example, if `inc` is a function that adds 1 to its argument,

```
print(compose(square, inc)(6))
```

49

returns 49.

Exercise 1.43. If f is a numerical function and n is a positive integer, then we can form the n th repeated application of f , which is defined to be the function whose value at x is $f(f(\dots f(x)\dots))$. For example, if f is the function $x \mapsto x+1$, then the n th repeated application of f is the function $x \mapsto x+n$. If f is the operation of squaring a number, then the n th repeated application of f is the function that raises its argument to the 2^n th power. Write a function that takes as inputs a function that computes f and a positive integer n and returns the function that computes the n th repeated application of f . Your function should be able to be used as follows:

```
print(repeated(square, 2)(5))
```

625

Hint: You may find it convenient to use `compose` from exercise 1.42.

Exercise 1.44. The idea of *smoothing* a function is an important concept in signal processing. If f is a function and dx is some small number, then the smoothed version of f is the function whose value at a point x is the average of $f(x-dx)$, $f(x)$, and $f(x+dx)$. Write a function `smooth` that takes as input a function that computes f and returns a function that computes the smoothed f . It is sometimes valuable to repeatedly smooth a function (that is,

smooth the smoothed function, and so on) to obtain the *n -fold smoothed function*. Show how to generate the n -fold smoothed function of any given function using `smooth` and `repeated` from exercise 1.43.

Exercise 1.45. We saw in section 1.3.3 that attempting to compute square roots by naively finding a fixed point of x/y does not converge, and that this can be fixed by average damping. The same method works for finding cube roots as fixed points of the average-damped x/y^2 . Unfortunately, the process does not work for fourth roots—a single average damp is not enough to make a fixed-point search for x/y^3 converge. On the other hand, if we average-damp twice (i.e., use the average damp of the average damp of x/y^3) the fixed-point search does converge. Do some experiments to determine how many average damps are required to compute n th roots as a fixed-point search based upon repeated average damping of x/y^{n-1} . Use this to implement a simple function for computing n th roots using `fixed_point`, `average_damp`, and the `repeated` function of exercise 1.43. Assume that any arithmetic operations you need are available as primitives.

Exercise 1.46. Several of the numerical methods described in this chapter are instances of an extremely general computational strategy known as *iterative improvement*. Iterative improvement says that, to compute something, we start with an initial guess for the answer, test if the guess is good enough, and otherwise improve the guess and continue the process using the improved guess as the new guess. Write a function `iterative_improve` that takes two functions as arguments: a method for telling whether a guess is good enough and a method for improving a guess. The function `iterative_improve` should return as its value a function that takes a guess as argument and keeps improving the guess until it is good enough. Rewrite the `sqrt` function of section 1.1.7 and the `fixed_point` function of section 1.3.3 in terms of `iterative_improve`.

2 Building Abstractions with Data

We now come to the decisive step of mathematical abstraction: we forget about what the symbols stand for. ... [The mathematician] need not be idle; there are many operations which he may carry out with these symbols, without ever having to look at the things they stand for. — Hermann Weyl, The Mathematical Way of Thinking

We concentrated in chapter 1 on computational processes and on the role of functions in program design. We saw how to use primitive data (numbers) and primitive operations (arithmetic operations), how to combine functions to form compound functions through composition, conditionals, and the use of parameters, and how to abstract processes by using function definitions. We saw that a function can be regarded as a pattern for the local evolution of a process, and we classified, reasoned about, and performed simple algorithmic analyses of some common patterns for processes as embodied in functions. We also saw that higher-order functions enhance the power of our language by enabling us to manipulate, and thereby to reason in terms of, general methods of computation. This is much of the essence of programming.

In this chapter we are going to look at more complex data. All the functions in chapter 1 operate on simple numerical data, and simple data are not sufficient for many of the problems we wish to address using computation. Programs are typically designed to model complex phenomena, and more often than not one must construct computational objects that have several parts in order to model real-world phenomena that have several aspects. Thus, whereas our focus in chapter 1 was on building abstractions by combining functions to form compound functions, we turn in this chapter to another key aspect of any programming language: the means it provides for building abstractions by combining data objects to form *compound data*.

Why do we want compound data in a programming language? For the same reasons that we want compound functions: to elevate the conceptual level at which we can design our programs, to increase the modularity of our designs, and to enhance the expressive power of our language. Just as the ability to declare functions enables us to deal with processes at a higher conceptual level than that of the primitive operations of the language, the ability to construct compound data objects enables us to deal with data at a higher conceptual level than that of the primitive data objects of the language.

Consider the task of designing a system to perform arithmetic with rational numbers. We could imagine an operation `add_rat` that takes two rational numbers and produces their sum. In terms of simple data, a rational number can be thought of as two integers: a numerator and a denominator. Thus, we could design a program in which each rational number would be represented by two integers (a numerator and a denominator) and where `add_rat` would be

implemented by two functions (one producing the numerator of the sum and one producing the denominator). But this would be awkward, because we would then need to explicitly keep track of which numerators corresponded to which denominators. In a system intended to perform many operations on many rational numbers, such bookkeeping details would clutter the programs substantially, to say nothing of what they would do to our minds. It would be much better if we could "glue together" a numerator and denominator to form a pair—a *compound data object*—that our programs could manipulate in a way that would be consistent with regarding a rational number as a single conceptual unit.

The use of compound data also enables us to increase the modularity of our programs. If we can manipulate rational numbers directly as objects in their own right, then we can separate the part of our program that deals with rational numbers per se from the details of how rational numbers may be represented as pairs of integers. The general technique of isolating the parts of a program that deal with how data objects are represented from the parts of a program that deal with how data objects are used is a powerful design methodology called *data abstraction*. We will see how data abstraction makes programs much easier to design, maintain, and modify.

The use of compound data leads to a real increase in the expressive power of our programming language. Consider the idea of forming a "linear combination" $ax+by$. We might like to write a function that would accept a , b , x , and y as arguments and return the value of $ax+by$. This presents no difficulty if the arguments are to be numbers, because we can readily declare the function

```
def linear_combination(a, b, x, y):  
    return a * x + b * y
```

But suppose we are not concerned only with numbers. Suppose we would like to describe a process that forms linear combinations whenever addition and multiplication are defined—for rational numbers, complex numbers, polynomials, or whatever. We could express this as a function of the form

```
def linear_combination(a, b, x, y):  
    return add(mul(a, x), mul(b, y))
```

where `add` and `mul` are not the primitive functions `+` and `*` but rather more complex things that will perform the appropriate operations for whatever kinds of data we pass in as the arguments `a`, `b`, `x`, and `y`. The key point is that the only thing `linear_combination` should need to know about `a`, `b`, `x`, and `y` is that the functions `add` and `mul` will perform the appropriate manipulations. From the perspective of the function `linear_combination`, it is irrelevant what `a`, `b`, `x`, and `y` are and even more irrelevant how they might happen to be represented in terms of more primitive data. This same example shows why it is important that our programming language provide the ability to manipulate compound objects directly: Without this,

there is no way for a function such as `linear_combination` to pass its arguments along to `add` and `mul` without having to know their detailed structure. [^1: The ability to directly manipulate functions provides an analogous increase in the expressive power of a programming language. For example, in section 1.3.1 we introduced the `sum` function, which takes a function `term` as an argument and computes the sum of the values of `term` over some specified interval. In order to define `sum`, it is crucial that we be able to speak of a function such as `term` as an entity in its own right, without regard for how `term` might be expressed with more primitive operations. Indeed, if we did not have the notion of "a function," it is doubtful that we would ever even think of the possibility of defining an operation such as `sum`. Moreover, insofar as performing the summation is concerned, the details of how `term` may be constructed from more primitive operations are irrelevant.]

We begin this chapter by implementing the rational-number arithmetic system mentioned above. This will form the background for our discussion of compound data and data abstraction. As with compound functions, the main issue to be addressed is that of abstraction as a technique for coping with complexity, and we will see how data abstraction enables us to erect suitable *abstraction barriers* between different parts of a program.

We will see that the key to forming compound data is that a programming language should provide some kind of "glue" so that data objects can be combined to form more complex data objects. There are many possible kinds of glue. Indeed, we will discover how to form compound data using no special "data" operations at all, only functions. This will further blur the distinction between "function" and "data," which was already becoming tenuous toward the end of chapter 1. We will also explore some conventional techniques for representing sequences and trees. One key idea in dealing with compound data is the notion of *closure*—that the glue we use for combining data objects should allow us to combine not only primitive data objects, but compound data objects as well. Another key idea is that compound data objects can serve as *conventional interfaces* for combining program modules in mix-and-match ways. We illustrate some of these ideas by presenting a simple graphics language that exploits closure.

We will then augment the representational power of our language by introducing *symbolic expressions*—data whose elementary parts can be arbitrary symbols rather than only numbers. We explore various alternatives for representing sets of objects. We will find that, just as a given numerical function can be computed by many different computational processes, there are many ways in which a given data structure can be represented in terms of simpler objects, and the choice of representation can have significant impact on the time and space requirements of processes that manipulate the data. We will investigate these ideas in the context of symbolic differentiation, the representation of sets, and the encoding of information.

Next we will take up the problem of working with data that may be represented differently by different parts of a program. This leads to the need to implement *generic operations*, which must handle many different types of data. Maintaining modularity in the presence of generic operations

requires more powerful abstraction barriers than can be erected with simple data abstraction alone. In particular, we introduce *data-directed programming* as a technique that allows individual data representations to be designed in isolation and then combined *additively* (i.e., without modification). To illustrate the power of this approach to system design, we close the chapter by applying what we have learned to the implementation of a package for performing symbolic arithmetic on polynomials, in which the coefficients of the polynomials can be integers, rational numbers, complex numbers, and even other polynomials.

2.1 Introduction to Data Abstraction

In section 1.1.8, we noted that a function used as an element in creating a more complex function could be regarded not only as a collection of particular operations but also as a functional abstraction. That is, the details of how the function was implemented could be suppressed, and the particular function itself could be replaced by any other function with the same overall behavior. In other words, we could make an abstraction that would separate the way the function would be used from the details of how the function would be implemented in terms of more primitive functions. The analogous notion for compound data is called *data abstraction*. Data abstraction is a methodology that enables us to isolate how a compound data object is used from the details of how it is constructed from more primitive data objects.

The basic idea of data abstraction is to structure the programs that are to use compound data objects so that they operate on "abstract data." That is, our programs should use data in such a way as to make no assumptions about the data that are not strictly necessary for performing the task at hand. At the same time, a "concrete" data representation is defined independent of the programs that use the data. The interface between these two parts of our system will be a set of functions, called *selectors* and *constructors*, that implement the abstract data in terms of the concrete representation. To illustrate this technique, we will consider how to design a set of functions for manipulating rational numbers.

2.1.1 Example: Arithmetic Operations for Rational Numbers

Suppose we want to do arithmetic with rational numbers. We want to be able to add, subtract, multiply, and divide them and to test whether two rational numbers are equal.

Let us begin by assuming that we already have a way of constructing a rational number from a numerator and a denominator. We also assume that, given a rational number, we have a way of extracting (or selecting) its numerator and its denominator. Let us further assume that the constructor and selectors are available as functions:

- `make_rat( $n$ , $d$ )` returns the rational number whose numerator is the integer `$n$` and whose denominator is the integer `$d$`.

- `numer( $x$ )` returns the numerator of the rational number `$x$`.
- `denom( $x$ )` returns the denominator of the rational number `$x$`.

We are using here a powerful strategy of synthesis: *wishful thinking*. We haven't yet said how a rational number is represented, or how the functions `numer`, `denom`, and `make_rat` should be implemented. Even so, if we did have these three functions, we could then add, subtract, multiply, divide, and test equality by using the following relations:

$$\begin{array}{l} \frac{n_1}{d_1} + \frac{n_2}{d_2} = \frac{n_1 d_2 + n_2 d_1}{d_1 d_2} \\ \frac{n_1}{d_1} - \frac{n_2}{d_2} = \frac{n_1 d_2 - n_2 d_1}{d_1 d_2} \\ \frac{n_1}{d_1} \cdot \frac{n_2}{d_2} = \frac{n_1 n_2}{d_1 d_2} \\ \frac{n_1}{d_1} / \frac{n_2}{d_2} = \frac{n_1 d_2}{d_1 n_2} \end{array}$$

$\frac{n_1}{d_1} = \frac{n_2}{d_2} \iff n_1 d_2 = n_2 d_1$

We can express these rules as functions:

```
def add_rat(x, y):
    return make_rat(numer(x) * denom(y) + numer(y) * denom(x),
                    denom(x) * denom(y))

def sub_rat(x, y):
    return make_rat(numer(x) * denom(y) - numer(y) * denom(x),
                    denom(x) * denom(y))

def mul_rat(x, y):
    return make_rat(numer(x) * numer(y),
                    denom(x) * denom(y))

def div_rat(x, y):
    return make_rat(numer(x) * denom(y),
                    denom(x) * numer(y))

def equal_rat(x, y):
    return numer(x) * denom(y) == numer(y) * denom(x)
```

Now we have the operations on rational numbers defined in terms of the selector and constructor functions `numer`, `denom`, and `make_rat`. But we haven't yet defined these. What we need is some way to glue together a numerator and a denominator to form a rational number.

Pairs

To enable us to implement the concrete level of our data abstraction, our Python environment provides a compound structure called a *pair*, which can be constructed with the built-in function `pair`. This function takes two arguments and returns a compound data object that contains the two arguments as parts. Given a pair, we can extract the parts using the primitive functions `head` and `tail`. Thus, we can use `pair`, `head`, and `tail` as follows:

```
x = pair(1, 2)
```



```
print(head(x))
```

```
1
```

```
print(tail(x))
```

```
2
```

Notice that a pair is a data object that can be given a name and manipulated, just like a primitive data object. Moreover, `pair` can be used to form pairs whose elements are pairs, and so on:

```
x = pair(1, 2)
```

```
y = pair(3, 4)
```

```
z = pair(x, y)
```

```
print(head(head(z)))
```

```
1
```

```
print(head(tail(z)))
```

```
3
```

In section 2.2 we will see how this ability to combine pairs means that pairs can be used as general-purpose building blocks to create all sorts of complex data structures. The single compound-data primitive *pair*, implemented by the functions `pair`, `head`, and `tail`, is the only glue we need. Data objects constructed from pairs are called *list-structured* data.

Representing rational numbers

Pairs offer a natural way to complete the rational-number system. Simply represent a rational number as a pair of two integers: a numerator and a denominator. Then `make_rat`, `numer`, and `denom` are readily implemented as follows: [^2: Another way to define the selectors and constructor is The first definition associates the name `make_rat` with the value of the expression `pair`, which is the primitive function that constructs pairs. Thus `make_rat` and `pair` are names for the same primitive constructor. Defining selectors and constructors in this way is efficient: Instead of `make_rat` calling `pair`, `make_rat` is `pair`, so there is only one function called, not two,

when `make_rat` is called. On the other hand, doing this defeats debugging aids that trace function calls or put breakpoints on function calls: You may want to watch `make_rat` being called, but you certainly don't want to watch every call to `pair`. We have chosen not to use this style of definition in this book.]

```
make_rat = pair
numer = head
denom = tail
```

```
def make_rat(n, d): return pair(n, d)

def numer(x): return head(x)

def denom(x): return tail(x)
```

Also, in order to display the results of our computations, we can print rational numbers by printing the numerator, a slash, and the denominator. We use the built-in function `str` to turn any value (here a number) into a string. The operator `+` in Python is *overloaded*; it can be applied to two numbers or to two strings, and in the latter case it returns the result of *concatenating* the two strings.

```
def print_rat(x):
    print(str(numer(x)) + " / " + str(denom(x)))
```

Now we can try our rational-number functions:

```
one_half = make_rat(1, 2)

print(print_rat(one_half))
```

```
1 / 2
```

```
one_third = make_rat(1, 3)
```

```
print(print_rat(add_rat(one_half, one_third)))
```

```
5 / 6
```

```
print(print_rat(mul_rat(one_half, one_third)))
```

```
1 / 6
```

```
print(print_rat(add_rat(one_third, one_third)))
```

```
6 / 9
```

As the final example shows, our rational-number implementation does not reduce rational numbers to lowest terms. We can remedy this by changing `make_rat`. If we have a `gcd` function like the one in section 1.2.5 that produces the greatest common divisor of two integers, we can use `gcd` to reduce the numerator and the denominator to lowest terms before constructing the pair:

```
def make_rat(n, d):  
    g = gcd(n, d)  
    return pair(n // g, d // g)
```

Now we have

```
print_rat(add_rat(one_third, one_third))
```

```
2 / 3
```

as desired. This modification was accomplished by changing the constructor `make_rat` without changing any of the functions (such as `add_rat` and `mul_rat`) that implement the actual operations.

Exercise 2.1. Define a better version of `make_rat` that handles both positive and negative arguments. The function `make_rat` should normalize the sign so that if the rational number is positive, both the numerator and denominator are positive, and if the rational number is negative, only the numerator is negative.

2.1.2 Abstraction Barriers

Before continuing with more examples of compound data and data abstraction, let us consider some of the issues raised by the rational-number example. We defined the rational-number operations in terms of a constructor `make_rat` and selectors `numer` and `denom`. In general, the underlying idea of data abstraction is to identify for each type of data object a basic set of operations in terms of which all manipulations of data objects of that type will be expressed, and then to use only those operations in manipulating the data.

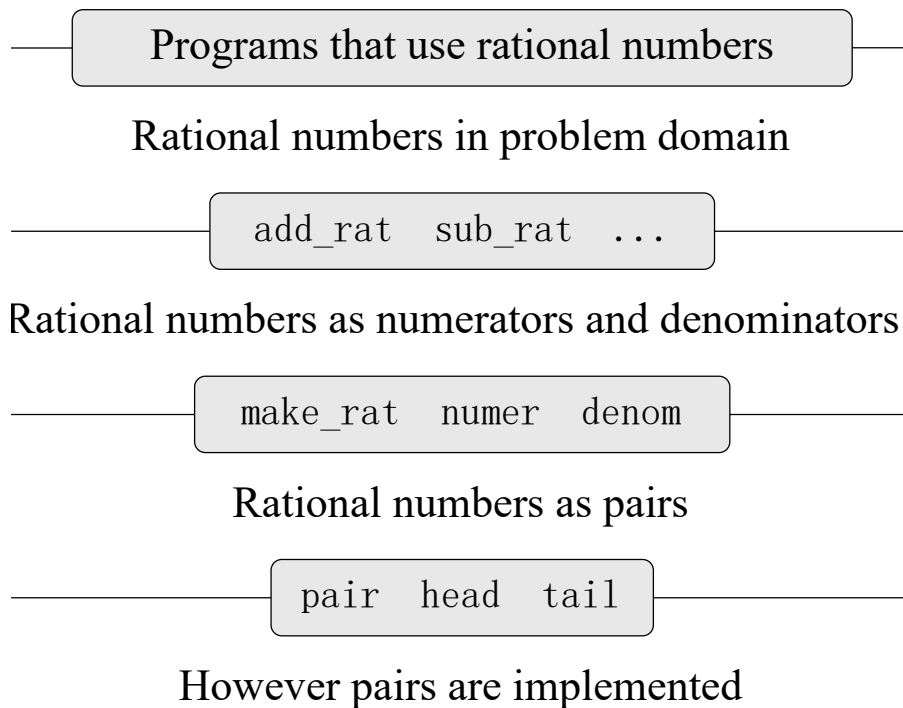


Figure 2.1. Data-abstraction barriers in the rational-number package.

We can envision the structure of the rational-number system as shown in figure 2.1. The horizontal lines represent *abstraction barriers* that isolate different "levels" of the system. At each level, the barrier separates the programs (above) that use the data abstraction from the programs (below) that implement the data abstraction. Programs that use rational numbers manipulate them solely in terms of the functions supplied "for public use" by the rational-number package:

`add_rat`, `sub_rat`, `mul_rat`, `div_rat`, and `equal_rat`. These, in turn, are implemented solely in terms of the constructor and selectors `make_rat`, `numer`, and `denom`, which themselves are implemented in terms of pairs. The details of how pairs are implemented are irrelevant to the rest of the rational-number package so long as pairs can be manipulated by the use of `pair`, `head`, and `tail`. In effect, functions at each level are the interfaces that define the abstraction barriers and connect the different levels.

This simple idea has many advantages. One advantage is that it makes programs much easier to maintain and to modify. Any complex data structure can be represented in a variety of ways with the primitive data structures provided by a programming language. Of course, the choice of representation influences the programs that operate on it; thus, if the representation were to be changed at some later time, all such programs might have to be modified accordingly. This task could be time-consuming and expensive in the case of large programs unless the dependence on the representation were to be confined by design to a very few program modules.

For example, an alternate way to address the problem of reducing rational numbers to lowest terms is to perform the reduction whenever we access the parts of a rational number, rather than when we construct it. This leads to different constructor and selector functions:

```

def make_rat(n, d):
    return pair(n, d)
def numer(x):
    g = gcd(head(x), tail(x))
    return head(x) // g
def denom(x):
    g = gcd(head(x), tail(x))
    return tail(x) // g

```

The difference between this implementation and the previous one lies in when we compute the `gcd`. If in our typical use of rational numbers we access the numerators and denominators of the same rational numbers many times, it would be preferable to compute the `gcd` when the rational numbers are constructed. If not, we may be better off waiting until access time to compute the `gcd`. In any case, when we change from one representation to the other, the functions `add_rat`, `sub_rat`, and so on do not have to be modified at all.

Constraining the dependence on the representation to a few interface functions helps us design programs as well as modify them, because it allows us to maintain the flexibility to consider alternate implementations. To continue with our simple example, suppose we are designing a rational-number package and we can't decide initially whether to perform the `gcd` at construction time or at selection time. The data-abstraction methodology gives us a way to defer that decision without losing the ability to make progress on the rest of the system.

Exercise 2.2. Consider the problem of representing line segments in a plane. Each segment is represented as a pair of points: a starting point and an ending point. Declare a constructor `make_segment` and selectors `start_segment` and `end_segment` that define the representation of segments in terms of points. Furthermore, a point can be represented as a pair of numbers: the x coordinate and the y coordinate. Accordingly, specify a constructor `make_point` and selectors `x_point` and `y_point` that define this representation. Finally, using your selectors and constructors, declare a function `midpoint_segment` that takes a line segment as argument and returns its midpoint (the point whose coordinates are the average of the coordinates of the endpoints). To try your functions, you'll need a way to print points:

```

def print_point(p):
    print("(" + str(x_point(p)) + ", "
          + str(y_point(p)) + ")")

```

Exercise 2.3. Implement a representation for rectangles in a plane. (Hint: You may want to make use of exercise 2.2.) In terms of your constructors and selectors, create functions that compute the perimeter and the area of a given rectangle. Now implement a different representation for rectangles. Can you design your system with suitable abstraction barriers, so that the same perimeter and area functions will work using either representation?

2.1.3 What Is Meant by Data?

We began the rational-number implementation in section 2.1.1 by implementing the rational-number operations `add_rat`, `sub_rat`, and so on in terms of three unspecified functions: `make_rat`, `numer`, and `denom`. At that point, we could think of the operations as being defined in terms of data objects—numerators, denominators, and rational numbers—whose behavior was specified by the latter three functions.

But exactly what is meant by *data*? It is not enough to say "whatever is implemented by the given selectors and constructors." Clearly, not every arbitrary set of three functions can serve as an appropriate basis for the rational-number implementation. We need to guarantee that, if we construct a rational number `x` from a pair of integers `n` and `d`, then extracting the `numer` and the `denom` of `x` and dividing them should yield the same result as dividing `n` by `d`. In other words, `make_rat`, `numer`, and `denom` must satisfy the condition that, for any integer `n` and any nonzero integer `d`, if `x` is `make_rat(n, d)`, then

$$\begin{array}{l} \text{\texttt{\texttt{numer}}(\text{\texttt{x}})} \text{\texttt{\texttt{denom}}(\text{\texttt{x}})} = \text{\texttt{\texttt{numer}}(\text{\texttt{n}})} \\ \text{\texttt{\texttt{denom}}(\text{\texttt{x}})} \end{array} \text{\texttt{\texttt{denom}}(\text{\texttt{d}})}$$

In fact, this is the only condition `make_rat`, `numer`, and `denom` must fulfill in order to form a suitable basis for a rational-number representation. In general, we can think of data as defined by some collection of selectors and constructors, together with specified conditions that these functions must fulfill in order to be a valid representation. [^3: Surprisingly, this idea is very difficult to formulate rigorously. There are two approaches to giving such a formulation. One, pioneered by C. A. R. Hoare (1972), is known as the method of *abstract models*. It formalizes the "functions plus conditions" specification as outlined in the rational-number example above. Note that the condition on the rational-number representation was stated in terms of facts about integers (equality and division). In general, abstract models define new kinds of data objects in terms of previously defined types of data objects. Assertions about data objects can therefore be checked by reducing them to assertions about previously defined data objects. Another approach, introduced by Zilles at MIT, by Goguen, Thatcher, Wagner, and Wright at IBM (see Thatcher, Wagner, and Wright 1978), and by Guttag at Toronto (see Guttag 1977), is called *algebraic specification*. It regards the "functions" as elements of an abstract algebraic system whose behavior is specified by axioms that correspond to our "conditions," and uses the techniques of abstract algebra to check assertions about data objects. Both methods are surveyed in the paper by Liskov and Zilles (1975).]

This point of view can serve to define not only "high-level" data objects, such as rational numbers, but lower-level objects as well. Consider the notion of a pair, which we used in order to define our rational numbers. We never actually said what a pair was, only that the language supplied functions `pair`, `head`, and `tail` for operating on pairs. But the only thing we need to know about these three operations is that if we glue two objects together using `pair` we can retrieve the objects using `head` and `tail`. That is, the operations satisfy the condition that, for any

objects `x` and `y`, if `z` is `pair(x, y)` then `head(z)` is `x` and `tail(z)` is `y`. Indeed, we mentioned that these three functions are included as primitives in our language. However, any triple of functions that satisfies the above condition can be used as the basis for implementing pairs. This point is illustrated strikingly by the fact that we could implement `pair`, `head`, and `tail` without using any data structures at all but only using functions. Here are the definitions:

[^4: The function `error` introduced in section 1.3.3 takes as optional second argument a string that gets displayed before the first argument—for example, if `m` is 42:]

```
Error in line 7: argument not 0 or 1 -- pair: 42
```

```
def pair(x, y):
    def dispatch(m):
        return (x if m == 0
                else y if m == 1
                else error("argument not 0 or 1 -- pair", m))
    return dispatch
def head(z): return z(0)

def tail(z): return z(1)
```

This use of functions corresponds to nothing like our intuitive notion of what data should be. Nevertheless, all we need to do to show that this is a valid way to represent pairs is to verify that these functions satisfy the condition given above.

The subtle point to notice is that the value returned by `pair(x, y)` is a function—namely the internally defined function `dispatch`, which takes one argument and returns either `x` or `y` depending on whether the argument is 0 or 1. Correspondingly, `head(z)` is defined to apply `z` to 0. Hence, if `z` is the function formed by `pair(x, y)`, then `z` applied to 0 will yield `x`. Thus, we have shown that `head(pair(x, y))` yields `x`, as desired. Similarly, `tail(pair(x, y))` applies the function returned by `pair(x, y)` to 1, which returns `y`. Therefore, this functional implementation of pairs is a valid implementation, and if we access pairs using only `pair`, `head`, and `tail` we cannot distinguish this implementation from one that uses "real" data structures.

The point of exhibiting the functional representation of pairs is not that our language works this way (an efficient implementation of pairs might use Python's native *list* data structure) but that it could work this way. The functional representation, although obscure, is a perfectly adequate way to represent pairs, since it fulfills the only conditions that pairs need to fulfill. This example also demonstrates that the ability to manipulate functions as objects automatically provides the ability to represent compound data. This may seem a curiosity now, but functional representations of data will play a central role in our programming repertoire. This style of programming is often called *message passing*, and we will be using it as a basic tool in chapter 3 when we address the issues of modeling and simulation.

Exercise 2.4. Here is an alternative functional representation of pairs. For this representation, verify that `head(pair(x, y))` yields `x` for any objects `x` and `y`.

```
def pair(x, y):
    return lambda m: m(x, y)
def head(z):
    return z(lambda p, q: p)
```

What is the corresponding definition of `tail`? (Hint: To verify that this works, make use of the substitution model of section 1.1.5.)

Exercise 2.5. Show that we can represent pairs of nonnegative integers using only numbers and arithmetic operations if we represent the pair a and b as the integer that is the product $2^a 3^b$. Give the corresponding definitions of the functions `pair`, `head`, and `tail`.

Exercise 2.6. In case representing pairs as functions (exercise 2.4) wasn't mind-boggling enough, consider that, in a language that can manipulate functions, we can get by without numbers (at least insofar as nonnegative integers are concerned) by implementing 0 and the operation of adding 1 as

```
zero = lambda f: lambda x: x
def add_1(n):
    return lambda f: lambda x: f(n(f)(x))
```

This representation is known as *Church numerals*, after its inventor, Alonzo Church, the logician who invented the λ calculus.

Define `one` and `two` directly (not in terms of `zero` and `add_1`). (Hint: Use substitution to evaluate `add_1(zero)`). Give a direct definition of the addition function `plus` (not in terms of repeated application of `add_1`).

2.1.4 Extended Exercise: Interval Arithmetic

Alyssa P. Hacker is designing a system to help people solve engineering problems. One feature she wants to provide in her system is the ability to manipulate inexact quantities (such as measured parameters of physical devices) with known precision, so that when computations are done with such approximate quantities the results will be numbers of known precision.

Electrical engineers will be using Alyssa's system to compute electrical quantities. It is sometimes necessary for them to compute the value of a parallel equivalent resistance R_p of two resistors R_1 and R_2 using the formula

$$R_p = \frac{1}{1/R_1 + 1/R_2}$$

Resistance values are usually known only up to some tolerance guaranteed by the manufacturer of the resistor. For example, if you buy a resistor labeled "6.8 ohms with 10% tolerance" you can only be sure that the resistor has a resistance between $6.8 - 0.68 = 6.12\Omega$ and $6.8 + 0.68 = 7.48\Omega$. Thus, if you have a 6.8-ohm 10% resistor in parallel with a 4.7-ohm 5% resistor, the resistance of the combination can range from about 2.58 ohms (if the two resistors are at the lower bounds) to about 2.97 ohms (if the two resistors are at the upper bounds).

Alyssa's idea is to implement "interval arithmetic" as a set of arithmetic operations for combining "intervals" (objects that represent the range of possible values of an inexact quantity). The result of adding, subtracting, multiplying, or dividing two intervals is itself an interval, representing the range of the result.

Alyssa postulates the existence of an abstract object called an "interval" that has two endpoints: a lower bound and an upper bound. She also presumes that, given the endpoints of an interval, she can construct the interval using the data constructor `make_interval`. Alyssa first writes a function for adding two intervals. She reasons that the minimum value the sum could be is the sum of the two lower bounds and the maximum value it could be is the sum of the two upper bounds:

```
def add_interval(x, y):
    return make_interval(lower_bound(x) + lower_bound(y),
                        upper_bound(x) + upper_bound(y))
```

Alyssa also works out the product of two intervals by finding the minimum and the maximum of the products of the bounds and using them as the bounds of the resulting interval. (The functions `min` and `max` are primitives that find the minimum or maximum of any number of arguments.)

```
def mul_interval(x, y):
    p1 = lower_bound(x) * lower_bound(y)
    p2 = lower_bound(x) * upper_bound(y)
    p3 = upper_bound(x) * lower_bound(y)
    p4 = upper_bound(x) * upper_bound(y)
    return make_interval(min(p1, p2, p3, p4),
                        max(p1, p2, p3, p4))
```

To divide two intervals, Alyssa multiplies the first by the reciprocal of the second. Note that the bounds of the reciprocal interval are the reciprocal of the upper bound and the reciprocal of the lower bound, in that order.

```
def div_interval(x, y):
    return mul_interval(x, make_interval(1 / upper_bound(y),
                                        1 / lower_bound(y)))
```

Exercise 2.7. Alyssa's program is incomplete because she has not specified the implementation of the interval abstraction. Here is a definition of the interval constructor:

```
def make_interval(x, y): return pair(x, y)
```

Define selectors `upper_bound` and `lower_bound` to complete the implementation.

Exercise 2.8. Using reasoning analogous to Alyssa's, describe how the difference of two intervals may be computed. Define a corresponding subtraction function, called `sub_interval`.

Exercise 2.9. The *width* of an interval is half of the difference between its upper and lower bounds. The width is a measure of the uncertainty of the number specified by the interval. For some arithmetic operations the width of the result of combining two intervals is a function only of the widths of the argument intervals, whereas for others the width of the combination is not a function of the widths of the argument intervals. Show that the width of the sum (or difference) of two intervals is a function only of the widths of the intervals being added (or subtracted). Give examples to show that this is not true for multiplication or division.

Exercise 2.10. Ben Bitdiddle, an expert systems programmer, looks over Alyssa's shoulder and comments that it is not clear what it means to divide by an interval that spans zero. Modify Alyssa's program to check for this condition and to signal an error if it occurs.

Exercise 2.11. In passing, Ben also cryptically comments: "By testing the signs of the endpoints of the intervals, it is possible to break `mul_interval` into nine cases, only one of which requires more than two multiplications." Rewrite this function using Ben's suggestion.

After debugging her program, Alyssa shows it to a potential user, who complains that her program solves the wrong problem. He wants a program that can deal with numbers represented as a center value and an additive tolerance; for example, he wants to work with intervals such as $\$3.5 \pm 0.15$ rather than $[\$3.35, \$3.65]$. Alyssa returns to her desk and fixes this problem by supplying an alternate constructor and alternate selectors:

```
def make_center_width(c, w):
    return make_interval(c - w, c + w)
def center(i):
    return (lower_bound(i) + upper_bound(i)) / 2
def width(i):
    return (upper_bound(i) - lower_bound(i)) / 2
```

Unfortunately, most of Alyssa's users are engineers. Real engineering situations usually involve measurements with only a small uncertainty, measured as the ratio of the width of the interval to the midpoint of the interval. Engineers usually specify percentage tolerances on the parameters of devices, as in the resistor specifications given earlier.

Exercise 2.12. Define a constructor `make_center_percent` that takes a center and a percentage tolerance and produces the desired interval. You must also define a selector `percent` that

produces the percentage tolerance for a given interval. The `center` selector is the same as the one shown above.

Exercise 2.13. Show that under the assumption of small percentage tolerances there is a simple formula for the approximate percentage tolerance of the product of two intervals in terms of the tolerances of the factors. You may simplify the problem by assuming that all numbers are positive.

After considerable work, Alyssa P. Hacker delivers her finished system. Several years later, after she has forgotten all about it, she gets a frenzied call from an irate user, Lem E. Tweakit. It seems that Lem has noticed that the formula for parallel resistors can be written in two algebraically equivalent ways:

$$\frac{R_1 R_2}{R_1 + R_2}$$

and

$$\frac{1}{1/R_1 + 1/R_2}$$

He has written the following two programs, each of which computes the parallel-resistors formula differently:

```
def par1(r1, r2):
    return div_interval(mul_interval(r1, r2),
                        add_interval(r1, r2))

def par2(r1, r2):
    one = make_interval(1, 1)
    return div_interval(one,
                        add_interval(div_interval(one, r1),
                                    div_interval(one, r2)))
```

Lem complains that Alyssa's program gives different answers for the two ways of computing. This is a serious complaint.

Exercise 2.14. Demonstrate that Lem is right. Investigate the behavior of the system on a variety of arithmetic expressions. Make some intervals `A` and `B`, and use them in computing the expressions `A/A` and `A/B`. You will get the most insight by using intervals whose width is a small percentage of the center value. Examine the results of the computation in center-percent form (see exercise 2.12).

Exercise 2.15. Eva Lu Ator, another user, has also noticed the different intervals computed by different but algebraically equivalent expressions. She says that a formula to compute with intervals using Alyssa's system will produce tighter error bounds if it can be written in such a form that no name that represents an uncertain number is repeated. Thus, she says, `par2` is a "better" program for parallel resistances than `par1`. Is she right? Why?

Exercise 2.16. Explain, in general, why equivalent algebraic expressions may lead to different answers. Can you devise an interval-arithmetic package that does not have this shortcoming, or is this task impossible? (Warning: This problem is very difficult.)

2.2 Hierarchical Data and the Closure Property

As we have seen, pairs provide a primitive "glue" that we can use to construct compound data objects. Figure 2.2 shows a standard way to visualize a pair—in this case, the pair formed by `pair(1, 2)`. In this representation, which is called *box-and-pointer notation*, each compound object is shown as a *pointer* to a box. The box for a pair has two parts, the left part containing the head of the pair and the right part containing the tail.

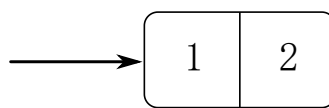


Figure 2.2. Box-and-pointer representation of `pair(1, 2)`.

We have already seen that `pair` can be used to combine not only numbers but pairs as well. (You made use of this fact, or should have, in doing exercises 2.2 and 2.3.) As a consequence, pairs provide a universal building block from which we can construct all sorts of data structures.

Figure 2.3 shows two ways to use pairs to combine the numbers 1, 2, 3, and 4.

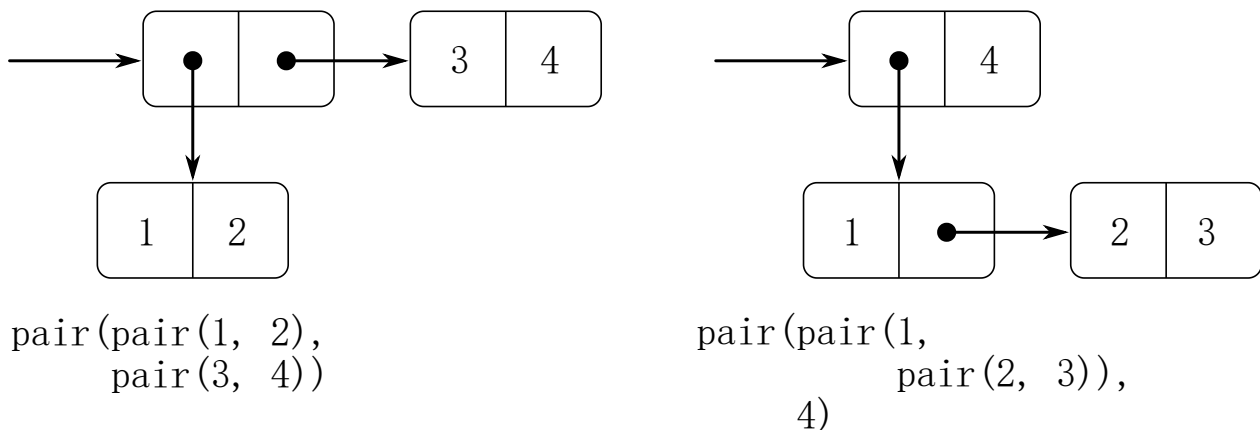


Figure 2.3. Two ways to combine 1, 2, 3, and 4 using pairs.

The ability to create pairs whose elements are pairs is the essence of linked-list structure's importance as a representational tool. We refer to this ability as the *closure property* of `pair`. In general, an operation for combining data objects satisfies the closure property if the results of combining things with that operation can themselves be combined using the same operation. [^5: The use of the word "closure" here comes from abstract algebra, where a set of elements is said to be closed under an operation if applying the operation to elements in the set produces an element that is again an element of the set. The programming languages community also (unfortunately) uses the word "closure" to describe a totally unrelated concept: A closure is an implementation

technique for representing functions with free names. We do not use the word "closure" in this second sense in this book.] Closure is the key to power in any means of combination because it permits us to create *hierarchical* structures—structures made up of parts, which themselves are made up of parts, and so on.

From the outset of chapter 1, we've made essential use of closure in dealing with functions, because all but the very simplest programs rely on the fact that the elements of a combination can themselves be combinations. In this section, we take up the consequences of closure for compound data. We describe some conventional techniques for using pairs to represent sequences and trees, and we exhibit a graphics language that illustrates closure in a vivid way.

2.2.1 Representing Sequences

One of the useful structures we can build with pairs is a *sequence*—an ordered collection of data objects. There are, of course, many ways to represent sequences in terms of pairs. One particularly straightforward representation is illustrated in figure 2.4, where the sequence 1, 2, 3, 4 is represented as a chain of pairs. The `head` of each pair is the corresponding item in the chain, and the `tail` of the pair is the next pair in the chain. The `tail` of the final pair signals the end of the sequence, represented in box-and-pointer diagrams as a diagonal line and in programs as Python's primitive value `None`. The entire sequence is constructed by nested `pair` operations:

```
pair(1,
    pair(2,
        pair(3,
            pair(4, None))))
```

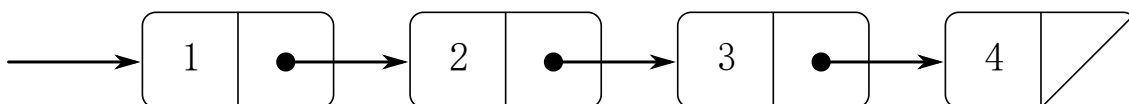


Figure 2.4. The sequence 1, 2, 3, 4 represented as a chain of pairs.

Such a sequence of pairs, formed by nested `pair` applications, is called a *linked list*, and our Python environment provides a built-in function called `llist` to help in constructing linked lists. [^6: In this book, we use *linked list* to mean a chain of pairs terminated by the end-of-linked-list marker. In contrast, the term *linked-list structure* refers to any data structure made out of pairs, not just to linked lists.] The above sequence could be produced by `llist(1, 2, 3, 4)`. In general,

`llist( $a_1$ ,  $a_2$ , ...,  $a_n$ )`

is equivalent to

`pair( $a_1$ , pair( $a_2$ , pair(..., pair( $a_n$ , None)...)...))`

Our interpreter prints pairs using a textual representation of box-and-pointer diagrams that we call *box notation*. The result of `pair(1, 2)` is printed as `[1, 2]`, and the data object in figure 2.4 is printed as `[1, [2, [3, [4, None]]]]`:

```
one_through_four = llist(1, 2, 3, 4)
```

```
print(one_through_four)
```

```
[1, [2, [3, [4, None]]]]
```

We can think of `head` as selecting the first item in the linked list, and of `tail` as selecting the linked-list component consisting of all but the first item. Nested applications of `head` and `tail` can be used to extract the second, third, and subsequent items in the linked list. The constructor `pair` makes a linked list like the original one, but with an additional item at the beginning.

```
print(head(one_through_four))
```

```
1
```

```
print(tail(one_through_four))
```

```
[2, [3, [4, None]]]
```

```
print(head(tail(one_through_four)))
```

```
2
```

```
print(pair(10, one_through_four))
```

```
[10, [1, [2, [3, [4, None]]]]]
```

```
print(pair(5, one_through_four))
```

```
[5, [1, [2, [3, [4, None]]]]]
```

The value `None`, used to terminate the chain of pairs, can be thought of as a sequence of no elements, the *empty linked list*. [^7: The value `None` is used in Python for various purposes, as we

shall see in chapter 3.]

Box notation is sometimes difficult to read. In this book, when we want to indicate the linked-list nature of a data structure, we will employ the alternative *linked-list notation*: Whenever possible, linked-list notation uses applications of `l1ist` whose evaluation would result in the desired structure. For example, instead of the box notation

```
[1, [[2, 3], [[4, [5, None]], [6, None]]]]
```

we write

```
l1ist(1, [2, 3], l1ist(4, 5), 6)
```

in linked-list notation. [^8: Our Python environment provides a built-in function `print_l1ist` that works like the built-in function `print`, except that it uses linked-list notation instead of box notation.]

Linked-list operations

The use of pairs to represent sequences of elements as linked lists is accompanied by conventional programming techniques for manipulating linked lists by successively using `tail` to walk down the linked lists. For example, the function `l1ist_ref` takes as arguments a linked list and a number n and returns the n th item of the linked list. It is customary to number the elements of the linked list beginning with 0. The method for computing `l1ist_ref` is the following:

- For $n=0$, `l1ist_ref` should return the `head` of the linked list.
- Otherwise, `l1ist_ref` should return the $(n-1)$ st item of the `tail` of the linked list.

```
def l1ist_ref(items, n):  
    return (head(items) if n == 0  
            else l1ist_ref(tail(items), n - 1))
```

```
squares = l1ist(1, 4, 9, 16, 25)  
  
print(l1ist_ref(squares, 3))
```

16

Often we walk down the whole linked list. To aid in this, our Python environment includes a built-in predicate `is_none`, which tests whether its argument is the empty linked list. The function `length`, which returns the number of items in a linked list, illustrates this typical pattern of use:

```
def length(items):
    return (0 if is_none(items)
            else 1 + length(tail(items)))
```

```
odds = llist(1, 3, 5, 7)

print(length(odds))
```

4

The `length` function implements a simple recursive plan. The reduction step is:

- The `length` of any linked list is 1 plus the `length` of the `tail` of the linked list. This is applied successively until we reach the base case:
- The `length` of the empty linked list is 0. We could also compute `length` in an iterative style:

```
def length(items):
    def length_iter(a, count):
        return (count if is_none(a)
                else length_iter(tail(a), count + 1))
    return length_iter(items, 0)
```

Another conventional programming technique is to construct an answer linked list by adjoining elements to the front of the linked list with `pair` while walking down a linked list using `tail`, as in the function `append`, which takes two linked lists as arguments and combines their elements to make a new linked list:

```
print_llist(append(squares, odds))
```

```
llist(1, 4, 9, 16, 25, 1, 3, 5, 7)
```

```
print_llist(append(odds, squares))
```

```
llist(1, 3, 5, 7, 1, 4, 9, 16, 25)
```

The function `append` is also implemented using a recursive plan. To append linked lists `list1` and `list2`, do the following:

- If `list1` is the empty linked list, then the result is just `list2`.

- Otherwise, append the `tail` of `list1` and `list2`, and adjoin the `head` of `list1` to the result:

```
def append(list1, list2):
    return (list2 if is_none(list1)
            else pair(head(list1), append(tail(list1), list2)))
```

Exercise 2.17. Define a function `last_pair` that returns the linked list that contains only the last element of a given (nonempty) linked list:

```
print_llist(last_pair(llist(23, 72, 149, 34)))
```

```
llist(34)
```

Exercise 2.18. Define a function `reverse` that takes a linked list as argument and returns a linked list of the same elements in reverse order:

```
print_llist(reverse(llist(1, 4, 9, 16, 25)))
```

```
llist(25, 16, 9, 4, 1)
```

Exercise 2.19. Consider the change-counting program of section 1.2.2. It would be nice to be able to easily change the currency used by the program, so that we could compute the number of ways to change a British pound, for example. As the program is written, the knowledge of the currency is distributed partly into the function `first_denomination` and partly into the function `count_change` (which knows that there are five kinds of U.S. coins). It would be nicer to be able to supply a linked list of coins to be used for making change.

We want to rewrite the function `cc` so that its second argument is a linked list of the values of the coins to use rather than an integer specifying which coins to use. We could then have linked lists that defined each kind of currency:

```
us_coins = llist(50, 25, 10, 5, 1)
uk_coins = llist(100, 50, 20, 10, 5, 2, 1)
```

We could then call `cc` as follows:

```
print(cc(100, us_coins))
```

```
292
```

To do this will require changing the program `cc` somewhat. It will still have the same form, but it will access its second argument differently, as follows:

```
def cc(amount, coin_values):
    return (1 if amount == 0
            else 0 if amount < 0 or no_more(coin_values)
            else cc(amount, except_first_denomination(coin_values)) +
                cc(amount - first_denomination(coin_values), coin_values))
```

Define the functions `first_denomination`, `except_first_denomination`, and `no_more` in terms of primitive operations on linked list structures. Does the order of the linked list `coin_values` affect the answer produced by `cc`? Why or why not?

Exercise 2.20. In the presence of higher-order functions, it is not strictly necessary for functions to have multiple parameters; one would suffice. If we have a function such as `plus` that naturally requires two arguments, we could write a variant of the function to which we pass the arguments one at a time. An application of the variant to the first argument could return a function that we can then apply to the second argument, and so on. This practice—called *currying* and named after the American mathematician and logician Haskell Brooks Curry—is quite common in programming languages such as Haskell and OCaml. In Python, a curried version of `plus` looks as follows.

```
def plus_curried(x):
    return lambda y: x + y
```

Write a function `brooks` that takes a curried function as first argument and as second argument a linked list of arguments to which the curried function is then applied, one by one, in the given order. For example, the following application of `brooks` should have the same effect as `plus_curried(3)(4)`:

```
print(brooks(plus_curried, llist(3, 4)))
```

7

While we are at it, we might as well curry the function `brooks`! Write a function `brooks_curried` that can be applied as follows:

```
print(brooks_curried(llist(plus_curried, 3, 4)))
```

7

With this function `brooks_curried`, what are the results of evaluating the following two statements?

```
brooks_curried(llist(brooks_curried,
                    llist(plus_curried, 3, 4)))
```

```
brooks_curried(llist(brooks_curried,
                    llist(brooks_curried,
                        llist(plus_curried, 3, 4))))
```

Mapping over linked lists

One extremely useful operation is to apply some transformation to each element in a linked list and generate the linked list of results. For instance, the following function scales each number in a linked list by a given factor:

```
def scale_linked_list(items, factor):
    return (None if is_none(items)
            else pair(head(items) * factor,
                      scale_linked_list(tail(items), factor)))
```

```
print(scale_linked_list(llist(1, 2, 3, 4, 5), 10))
```

```
[10, [20, [30, [40, [50, None]]]]]
```

We can abstract this general idea and capture it as a common pattern expressed as a higher-order function, just as in section 1.3. The higher-order function here is called `map`. The function `map` takes as arguments a function of one argument and a linked list, and returns a linked list of the results produced by applying the function to each element in the linked list:

```
def map(fun, items):
    return (None if is_none(items)
            else pair(fun(head(items)),
                      map(fun, tail(items))))
```

```
print(map(abs, llist(-10, 2.5, -11.6, 17)))
```

```
[10, [2.5, [11.6, [17, None]]]]
```

```
print(map(lambda x: x * x, llist(1, 2, 3, 4)))
```

```
[1, [4, [9, [16, None]]]]
```

Now we can give a new definition of `scale_linked_list` in terms of `map` :

```
def scale_linked_list(items, factor):  
    return map(lambda x: x * factor, items)
```

The function `map` is an important construct, not only because it captures a common pattern, but because it establishes a higher level of abstraction in dealing with linked lists. In the original definition of `scale_linked_list`, the recursive structure of the program draws attention to the element-by-element processing of the linked list. Defining `scale_linked_list` in terms of `map` suppresses that level of detail and emphasizes that scaling transforms a linked list of elements to a linked list of results. The difference between the two definitions is not that the computer is performing a different process (it isn't) but that we think about the process differently. In effect, `map` helps establish an abstraction barrier that isolates the implementation of functions that transform linked lists from the details of how the elements of the linked list are extracted and combined. Like the barriers shown in figure 2.1, this abstraction gives us the flexibility to change the low-level details of how sequences are implemented, while preserving the conceptual framework of operations that transform sequences to sequences. Section 2.2.3 expands on this use of sequences as a framework for organizing programs.

Exercise 2.21. The function `square_linked_list` takes a linked list of numbers as argument and returns a linked list of the squares of those numbers.

```
print(square_linked_list(llist(1, 2, 3, 4)))
```

```
[1, [4, [9, [16, None]]]]
```

Here are two different definitions of `square_linked_list`. Complete both of them by filling in the missing expressions:

```
def square_linked_list(items):  
    return (None if is_none(items)  
            else pair(??, ??))
```

```
def square_linked_list(items):  
    return map(??, ??)
```

Exercise 2.22. Louis Reasoner tries to rewrite the first `square_linked_list` function of exercise 2.21 so that it evolves an iterative process:

```
def square_linked_list(items):
    def iter(things, answer):
        return (answer if is_none(things)
                else iter(tail(things),
                          pair(square(head(things)),
                              answer)))
    return iter(items, None)
```

Unfortunately, defining `square_linked_list` this way produces the answer linked list in the reverse order of the one desired. Why?

Louis then tries to fix his bug by interchanging the arguments to `pair` :

```
def square_linked_list(items):
    def iter(things, answer):
        return (answer if is_none(things)
                else iter(tail(things),
                          pair(answer,
                              square(head(things)))))
    return iter(items, None)
```

This doesn't work either. Explain.

Exercise 2.23. The function `for_each` is similar to `map`. It takes as arguments a function and a linked list of elements. However, rather than forming a linked list of the results, `for_each` just applies the function to each of the elements in turn, from left to right. The values returned by applying the function to the elements are not used at all—`for_each` is used with functions that perform an action, such as printing. For example,

```
print(for_each(lambda x: print(x), llist(57, 321, 88)))
```

```
57
321
88
```

The value returned by the call to `for_each` (not illustrated above) can be something arbitrary, such as `true`. Give an implementation of `for_each`.

2.2.2 Hierarchical Structures

The representation of sequences in terms of linked lists generalizes naturally to represent sequences whose elements may themselves be sequences. For example, we can regard the object `[[1, [2, None]], [3, [4, None]]]` constructed by

```
pair(llist(1, 2), llist(3, 4))
```

as a linked list of three items, the first of which is itself a linked list, `[1, [2, None]]`. Figure 2.5 shows the representation of this structure in terms of pairs.

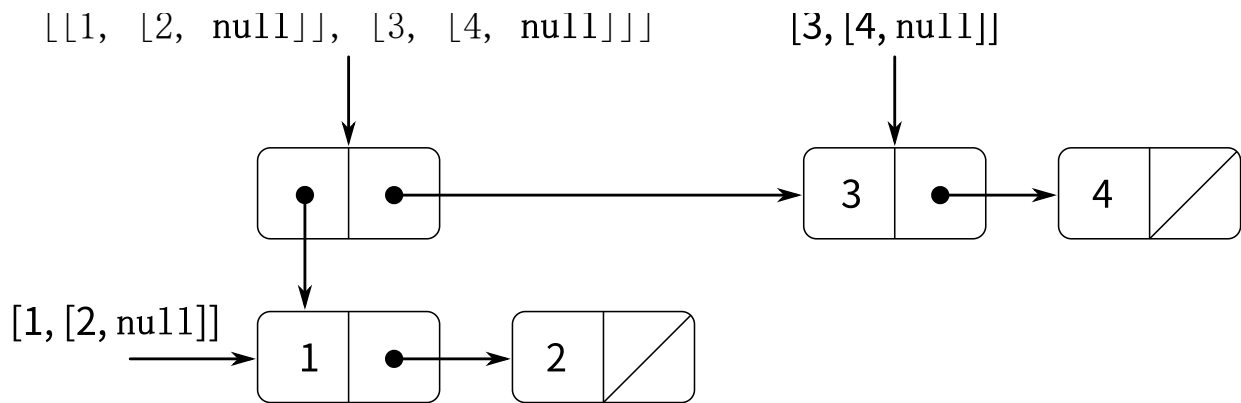


Figure 2.5. Structure formed by `pair(llist(1, 2), llist(3, 4))`.

Another way to think of sequences whose elements are sequences is as *trees*. The elements of the sequence are the branches of the tree, and elements that are themselves sequences are subtrees. Figure 2.6 shows the structure in figure 2.5 viewed as a tree.

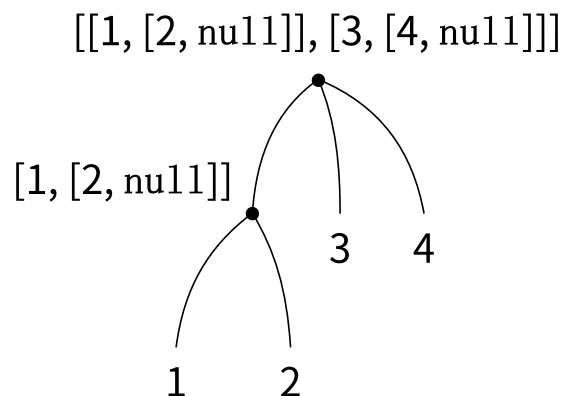


Figure 2.6. The linked-list structure in figure 2.5 viewed as a tree.

Recursion is a natural tool for dealing with tree structures, since we can often reduce operations on trees to operations on their branches, which reduce in turn to operations on the branches of the branches, and so on, until we reach the leaves of the tree. As an example, compare the `length` function of section 2.2.1 with the `count_leaves` function, which returns the total number of leaves of a tree:

```
x = pair(llist(1, 2), llist(3, 4))
```

```
print(length(x))
```

3

```
print(count_leaves(x))
```

4

```
print_llist(llist(x, x))
```

```
llist(llist(llist(1, 2), 3, 4), llist(llist(1, 2), 3, 4))
```

```
print(length(llist(x, x)))
```

2

```
print(count_leaves(llist(x, x)))
```

8

To implement `count_leaves`, recall the recursive plan for computing `length`:

- The `length` of a linked list `x` is 1 plus the `length` of the `tail` of `x`.
- The `length` of the empty linked list is 0. The function `count_leaves` is similar. The value for the empty linked list is the same:
- `count_leaves` of the empty linked list is 0. But in the reduction step, where we strip off the `head` of the linked list, we must take into account that the `head` may itself be a tree whose leaves we need to count. Thus, the appropriate reduction step is
- `count_leaves` of a tree `x` is `count_leaves` of the `head` of `x` plus `count_leaves` of the `tail` of `x`. Finally, by taking `head`s we reach actual leaves, so we need another base case:
- `count_leaves` of a leaf is 1. To aid in writing recursive functions on trees, our Python environment provides the built-in predicate `is_pair`, which tests whether its argument is a pair. Here is the complete function: [^9: The order of the two predicates matters, since `None` satisfies `is_none` and also is not a pair.]

```
def count_leaves(x):  
    return (0 if is_none(x)  
            else 1 if not is_pair(x)  
            else count_leaves(head(x)) + count_leaves(tail(x)))
```

Exercise 2.24. Suppose we evaluate the expression `l1ist(1, l1ist(2, l1ist(3, 4)))`. Give the result printed by the interpreter, the corresponding box-and-pointer structure, and the interpretation of this as a tree (as in figure 2.6).

Exercise 2.25. Give combinations of `head` s and `tail` s that will pick 7 from each of the following linked lists, given in linked-list notation:

```
l1ist(1, 3, l1ist(5, 7), 9)

l1ist(l1ist(7))

l1ist(1, l1ist(2, l1ist(3, l1ist(4, l1ist(5, l1ist(6, 7))))))
```

Exercise 2.26. Suppose we define `x` and `y` to be two linked lists:

```
x = l1ist(1, 2, 3)

y = l1ist(4, 5, 6)
```

What is the result of evaluating each of the following expressions, in box notation and linked-list notation?

```
append(x, y)
```

```
pair(x, y)
```

```
l1ist(x, y)
```

Exercise 2.27. Modify your `reverse` function of exercise 2.18 to produce a `deep_reverse` function that takes a linked list as argument and returns as its value the linked list with its elements reversed and with all sublists deep-reversed as well. For example,

```
x = l1ist(l1ist(1, 2), l1ist(3, 4))
```

```
print_l1ist(x)
```

```
l1ist(l1ist(1, 2), l1ist(3, 4))
```

```
print_l1ist(reverse(x))
```



```
l1list(l1list(3, 4), l1list(1, 2))
```

```
print_l1list(deep_reverse(x))
```

```
l1list(l1list(4, 3), l1list(2, 1))
```

Exercise 2.28. Write a function `fringe` that takes as argument a tree (represented as a linked list) and returns a linked list whose elements are all the leaves of the tree arranged in left-to-right order. For example,

```
x = l1list(l1list(1, 2), l1list(3, 4))
```

```
print_l1list(fringe(x))
```

```
l1list(1, 2, 3, 4)
```

```
print_l1list(fringe(l1list(x, x)))
```

```
l1list(1, 2, 3, 4, 1, 2, 3, 4)
```

Exercise 2.29. A binary mobile consists of two branches, a left branch and a right branch. Each branch is a rod of a certain length, from which hangs either a weight or another binary mobile. We can represent a binary mobile using compound data by constructing it from two branches (for example, using `l1list`):

```
def make_mobile(left, right):  
    return l1list(left, right)
```

A branch is constructed from a `length` (which must be a number) together with a `structure`, which may be either a number (representing a simple weight) or another mobile:

```
def make_branch(length, structure):  
    return l1list(length, structure)
```

a. Write the corresponding selectors `left_branch` and `right_branch`, which return the branches of a mobile, and `branch_length` and `branch_structure`, which return the components of a branch. b. Using your selectors, define a function `total_weight` that returns the total weight of a mobile. c. A mobile is said to be *balanced* if the torque applied by its top-left branch is equal to that applied by its top-right branch (that is, if the length of the left rod multiplied by the weight

hanging from that rod is equal to the corresponding product for the right side) and if each of the submobiles hanging off its branches is balanced. Design a predicate that tests whether a binary mobile is balanced. d. Suppose we change the representation of mobiles so that the constructors are

```
def make_mobile(left, right):
    return pair(left, right)
def make_branch(length, structure):
    return pair(length, structure)
```

How much do you need to change your programs to convert to the new representation?

Mapping over trees

Just as `map` is a powerful abstraction for dealing with sequences, `map` together with recursion is a powerful abstraction for dealing with trees. For instance, the `scale_tree` function, analogous to `scale_linked_list` of section 2.2.1, takes as arguments a numeric factor and a tree whose leaves are numbers. It returns a tree of the same shape, where each number is multiplied by the factor. The recursive plan for `scale_tree` is similar to the one for `count_leaves`:

```
def scale_tree(tree, factor):
    return (None if is_none(tree)
            else tree * factor if not is_pair(tree)
            else pair(scale_tree(head(tree), factor),
                      scale_tree(tail(tree), factor)))
```

```
print_llist(scale_tree(llist(1, llist(2, llist(3, 4), 5), llist(6, 7)),
                      10))
```

```
llist(10, llist(20, llist(30, 40), 50), llist(60, 70))
```

Another way to implement `scale_tree` is to regard the tree as a sequence of sub-trees and use `map`. We map over the sequence, scaling each sub-tree in turn, and return the linked list of results. In the base case, where the tree is a leaf, we simply multiply by the factor:

```
def scale_tree(tree, factor):
    return map(lambda sub_tree: (scale_tree(sub_tree, factor)
                                if is_pair(sub_tree)
                                else sub_tree * factor),
              tree)
```

Many tree operations can be implemented by similar combinations of sequence operations and recursion.

Exercise 2.30. Declare a function `square_tree` analogous to the `square_linked_list` function of exercise 2.21. That is, `square_tree` should behave as follows:

```
print_llist(square_tree(llist(1,
                             llist(2, llist(3, 4), 5),
                             llist(6, 7))))
```

```
llist(1, llist(4, llist(9, 16), 25), llist(36, 49)))
```

Declare `square_tree` both directly (i.e., without using any higher-order functions) and also by using `map` and recursion.

Exercise 2.31. Abstract your answer to exercise 2.30 to produce a function `tree_map` with the property that `square_tree` could be declared as

```
def square_tree(tree): return tree_map(square, tree)
```

Exercise 2.32. We can represent a set as a linked list of distinct elements, and we can represent the set of all subsets of the set as a linked list of linked lists. For example, if the set is `llist(1, 2, 3)`, then the set of all subsets is

```
llist(None, llist(3), llist(2), llist(2, 3),
        llist(1), llist(1, 3), llist(1, 2),
        llist(1, 2, 3))
```

Complete the following declaration of a function that generates the set of subsets of a set and give a clear explanation of why it works:

```
def subsets(s):
    if is_none(s):
        return llist(None)
    else:
        rest = subsets(tail(s))
        return append(rest, map(??, rest))
```

2.2.3 Sequences as Conventional Interfaces

In working with compound data, we've stressed how data abstraction permits us to design programs without becoming enmeshed in the details of data representations, and how abstraction preserves for us the flexibility to experiment with alternative representations. In this section, we introduce another powerful design principle for working with data structures—the use of *conventional interfaces*.

In section 1.3 we saw how program abstractions, implemented as higher-order functions, can capture common patterns in programs that deal with numerical data. Our ability to formulate analogous operations for working with compound data depends crucially on the style in which we manipulate our data structures. Consider, for example, the following function, analogous to the `count_leaves` function of section 2.2.2, which takes a tree as argument and computes the sum of the squares of the leaves that are odd:

```
def sum_odd_squares(tree):
    return (0 if is_none(tree)
            else (square(tree) if is_odd(tree)
                  else 0) if not is_pair(tree)
            else sum_odd_squares(head(tree)) +
                  sum_odd_squares(tail(tree)))
```

On the surface, this function is very different from the following one, which constructs a linked list of all the even Fibonacci numbers $\text{Fib}(k)$, where k is less than or equal to a given integer n :

```
def even_fibs(n):
    def next(k):
        if k > n:
            return None
        else:
            f = fib(k)
            return (pair(f, next(k + 1)) if is_even(f)
                    else next(k + 1))
    return next(0)
```

Despite the fact that these two functions are structurally very different, a more abstract description of the two computations reveals a great deal of similarity. The first program

- enumerates the leaves of a tree;
- filters them, selecting the odd ones;
- squares each of the selected ones; and
- accumulates the results using `+`, starting with 0. The second program
- enumerates the integers from 0 to n ;
- computes the Fibonacci number for each integer;
- filters them, selecting the even ones; and
- accumulates the results using `pair`, starting with the empty linked list.

A signal-processing engineer would find it natural to conceptualize these processes in terms of signals flowing through a cascade of stages, each of which implements part of the program plan, as shown in figure 2.7. In `sum_odd_squares`, we begin with an *enumerator*, which generates a "signal" consisting of the leaves of a given tree. This signal is passed through a *filter*, which eliminates all but the odd elements. The resulting signal is in turn passed through a *map*, which is a

"transducer" that applies the `square` function to each element. The output of the map is then fed to an *accumulator*, which combines the elements using `+`, starting from an initial 0. The plan for `even_fibs` is analogous.

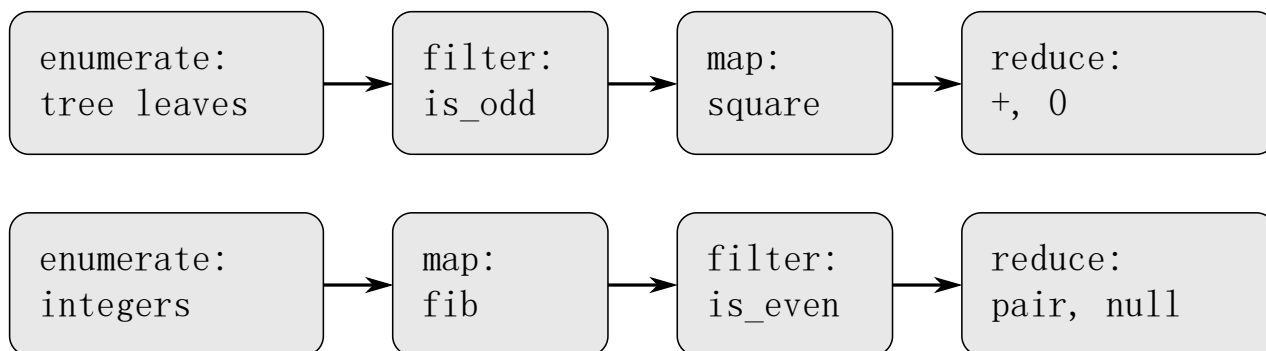


Figure 2.7. The signal-flow plans for the functions `sum_odd_squares` (top) and `even_fibs` (bottom) reveal the commonality between the two programs.

Unfortunately, the two function definitions above fail to exhibit this signal-flow structure. For instance, if we examine the `sum_odd_squares` function, we find that the enumeration is implemented partly by the `is_none` and `is_pair` tests and partly by the tree-recursive structure of the function. Similarly, the accumulation is found partly in the tests and partly in the addition used in the recursion. In general, there are no distinct parts of either function that correspond to the elements in the signal-flow description. Our two functions decompose the computations in a different way, spreading the enumeration over the program and mingling it with the map, the filter, and the accumulation. If we could organize our programs to make the signal-flow structure manifest in the functions we write, this would increase the conceptual clarity of the resulting program.

Sequence Operations

The key to organizing programs so as to more clearly reflect the signal-flow structure is to concentrate on the "signals" that flow from one stage in the process to the next. If we represent these signals as linked lists, then we can use linked-list operations to implement the processing at each of the stages. For instance, we can implement the mapping stages of the signal-flow diagrams using the `map` function from section 2.2.1:

```
print_llist(map(square, llist(1, 2, 3, 4, 5)))
```

```
llist(1, 4, 9, 16, 25)
```

Filtering a sequence to select only those elements that satisfy a given predicate is accomplished by

```
def filter(predicate, sequence):
    return (None if is_none(sequence)
            else pair(head(sequence),
                      filter(predicate, tail(sequence)))
            if predicate(head(sequence))
            else filter(predicate, tail(sequence)))
```

For example,

```
print_llist(filter(is_odd, llist(1, 2, 3, 4, 5)))
```

```
llist(1, 3, 5)
```

Accumulations can be implemented by

```
def reduce(op, initial, sequence):
    return (initial if is_none(sequence)
            else op(head(sequence),
                    reduce(op, initial, tail(sequence))))
```

```
print(reduce(plus, 0, llist(1, 2, 3, 4, 5)))
```

```
15
```

```
print(reduce(times, 1, llist(1, 2, 3, 4, 5)))
```

```
120
```

```
print_llist(reduce(pair, None, llist(1, 2, 3, 4, 5)))
```

```
llist(1, 2, 3, 4, 5)
```

All that remains to implement signal-flow diagrams is to enumerate the sequence of elements to be processed. For `even_fibs`, we need to generate the sequence of integers in a given range, which we can do as follows:

```
def enumerate_interval(low, high):
    return (None if low > high
            else pair(low,
                      enumerate_interval(low + 1, high)))
```

```
print_llist(enumerate_interval(2, 7))
```

```
llist(2, 3, 4, 5, 6, 7)
```

To enumerate the leaves of a tree, we can use [^10: This is, in fact, precisely the `fringe` function from exercise 2.28. Here we've renamed it to emphasize that it is part of a family of general sequence-manipulation functions.]

```
def enumerate_tree(tree):
    return (None if is_none(tree)
            else llist(tree) if not is_pair(tree)
            else append(enumerate_tree(head(tree)),
                        enumerate_tree(tail(tree))))
```

```
print_llist(enumerate_tree(llist(1, llist(2, llist(3, 4)), 5)))
```

```
llist(1, 2, 3, 4, 5)
```

Now we can reformulate `sum_odd_squares` and `even_fibs` as in the signal-flow diagrams. For `sum_odd_squares`, we enumerate the sequence of leaves of the tree, filter this to keep only the odd numbers in the sequence, square each element, and sum the results:

```
def sum_odd_squares(tree):
    return reduce(plus,
                 0,
                 map(square,
                     filter(is_odd,
                           enumerate_tree(tree))))
```

For `even_fibs`, we enumerate the integers from 0 to n , generate the Fibonacci number for each of these integers, filter the resulting sequence to keep only the even elements, and accumulate the results into a linked list:

```
def even_fibs(n):
    return reduce(pair,
                 None,
                 filter(is_even,
                       map(fib,
                           enumerate_interval(0, n))))
```

The value of expressing programs as sequence operations is that this helps us make program designs that are modular, that is, designs that are constructed by combining relatively independent

pieces. We can encourage modular design by providing a library of standard components together with a conventional interface for connecting the components in flexible ways.

Modular construction is a powerful strategy for controlling complexity in engineering design. In real signal-processing applications, for example, designers regularly build systems by cascading elements selected from standardized families of filters and transducers. Similarly, sequence operations provide a library of standard program elements that we can mix and match. For instance, we can reuse pieces from the `sum_odd_squares` and `even_fibs` functions in a program that constructs a linked list of the squares of the first $n+1$ Fibonacci numbers:

```
def list_fib_squares(n):
    return reduce(pair,
                  None,
                  map(square,
                      map(fib,
                          enumerate_interval(0, n))))
```

```
print_llist(list_fib_squares(10))
```

```
llist(0, 1, 1, 4, 9, 25, 64, 169, 441, 1156, 3025)
```

We can rearrange the pieces and use them in computing the product of the squares of the odd integers in a sequence:

```
def product_of_squares_of_odd_elements(sequence):
    return reduce(times,
                  1,
                  map(square,
                      filter(is_odd, sequence)))
```

```
print(product_of_squares_of_odd_elements(llist(1, 2, 3, 4, 5)))
```

```
225
```

We can also formulate conventional data-processing applications in terms of sequence operations. Suppose we have a sequence of personnel records and we want to find the salary of the highest-paid programmer. Assume that we have a selector `salary` that returns the salary of a record, and a predicate `is_programmer` that tests if a record is for a programmer. Then we can write


```
def salary_of_highest_paid_programmer(records):
    return reduce(max,
                  0,
                  map(salary,
                      filter(is_programmer, records)))
```

These examples give just a hint of the vast range of operations that can be expressed as sequence operations. [^11: Richard Waters (1979) developed a program that automatically analyzes traditional Fortran programs, viewing them in terms of maps, filters, and accumulations. He found that fully 90 percent of the code in the Fortran Scientific Subroutine Package fits neatly into this paradigm. One of the reasons for the success of Lisp as a programming language is that linked lists provide a standard medium for expressing ordered collections so that they can be manipulated using higher-order operations. Many modern languages, such as Python, have learned this lesson.]

Sequences, implemented here as linked lists, serve as a conventional interface that permits us to combine processing modules. Additionally, when we uniformly represent structures as sequences, we have localized the data-structure dependencies in our programs to a small number of sequence operations. By changing these, we can experiment with alternative representations of sequences, while leaving the overall design of our programs intact. We will exploit this capability in section 3.5, when we generalize the sequence-processing paradigm to admit infinite sequences.

Exercise 2.33. Fill in the missing expressions to complete the following definitions of some basic linked-list-manipulation operations as accumulations:

```
def map(f, sequence):
    return reduce(lambda x, y: ??,
                  None, sequence)

def append(seq1, seq2):
    return reduce(pair, ??, ??)

def length(sequence):
    return reduce(??, 0, sequence)
```

Exercise 2.34. Evaluating a polynomial in x at a given value of x can be formulated as an accumulation. We evaluate the polynomial

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

using a well-known algorithm called *Horner's rule*, which structures the computation as

$$(\cdots (a_n x + a_{n-1})x + \cdots + a_1)x + a_0$$

In other words, we start with a_n , multiply by x , add a_{n-1} , multiply by x , and so on, until we reach a_0 . [^12: According to Knuth (1997b), this rule was formulated by W. G. Horner early in the nineteenth century, but the method was actually used by Newton over a hundred years earlier. Horner's rule evaluates the polynomial using fewer additions and multiplications than does

the straightforward method of first computing $a_n x^n$, then adding $a_{n-1}x^{n-1}$, and so on. In fact, it is possible to prove that any algorithm for evaluating arbitrary polynomials must use at least as many additions and multiplications as does Horner's rule, and thus Horner's rule is an optimal algorithm for polynomial evaluation. This was proved (for the number of additions) by A. M. Ostrowski in a 1954 paper that essentially founded the modern study of optimal algorithms. The analogous statement for multiplications was proved by V. Y. Pan in 1966. The book by Borodin and Munro (1975) provides an overview of these and other results about optimal algorithms.] Fill in the following template to produce a function that evaluates a polynomial using Horner's rule. Assume that the coefficients of the polynomial are arranged in a sequence, from a_0 through a_n .

```
def horner_eval(x, coefficient_sequence):
    return reduce(lambda this_coeff, higher_terms: ??,
                  0,
                  coefficient_sequence)
```

For example, to compute $1+3x+5x^3+x^5$ at $x=2$ you would evaluate

```
print(horner_eval(2, llist(1, 3, 0, 5, 0, 1)))
```

Exercise 2.35. Redefine `count_leaves` from section 2.2.2 as an accumulation:

```
def count_leaves(t):
    return reduce(??, ??, map(??, ??))
```

Exercise 2.36. The function `reduce_n` is similar to `reduce` except that it takes as its third argument a sequence of sequences, which are all assumed to have the same number of elements. It applies the designated accumulation function to combine all the first elements of the sequences, all the second elements of the sequences, and so on, and returns a sequence of the results. For instance, if `s` is a sequence containing four sequences

```
llist(llist(1, 2, 3), llist(4, 5, 6), llist(7, 8, 9), llist(10, 11, 12))
```

then the value of `reduce_n(plus, 0, s)` should be the sequence `llist(22, 26, 30)`. Fill in the missing expressions in the following definition of `reduce_n`:

```
def reduce_n(op, init, seqs):
    return (None if is_none(head(seqs))
            else pair(reduce(op, init, ??),
                      reduce_n(op, init, ???)))
```

Exercise 2.37. Suppose we represent vectors $v=(v_i)$ as sequences of numbers, and matrices $m=(m_{ij})$ as sequences of vectors (the rows of the matrix). For example, the matrix

$\left[\begin{array}{llll} 1 & 2 & 3 & 4 \\ 4 & 5 & 6 & 6 \\ 6 & 7 & 8 & 9 \end{array} \right]$

is represented as the following sequence:

```
l1list(l1list(1, 2, 3, 4),
      l1list(4, 5, 6, 6),
      l1list(6, 7, 8, 9))
```

With this representation, we can use sequence operations to concisely express the basic matrix and vector operations. These operations (which are described in any book on matrix algebra) are the following:

| `dot_product( $v$ , $w$ )` | returns the sum $\sum_i v_i w_i$; | | --- | --- | |
`matrix_times_vector( $m$ , $v$ )` | returns the vector t , where $t_i = \sum_j m_{ij} v_j$; | |
`matrix_times_matrix( $m$ , $n$ )` | returns the matrix p , where $p_{ij} = \sum_k m_{ik} n_{kj}$; | |
`transpose( $m$ )` | returns the matrix n , where $n_{ij} = m_{ji}$. |

We can define the dot product as [^13: This definition uses the function `reduce_n` from exercise 2.36.]

```
def dot_product(v, w):
    return reduce(plus, 0, reduce_n(times, 1, l1list(v, w)))
```

Fill in the missing expressions in the following functions for computing the other matrix operations. (The function `reduce_n` is declared in exercise 2.36.)

```
def matrix_times_vector(m, v):
    return map(??, m)
def transpose(mat):
    return reduce_n(??, ??, mat)
def matrix_times_matrix(m, n):
    cols = transpose(n)
    return map(??, m)
```

Exercise 2.38. The `reduce` function is also known as `fold_right`, because it combines the first element of the sequence with the result of combining all the elements to the right. There is also a `fold_left`, which is similar to `fold_right`, except that it combines elements working in the opposite direction:

```
def fold_left(op, initial, sequence):
    def iter(result, rest):
        return (result if is_none(rest)
                else iter(op(result, head(rest)),
                          tail(rest)))
    return iter(initial, sequence)
```

What are the values of

```
print(fold_right(divide, 1, llist(1, 2, 3)))
```

```
print(fold_left(divide, 1, llist(1, 2, 3)))
```

```
print(fold_right(llist, None, llist(1, 2, 3)))
```

```
print(fold_left(llist, None, llist(1, 2, 3)))
```

Give a property that `op` should satisfy to guarantee that `fold_right` and `fold_left` will produce the same values for any sequence.

Exercise 2.39. Complete the following definitions of `reverse` (exercise 2.18) in terms of `fold_right` and `fold_left` from exercise 2.38:

```
def reverse(sequence):  
    return fold_right(lambda x, y: ??, None, sequence)
```

```
def reverse(sequence):  
    return fold_left(lambda x, y: ??, None, sequence)
```

Nested Mappings

We can extend the sequence paradigm to include many computations that are commonly expressed using nested loops. [¹⁴ This approach to nested mappings was shown to us by David Turner, whose languages KRC and Miranda provide elegant formalisms for dealing with these constructs. The examples in this section (see also exercise 2.42) are adapted from Turner 1981. In section 3.5.3, we'll see how this approach generalizes to infinite sequences.] Consider this problem: Given a positive integer n , find all ordered pairs of distinct positive integers i and j , where $1 \leq j < i \leq n$, such that $i + j$ is prime. For example, if n is 6, then the pairs are the following:

```
$$$ \begin{array}{c|cccccc} i & 2 & 3 & 4 & 5 & 6 & 6 \\ j & 1 & 2 & 1 & 3 & 2 & 1 \\ \hline i+j & 3 & 5 & 5 & 7 & 7 & 11 \end{array} $$$
```

A natural way to organize this computation is to generate the sequence of all ordered pairs of positive integers less than or equal to n , filter to select those pairs whose sum is prime, and then, for each pair (i, j) that passes through the filter, produce the triple $(i, j, i+j)$.

Here is a way to generate the sequence of pairs: For each integer $i \leq n$, enumerate the integers $j < i$, and for each such i and j generate the pair (i, j) . In terms of sequence operations, we map along the sequence `enumerate_interval(1, n)`. For each i in this sequence, we map along the sequence `enumerate_interval(1, i - 1)`. For each j in this latter sequence, we generate the pair `l1ist(i, j)`. This gives us a sequence of pairs for each i . Combining all the sequences for all the i (by accumulating with `append`) produces the required sequence of pairs: [^15: We're representing a pair here as a linked list of two elements rather than as an ordinary pair. Thus, the "pair" (i, j) is represented as `l1ist(i, j)`, not `pair(i, j)`.]

```
print(reduce(append,
            None,
            map(lambda i: map(lambda j: l1ist(i, j),
                             enumerate_interval(1, i - 1)),
                enumerate_interval(1, n))))
```

The combination of mapping and accumulating with `append` is so common in this sort of program that we will isolate it as a separate function:

```
def flatmap(f, seq):
    return reduce(append, None, map(f, seq))
```

Now filter this sequence of pairs to find those whose sum is prime. The filter predicate is called for each element of the sequence; its argument is a pair and it must extract the integers from the pair. Thus, the predicate to apply to each element in the sequence is

```
def is_prime_sum(pair):
    return is_prime(head(pair) + head(tail(pair)))
```

Finally, generate the sequence of results by mapping over the filtered pairs using the following function, which constructs a triple consisting of the two elements of the pair along with their sum:

```
def make_pair_sum(pair):
    return l1ist(head(pair), head(tail(pair)),
                head(pair) + head(tail(pair)))
```

Combining all these steps yields the complete function:

```
def prime_sum_pairs(n):
    return map(make_pair_sum,
              filter(is_prime_sum,
                    flatmap(lambda i: map(lambda j: l1ist(i, j),
                                         enumerate_interval(1, i - 1)),
                            enumerate_interval(1, n))))
```

Nested mappings are also useful for sequences other than those that enumerate intervals. Suppose we wish to generate all the permutations of a set S ; that is, all the ways of ordering the items in the set. For instance, the permutations of $\{1, 2, 3\}$ are $\{1, 2, 3\}$, $\{1, 3, 2\}$, $\{2, 1, 3\}$, $\{2, 3, 1\}$, $\{3, 1, 2\}$, and $\{3, 2, 1\}$. Here is a plan for generating the permutations of S : For each item x in S , recursively generate the sequence of permutations of $S - x$, [¹⁶: The set $S - x$ is the set of all elements of S , excluding x .] and adjoin x to the front of each one. This yields, for each x in S , the sequence of permutations of S that begin with x . Combining these sequences for all x gives all the permutations of S : [¹⁷: The character `#` in Python programs is used to introduce *comments*. Everything from `#` to the end of the line is ignored by the interpreter. In this book we don't use many comments; we try to make our programs self-documenting by using descriptive names.]

```
def permutations(s):
    return (llist(None)      # sequence containing empty set
            if is_none(s)    # empty set?
            else flatmap(lambda x: map(lambda p: pair(x, p),
                                       permutations(remove(x, s))),
                          s))
```

Notice how this strategy reduces the problem of generating permutations of S to the problem of generating the permutations of sets with fewer elements than S . In the terminal case, we work our way down to the empty linked list, which represents a set of no elements. For this, we generate `llist(None)`, which is a sequence with one item, namely the set with no elements. The `remove` function used in `permutations` returns all the items in a given sequence except for a given item. This can be expressed as a simple filter:

```
def remove(item, sequence):
    return filter(lambda x: x != item,
                  sequence)
```

Exercise 2.40. Write a function `unique_pairs` that, given an integer n , generates the sequence of pairs (i, j) with $1 \leq j < i \leq n$. Use `unique_pairs` to simplify the definition of `prime_sum_pairs` given above.

Exercise 2.41. Write a function to find all ordered triples of distinct positive integers i , j , and k less than or equal to a given integer n that sum to a given integer s .

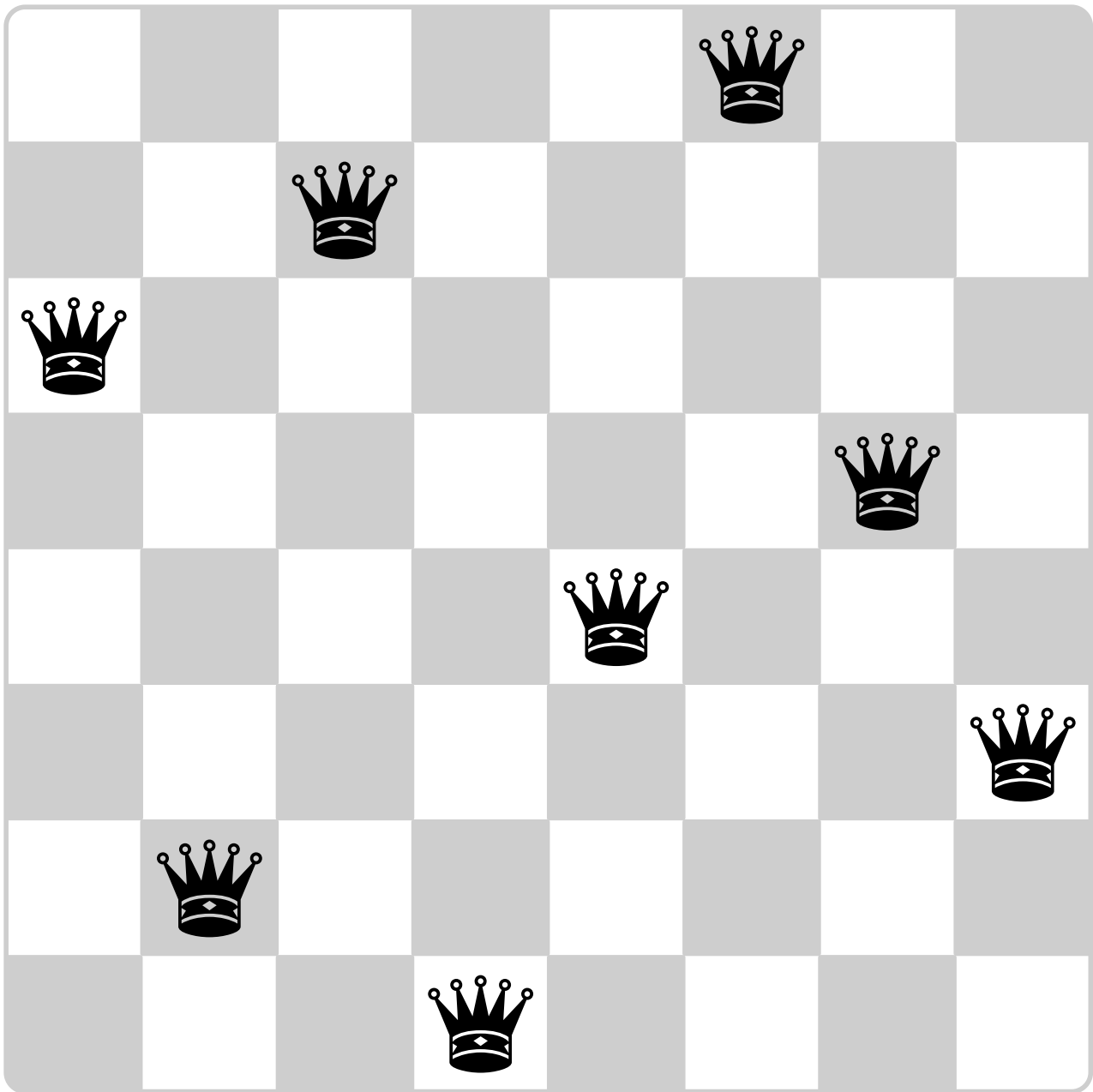


Figure 2.8. A solution to the eight-queens puzzle.

Exercise 2.42. The "eight-queens puzzle" asks how to place eight queens on a chessboard so that no queen is in check from any other (i.e., no two queens are in the same row, column, or diagonal). One possible solution is shown in figure 2.8. One way to solve the puzzle is to work across the board, placing a queen in each column. Once we have placed $k-1$ queens, we must place the k th queen in a position where it does not check any of the queens already on the board. We can formulate this approach recursively: Assume that we have already generated the sequence of all possible ways to place $k-1$ queens in the first $k-1$ columns of the board. For each of these ways, generate an extended set of positions by placing a queen in each row of the k th column. Now filter these, keeping only the positions for which the queen in the k th column is safe with respect to the other queens. This produces the sequence of all ways to place k queens in the first

k columns. By continuing this process, we will produce not only one solution, but all solutions to the puzzle.

We implement this solution as a function `queens`, which returns a sequence of all solutions to the problem of placing n queens on an $n \times n$ chessboard. The function `queens` has an internal function `queens_cols` that returns the sequence of all ways to place queens in the first k columns of the board.

```
def queens(board_size):
    def queen_cols(k):
        return (l1st(empty_board) if k == 0
                else filter(lambda positions: is_safe(k, positions),
                            flatmap(lambda rest_of_queens:
                                    map(lambda new_row:
                                        adjoin_position(new_row, k,
                                                        rest_of_queens),
                                        enumerate_interval(1, board_size)),
                                    queen_cols(k - 1))))
    return queen_cols(board_size)
```

In this function `rest_of_queens` is a way to place $k-1$ queens in the first $k-1$ columns, and `new_row` is a proposed row in which to place the queen for the k th column. Complete the program by implementing the representation for sets of board positions, including the function `adjoin_position`, which adjoins a new row-column position to a set of positions, and `empty_board`, which represents an empty set of positions. You must also write the function `is_safe`, which determines for a set of positions whether the queen in the k th column is safe with respect to the others. (Note that we need only check whether the new queen is safe—the other queens are already guaranteed safe with respect to each other.)

Exercise 2.43. Louis Reasoner is having a terrible time doing exercise 2.42. His `queens` function seems to work, but it runs extremely slowly. (Louis never does manage to wait long enough for it to solve even the 6×6 case.) When Louis asks Eva Lu Ator for help, she points out that he has interchanged the order of the nested mappings in the `flatmap`, writing it as

```
flatmap(lambda new_row:
         map(lambda rest_of_queens:
             adjoin_position(new_row, k, rest_of_queens),
             queen_cols(k - 1)),
         enumerate_interval(1, board_size))
```

Explain why this interchange makes the program run slowly. Estimate how long it will take Louis's program to solve the eight-queens puzzle, assuming that the program in exercise 2.42 solves the puzzle in time T .

2.2.4 Example: A Picture Language

This section presents a simple language for drawing pictures that illustrates the power of data abstraction and closure, and also exploits higher-order functions in an essential way. The language is designed to make it easy to experiment with patterns such as the ones in figure 2.9, which are composed of repeated elements that are shifted and scaled. [^18: The picture language is based on the language Peter Henderson created to construct images like M.C. Escher's "Square Limit" woodcut (see Henderson 1982). The woodcut incorporates a repeated scaled pattern, similar to the arrangements drawn using the `square_limit` function in this section.] In this language, the data objects being combined are represented as functions rather than as linked-list structure. Just as `pair`, which satisfies the closure property, allowed us to easily build arbitrarily complicated linked-list structure, the operations in this language, which also satisfy the closure property, allow us to easily build arbitrarily complicated patterns.

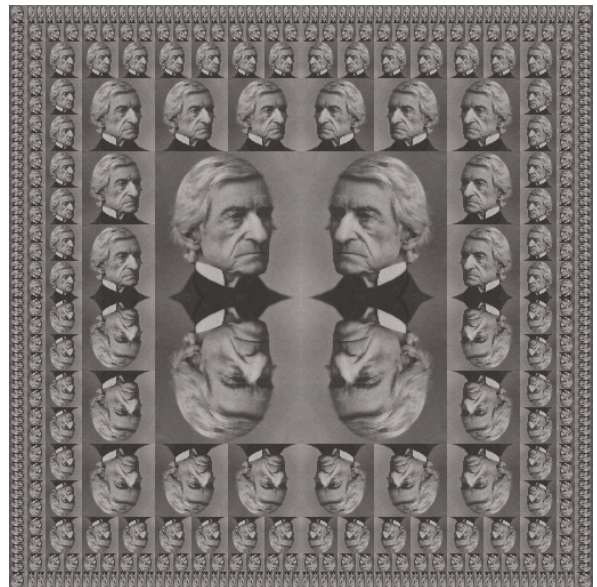
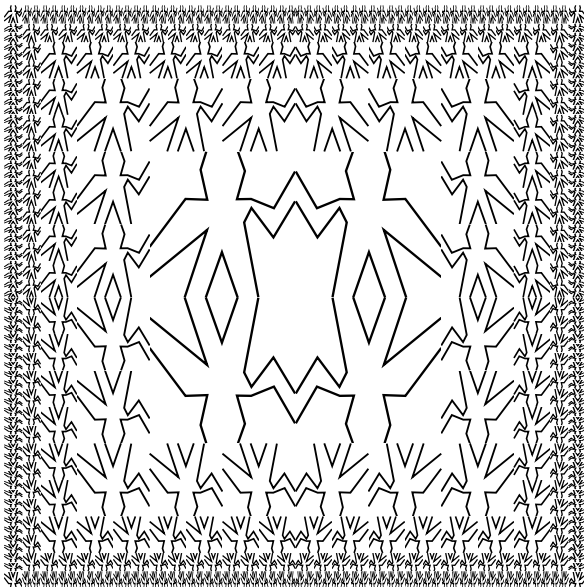


Figure 2.9. Designs generated with the picture language.

The picture language

When we began our study of programming in section 1.1, we emphasized the importance of describing a language by focusing on the language's primitives, its means of combination, and its means of abstraction. We'll follow that framework here.

Part of the elegance of this picture language is that there is only one kind of element, called a *painter*. A painter draws an image that is shifted and scaled to fit within a designated parallelogram-shaped frame. For example, there's a primitive painter we'll call `wave` that makes a crude line drawing, as shown in figure 2.10.

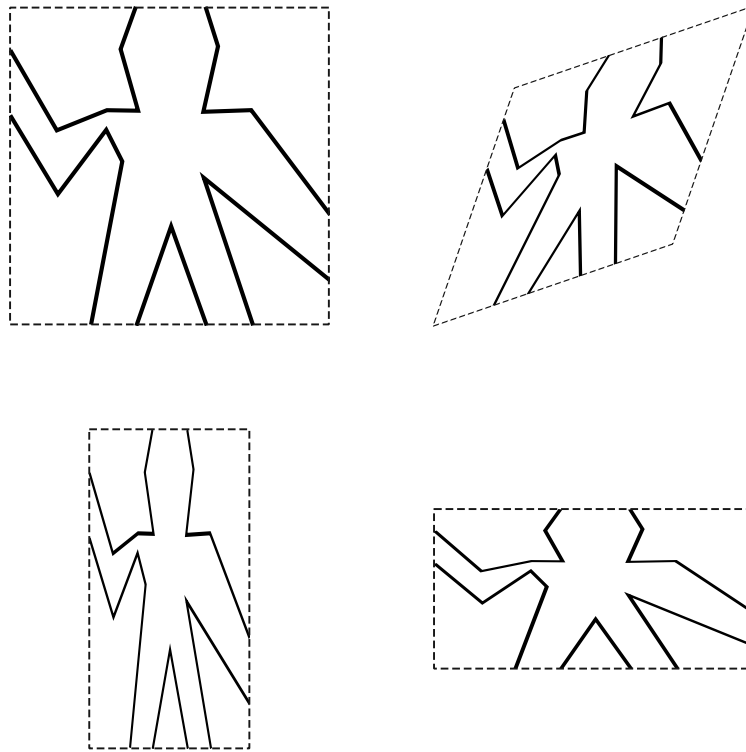


Figure 2.10. Images produced by the `wave` painter, with respect to four different frames. The frames, shown with dashed lines, are not part of the images.

The actual shape of the drawing depends on the frame—all four images in figure 2.10 are produced by the same `wave` painter, but with respect to four different frames. Painters can be more elaborate than this: The primitive painter called `rogers` paints a picture of MIT's founder, William Barton Rogers, as shown in figure 2.11. [¹⁹: William Barton Rogers (1804–1882) was the founder and first president of MIT. A geologist and talented teacher, he taught at William and Mary College and at the University of Virginia. In 1859 he moved to Boston, where he had more time for research, worked on a plan for establishing a "polytechnic institute," and served as Massachusetts's first State Inspector of Gas Meters. When MIT was established in 1861, Rogers was elected its first president. Rogers espoused an ideal of "useful learning" that was different from the university education of the time, with its overemphasis on the classics, which, as he wrote, "stand in the way of the broader, higher and more practical instruction and discipline of the natural and social sciences." This education was likewise to be different from narrow trade-school education. In Rogers's words: > The world-enforced distinction between the practical and the scientific worker is utterly futile, and the whole experience of modern times has demonstrated its utter worthlessness. Rogers served as president of MIT until 1870, when he resigned due to ill health. In 1878 the second president of MIT, John Runkle, resigned under the pressure of a financial crisis brought on by the Panic of 1873 and strain of fighting off attempts by Harvard to take over MIT. Rogers returned to hold the office of president until 1881. Rogers collapsed and died while addressing MIT's graduating class at the commencement exercises of 1882. Runkle quoted Rogers's last words in a memorial address delivered that same year: > "As I stand here today and see what the Institute is, ... I call to mind the beginnings of science. I remember one hundred and fifty years ago Stephen Hales published a pamphlet on the subject of illuminating gas, in which he stated that his

researches had demonstrated that 128 grains of bituminous coal—" > > > "Bituminous coal," these were his last words on earth. Here he bent forward, as if consulting some notes on the table before him, then slowly regaining an erect position, threw up his hands, and was translated from the scene of his earthly labors and triumphs to "the tomorrow of death," where the mysteries of life are solved, and the disembodied spirit finds unending satisfaction in contemplating the new and still unfathomable mysteries of the infinite future. In the words of Francis A. Walker (MIT's third president): > All his life he had borne himself most faithfully and heroically, and he died as so good a knight would surely have wished, in harness, at his post, and in the very part and act of public duty.] The four images in figure 2.11 are drawn with respect to the same four frames as the `wave` images in figure 2.10.

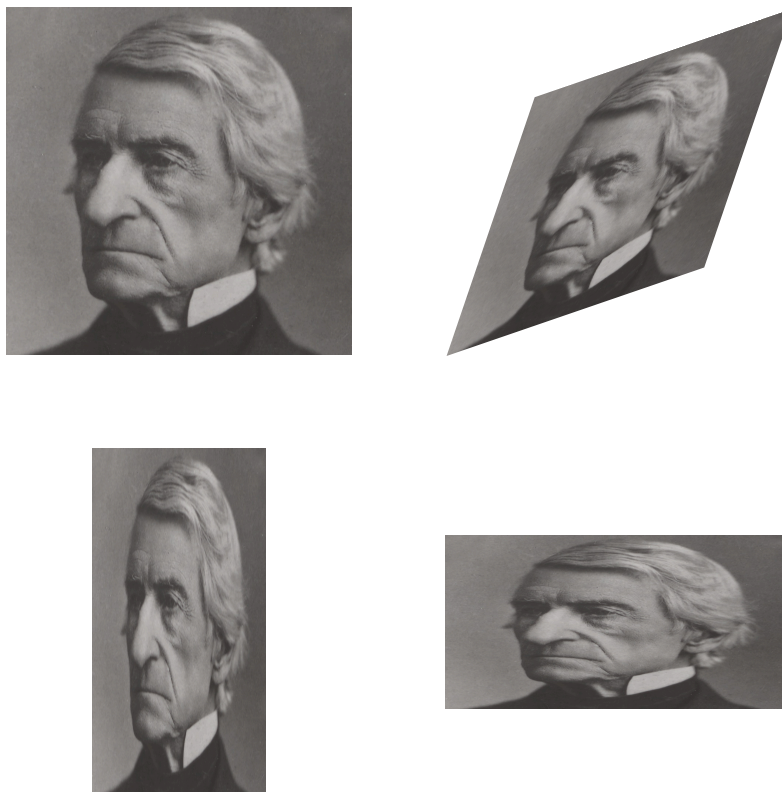


Figure 2.11. Images of William Barton Rogers, founder and first president of MIT, painted with respect to the same four frames as in figure 2.10 (original image courtesy MIT Museum).

To combine images, we use various operations that construct new painters from given painters. For example, the `beside` operation takes two painters and produces a new, compound painter that draws the first painter's image in the left half of the frame and the second painter's image in the right half of the frame. Similarly, `below` takes two painters and produces a compound painter that draws the first painter's image below the second painter's image. Some operations transform a single painter to produce a new painter. For example, `flip_vert` takes a painter and produces a painter that draws its image upside-down, and `flip_horiz` produces a painter that draws the original painter's image left-to-right reversed.

Figure 2.12 shows the drawing of a painter called `wave4` that is built up in two stages starting from `wave` :

```

wave2 = beside(wave, flip_vert(wave))
wave4 = below(wave2, wave2)

```

In building up a complex image in this manner we are exploiting the fact that painters are closed under the language's means of combination. The `beside` or `below` of two painters is itself a painter; therefore, we can use it as an element in making more complex painters. As with building up linked-list structure using `pair`, the closure of our data under the means of combination is crucial to the ability to create complex structures while using only a few operations.

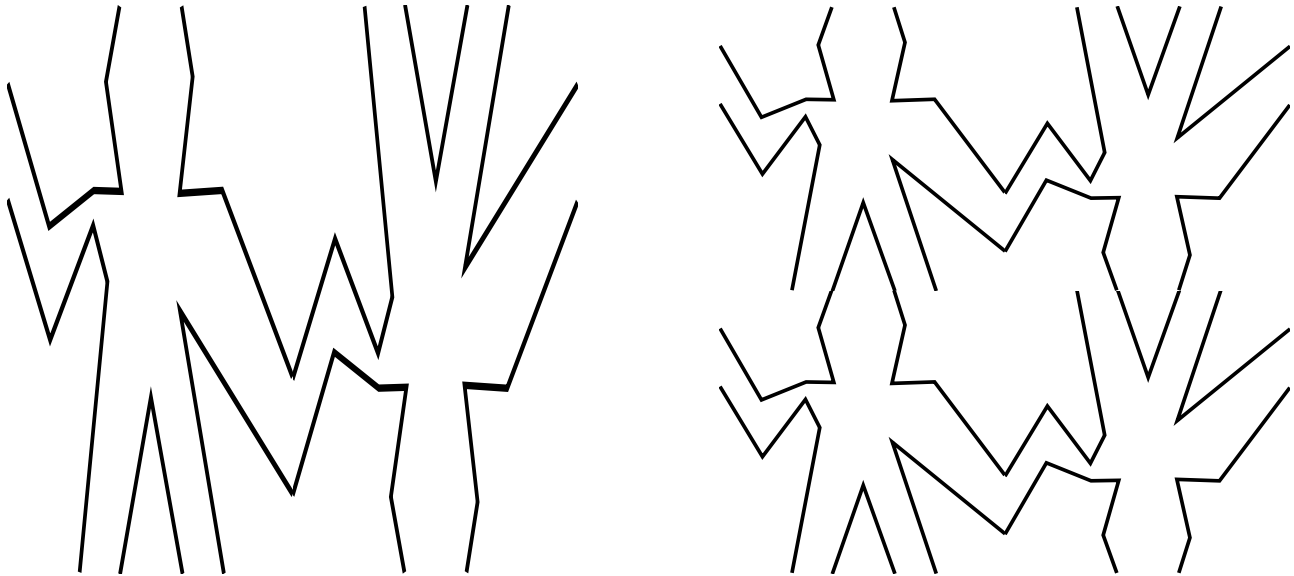


Figure 2.12. Creating a complex figure, starting from the `wave` painter of figure 2.10.

```

$ $ wave2 = wave4 = beside(wave, flip_vert(wave)) below(wave2, wave2)

```

Once we can combine painters, we would like to be able to abstract typical patterns of combining painters. We will implement the painter operations as Python functions. This means that we don't need a special abstraction mechanism in the picture language: Since the means of combination are ordinary Python functions, we automatically have the capability to do anything with painter operations that we can do with functions. For example, we can abstract the pattern in `wave4` as

```

def flipped_pairs(painter):
    painter2 = beside(painter, flip_vert(painter))
    return below(painter2, painter2)

```

and declare `wave4` as an instance of this pattern:

```

wave4 = flipped_pairs(wave)

```

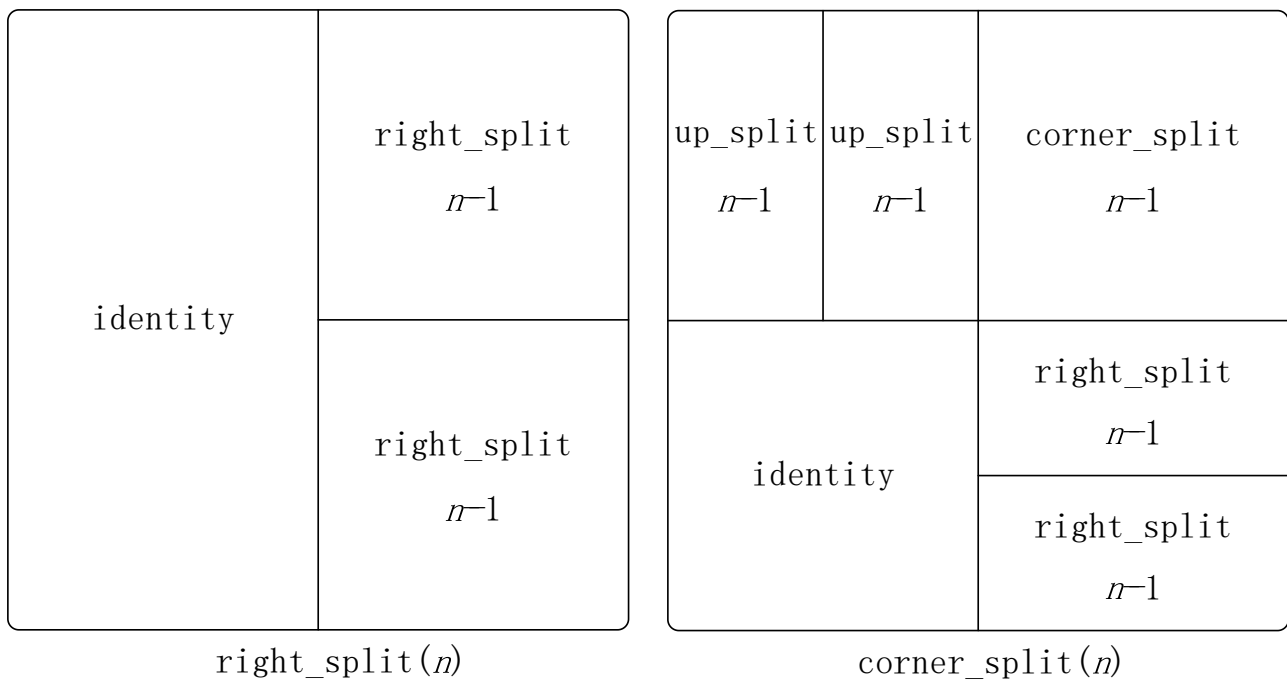


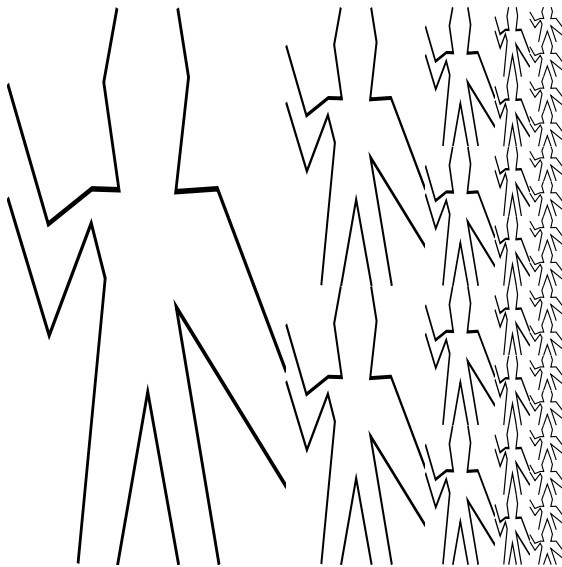
Figure 2.13. Recursive plans for `right_split` and `corner_split`.

We can also define recursive operations. Here's one that makes painters split and branch towards the right as shown in figures 2.13 and 2.14:

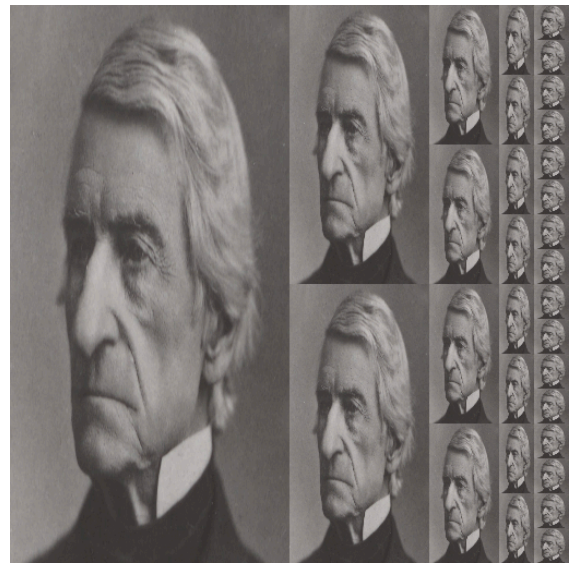
```
def right_split(painter, n):
    if n == 0:
        return painter
    else:
        smaller = right_split(painter, n - 1)
        return beside(painter, below(smaller, smaller))
```

We can produce balanced patterns by branching upwards as well as towards the right (see exercise 2.44 and figures 2.13 and 2.14):

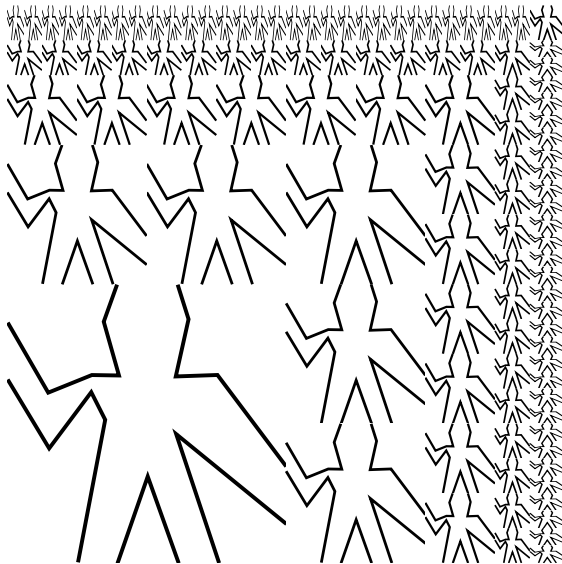
```
def corner_split(painter, n):
    if n == 0:
        return painter
    else:
        up = up_split(painter, n - 1)
        right = right_split(painter, n - 1)
        top_left = beside(up, up)
        bottom_right = below(right, right)
        corner = corner_split(painter, n - 1)
        return beside(below(painter, top_left),
                      below(bottom_right, corner))
```

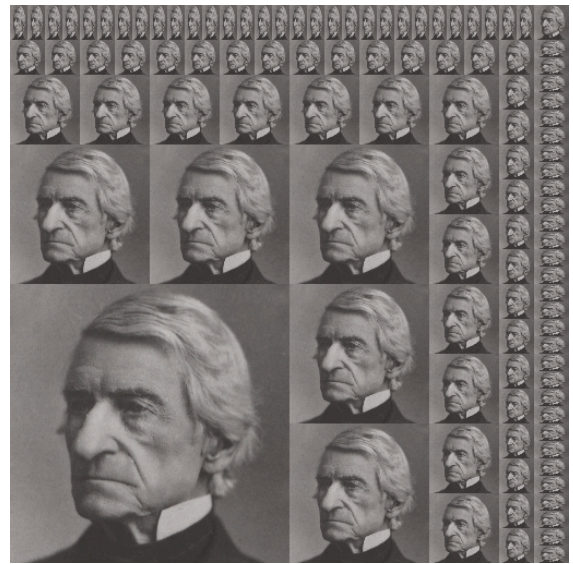
`right_split(wave, 4)`



`right_split(rogers, 4)`



`corner_split(wave, 4)`



`corner_split(rogers, 4)`

Figure 2.14. The recursive operation `right_split` applied to the painters `wave` and `rogers`. Combining four `corner_split` figures produces symmetric `square_limit` as shown in figure 2.9.

By placing four copies of a `corner_split` appropriately, we obtain a pattern called `square_limit`, whose application to `wave` and `rogers` is shown in figure 2.9:

```
def square_limit(painter, n):
    quarter = corner_split(painter, n)
    half = beside(flip_horiz(quarter), quarter)
    return below(flip_vert(half), half)
```

Exercise 2.44. Declare the function `up_split` used by `corner_split`. It is similar to `right_split`, except that it switches the roles of `below` and `beside`.

Higher-order operations

In addition to abstracting patterns of combining painters, we can work at a higher level, abstracting patterns of combining painter operations. That is, we can view the painter operations as elements to manipulate and can write means of combination for these elements—functions that take painter operations as arguments and create new painter operations.

For example, `flipped_pairs` and `square_limit` each arrange four copies of a painter's image in a square pattern; they differ only in how they orient the copies. One way to abstract this pattern of painter combination is with the following function, which takes four one-argument painter operations and produces a painter operation that transforms a given painter with those four operations and arranges the results in a square. [^20: The painter operation returned by `square_of_four` consists of several statements, so it cannot be written as a lambda expression, whose body in Python must be a single expression. Instead, we use a local function declaration with the name `combine`.] The functions `tl`, `tr`, `bl`, and `br` are the transformations to apply to the top left copy, the top right copy, the bottom left copy, and the bottom right copy, respectively.

```
def square_of_four(tl, tr, bl, br):
    def combine(painter):
        top = beside(tl(painter), tr(painter))
        bottom = beside(bl(painter), br(painter))
        return below(bottom, top)
    return combine
```

Then `flipped_pairs` can be defined in terms of `square_of_four` as follows: [^21: Equivalently, we could write]

```
flipped_pairs = square_of_four(identity, flip_vert,
                                identity, flip_vert)
```

```
def flipped_pairs(painter):
    combine4 = square_of_four(identity, flip_vert,
                              identity, flip_vert)
    return combine4(painter)
```

and `square_limit` can be expressed as [^22: The function `rotate180` rotates a painter by 180 degrees. Instead of `rotate180` we could say `compose(flip_vert, flip_horiz)`, using the `compose` function from exercise 1.42.]

```
def square_limit(painter, n):
    combine4 = square_of_four(flip_horiz, identity,
                              rotate180, flip_vert)
    return combine4(corner_split(painter, n))
```

Exercise 2.45. The functions `right_split` and `up_split` can be expressed as instances of a general splitting operation. Declare a function `split` with the property that evaluating

```
right_split = split(beside, below)
up_split = split(below, beside)
```

produces functions `right_split` and `up_split` with the same behaviors as the ones already declared.

Frames

Before we can show how to implement painters and their means of combination, we must first consider frames. A frame can be described by three vectors—an origin vector and two edge vectors. The origin vector specifies the offset of the frame's origin from some absolute origin in the plane, and the edge vectors specify the offsets of the frame's corners from its origin. If the edges are perpendicular, the frame will be rectangular. Otherwise the frame will be a more general parallelogram.

Figure 2.15 shows a frame and its associated vectors. In accordance with data abstraction, we need not be specific yet about how frames are represented, other than to say that there is a constructor `make_frame`, which takes three vectors and produces a frame, and three corresponding selectors `origin_frame`, `edge1_frame`, and `edge2_frame` (see exercise 2.47).

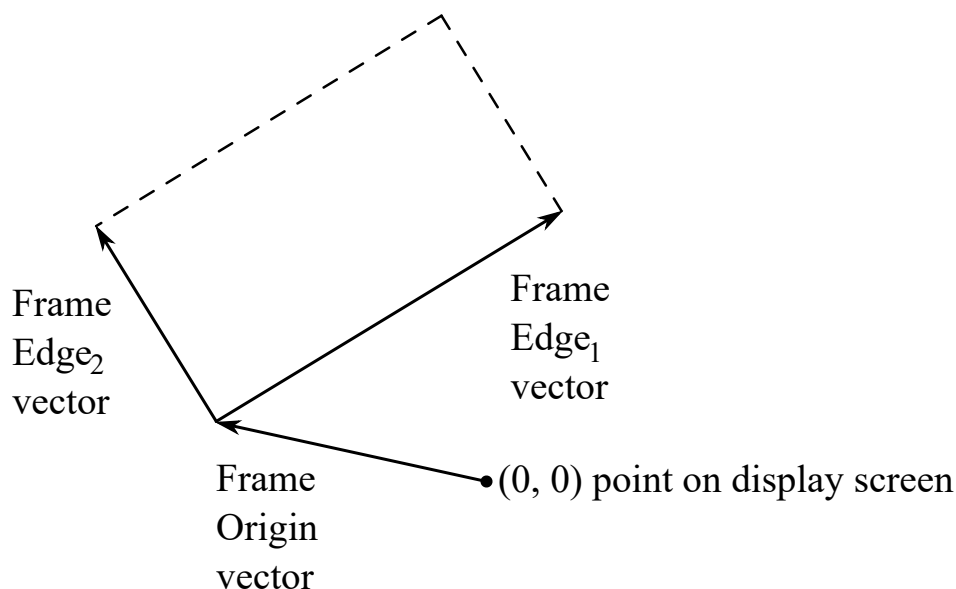


Figure 2.15. A frame is described by three vectors—an origin and two edges.

We will use coordinates in the unit square ($0 \leq x, y \leq 1$) to specify images. With each frame, we associate a *frame coordinate map*, which will be used to shift and scale images to fit the frame. The map transforms the unit square into the frame by mapping the vector $\mathbf{v} = (x, y)$ to the vector sum

$$x \cdot \text{Edge}_1(\text{Frame}) + y \cdot \text{Edge}_2(\text{Frame})$$

For example, $(0, 0)$ is mapped to the origin of the frame, $(1, 1)$ to the vertex diagonally opposite the origin, and $(0.5, 0.5)$ to the center of the frame. We can create a frame's coordinate map with the following function: [^23: The function `frame_coord_map` uses the vector operations described in exercise 2.46 below, which we assume have been implemented using some representation for vectors. Because of data abstraction, it doesn't matter what this vector representation is, so long as the vector operations behave correctly.]

```
def frame_coord_map(frame):
    return lambda v: add_vect(origin_frame(frame),
                              add_vect(scale_vect(xcor_vect(v),
                                                  edge1_frame(frame)),
                              scale_vect(ycor_vect(v),
                                          edge2_frame(frame))))
```

Observe that applying `frame_coord_map` to a frame returns a function that, given a vector, returns a vector. If the argument vector is in the unit square, the result vector will be in the frame. For example,

```
print(frame_coord_map(a_frame)(make_vect(0, 0)))
```

returns the same vector as

```
print(origin_frame(a_frame))
```

Exercise 2.46. A two-dimensional vector v running from the origin to a point can be represented as a pair consisting of an x -coordinate and a y -coordinate. Implement a data abstraction for vectors by giving a constructor `make_vect` and corresponding selectors `xcor_vect` and `ycor_vect`. In terms of your selectors and constructor, implement functions `add_vect`, `sub_vect`, and `scale_vect` that perform the operations vector addition, vector subtraction, and multiplying a vector by a scalar:

$$\begin{array}{l} (x_1, y_1) + (x_2, y_2) = (x_1 + x_2, y_1 + y_2) \\ (x_1, y_1) - (x_2, y_2) = (x_1 - x_2, y_1 - y_2) \\ s \cdot (x, y) = (sx, sy) \end{array}$$

Exercise 2.47. Here are two possible constructors for frames:

```
def make_frame(origin, edge1, edge2):
    return llist(origin, edge1, edge2)

def make_frame(origin, edge1, edge2):
    return pair(origin, pair(edge1, edge2))
```

For each constructor supply the appropriate selectors to produce an implementation for frames.

Painters

A painter is represented as a function that, given a frame as argument, draws a particular image shifted and scaled to fit the frame. That is to say, if `p` is a painter and `f` is a frame, then we produce `p`'s image in `f` by calling `p` with `f` as argument.

The details of how primitive painters are implemented depend on the particular characteristics of the graphics system and the type of image to be drawn. For instance, suppose we have a function `draw_line` that draws a line on the screen between two specified points. Then we can create painters for line drawings, such as the `wave` painter in figure 2.10, from linked lists of line segments as follows: [^24: The function `segments_toPainter` uses the representation for line segments described in exercise 2.48 below. It also uses the `for_each` function described in exercise 2.23.]

```
def segments_toPainter(segment_list):
    return lambda frame:
        for_each(lambda segment:
            draw_line(
                frame_coord_map(frame)
                    (start_segment(segment)),
                frame_coord_map(frame)
                    (end_segment(segment))),
            segment_list)
```

The segments are given using coordinates with respect to the unit square. For each segment in the linked list, the painter transforms the segment endpoints with the frame coordinate map and draws a line between the transformed points.

Representing painters as functions erects a powerful abstraction barrier in the picture language. We can create and intermix all sorts of primitive painters, based on a variety of graphics capabilities. The details of their implementation do not matter. Any function can serve as a painter, provided that it takes a frame as argument and draws something scaled to fit the frame. [^25: For example, the `rogers` painter of figure 2.11 was constructed from a gray-level image. For each point in a given frame, the `rogers` painter determines the point in the image that is mapped to it under the frame coordinate map, and shades it accordingly. By allowing different types of painters, we are capitalizing on the abstract data idea discussed in section 2.1.3, where we argued that a rational-number representation could be anything at all that satisfies an appropriate condition. Here we're using the fact that a painter can be implemented in any way at all, so long as it draws something in the designated frame. Section 2.1.3 also showed how pairs could be implemented as functions. Painters are our second example of a functional representation for data.]

Exercise 2.48. A directed line segment in the plane can be represented as a pair of vectors—the vector running from the origin to the start-point of the segment, and the vector running from the origin to the end-point of the segment. Use your vector representation from exercise 2.46 to define a representation for segments with a constructor `make_segment` and selectors `start_segment` and `end_segment`.

Exercise 2.49. Use `segments_toPainter` to define the following primitive painters: a. The painter that draws the outline of the designated frame. b. The painter that draws an "X" by connecting opposite corners of the frame. c. The painter that draws a diamond shape by connecting the midpoints of the sides of the frame. d. The `wave` painter.

Transforming and combining painters

An operation on painters (such as `flip_vert` or `beside`) works by creating a painter that invokes the original painters with respect to frames derived from the argument frame. Thus, for example, `flip_vert` doesn't have to know how a painter works in order to flip it—it just has to know how to turn a frame upside down: The flipped painter just uses the original painter, but in the inverted frame.

Painter operations are based on the function `transformPainter`, which takes as arguments a painter and information on how to transform a frame and produces a new painter. The transformed painter, when called on a frame, transforms the frame and calls the original painter on the transformed frame. The arguments to `transformPainter` are points (represented as vectors) that specify the corners of the new frame: When mapped into the frame, the first point specifies the new frame's origin and the other two specify the ends of its edge vectors. Thus, arguments within the unit square specify a frame contained within the original frame.

```
def transformPainter(painter, origin, corner1, corner2):
    def transformed(frame):
        m = frame_coord_map(frame)
        new_origin = m(origin)
        return painter(make_frame(
            new_origin,
            sub_vect(m(corner1), new_origin),
            sub_vect(m(corner2), new_origin)))
    return transformed
```

Here's how to flip painter images vertically:

```
def flip_vert(painter):
    return transformPainter(painter,
        make_vect(0, 1), # new origin
        make_vect(1, 1), # new end of edge1
        make_vect(0, 0)); # new end of edge2
```

Using `transformPainter`, we can easily define new transformations. For example, we can declare a painter that shrinks its image to the upper-right quarter of the frame it is given:

```
def shrink_to_upper_right(painter):
    return transformPainter(painter,
                            make_vect(0.5, 0.5),
                            make_vect(1, 0.5),
                            make_vect(0.5, 1))
```

Other transformations rotate images counterclockwise by 90 degrees [^26: The function `rotate90` is a pure rotation only for square frames, because it also stretches and shrinks the image to fit into the rotated frame.]

```
def rotate90(painter):
    return transformPainter(painter,
                            make_vect(1, 0),
                            make_vect(1, 1),
                            make_vect(0, 0))
```

or squash images towards the center of the frame: [^27: The diamond-shaped images in figures 2.10 and 2.11 were created with `squash_inwards` applied to `wave` and `rogers`.]

```
def squash_inwards(painter):
    return transformPainter(painter,
                            make_vect(0, 0),
                            make_vect(0.65, 0.35),
                            make_vect(0.35, 0.65))
```

Frame transformation is also the key to defining means of combining two or more painters. The `beside` function, for example, takes two painters, transforms them to paint in the left and right halves of an argument frame respectively, and produces a new, compound painter. When the compound painter is given a frame, it calls the first transformed painter to paint in the left half of the frame and calls the second transformed painter to paint in the right half of the frame:

```

def beside(painter1, painter2):
    split_point = make_vect(0.5, 0)
    paint_left = transform_painter(painter1,
                                   make_vect(0, 0),
                                   split_point,
                                   make_vect(0, 1))
    paint_right = transform_painter(painter2,
                                    split_point,
                                    make_vect(1, 0),
                                    make_vect(0.5, 1))

    def painter(frame):
        paint_left(frame)
        paint_right(frame)
    return painter

```

Observe how the painter data abstraction, and in particular the representation of painters as functions, makes `beside` easy to implement. The `beside` function need not know anything about the details of the component painters other than that each painter will draw something in its designated frame.

Exercise 2.50. Declare the transformation `flip_horiz`, which flips painters horizontally, and transformations that rotate painters counterclockwise by 180 degrees and 270 degrees.

Exercise 2.51. Declare the `below` operation for painters. The function `below` takes two painters as arguments. The resulting painter, given a frame, draws with the first painter in the bottom of the frame and with the second painter in the top. Define `below` in two different ways—first by writing a function that is analogous to the `beside` function given above, and again in terms of `beside` and suitable rotation operations (from exercise 2.50).

Levels of language for robust design

The picture language exploits some of the critical ideas we've introduced about abstraction with functions and data. The fundamental data abstractions, painters, are implemented using functional representations, which enables the language to handle different basic drawing capabilities in a uniform way. The means of combination satisfy the closure property, which permits us to easily build up complex designs. Finally, all the tools for abstracting functions are available to us for abstracting means of combination for painters.

We have also obtained a glimpse of another crucial idea about languages and program design. This is the approach of *stratified design*, the notion that a complex system should be structured as a sequence of levels that are described using a sequence of languages. Each level is constructed by combining parts that are regarded as primitive at that level, and the parts constructed at each level are used as primitives at the next level. The language used at each level of a stratified design has primitives, means of combination, and means of abstraction appropriate to that level of detail.

Stratified design pervades the engineering of complex systems. For example, in computer engineering, resistors and transistors are combined (and described using a language of analog circuits) to produce parts such as and-gates and or-gates, which form the primitives of a language for digital-circuit design. [^28: Section 3.3.4 describes one such language.] These parts are combined to build processors, bus structures, and memory systems, which are in turn combined to form computers, using languages appropriate to computer architecture. Computers are combined to form distributed systems, using languages appropriate for describing network interconnections, and so on.

As a tiny example of stratification, our picture language uses primitive elements (primitive painters) that specify points and lines to provide the shapes of a painter like `rogers`. The bulk of our description of the picture language focused on combining these primitives, using geometric combiners such as `beside` and `below`. We also worked at a higher level, regarding `beside` and `below` as primitives to be manipulated in a language whose operations, such as `square_of_four`, capture common patterns of combining geometric combiners.

Stratified design helps make programs *robust*, that is, it makes it likely that small changes in a specification will require correspondingly small changes in the program. For instance, suppose we wanted to change the image based on `wave` shown in figure 2.9. We could work at the lowest level to change the detailed appearance of the `wave` element; we could work at the middle level to change the way `corner_split` replicates the `wave`; we could work at the highest level to change how `square_limit` arranges the four copies of the corner. In general, each level of a stratified design provides a different vocabulary for expressing the characteristics of the system, and a different kind of ability to change it.

Exercise 2.52. Make changes to the square limit of `wave` shown in figure 2.9 by working at each of the levels described above. In particular: a. Add some segments to the primitive `wave` painter of exercise 2.49 (to add a smile, for example). b. Change the pattern constructed by `corner_split` (for example, by using only one copy of the `up_split` and `right_split` images instead of two). c. Modify the version of `square_limit` that uses `square_of_four` so as to assemble the corners in a different pattern. (For example, you might make the big Mr. Rogers look outward from each corner of the square.)

2.3 Symbolic Data

All the compound data objects we have used so far were constructed ultimately from numbers. In this section we extend the representational capability of our language by introducing the ability to work with strings of characters as data.

2.3.1 Strings

So far, we have used strings in order to display messages, using the functions `display` and `error` (as for example in exercise 1.22). We can form compound data using strings and have linked lists such as

```
l1list("a", "b", "c", "d")
l1list(23, 45, 17)
l1list(l1list("Jakob", 27), l1list("Lova", 9), l1list("Luisa", 24))
```

In order to distinguish strings from names, we surround them with double quotation marks. For example, the Python expression `z` denotes the value of the name `z`, whereas the Python expression `"z"` denotes a string that consists of a single character, namely the last letter in the English alphabet in lower case.

Via quotation marks, we can distinguish between strings and names:

```
a = 1
b = 2
```

```
print(l1list(a, b))
```

```
[1, [2, None]]
```

```
print(l1list("a", "b"))
```

```
["a", ["b", None]]
```

```
print(l1list("a", b))
```

```
["a", [2, None]]
```

In section 1.1.6, we introduced `==` and `!=` as built-in predicates on numbers. From now on, we shall allow two strings as operands of `==` and `!=`. The predicate `==` returns True if and only if the two strings are the same, and `!=` returns True if and only if the two strings are not the same. [^29: We can consider two strings to be "the same" if they consist of the same characters in the same order. Such a definition skirts a deep issue that we are not yet ready to address: the meaning of "sameness" in a programming language. We will return to this in chapter 3 (section 3.1.3).] Using `==`, we can implement a useful function called `member`. This takes two arguments: a string and a linked list of strings or a number and a linked list of numbers. If the first argument is not

contained in the linked list (i.e., is not `==` to any item in the linked list), then `member` returns `None`. Otherwise, it returns the sublist of the linked list beginning with the first occurrence of the string or number:

```
def member(item, x):
    return (None if is_none(x)
            else x if item == head(x)
            else member(item, tail(x)))
```

For example, the value of

```
print(member("apple", llist("pear", "banana", "prune")))
```

is `None`, whereas the value of

```
print(member("apple", llist("x", "y", "apple", "pear")))
```

is `llist("apple", "pear")`.

Two linked lists are said to be *equal* if they contain equal elements arranged in the same order, and Python's `==` operator supports this notion of *structural equality*. For example,

```
llist("this", "is", "a", "linked", "list") == llist("this", "is", "a", "linked", "list")
```

is `True`, but

```
llist("this", "is", "a", "linked", "list") == llist("this", llist("is", "a"), "linked", "list")
```

is `False`. To be more precise, Python defines the equality operator `==` recursively in terms of the basic `==` equality of numbers and strings by saying that `a` and `b` are equal if they are both strings or both numbers and they are equal, or if they are both pairs such that `head(a)` is equal to `head(b)` and `tail(a)` is equal to `tail(b)`. The `member` function above uses `==` and therefore tests for structural equality.

Exercise 2.53. What is the result of evaluating each of the following expressions, in box notation and linked-list notation?

```
llist("a", "b", "c")
```

```
llist(llist("george"))
```



```
tail(llist(llist("x1", "x2"), llist("y1", "y2")))
```

```
tail(head(llist(llist("x1", "x2"), llist("y1", "y2"))))
```

```
member("red", llist("blue", "shoes", "yellow", "socks"))
```

```
member("red", llist("red", "shoes", "blue", "socks"))
```

Exercise 2.54. Implement a function `equal` that behaves exactly like `==`, such that `equal` applies `==` only to numbers and strings.

Exercise 2.55. The Python interpreter reads the characters after a double quotation mark `"` until it finds another double quotation mark. All characters between the two are part of the string, excluding the double quotation marks themselves. But what if we want a string to contain double quotation marks? For this purpose, Python also allows *single* quotation marks to delimit strings, as for example in `'say your name aloud'`. Within singly-quoted strings, we can use double quotation marks, and vice versa, so `'say "your name" aloud'` and `"say 'your name' aloud"` are valid strings that have different characters at positions 4 and 14, if we start counting at 0. Depending on the font in use, two single quotation marks might not be easily distinguishable from a double quotation mark. Can you spot which is which and work out the value of the following expression?

```
print('"' == "'')
```

2.3.2 Example: Symbolic Differentiation

As an illustration of symbol manipulation and a further illustration of data abstraction, consider the design of a function that performs symbolic differentiation of algebraic expressions. We would like the function to take as arguments an algebraic expression and a variable and to return the derivative of the expression with respect to the variable. For example, if the arguments to the function are $ax^2 + bx + c$ and x , the function should return $2ax + b$. Symbolic differentiation is of special historical significance in the programming language Lisp. [³⁰: The original version of this book used the programming language Scheme, a dialect of Lisp.] It was one of the motivating examples behind the development of a computer language for symbol manipulation. Furthermore, it marked the beginning of the line of research that led to the development of powerful systems for symbolic mathematical work, which are today routinely used by applied mathematicians and physicists.

In developing the symbolic-differentiation program, we will follow the same strategy of data abstraction that we followed in developing the rational-number system of section 2.1.1. That is, we

will first define a differentiation algorithm that operates on abstract objects such as "sums," "products," and "variables" without worrying about how these are to be represented. Only afterward will we address the representation problem.

The differentiation program with abstract data

To keep things simple, we will consider a very simple symbolic-differentiation program that handles expressions that are built up using only the operations of addition and multiplication with two arguments. Differentiation of any such expression can be carried out by applying the following reduction rules:

$$\begin{array}{l} \frac{dc}{dx} = 0 \quad \text{for } c \text{ a constant or a variable different from } x \\ \frac{dx}{dx} = 1 \\ \frac{d(u+v)}{dx} = \frac{du}{dx} + \frac{dv}{dx} \\ \frac{d(uv)}{dx} = u \left(\frac{dv}{dx} \right) + v \left(\frac{du}{dx} \right) \end{array}$$

Observe that the latter two rules are recursive in nature. That is, to obtain the derivative of a sum we first find the derivatives of the terms and add them. Each of the terms may in turn be an expression that needs to be decomposed. Decomposing into smaller and smaller pieces will eventually produce pieces that are either constants or variables, whose derivatives will be either 0 or 1.

To embody these rules in a function we indulge in a little wishful thinking, as we did in designing the rational-number implementation. If we had a means for representing algebraic expressions, we should be able to tell whether an expression is a sum, a product, a constant, or a variable. We should be able to extract the parts of an expression. For a sum, for example, we want to be able to extract the addend (first term) and the augend (second term). We should also be able to construct expressions from parts. Let us assume that we already have functions to implement the following selectors, constructors, and predicates:

```
| is_variable(e) | Is e a variable? | --- | --- | is_same_variable(v1, v2) | Are v1 and v2
the same variable? | is_sum(e) | Is e a sum? | addend(e) | Addend of the sum e. |
augend(e) | Augend of the sum e. | make_sum(a1, a2) | Construct the sum of a1 and a2. |
is_product(e) | Is e a product? | multiplier(e) | Multiplier of the product e. |
multiplicand(e) | Multiplicand of the product e. | make_product(m1, m2) | Construct the
product of m1 and m2. |
```

Using these, and the built-in predicate `is_number`, which identifies numbers, we can express the differentiation rules as the following function:

```
def deriv(exp, variable):
    return (0
            if is_number(exp)
            else (1 if is_same_variable(exp, variable) else 0)
            if is_variable(exp)
            else make_sum(deriv(addend(exp), variable),
                          deriv(augend(exp), variable))
            if is_sum(exp)
            else make_sum(make_product(multiplier(exp),
                                       deriv(multiplicand(exp),
                                             variable)),
                          make_product(deriv(multiplier(exp),
                                             variable),
                                       multiplicand(exp)))
            if is_product(exp)
            else error("unknown expression type -- deriv", exp))
```

This `deriv` function incorporates the complete differentiation algorithm. Since it is expressed in terms of abstract data, it will work no matter how we choose to represent algebraic expressions, as long as we design a proper set of selectors and constructors. This is the issue we must address next.

Representing algebraic expressions

We can imagine many ways to use linked list structure to represent algebraic expressions. For example, we could use linked lists of symbols that mirror the usual algebraic notation, representing $ax+b$ as `llist("a", "*", "x", "+", "b")`. However, it will be more convenient if we reflect the mathematical structure of the expression in the Python value representing it; that is, to represent $ax+b$ as `llist("+", llist("*", "a", "x"), "b")`. Placing a binary operator in front of its operands is called *prefix notation*, in contrast with the infix notation introduced in section 1.1.1. With prefix notation, our data representation for the differentiation problem is as follows:

- The variables are just strings. They are identified by the built-in predicate `is_string` :

```
def is_variable(x): return is_string(x)
```

- Two variables are the same if the strings representing them are equal:

```
def is_same_variable(v1, v2):
    return is_variable(v1) and is_variable(v2) and v1 == v2
```

- Sums and products are constructed as linked lists:

```
def make_sum(a1, a2): return llist("+", a1, a2)

def make_product(m1, m2): return llist("*", m1, m2)
```

- A sum is a linked list whose first element is the string "+" :

```
def is_sum(x):
    return is_pair(x) and head(x) == "+"
```

- The addend is the second item of the sum linked list:

```
def addend(s): return head(tail(s))
```

- The augend is the third item of the sum linked list:

```
def augend(s): return head(tail(tail(s)))
```

- A product is a linked list whose first element is the string "*" :

```
def is_product(x):
    return is_pair(x) and head(x) == "*"
```

- The multiplier is the second item of the product linked list:

```
def multiplier(s): return head(tail(s))
```

- The multiplicand is the third item of the product linked list:

```
def multiplicand(s): return head(tail(tail(s)))
```

Thus, we need only combine these with the algorithm as embodied by `deriv` in order to have a working symbolic-differentiation program. Let us look at some examples of its behavior:

```
print_llist(deriv(llist("+", "x", 3), "x"))
```

```
llist("+", 1, 0)
```

```
print_llist(deriv(llist("*", "x", "y"), "x"))
```

```
l1ist("+", l1ist("*", "x", 0), l1ist("*", 1, "y"))
```

```
print_l1ist(deriv(l1ist("*", l1ist("*", "x", "y"),
                    l1ist("+", "x", 3)), "x"))
```

```
l1ist("+", l1ist("*", l1ist("*", "x", "y"), l1ist("+", 1, 0)),
      l1ist("*", l1ist("+", l1ist("*", "x", 0),
                          l1ist("*", 1, "y")),
      l1ist("+", "x", 3)))
```

The program produces answers that are correct; however, they are unsimplified. It is true that

$$\frac{d(xy)}{dx} = x \cdot 0 + 1 \cdot y$$

but we would like the program to know that $x \cdot 0 = 0$, $1 \cdot y = y$, and $0 + y = y$. The answer for the second example should have been simply y . As the third example shows, this becomes a serious issue when the expressions are complex.

Our difficulty is much like the one we encountered with the rational-number implementation: we haven't reduced answers to simplest form. To accomplish the rational-number reduction, we needed to change only the constructors and the selectors of the implementation. We can adopt a similar strategy here. We won't change `deriv` at all. Instead, we will change `make_sum` so that if both summands are numbers, `make_sum` will add them and return their sum. Also, if one of the summands is 0, then `make_sum` will return the other summand.

```
def make_sum(a1, a2):
    return (a2
            if number_equal(a1, 0)
            else a1
            if number_equal(a2, 0)
            else a1 + a2
            if is_number(a1) and is_number(a2)
            else l1ist("+", a1, a2))
```

This uses the function `number_equal`, which checks whether an expression is equal to a given number:

```
def number_equal(exp, num):
    return is_number(exp) and exp == num
```

Similarly, we will change `make_product` to build in the rules that 0 times anything is 0 and 1 times anything is the thing itself:

```
def make_product(m1, m2):
    return (0
            if number_equal(m1, 0) or number_equal(m2, 0)
            else m2
            if number_equal(m1, 1)
            else m1
            if number_equal(m2, 1)
            else m1 * m2
            if is_number(m1) and is_number(m2)
            else llist("?", m1, m2))
```

Here is how this version works on our three examples:

```
print(deriv(llist("+", "x", 3), "x"))
```

```
1
```

```
print(deriv(llist("?", "x", "y"), "x"))
```

```
"y"
```

```
print_llist(deriv(llist("?", llist("?", "x", "y"),
                               llist("+", "x", 3)), "x"))
```

```
llist("+", llist("?", "x", "y"), llist("?", "y", llist("+", "x", 3)))
```

Although this is quite an improvement, the third example shows that there is still a long way to go before we get a program that puts expressions into a form that we might agree is "simplest." The problem of algebraic simplification is complex because, among other reasons, a form that may be simplest for one purpose may not be for another.

Exercise 2.56. Show how to extend the basic differentiator to handle more kinds of expressions. For instance, implement the differentiation rule

$$\frac{d(u^n)}{dx} = n u^{n-1} \left(\frac{du}{dx} \right)$$

by adding a new clause to the `deriv` program and defining appropriate functions `is_exp`, `base`, `exponent`, and `make_exp`. (You may use the string `"**"` to denote exponentiation.) Build in the rules that anything raised to the power 0 is 1 and anything raised to the power 1 is the thing itself.

Exercise 2.57. Extend the differentiation program to handle sums and products of arbitrary numbers of (two or more) terms. Then the last example above could be expressed as

```
deriv(llist("x", "y", llist("+", "x", 3)), "x")
```

Try to do this by changing only the representation for sums and products, without changing the `deriv` function at all. For example, the `addend` of a sum would be the first term, and the `augend` would be the sum of the rest of the terms.

Exercise 2.58. Suppose we want to modify the differentiation program so that it works with ordinary mathematical notation, in which `+` and `*` are infix rather than prefix operators. Since the differentiation program is defined in terms of abstract data, we can modify it to work with different representations of expressions solely by changing the predicates, selectors, and constructors that define the representation of the algebraic expressions on which the differentiator is to operate. a. Show how to do this in order to differentiate algebraic expressions presented in infix form, as in this example:

```
llist("x", "+", llist(3, "*", llist("x", "+", llist("y", "+", 2))))
```

To simplify the task, assume that `+` and `*` always take two arguments and that expressions are fully parenthesized. b. The problem becomes substantially harder if we allow a notation closer to ordinary infix notation, which omits unnecessary parentheses and assumes that multiplication has higher precedence than addition, as in this example:

```
llist("x", "+", 3, "*", llist("x", "+", "y", "+", 2))
```

Can you design appropriate predicates, selectors, and constructors for this notation such that our derivative program still works?

2.3.3 Example: Representing Sets

In the previous examples we built representations for two kinds of compound data objects: rational numbers and algebraic expressions. In one of these examples we had the choice of simplifying (reducing) the expressions at either construction time or selection time, but other than that the choice of a representation for these structures in terms of linked lists was straightforward. When we turn to the representation of sets, the choice of a representation is not so obvious. Indeed, there are a number of possible representations, and they differ significantly from one another in several ways.

Informally, a set is simply a collection of distinct objects. To give a more precise definition we can employ the method of data abstraction. That is, we define "set" by specifying the operations that are to be used on sets. These are `union_set`, `intersection_set`, `is_element_of_set`, and `adjoin_set`. The function `is_element_of_set` is a predicate that determines whether a given element is a member of a set. The function `adjoin_set` takes an object and a set as arguments

and returns a set that contains the elements of the original set and also the adjoined element. The function `union_set` computes the union of two sets, which is the set containing each element that appears in either argument. The function `intersection_set` computes the intersection of two sets, which is the set containing only elements that appear in both arguments. From the viewpoint of data abstraction, we are free to design any representation that implements these operations in a way consistent with the interpretations given above. [^31: If we want to be more formal, we can specify "consistent with the interpretations given above" to mean that the operations satisfy a collection of rules such as these: - For any set `S` and any object `x`, `is_element_of_set(x, adjoin_set(x, S))` is true (informally: "Adjoining an object to a set produces a set that contains the object"). - For any sets `S` and `T` and any object `x`, `is_element_of_set(x, union_set(S, T))` is equal to `is_element_of_set(x, S)` or `is_element_of_set(x, T)` (informally: "The elements of `union_set(S, T)` are the elements that are in `S` or in `T`"). - For any object `x`, `is_element_of_set(x, None)` is false (informally: "No object is an element of the empty set").]

Sets as unordered linked lists

One way to represent a set is as a linked list of its elements in which no element appears more than once. The empty set is represented by the empty linked list. In this representation, `is_element_of_set` is similar to the function `member` of section . It uses `equal` instead of `==` so that the set elements need not be just numbers or strings:

```
def is_element_of_set(x, set):
    return (False
            if is_none(set)
            else True
            if x == head(set)
            else is_element_of_set(x, tail(set)))
```

Using this, we can write `adjoin_set`. If the object to be adjoined is already in the set, we just return the set. Otherwise, we use `pair` to add the object to the linked list that represents the set:

```
def adjoin_set(x, set):
    return (set
            if is_element_of_set(x, set)
            else pair(x, set))
```

For `intersection_set` we can use a recursive strategy. If we know how to form the intersection of `set2` and the `tail` of `set1`, we only need to decide whether to include the `head` of `set1` in this. But this depends on whether `head(set1)` is also in `set2`. Here is the resulting function:


```
def intersection_set(set1, set2):
    return (None
            if is_none(set1) or is_none(set2)
            else pair(head(set1), intersection_set(tail(set1), set2))
            if is_element_of_set(head(set1), set2)
            else intersection_set(tail(set1), set2))
```

In designing a representation, one of the issues we should be concerned with is efficiency. Consider the number of steps required by our set operations. Since they all use `is_element_of_set`, the speed of this operation has a major impact on the efficiency of the set implementation as a whole. Now, in order to check whether an object is a member of a set, `is_element_of_set` may have to scan the entire set. (In the worst case, the object turns out not to be in the set.) Hence, if the set has n elements, `is_element_of_set` might take up to n steps. Thus, the number of steps required grows as $\Theta(n)$. The number of steps required by `adjoin_set`, which uses this operation, also grows as $\Theta(n)$. For `intersection_set`, which does an `is_element_of_set` check for each element of `set1`, the number of steps required grows as the product of the sizes of the sets involved, or $\Theta(n^2)$ for two sets of size n . The same will be true of `union_set`.

Exercise 2.59. Implement the `union_set` operation for the unordered-linked-list representation of sets.

Exercise 2.60. We specified that a set would be represented as a linked list with no duplicates. Now suppose we allow duplicates. For instance, the set $\{1,2,3\}$ could be represented as the linked list `llist(2, 3, 2, 1, 3, 2, 2)`. Design functions `is_element_of_set`, `adjoin_set`, `union_set`, and `intersection_set` that operate on this representation. How does the efficiency of each compare with the corresponding function for the non-duplicate representation? Are there applications for which you would use this representation in preference to the non-duplicate one?

Sets as ordered linked lists

One way to speed up our set operations is to change the representation so that the set elements are listed in increasing order. To do this, we need some way to compare two objects so that we can say which is bigger. For example, we could compare strings lexicographically, or we could agree on some method for assigning a unique number to an object and then compare the elements by comparing the corresponding numbers. To keep our discussion simple, we will consider only the case where the set elements are numbers, so that we can compare elements using `>` and `<`. We will represent a set of numbers by listing its elements in increasing order. Whereas our first representation above allowed us to represent the set $\{1,3,6,10\}$ by listing the elements in any order, our new representation allows only the linked list `llist(1, 3, 6, 10)`.

One advantage of ordering shows up in `is_element_of_set` : In checking for the presence of an item, we no longer have to scan the entire set. If we reach a set element that is larger than the item we are looking for, then we know that the item is not in the set:

```
def is_element_of_set(x, set):
    return (False
            if is_none(set)
            else True
            if x == head(set)
            else False
            if x < head(set)
            # x > head(set)
            else is_element_of_set(x, tail(set)))
```

How many steps does this save? In the worst case, the item we are looking for may be the largest one in the set, so the number of steps is the same as for the unordered representation. On the other hand, if we search for items of many different sizes we can expect that sometimes we will be able to stop searching at a point near the beginning of the linked list and that other times we will still need to examine most of the linked list. On the average we should expect to have to examine about half of the items in the set. Thus, the average number of steps required will be about $n/2$. This is still $\Theta(n)$ growth, but it does save us, on the average, a factor of 2 in number of steps over the previous implementation.

We obtain a more impressive speedup with `intersection_set` . In the unordered representation this operation required $\Theta(n^2)$ steps, because we performed a complete scan of `set2` for each element of `set1` . But with the ordered representation, we can use a more clever method. Begin by comparing the initial elements, `x1` and `x2` , of the two sets. If `x1` equals `x2` , then that gives an element of the intersection, and the rest of the intersection is the intersection of the `tail` s of the two sets. Suppose, however, that `x1` is less than `x2` . Since `x2` is the smallest element in `set2` , we can immediately conclude that `x1` cannot appear anywhere in `set2` and hence is not in the intersection. Hence, the intersection is equal to the intersection of `set2` with the `tail` of `set1` . Similarly, if `x2` is less than `x1` , then the intersection is given by the intersection of `set1` with the `tail` of `set2` . Here is the function:

```
def intersection_set(set1, set2):
    if is_none(set1) or is_none(set2):
        return None
    else:
        x1 = head(set1)
        x2 = head(set2)
        return (pair(x1, intersection_set(tail(set1), tail(set2)))
                if x1 == x2
                else intersection_set(tail(set1), set2)
                if x1 < x2
                # x2 < x1
                else intersection_set(set1, tail(set2)))
```

To estimate the number of steps required by this process, observe that at each step we reduce the intersection problem to computing intersections of smaller sets—removing the first element from `set1` or `set2` or both. Thus, the number of steps required is at most the sum of the sizes of `set1` and `set2`, rather than the product of the sizes as with the unordered representation. This is $\Theta(n)$ growth rather than $\Theta(n^2)$ —a considerable speedup, even for sets of moderate size.

Exercise 2.61. Give an implementation of `adjoin_set` using the ordered representation. By analogy with `is_element_of_set` show how to take advantage of the ordering to produce a function that requires on the average about half as many steps as with the unordered representation.

Exercise 2.62. Give a $\Theta(n)$ implementation of `union_set` for sets represented as ordered linked lists.

Sets as binary trees

We can do better than the ordered-linked-list representation by arranging the set elements in the form of a tree. Each node of the tree holds one element of the set, called the "entry" at that node, and a link to each of two other (possibly empty) nodes. The "left" link points to elements smaller than the one at the node, and the "right" link to elements greater than the one at the node.

Figure 2.16 shows some trees that represent the set $\{1,3,5,7,9,11\}$. The same set may be represented by a tree in a number of different ways. The only thing we require for a valid representation is that all elements in the left subtree be smaller than the node entry and that all elements in the right subtree be larger.

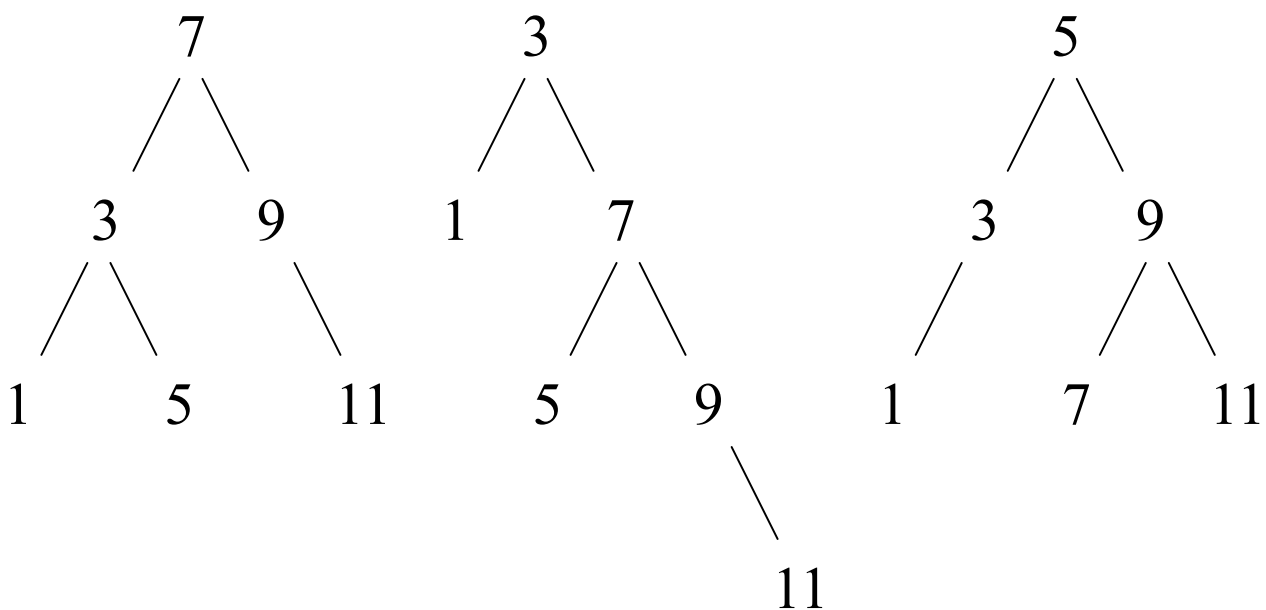


Figure 2.16. Various binary trees that represent the set $\{1,3,5,7,9,11\}$.

The advantage of the tree representation is this: Suppose we want to check whether a number x is contained in a set. We begin by comparing x with the entry in the top node. If x is less than this, we know that we need only search the left subtree; if x is greater, we need only search the right subtree. Now, if the tree is "balanced," each of these subtrees will be about half the size of the original. Thus, in one step we have reduced the problem of searching a tree of size n to searching a tree of size $n/2$. Since the size of the tree is halved at each step, we should expect that the number of steps needed to search a tree of size n grows as $\Theta(\log n)$. [³²: Halving the size of the problem at each step is the distinguishing characteristic of logarithmic growth, as we saw with the fast-exponentiation algorithm of section 1.2.4 and the half-interval search method of section 1.3.3.] For large sets, this will be a significant speedup over the previous representations.

We can represent trees by using linked lists. Each node will be a linked list of three items: the entry at the node, the left subtree, and the right subtree. A left or a right subtree of the empty linked list will indicate that there is no subtree connected there. We can describe this representation by the following functions: [³³: We are representing sets in terms of trees, and trees in terms of linked lists—in effect, a data abstraction built upon a data abstraction. We can regard the functions `entry`, `left_branch`, `right_branch`, and `make_tree` as a way of isolating the abstraction of a "binary tree" from the particular way we might wish to represent such a tree in terms of linked-list structure.]

```
def entry(tree): return head(tree)

def left_branch(tree): return head(tail(tree))

def right_branch(tree): return head(tail(tail(tree)))

def make_tree(entry, left, right):
    return llist(entry, left, right)
```

Now we can write `is_element_of_set` using the strategy described above:

```
def is_element_of_set(x, set):
    return (False
            if is_none(set)
            else True
            if x == entry(set)
            else is_element_of_set(x, left_branch(set))
            if x < entry(set)
            # x > entry(set)
            else is_element_of_set(x, right_branch(set)))
```

Adjoining an item to a set is implemented similarly and also requires $\Theta(\log n)$ steps. To adjoin an item `x`, we compare `x` with the node entry to determine whether `x` should be added to the right or to the left branch, and having adjoined `x` to the appropriate branch we piece this

newly constructed branch together with the original entry and the other branch. If `x` is equal to the entry, we just return the node. If we are asked to adjoin `x` to an empty tree, we generate a tree that has `x` as the entry and empty right and left branches. Here is the function:

```
def adjoin_set(x, set):
    return (make_tree(x, None, None)
            if is_none(set)
            else set
            if x == entry(set)
            else make_tree(entry(set),
                           adjoin_set(x, left_branch(set)),
                           right_branch(set))
            if x < entry(set)
            # x > entry(set)
            else make_tree(entry(set),
                           left_branch(set),
                           adjoin_set(x, right_branch(set))))
```

The above claim that searching the tree can be performed in a logarithmic number of steps rests on the assumption that the tree is "balanced," i.e., that the left and the right subtree of every tree have approximately the same number of elements, so that each subtree contains about half the elements of its parent. But how can we be certain that the trees we construct will be balanced? Even if we start with a balanced tree, adding elements with `adjoin_set` may produce an unbalanced result. Since the position of a newly adjoined element depends on how the element compares with the items already in the set, we can expect that if we add elements "randomly" the tree will tend to be balanced on the average. But this is not a guarantee. For example, if we start with an empty set and adjoin the numbers 1 through 7 in sequence we end up with the highly unbalanced tree shown in figure 2.17. In this tree all the left subtrees are empty, so it has no advantage over a simple ordered linked list. One way to solve this problem is to define an operation that transforms an arbitrary tree into a balanced tree with the same elements. Then we can perform this transformation after every few `adjoin_set` operations to keep our set in balance. There are also other ways to solve this problem, most of which involve designing new data structures for which searching and insertion both can be done in $\Theta(\log n)$ steps. [³⁴: Examples of such structures include *B-trees* and *red-black trees*. There is a large literature on data structures devoted to this problem. See Cormen, Leiserson, Rivest, and Stein 2022.]

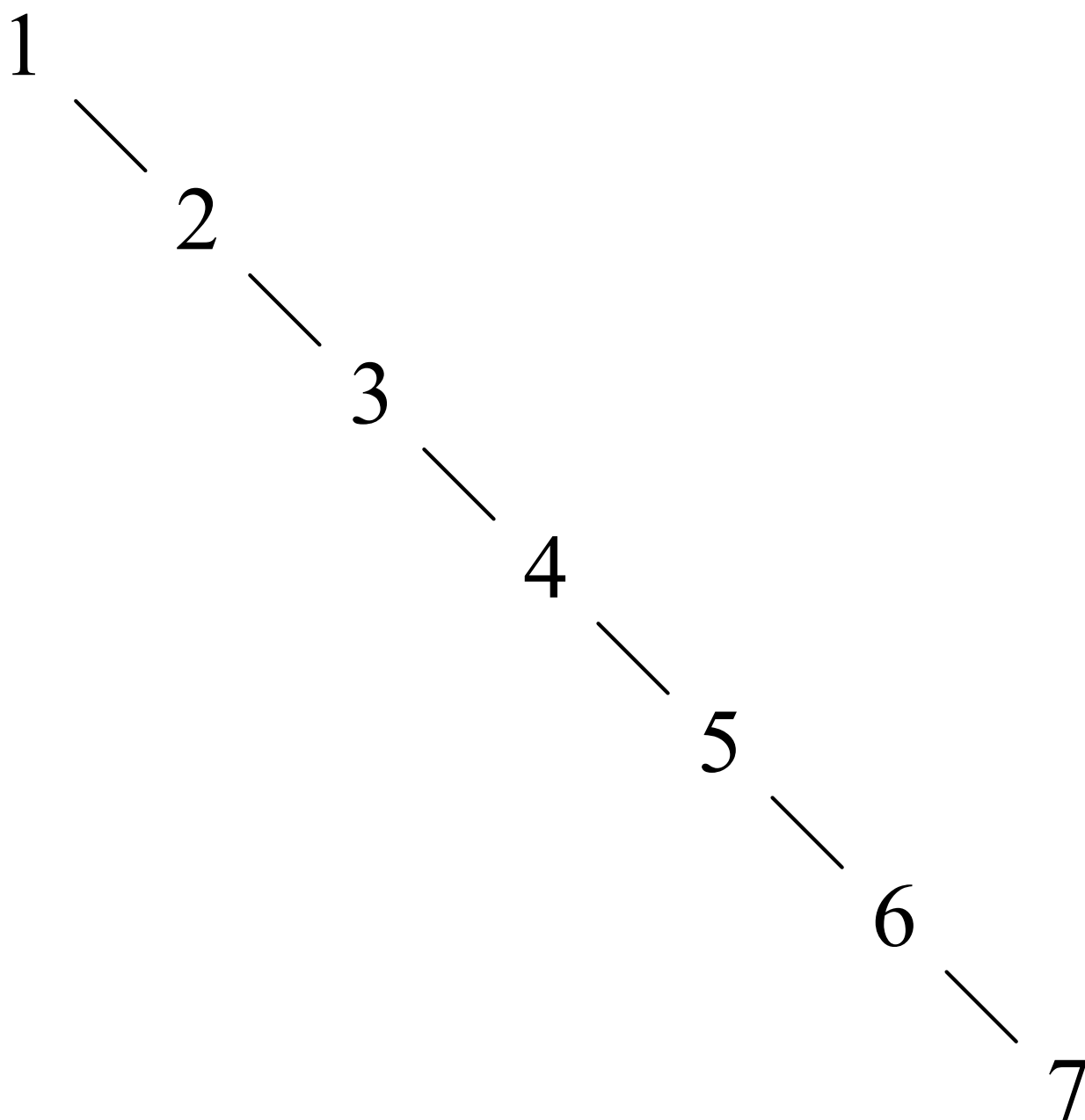


Figure 2.17. Unbalanced tree produced by adjoining 1 through 7 in sequence.

Exercise 2.63. Each of the following two functions converts a binary tree to a linked list.

```
def tree_to_linked_list_1(tree):  
    return (None  
            if is_none(tree)  
            else append(tree_to_linked_list_1(left_branch(tree)),  
                        pair(entry(tree),  
                            tree_to_linked_list_1(right_branch(tree)))))
```

```

def tree_to_linked_list_2(tree):
    def copy_to_linked_list(tree, result_list):
        return (result_list
                if is_none(tree)
                else copy_to_linked_list(left_branch(tree),
                                         pair(entry(tree),
                                               copy_to_linked_list(right_branch(tree),
                                                                    result_list))))
    return copy_to_linked_list(tree, None)

```

a. Do the two functions produce the same result for every tree? If not, how do the results differ? What linked lists do the two functions produce for the trees in figure 2.16? b. Do the two functions have the same order of growth in the number of steps required to convert a balanced tree with n elements to a linked list? If not, which one grows more slowly?

Exercise 2.64. The following function `linked_list_to_tree` converts an ordered linked list to a balanced binary tree. The helper function `partial_tree` takes as arguments an integer n and linked list of at least n elements and constructs a balanced tree containing the first n elements of the linked list. The result returned by `partial_tree` is a pair (formed with `pair`) whose `head` is the constructed tree and whose `tail` is the linked list of elements not included in the tree.

```

def linked_list_to_tree(elements):
    return head(partial_tree(elements, length(elements)))
def partial_tree(elts, n):
    if n == 0:
        return pair(None, elts)
    else:
        left_size = (n - 1) // 2
        left_result = partial_tree(elts, left_size)
        left_tree = head(left_result)
        non_left_elts = tail(left_result)
        right_size = n - (left_size + 1)
        this_entry = head(non_left_elts)
        right_result = partial_tree(tail(non_left_elts), right_size)
        right_tree = head(right_result)
        remaining_elts = tail(right_result)
        return pair(make_tree(this_entry, left_tree, right_tree),
                    remaining_elts)

```

a. Write a short paragraph explaining as clearly as you can how `partial_tree` works. Draw the tree produced by `linked_list_to_tree` for the linked list `llist(1, 3, 5, 7, 9, 11)`. b. What is the order of growth in the number of steps required by `linked_list_to_tree` to convert a linked list of n elements?

Exercise 2.65. Use the results of exercises 2.63 and 2.64 to give $\Theta(n)$ implementations of `union_set` and `intersection_set` for sets implemented as (balanced) binary trees. [35: Exercises 2.63–2.65 are due to Paul Hilfinger.]

Sets and information retrieval

We have examined options for using linked lists to represent sets and have seen how the choice of representation for a data object can have a large impact on the performance of the programs that use the data. Another reason for concentrating on sets is that the techniques discussed here appear again and again in applications involving information retrieval.

Consider a data base containing a large number of individual records, such as the personnel files for a company or the transactions in an accounting system. A typical data-management system spends a large amount of time accessing or modifying the data in the records and therefore requires an efficient method for accessing records. This is done by identifying a part of each record to serve as an identifying *key*. A key can be anything that uniquely identifies the record. For a personnel file, it might be an employee's ID number. For an accounting system, it might be a transaction number. Whatever the key is, when we define the record as a data structure we should include a `key` selector function that retrieves the key associated with a given record.

Now we represent the data base as a set of records. To locate the record with a given key we use a function `lookup`, which takes as arguments a key and a data base and which returns the record that has that key, or false if there is no such record. The function `lookup` is implemented in almost the same way as `is_element_of_set`. For example, if the set of records is implemented as an unordered linked list we could use

```
def lookup(given_key, set_of_records):
    return (False
            if is_none(set_of_records)
            else head(set_of_records)
            if given_key == key(head(set_of_records))
            else lookup(given_key, tail(set_of_records)))
```

Of course, there are better ways to represent large sets than as unordered linked lists. Information-retrieval systems in which records have to be "randomly accessed" are typically implemented by a tree-based method, such as the binary-tree representation discussed previously. In designing such a system the methodology of data abstraction can be a great help. The designer can create an initial implementation using a simple, straightforward representation such as unordered linked lists. This will be unsuitable for the eventual system, but it can be useful in providing a "quick and dirty" data base with which to test the rest of the system. Later on, the data representation can be modified to be more sophisticated. If the data base is accessed in terms of abstract selectors and constructors, this change in representation will not require any changes to the rest of the system.

Exercise 2.66. Implement the `lookup` function for the case where the set of records is structured as a binary tree, ordered by the numerical values of the keys.

2.3.4 Example: Huffman Encoding Trees

This section provides practice in the use of linked list structure and data abstraction to manipulate sets and trees. The application is to methods for representing data as sequences of ones and zeros (bits). For example, the ASCII standard code used to represent text in computers encodes each character as a sequence of seven bits. Using seven bits allows us to distinguish 2^7 , or 128, possible different characters. In general, if we want to distinguish n different symbols, we will need to use $\lceil \log_2 n \rceil$ bits per symbol. If all our messages are made up of the eight symbols A, B, C, D, E, F, G, and H, we can choose a code with three bits per character, for example

```
| A | 000 || C | 010 || E | 100 || G | 110 || --- | --- | --- | --- | --- | --- | --- | --- | --- | --- || B | 001 |
| D | 011 || F | 101 || H | 111 |
```

With this code, the message

```
| BACADAEAFABBAAGAH || --- |
```

is encoded as the string of 54 bits

```
| 001000010000011000100000101000001001000000000110000111 || --- |
```

Codes such as ASCII and the A-through-H code above are known as *fixed-length* codes, because they represent each symbol in the message with the same number of bits. It is sometimes advantageous to use *variable-length* codes, in which different symbols may be represented by different numbers of bits. For example, Morse code does not use the same number of dots and dashes for each letter of the alphabet. In particular, E, the most frequent letter, is represented by a single dot. In general, if our messages are such that some symbols appear very frequently and some very rarely, we can encode data more efficiently (i.e., using fewer bits per message) if we assign shorter codes to the frequent symbols. Consider the following alternative code for the letters A through H:

```
| A | 0 || C | 1010 || E | 1100 || G | 1110 || --- | --- | --- | --- | --- | --- | --- | --- | --- | --- || B | 100
|| D | 1011 || F | 1101 || H | 1111 |
```

With this code, the same message as above is encoded as the string

```
| 100010100101101100011010100100000111001111 || --- |
```

This string contains 42 bits, so it saves more than 20% in space in comparison with the fixed-length code shown above.

One of the difficulties of using a variable-length code is knowing when you have reached the end of a symbol in reading a sequence of zeros and ones. Morse code solves this problem by using a special *separator code* (in this case, a pause) after the sequence of dots and dashes for each letter. Another solution is to design the code in such a way that no complete code for any symbol is the

beginning (or *prefix*) of the code for another symbol. Such a code is called a *prefix code*. In the example above, A is encoded by 0 and B is encoded by 100, so no other symbol can have a code that begins with 0 or with 100.

In general, we can attain significant savings if we use variable-length prefix codes that take advantage of the relative frequencies of the symbols in the messages to be encoded. One particular scheme for doing this is called the Huffman encoding method, after its discoverer, David Huffman. A Huffman code can be represented as a binary tree whose leaves are the symbols that are encoded. At each non-leaf node of the tree there is a set containing all the symbols in the leaves that lie below the node. In addition, each symbol at a leaf is assigned a weight (which is its relative frequency), and each non-leaf node contains a weight that is the sum of all the weights of the leaves lying below it. The weights are not used in the encoding or the decoding process. We will see below how they are used to help construct the tree.

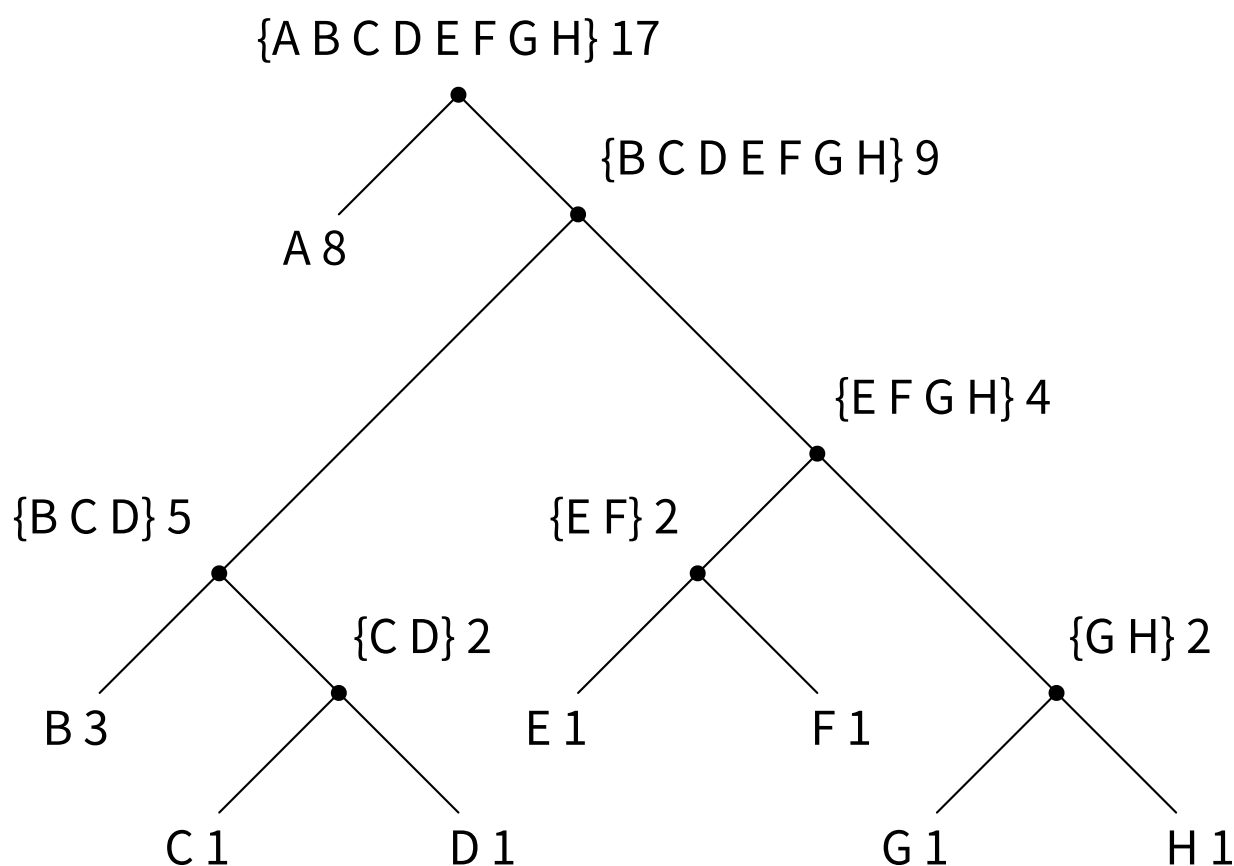


Figure 2.18. A Huffman encoding tree.

Figure 2.18 shows the Huffman tree for the A-through-H code given above. The weights at the leaves indicate that the tree was designed for messages in which A appears with relative frequency 8, B with relative frequency 3, and the other letters each with relative frequency 1.

Given a Huffman tree, we can find the encoding of any symbol by starting at the root and moving down until we reach the leaf that holds the symbol. Each time we move down a left branch we add a 0 to the code, and each time we move down a right branch we add a 1. (We decide which branch to follow by testing to see which branch either is the leaf node for the symbol or contains the

symbol in its set.) For example, starting from the root of the tree in figure 2.18, we arrive at the leaf for D by following a right branch, then a left branch, then a right branch, then a right branch; hence, the code for D is 1011.

To decode a bit sequence using a Huffman tree, we begin at the root and use the successive zeros and ones of the bit sequence to determine whether to move down the left or the right branch. Each time we come to a leaf, we have generated a new symbol in the message, at which point we start over from the root of the tree to find the next symbol. For example, suppose we are given the tree above and the sequence 10001010. Starting at the root, we move down the right branch (since the first bit of the string is 1), then down the left branch (since the second bit is 0), then down the left branch (since the third bit is also 0). This brings us to the leaf for B, so the first symbol of the decoded message is B. Now we start again at the root, and we make a left move because the next bit in the string is 0. This brings us to the leaf for A. Then we start again at the root with the rest of the string 1010, so we move right, left, right, left and reach C. Thus, the entire message is BAC.

Generating Huffman trees

Given an "alphabet" of symbols and their relative frequencies, how do we construct the "best" code? (In other words, which tree will encode messages with the fewest bits?) Huffman gave an algorithm for doing this and showed that the resulting code is indeed the best variable-length code for messages where the relative frequency of the symbols matches the frequencies with which the code was constructed. We will not prove this optimality of Huffman codes here, but we will show how Huffman trees are constructed. [³⁶ See Hamming 1980 for a discussion of the mathematical properties of Huffman codes.]

The algorithm for generating a Huffman tree is very simple. The idea is to arrange the tree so that the symbols with the lowest frequency appear farthest away from the root. Begin with the set of leaf nodes, containing symbols and their frequencies, as determined by the initial data from which the code is to be constructed. Now find two leaves with the lowest weights and merge them to produce a node that has these two nodes as its left and right branches. The weight of the new node is the sum of the two weights. Remove the two leaves from the original set and replace them by this new node. Now continue this process. At each step, merge two nodes with the smallest weights, removing them from the set and replacing them with a node that has these two as its left and right branches. The process stops when there is only one node left, which is the root of the entire tree. Here is how the Huffman tree of figure 2.18 was generated:

```
| Initial leaves | A(8) (B 3) (C 1) (D 1) (E 1) (F 1) (G 1) (H 1) | --- | --- | |
| Merge | A(8) (B 3) | A(8) (B 3) (C D)(2) | A(8) (B 3) (E F)(2) | A(8) (B 3) (C D)(2) (E F)(2) |
| Merge | A(8) (B 3) (C D)(2) (E F)(2) | A(8) (B 3) (C D)(2) (E F)(2) (G H)(2) |
| Merge | A(8) (B 3) (C D)(2) (E F)(2) (G H)(2) | A(8) (B 3) (C D E F)(5) (G H)(2) |
| Merge | A(8) (B 3) (C D E F)(5) (G H)(2) | A(8) (B 3) (C D E F)(5) (G H)(2) (G H)(2) |
| Merge | A(8) (B 3) (C D E F)(5) (G H)(2) (G H)(2) | A(8) (B 3) (C D E F)(5) (G H)(2) (G H)(2) (G H)(2) |
| Merge | A(8) (B 3) (C D E F)(5) (G H)(2) (G H)(2) (G H)(2) | A(8) (B 3) (C D E F)(5) (G H)(2) (G H)(2) (G H)(2) (G H)(2) |
| Final merge | A(8) (B 3) (C D E F)(5) (G H)(2) (G H)(2) (G H)(2) (G H)(2) (G H)(2) (G H)(2) (G H)(2) |
```

The algorithm does not always specify a unique tree, because there may not be unique smallest-weight nodes at each step. Also, the choice of the order in which the two nodes are merged (i.e., which will be the right branch and which will be the left branch) is arbitrary.

Representing Huffman trees

In the exercises below we will work with a system that uses Huffman trees to encode and decode messages and generates Huffman trees according to the algorithm outlined above. We will begin by discussing how trees are represented.

Leaves of the tree are represented by a linked list consisting of the string "leaf", the symbol at the leaf, and the weight:

```
def make_leaf(symbol, weight):
    return llist("leaf", symbol, weight)
def is_leaf(object):
    return head(object) == "leaf"
def symbol_leaf(x): return head(tail(x))
def weight_leaf(x): return head(tail(tail(x)))
```

A general tree will be a linked list of a string "code_tree", a left branch, a right branch, a set of symbols, and a weight. The set of symbols will be simply a linked list of the symbols, rather than some more sophisticated set representation. When we make a tree by merging two nodes, we obtain the weight of the tree as the sum of the weights of the nodes, and the set of symbols as the union of the sets of symbols for the nodes. Since our symbol sets are represented as linked lists, we can form the union by using the `append` function we defined in section 2.2.1:

```
def make_code_tree(left, right):
    return llist("code_tree", left, right,
                append(symbols(left), symbols(right)),
                weight(left) + weight(right))
```

If we make a tree in this way, we have the following selectors:

```

def left_branch(tree): return head(tail(tree))

def right_branch(tree): return head(tail(tail(tree)))

def symbols(tree):
    return (l1ist(symbol_leaf(tree))
            if is_leaf(tree)
            else head(tail(tail(tail(tree)))))

def weight(tree):
    return (weight_leaf(tree)
            if is_leaf(tree)
            else head(tail(tail(tail(tail(tree)))))

```

The functions `symbols` and `weight` must do something slightly different depending on whether they are called with a leaf or a general tree. These are simple examples of *generic functions* (functions that can handle more than one kind of data), which we will have much more to say about in sections 2.4 and 2.5.

The decoding function

The following function implements the decoding algorithm. It takes as arguments a linked list of zeros and ones, together with a Huffman tree.

```

def decode(bits, tree):
    def decode_1(bits, current_branch):
        if is_none(bits):
            return None
        else:
            next_branch = choose_branch(head(bits),
                                       current_branch)
            return (pair(symbol_leaf(next_branch),
                        decode_1(tail(bits), tree))
                    if is_leaf(next_branch)
                    else decode_1(tail(bits), next_branch))
    return decode_1(bits, tree)

def choose_branch(bit, branch):
    return (left_branch(branch)
            if bit == 0
            else right_branch(branch)
            if bit == 1
            else error("bad bit -- choose_branch", bit))

```

The function `decode_1` takes two arguments: the linked list of remaining bits and the current position in the tree. It keeps moving "down" the tree, choosing a left or a right branch according to whether the next bit in the linked list is a zero or a one. (This is done with the function `choose_branch`.) When it reaches a leaf, it returns the symbol at that leaf as the next symbol in the message by adjoining it to the result of decoding the rest of the message, starting at

the root of the tree. Note the error check in the final clause of `choose_branch`, which complains if the function finds something other than a zero or a one in the input data.

Sets of weighted elements

In our representation of trees, each non-leaf node contains a set of symbols, which we have represented as a simple linked list. However, the tree-generating algorithm discussed above requires that we also work with sets of leaves and trees, successively merging the two smallest items. Since we will be required to repeatedly find the smallest item in a set, it is convenient to use an ordered representation for this kind of set.

We will represent a set of leaves and trees as a linked list of elements, arranged in increasing order of weight. The following `adjoin_set` function for constructing sets is similar to the one described in exercise 2.61; however, items are compared by their weights, and the element being added to the set is never already in it.

```
def adjoin_set(x, set):
    return (llist(x)
            if is_none(set)
            else pair(x, set)
            if weight(x) < weight(head(set))
            else pair(head(set), adjoin_set(x, tail(set))))
```

The following function takes a linked list of symbol-frequency pairs such as

```
llist(llist("A", 4), llist("B", 2), llist("C", 1), llist("D", 1))
```

and constructs an initial ordered set of leaves, ready to be merged according to the Huffman algorithm:

```
def make_leaf_set(pairs):
    if is_none(pairs):
        return None
    else:
        first_pair = head(pairs)
        return adjoin_set(
            make_leaf(head(first_pair), # symbol
                    head(tail(first_pair))), # frequency
            make_leaf_set(tail(pairs)))
```

Exercise 2.67. Declare an encoding tree and a sample message:

```

sample_tree = make_code_tree(make_leaf("A", 4),
                             make_code_tree(make_leaf("B", 2),
                                             make_code_tree(
                                                make_leaf("D", 1),
                                                make_leaf("C", 1))))
sample_message = llist(0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0)

```

Use the `decode` function to decode the message, and give the result.

Exercise 2.68. The `encode` function takes as arguments a message and a tree and produces the linked list of bits that gives the encoded message.

```

def encode(message, tree):
    return (None
            if is_none(message)
            else append(encode_symbol(head(message), tree),
                        encode(tail(message), tree)))

```

The function `encode_symbol`, which you must write, returns the linked list of bits that encodes a given symbol according to a given tree. You should design `encode_symbol` so that it signals an error if the symbol is not in the tree at all. Test your function by encoding the result you obtained in exercise 2.67 with the sample tree and seeing whether it is the same as the original sample message.

Exercise 2.69. The following function takes as its argument a linked list of symbol-frequency pairs (where no symbol appears in more than one pair) and generates a Huffman encoding tree according to the Huffman algorithm.

```

def generate_huffman_tree(pairs):
    return successive_merge(make_leaf_set(pairs))

```

The function `make_leaf_set` that transforms the linked list of pairs into an ordered set of leaves is given above. Write the function `successive_merge` using `make_code_tree` to successively merge the smallest-weight elements of the set until there is only one element left, which is the desired Huffman tree. (This function is slightly tricky, but not really complicated. If you find yourself designing a complex function, then you are almost certainly doing something wrong. You can take significant advantage of the fact that we are using an ordered set representation.)

Exercise 2.70. The following eight-symbol alphabet with associated relative frequencies was designed to efficiently encode the lyrics of 1950s rock songs. (Note that the "symbols" of an "alphabet" need not be individual letters.)

```

| A | 2 | | NA | 16 | | --- | --- | --- | --- | --- | | BOOM | 1 | | SHA | 3 | | GET | 2 | | YIP | 9 | | JOB | 2 | |
WAH | 1 |

```

Use `generate_huffman_tree` (exercise 2.69) to generate a corresponding Huffman tree, and use `encode` (exercise 2.68) to encode the following message:

| Get a job | | --- | | Sha na na na na na na na na | | Get a job | | Sha na na na na na na na na | | Wah
yip yip yip yip yip yip yip yip yip | | Sha boom |

How many bits are required for the encoding? What is the smallest number of bits that would be needed to encode this song if we used a fixed-length code for the eight-symbol alphabet?

Exercise 2.71. Suppose we have a Huffman tree for an alphabet of n symbols, and that the relative frequencies of the symbols are 1, 2, 4, ..., 2^{n-1} . Sketch the tree for $n=5$; for $n=10$. In such a tree (for general n) how many bits are required to encode the most frequent symbol? the least frequent symbol?

Exercise 2.72. Consider the encoding function that you designed in exercise 2.68. What is the order of growth in the number of steps needed to encode a symbol? Be sure to include the number of steps needed to search the symbol linked list at each node encountered. To answer this question in general is difficult. Consider the special case where the relative frequencies of the n symbols are as described in exercise 2.71, and give the order of growth (as a function of n) of the number of steps needed to encode the most frequent and least frequent symbols in the alphabet.

2.4 Multiple Representations for Abstract Data

We have introduced data abstraction, a methodology for structuring systems in such a way that much of a program can be specified independent of the choices involved in implementing the data objects that the program manipulates. For example, we saw in section 2.1.1 how to separate the task of designing a program that uses rational numbers from the task of implementing rational numbers in terms of the computer language's primitive mechanisms for constructing compound data. The key idea was to erect an abstraction barrier—in this case, the selectors and constructors for rational numbers (`make_rat`, `numer`, `denom`)—that isolates the way rational numbers are used from their underlying representation in terms of linked-list structure. A similar abstraction barrier isolates the details of the functions that perform rational arithmetic (`add_rat`, `sub_rat`, `mul_rat`, and `div_rat`) from the "higher-level" functions that use rational numbers. The resulting program has the structure shown in figure 2.1.

These data-abstraction barriers are powerful tools for controlling complexity. By isolating the underlying representations of data objects, we can divide the task of designing a large program into smaller tasks that can be performed separately. But this kind of data abstraction is not yet powerful enough, because it may not always make sense to speak of "the underlying representation" for a data object.

For one thing, there might be more than one useful representation for a data object, and we might like to design systems that can deal with multiple representations. To take a simple example, complex numbers may be represented in two almost equivalent ways: in rectangular form (real and imaginary parts) and in polar form (magnitude and angle). Sometimes rectangular form is more appropriate and sometimes polar form is more appropriate. Indeed, it is perfectly plausible to imagine a system in which complex numbers are represented in both ways, and in which the functions for manipulating complex numbers work with either representation.

More importantly, programming systems are often designed by many people working over extended periods of time, subject to requirements that change over time. In such an environment, it is simply not possible for everyone to agree in advance on choices of data representation. So in addition to the data-abstraction barriers that isolate representation from use, we need abstraction barriers that isolate different design choices from each other and permit different choices to coexist in a single program. Furthermore, since large programs are often created by combining preexisting modules that were designed in isolation, we need conventions that permit programmers to incorporate modules into larger systems *additively*, that is, without having to redesign or reimplement these modules.

In this section, we will learn how to cope with data that may be represented in different ways by different parts of a program. This requires constructing *generic functions*—functions that can operate on data that may be represented in more than one way. Our main technique for building generic functions will be to work in terms of data objects that have *type tags*, that is, data objects that include explicit information about how they are to be processed. We will also discuss *data-directed* programming, a powerful and convenient implementation strategy for additively assembling systems with generic operations.

We begin with the simple complex-number example. We will see how type tags and data-directed style enable us to design separate rectangular and polar representations for complex numbers while maintaining the notion of an abstract "complex-number" data object. We will accomplish this by defining arithmetic functions for complex numbers (`add_complex` , `sub_complex` , `mul_complex` , and `div_complex`) in terms of generic selectors that access parts of a complex number independent of how the number is represented. The resulting complex-number system, as shown in figure 2.19, contains two different kinds of abstraction barriers. The "horizontal" abstraction barriers play the same role as the ones in figure 2.1. They isolate "higher-level" operations from "lower-level" representations. In addition, there is a "vertical" barrier that gives us the ability to separately design and install alternative representations.

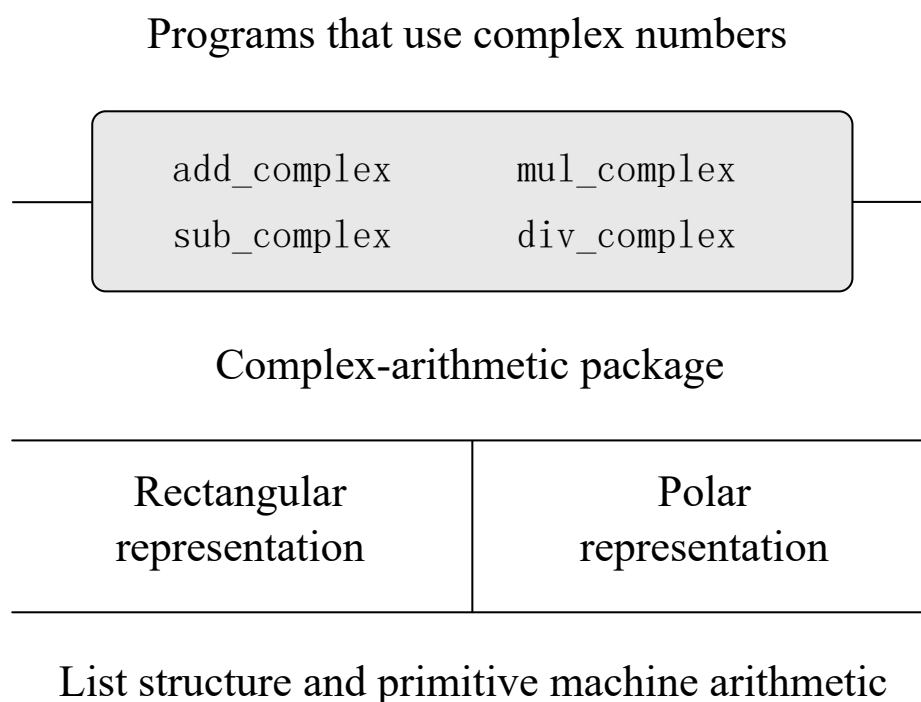


Figure 2.19. Data-abstraction barriers in the complex-number system.

In section 2.5 we will show how to use type tags and data-directed style to develop a generic arithmetic package. This provides functions (`add` , `mul` , and so on) that can be used to manipulate all sorts of "numbers" and can be easily extended when a new kind of number is needed. In section 2.5.3, we'll show how to use generic arithmetic in a system that performs symbolic algebra.

2.4.1 Representations for Complex Numbers

We will develop a system that performs arithmetic operations on complex numbers as a simple but unrealistic example of a program that uses generic operations. We begin by discussing two plausible representations for complex numbers as ordered pairs: rectangular form (real part and imaginary part) and polar form (magnitude and angle). [^37: In actual computational systems, rectangular form is preferable to polar form most of the time because of roundoff errors in conversion between rectangular and polar form. This is why the complex-number example is unrealistic. Nevertheless, it provides a clear illustration of the design of a system using generic operations and a good introduction to the more substantial systems to be developed later in this chapter.] Section 2.4.2 will show how both representations can be made to coexist in a single system through the use of type tags and generic operations.

Like rational numbers, complex numbers are naturally represented as ordered pairs. The set of complex numbers can be thought of as a two-dimensional space with two orthogonal axes, the "real" axis and the "imaginary" axis. (See figure 2.20.) From this point of view, the complex number $z = x + iy$ (where $i^2 = -1$) can be thought of as the point in the plane whose real coordinate is x and whose imaginary coordinate is y . Addition of complex numbers reduces in this representation to addition of coordinates:

$$\begin{array}{l} \text{Real-part}(z_1+z_2) \text{ \& } = \text{ \& } \text{Real-part}(z_1)+\text{Real-part}(z_2) \text{ \& } \\ \text{Imaginary-part}(z_1+z_2) \text{ \& } = \text{ \& } \text{Imaginary-part}(z_1)+\text{Imaginary-part}(z_2) \end{array}$$

When multiplying complex numbers, it is more natural to think in terms of representing a complex number in polar form, as a magnitude and an angle (r and A in figure 2.20). The product of two complex numbers is the vector obtained by stretching one complex number by the length of the other and then rotating it through the angle of the other:

$$\begin{array}{l} \text{Magnitude}(z_1 \cdot z_2) \text{ \& } = \text{ \& } \text{Magnitude}(z_1) \cdot \text{Magnitude}(z_2) \text{ \& } \\ \text{Angle}(z_1 \cdot z_2) \text{ \& } = \text{ \& } \text{Angle}(z_1) + \text{Angle}(z_2) \end{array}$$

Thus, there are two different representations for complex numbers, which are appropriate for different operations. Yet, from the viewpoint of someone writing a program that uses complex numbers, the principle of data abstraction suggests that all the operations for manipulating complex numbers should be available regardless of which representation is used by the computer. For example, it is often useful to be able to find the magnitude of a complex number that is specified by rectangular coordinates. Similarly, it is often useful to be able to determine the real part of a complex number that is specified by polar coordinates.

Imaginary

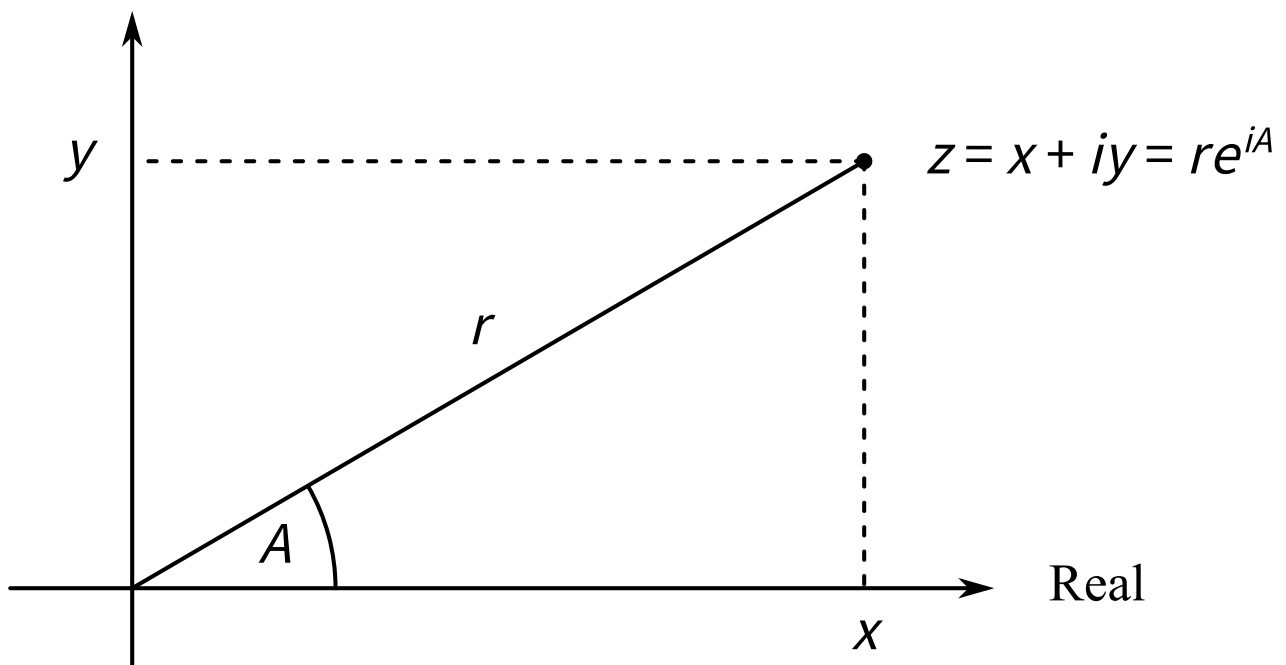


Figure 2.20. Complex numbers as points in the plane.

To design such a system, we can follow the same data-abstraction strategy we followed in designing the rational-number package in section 2.1.1. Assume that the operations on complex numbers are implemented in terms of four selectors: `real_part`, `imag_part`, `magnitude`, and

`angle` . Also assume that we have two functions for constructing complex numbers:
`make_from_real_imag` returns a complex number with specified real and imaginary parts, and
`make_from_mag_ang` returns a complex number with specified magnitude and angle. These functions have the property that, for any complex number `z` , both

```
make_from_real_imag(real_part(z), imag_part(z))
```

and

```
make_from_mag_ang(magnitude(z), angle(z))
```

produce complex numbers that are equal to `z` .

Using these constructors and selectors, we can implement arithmetic on complex numbers using the "abstract data" specified by the constructors and selectors, just as we did for rational numbers in section 2.1.1. As shown in the formulas above, we can add and subtract complex numbers in terms of real and imaginary parts while multiplying and dividing complex numbers in terms of magnitudes and angles:

```
def add_complex(z1, z2):
    return make_from_real_imag(real_part(z1) + real_part(z2),
                               imag_part(z1) + imag_part(z2))

def sub_complex(z1, z2):
    return make_from_real_imag(real_part(z1) - real_part(z2),
                               imag_part(z1) - imag_part(z2))

def mul_complex(z1, z2):
    return make_from_mag_ang(magnitude(z1) * magnitude(z2),
                             angle(z1) + angle(z2))

def div_complex(z1, z2):
    return make_from_mag_ang(magnitude(z1) / magnitude(z2),
                             angle(z1) - angle(z2))
```

To complete the complex-number package, we must choose a representation and we must implement the constructors and selectors in terms of primitive numbers and primitive linked-list structure. There are two obvious ways to do this: We can represent a complex number in "rectangular form" as a pair (real part, imaginary part) or in "polar form" as a pair (magnitude, angle). Which shall we choose?

In order to make the different choices concrete, imagine that there are two programmers, Ben Bitdiddle and Alyssa P. Hacker, who are independently designing representations for the complex-number system. Ben chooses to represent complex numbers in rectangular form. With this choice, selecting the real and imaginary parts of a complex number is straightforward, as is constructing a complex number with given real and imaginary parts. To find the magnitude and the angle, or to

construct a complex number with a given magnitude and angle, he uses the trigonometric relations

$$\begin{array}{l} x = r \cos A \quad \quad \quad r = \sqrt{x^2 + y^2} \\ y = r \sin A \quad \quad A = \arctan(y/x) \end{array}$$

which relate the real and imaginary parts (x , y) to the magnitude and the angle (r, A) . [38: The arctangent function referred to here, computed by Python's `math.atan2` function, is defined so as to take two arguments y and x and to return the angle whose tangent is y/x . The signs of the arguments determine the quadrant of the angle.] Ben's representation is therefore given by the following selectors and constructors:

```
def real_part(z): return head(z)

def imag_part(z): return tail(z)

def magnitude(z):
    return math_sqrt(square(real_part(z)) + square(imag_part(z)))
def angle(z):
    return math_atan2(imag_part(z), real_part(z))
def make_from_real_imag(x, y): return pair(x, y)

def make_from_mag_ang(r, a):
    return pair(r * math_cos(a), r * math_sin(a))
```

Alyssa, in contrast, chooses to represent complex numbers in polar form. For her, selecting the magnitude and angle is straightforward, but she has to use the trigonometric relations to obtain the real and imaginary parts. Alyssa's representation is:

```
def real_part(z):
    return magnitude(z) * math_cos(angle(z))
def imag_part(z):
    return magnitude(z) * math_sin(angle(z))
def magnitude(z): return head(z)

def angle(z): return tail(z)

def make_from_real_imag(x, y):
    return pair(math_sqrt(square(x) + square(y)),
                math_atan2(y, x))
def make_from_mag_ang(r, a): return pair(r, a)
```

The discipline of data abstraction ensures that the same implementation of `add_complex`, `sub_complex`, `mul_complex`, and `div_complex` will work with either Ben's representation or Alyssa's representation.

2.4.2 Tagged data

One way to view data abstraction is as an application of the "principle of least commitment." In implementing the complex-number system in section 2.4.1, we can use either Ben's rectangular representation or Alyssa's polar representation. The abstraction barrier formed by the selectors and constructors permits us to defer to the last possible moment the choice of a concrete representation for our data objects and thus retain maximum flexibility in our system design.

The principle of least commitment can be carried to even further extremes. If we desire, we can maintain the ambiguity of representation even *after* we have designed the selectors and constructors, and elect to use both Ben's representation *and* Alyssa's representation. If both representations are included in a single system, however, we will need some way to distinguish data in polar form from data in rectangular form. Otherwise, if we were asked, for instance, to find the `magnitude` of the pair `(3,4)`, we wouldn't know whether to answer 5 (interpreting the number in rectangular form) or 3 (interpreting the number in polar form). A straightforward way to accomplish this distinction is to include a *type tag*—the string `"rectangular"` or `"polar"`—as part of each complex number. Then when we need to manipulate a complex number we can use the tag to decide which selector to apply.

In order to manipulate tagged data, we will assume that we have functions `type_tag` and `contents` that extract from a data object the tag and the actual contents (the polar or rectangular coordinates, in the case of a complex number). We will also postulate a function `attach_tag` that takes a tag and contents and produces a tagged data object. A straightforward way to implement this is to use ordinary linked-list structure:

```
def attach_tag(type_tag, contents):
    return pair(type_tag, contents)
def type_tag(datum):
    return (head(datum)
            if is_pair(datum)
            else error("bad tagged datum -- type_tag", datum))
def contents(datum):
    return (tail(datum)
            if is_pair(datum)
            else error("bad tagged datum -- contents", datum))
```

Using `type_tag`, we can define predicates `is_rectangular` and `is_polar`, which recognize rectangular and polar numbers, respectively:

```
def is_rectangular(z):
    return type_tag(z) == "rectangular"
def is_polar(z):
    return type_tag(z) == "polar"
```

With type tags, Ben and Alyssa can now modify their code so that their two different representations can coexist in the same system. Whenever Ben constructs a complex number, he tags it as rectangular. Whenever Alyssa constructs a complex number, she tags it as polar. In addition, Ben and Alyssa must make sure that the names of their functions do not conflict. One way to do this is for Ben to append the suffix `rectangular` to the name of each of his representation functions and for Alyssa to append `polar` to the names of hers. Here is Ben's revised rectangular representation from section 2.4.1:

```
def real_part_rectangular(z): return head(z)

def imag_part_rectangular(z): return tail(z)

def magnitude_rectangular(z):
    return math_sqrt(square(real_part_rectangular(z)) +
                      square(imag_part_rectangular(z)))
def angle_rectangular(z):
    return math_atan2(imag_part_rectangular(z),
                     real_part_rectangular(z))
def make_from_real_imag_rectangular(x, y):
    return attach_tag("rectangular", pair(x, y))
def make_from_mag_ang_rectangular(r, a):
    return attach_tag("rectangular",
                     pair(r * math_cos(a), r * math_sin(a)))
```

and here is Alyssa's revised polar representation:

```
def real_part_polar(z):
    return magnitude_polar(z) * math_cos(angle_polar(z))
def imag_part_polar(z):
    return magnitude_polar(z) * math_sin(angle_polar(z))
def magnitude_polar(z): return head(z)

def angle_polar(z): return tail(z)

def make_from_real_imag_polar(x, y):
    return attach_tag("polar",
                     pair(math_sqrt(square(x) + square(y)),
                         math_atan2(y, x)))

def make_from_mag_ang_polar(r, a):
    return attach_tag("polar", pair(r, a))
```

Each generic selector is implemented as a function that checks the tag of its argument and calls the appropriate function for handling data of that type. For example, to obtain the real part of a complex number, `real_part` examines the tag to determine whether to use Ben's `real_part_rectangular` or Alyssa's `real_part_polar`. In either case, we use `contents` to extract the bare, untagged datum and send this to the rectangular or polar function as required:

```

def real_part(z):
    return (real_part_rectangular(contents(z))
            if is_rectangular(z)
            else real_part_polar(contents(z))
            if is_polar(z)
            else error("unknown type -- real_part", z))

def imag_part(z):
    return (imag_part_rectangular(contents(z))
            if is_rectangular(z)
            else imag_part_polar(contents(z))
            if is_polar(z)
            else error("unknown type -- imag_part", z))

def magnitude(z):
    return (magnitude_rectangular(contents(z))
            if is_rectangular(z)
            else magnitude_polar(contents(z))
            if is_polar(z)
            else error("unknown type -- magnitude", z))

def angle(z):
    return (angle_rectangular(contents(z))
            if is_rectangular(z)
            else angle_polar(contents(z))
            if is_polar(z)
            else error("unknown type -- angle", z))

```

To implement the complex-number arithmetic operations, we can use the same functions `add_complex`, `sub_complex`, `mul_complex`, and `div_complex` from section 2.4.1, because the selectors they call are generic, and so will work with either representation. For example, the function `add_complex` is still

```

def add_complex(z1, z2):
    return make_from_real_imag(real_part(z1) + real_part(z2),
                               imag_part(z1) + imag_part(z2))

```

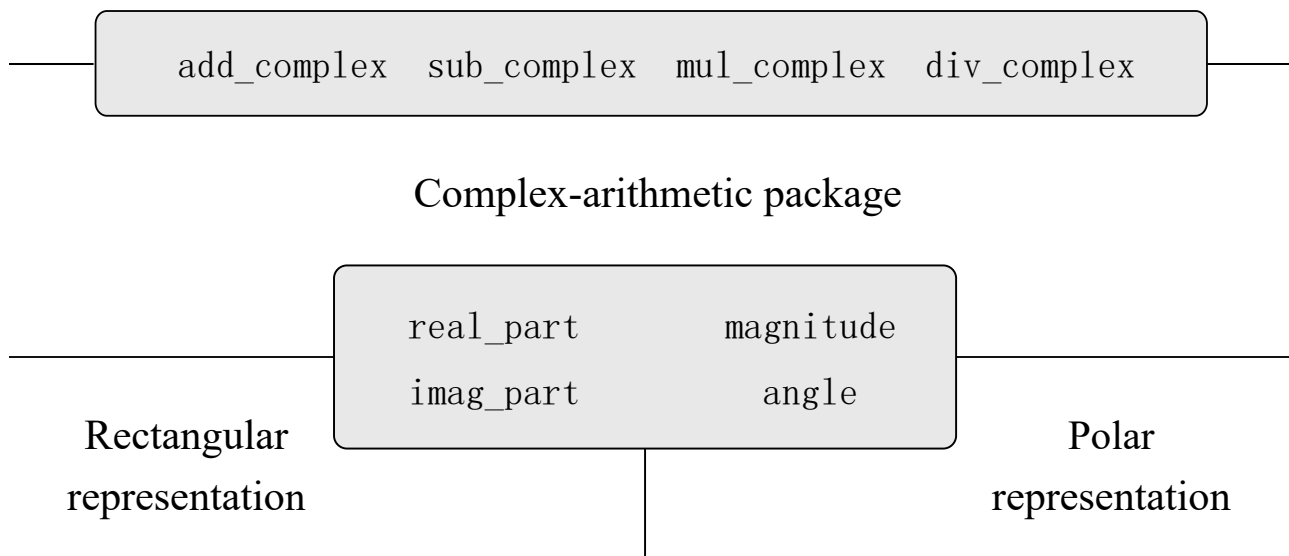
Finally, we must choose whether to construct complex numbers using Ben's representation or Alyssa's representation. One reasonable choice is to construct rectangular numbers whenever we have real and imaginary parts and to construct polar numbers whenever we have magnitudes and angles:

```

def make_from_real_imag(x, y):
    return make_from_real_imag_rectangular(x, y)
def make_from_mag_ang(r, a):
    return make_from_mag_ang_polar(r, a)

```


Programs that use complex numbers



List structure and primitive machine arithmetic

Figure 2.21. Structure of the generic complex-arithmetic system.

The resulting complex-number system has the structure shown in figure 2.21. The system has been decomposed into three relatively independent parts: the complex-number-arithmetic operations, Alyssa's polar implementation, and Ben's rectangular implementation. The polar and rectangular implementations could have been written by Ben and Alyssa working separately, and both of these can be used as underlying representations by a third programmer implementing the complex-arithmetic functions in terms of the abstract constructor-selector interface.

Since each data object is tagged with its type, the selectors operate on the data in a generic manner. That is, each selector is defined to have a behavior that depends upon the particular type of data it is applied to. Notice the general mechanism for interfacing the separate representations: Within a given representation implementation (say, Alyssa's polar package) a complex number is an untyped pair (magnitude, angle). When a generic selector operates on a number of `polar` type, it strips off the tag and passes the contents on to Alyssa's code. Conversely, when Alyssa constructs a number for general use, she tags it with a type so that it can be appropriately recognized by the higher-level functions. This discipline of stripping off and attaching tags as data objects are passed from level to level can be an important organizational strategy, as we shall see in section 2.5.

2.4.3 Data-Directed Programming and Additivity

The general strategy of checking the type of a datum and calling an appropriate function is called *dispatching on type*. This is a powerful strategy for obtaining modularity in system design. On the other hand, implementing the dispatch as in section 2.4.2 has two significant weaknesses. One

weakness is that the generic interface functions(`real_part` , `imag_part` , `magnitude` , and `angle`) must know about all the different representations. For instance, suppose we wanted to incorporate a new representation for complex numbers into our complex-number system. We would need to identify this new representation with a type, and then add a clause to each of the generic interface functions to check for the new type and apply the appropriate selector for that representation.

Another weakness of the technique is that even though the individual representations can be designed separately, we must guarantee that no two functions in the entire system have the same name. This is why Ben and Alyssa had to change the names of their original functions from section 2.4.1.

The issue underlying both of these weaknesses is that the technique for implementing generic interfaces is not *additive*. The person implementing the generic selector functions must modify those functions each time a new representation is installed, and the people interfacing the individual representations must modify their code to avoid name conflicts. In each of these cases, the changes that must be made to the code are straightforward, but they must be made nonetheless, and this is a source of inconvenience and error. This is not much of a problem for the complex-number system as it stands, but suppose there were not two but hundreds of different representations for complex numbers. And suppose that there were many generic selectors to be maintained in the abstract-data interface. Suppose, in fact, that no one programmer knew all the interface functions or all the representations. The problem is real and must be addressed in such programs as large-scale data-base-management systems.

What we need is a means for modularizing the system design even further. This is provided by the programming technique known as * data-directed programming*. To understand how data-directed programming works, begin with the observation that whenever we deal with a set of generic operations that are common to a set of different types we are, in effect, dealing with a two-dimensional table that contains the possible operations on one axis and the possible types on the other axis. The entries in the table are the functions that implement each operation for each type of argument presented. In the complex-number system developed in the previous section, the correspondence between operation name, data type, and actual function was spread out among the various conditional clauses in the generic interface functions. But the same information could have been organized in a table, as shown in figure 2.22.

Data-directed programming is the technique of designing programs to work with such a table directly. Previously, we implemented the mechanism that interfaces the complex-arithmetic code with the two representation packages as a set of functions that each perform an explicit dispatch on type. Here we will implement the interface as a single function that looks up the combination of the operation name and argument type in the table to find the correct function to apply, and then applies it to the contents of the argument. If we do this, then to add a new representation package to the system we need not change any existing functions; we need only add new entries to the table.

Operations	Types	
	Polar	Rectangular
real_part	real_part_polar	real_part_rectangular
imag_part	imag_part_polar	imag_part_rectangular
magnitude	magnitude_polar	magnitude_rectangular
angle	angle_polar	angle_rectangular

Figure 2.22. Table of operations for the complex-number system.

To implement this plan, assume that we have two functions, `put` and `get`, for manipulating the operation-and-type table:

- `put( op , type , item )` installs the *item* in the table, indexed by the *op* and the *type*.
- `get( op , type )` looks up the *op*, *type* entry in the table and returns the item found there. If no item is found, `get` returns a unique primitive value that is referred to by the keyword `None` and recognized by the built-in predicate `is_none`. For now, we can assume that `put` and `get` are included in our language. In chapter 3 (section 3.3.3) we will see how to implement these and other operations for manipulating tables.

Here is how data-directed programming can be used in the complex-number system. Ben, who developed the rectangular representation, implements his code just as he did originally. He defines a collection of functions or a *package*, and interfaces these to the rest of the system by adding entries to the table that tell the system how to operate on rectangular numbers. This is accomplished by calling the following function:

```

def install_rectangular_package():
    # internal functions
    def real_part(z): return head(z)
    def imag_part(z): return tail(z)
    def make_from_real_imag(x, y): return pair(x, y)
    def magnitude(z):
        return math_sqrt(square(real_part(z)) + square(imag_part(z)))
    def angle(z):
        return math_atan2(imag_part(z), real_part(z))
    def make_from_mag_ang(r, a):
        return pair(r * math_cos(a), r * math_sin(a))

    # interface to the rest of the system
    def tag(x): return attach_tag("rectangular", x)
    put("real_part", llist("rectangular"), real_part)
    put("imag_part", llist("rectangular"), imag_part)
    put("magnitude", llist("rectangular"), magnitude)
    put("angle", llist("rectangular"), angle)
    put("make_from_real_imag", "rectangular",
        lambda x, y: tag(make_from_real_imag(x, y)))
    put("make_from_mag_ang", "rectangular",
        lambda r, a: tag(make_from_mag_ang(r, a)))
    return "done"

```

Notice that the internal functions here are the same functions from section 2.4.1 that Ben wrote when he was working in isolation. No changes are necessary in order to interface them to the rest of the system. Moreover, since these function definitions are internal to the installation function, Ben needn't worry about name conflicts with other functions outside the rectangular package. To interface these to the rest of the system, Ben installs his `real_part` function under the operation name `real_part` and the type `llist("rectangular")`, and similarly for the other selectors. [³⁹: We use the linked list `llist("rectangular")` rather than the string `"rectangular"` to allow for the possibility of operations with multiple arguments, not all of the same type.] The interface also defines the constructors to be used by the external system. [⁴⁰: The type the constructors are installed under needn't be a linked list because a constructor is always used to make an object of one particular type.] These are identical to Ben's internally defined constructors, except that they attach the tag.

Alyssa's polar package is analogous:

```

def install_polar_package():
    # internal functions
    def magnitude(z): return head(z)
    def angle(z): return tail(z)
    def make_from_mag_ang(r, a): return pair(r, a)
    def real_part(z):
        return magnitude(z) * math_cos(angle(z))
    def imag_part(z):
        return magnitude(z) * math_sin(angle(z))
    def make_from_real_imag(x, y):
        return pair(math_sqrt(square(x) + square(y)),
                    math_atan2(y, x))

    # interface to the rest of the system
    def tag(x): return attach_tag("polar", x)
    put("real_part", llist("polar"), real_part)
    put("imag_part", llist("polar"), imag_part)
    put("magnitude", llist("polar"), magnitude)
    put("angle", llist("polar"), angle)
    put("make_from_real_imag", "polar",
        lambda x, y: tag(make_from_real_imag(x, y)))
    put("make_from_mag_ang", "polar",
        lambda r, a: tag(make_from_mag_ang(r, a)))
    return "done"

```

Even though Ben and Alyssa both still use their original functions defined with the same names as each other's (e.g., `real_part`), these declarations are now internal to different functions (see section), so there is no name conflict.

The complex-arithmetic selectors access the table by means of a general "operation" function called `apply_generic`, which applies a generic operation to some arguments. The function `apply_generic` looks in the table under the name of the operation and the types of the arguments and applies the resulting function if one is present: [^41: The function `apply_generic` uses the function `apply_in_underlying_javascript` given in section (footnote 2), which takes two arguments, a function and a linked list, and applies the function, using the elements in the linked list as arguments. For example, returns 10.]

```

apply_in_underlying_javascript(sum_of_squares, llist(1, 3))

```

```

def apply_generic(op, args):
    type_tags = map(type_tag, args)
    fun = get(op, type_tags)
    return (apply_in_underlying_javascript(fun, map(contents, args))
            if not is_none(fun)
            else error("no method for these types -- apply_generic",
                      llist(op, type_tags)))

```

Using `apply_generic`, we can define our generic selectors as follows:

```

def real_part(z): return apply_generic("real_part", llist(z))

def imag_part(z): return apply_generic("imag_part", llist(z))

def magnitude(z): return apply_generic("magnitude", llist(z))

def angle(z): return apply_generic("angle", llist(z))

```

Observe that these do not change at all if a new representation is added to the system.

We can also extract from the table the constructors to be used by the programs external to the packages in making complex numbers from real and imaginary parts and from magnitudes and angles. As in section 2.4.2, we construct rectangular numbers whenever we have real and imaginary parts, and polar numbers whenever we have magnitudes and angles:

```

def make_from_real_imag(x, y):
    return get("make_from_real_imag", "rectangular")(x, y)
def make_from_mag_ang(r, a):
    return get("make_from_mag_ang", "polar")(r, a)

```

Exercise 2.73. Section 2.3.2 described a program that performs symbolic differentiation:

```

def deriv(exp, variable):
    return (0
            if is_number(exp)
            else (1 if is_same_variable(exp, variable) else 0)
            if is_variable(exp)
            else make_sum(deriv(addend(exp), variable),
                          deriv(augend(exp), variable))
            if is_sum(exp)
            else make_sum(make_product(multiplier(exp),
                                       deriv(multiplicand(exp), variable)),
                          make_product(deriv(multiplier(exp), variable),
                                       multiplicand(exp)))
            if is_product(exp)
            # more rules can be added here
            else error("unknown expression type -- deriv", exp))

```

```

print(deriv(llist("*", llist("*", "x", "y"), llist("+", "x", 4)), "x"))

```

```

llist("+", llist("*", llist("*", "x", "y"), llist("+", 1, 0)),
      llist("*", llist("+", llist("*", "x", 0), llist("*", 1, "y")),
      llist("+", "x", 4)))

```

We can regard this program as performing a dispatch on the type of the expression to be differentiated. In this situation the "type tag" of the datum is the algebraic operator symbol (such

as "+") and the operation being performed is `deriv`. We can transform this program into data-directed style by rewriting the basic derivative function as

```
def deriv(exp, variable):
    return (0
            if is_number(exp)
            else (1 if is_same_variable(exp, variable) else 0)
            if is_variable(exp)
            else get("deriv", operator(exp))(operands(exp), variable))
def operator(exp): return head(exp)

def operands(exp): return tail(exp)
```

a. Explain what was done above. Why can't we assimilate the predicates `is_number` and `is_variable` into the data-directed dispatch? b. Write the functions for derivatives of sums and products, and the auxiliary code required to install them in the table used by the program above. c. Choose any additional differentiation rule that you like, such as the one for exponents (exercise 2.56), and install it in this data-directed system. d. In this simple algebraic manipulator the type of an expression is the algebraic operator that binds it together. Suppose, however, we indexed the functions in the opposite way, so that the dispatch line in `deriv` looked like

```
get(operator(exp), "deriv")(operands(exp), variable)
```

What corresponding changes to the derivative system are required?

Exercise 2.74. Insatiable Enterprises, Inc., is a highly decentralized conglomerate company consisting of a large number of independent divisions located all over the world. The company's computer facilities have just been interconnected by means of a clever network-interfacing scheme that makes the entire network appear to any user to be a single computer. Insatiable's president, in her first attempt to exploit the ability of the network to extract administrative information from division files, is dismayed to discover that, although all the division files have been implemented as data structures in Python, the particular data structure used varies from division to division. A meeting of division managers is hastily called to search for a strategy to integrate the files that will satisfy headquarters' needs while preserving the existing autonomy of the divisions.

Show how such a strategy can be implemented with data-directed programming. As an example, suppose that each division's personnel records consist of a single file, which contains a set of records keyed on employees' names. The structure of the set varies from division to division. Furthermore, each employee's record is itself a set (structured differently from division to division) that contains information keyed under identifiers such as `address` and `salary`. In particular: a. Implement for headquarters a `get_record` function that retrieves a specified employee's record from a specified personnel file. The function should be applicable to any division's file. Explain how the individual divisions' files should be structured. In particular, what type information must be supplied? b. Implement for headquarters a `get_salary` function that returns the salary information

from a given employee's record from any division's personnel file. How should the record be structured in order to make this operation work? c. Implement for headquarters a `find_employee_record` function. This should search all the divisions' files for the record of a given employee and return the record. Assume that this function takes as arguments an employee's name and a linked list of all the divisions' files. d. When Insatiable takes over a new company, what changes must be made in order to incorporate the new personnel information into the central system?

Message passing

The key idea of data-directed programming is to handle generic operations in programs by dealing explicitly with operation-and-type tables, such as the table in figure 2.22. The style of programming we used in section 2.4.2 organized the required dispatching on type by having each operation take care of its own dispatching. In effect, this decomposes the operation-and-type table into rows, with each generic operation function representing a row of the table.

An alternative implementation strategy is to decompose the table into columns and, instead of using "intelligent operations" that dispatch on data types, to work with "intelligent data objects" that dispatch on operation names. We can do this by arranging things so that a data object, such as a rectangular number, is represented as a function that takes as input the required operation name and performs the operation indicated. In such a discipline, `make_from_real_imag` could be written as

```
def make_from_real_imag(x, y):
    def dispatch(op):
        return (x
                if op == "real_part"
                else y
                if op == "imag_part"
                else math_sqrt(square(x) + square(y))
                if op == "magnitude"
                else math_atan2(y, x)
                if op == "angle"
                else error("unknown op -- make_from_real_imag", op))
    return dispatch
```

The corresponding `apply_generic` function, which applies a generic operation to an argument, now simply feeds the operation's name to the data object and lets the object do the work: [^42: One limitation of this organization is it permits only generic functions of one argument.]

```
def apply_generic(op, arg): return head(arg)(op)
```

Note that the value returned by `make_from_real_imag` is a function—the internal `dispatch` function. This is the function that is invoked when `apply_generic` requests an

operation to be performed.

This style of programming is called *message passing*. The name comes from the image that a data object is an entity that receives the requested operation name as a "message." We have already seen an example of message passing in section 2.1.3, where we saw how `pair`, `head`, and `tail` could be defined with no data objects but only functions. Here we see that message passing is not a mathematical trick but a useful technique for organizing systems with generic operations. In the remainder of this chapter we will continue to use data-directed programming, rather than message passing, to discuss generic arithmetic operations. In chapter 3 we will return to message passing, and we will see that it can be a powerful tool for structuring simulation programs.

Exercise 2.75. Implement the constructor `make_from_mag_ang` in message-passing style. This function should be analogous to the `make_from_real_imag` function given above.

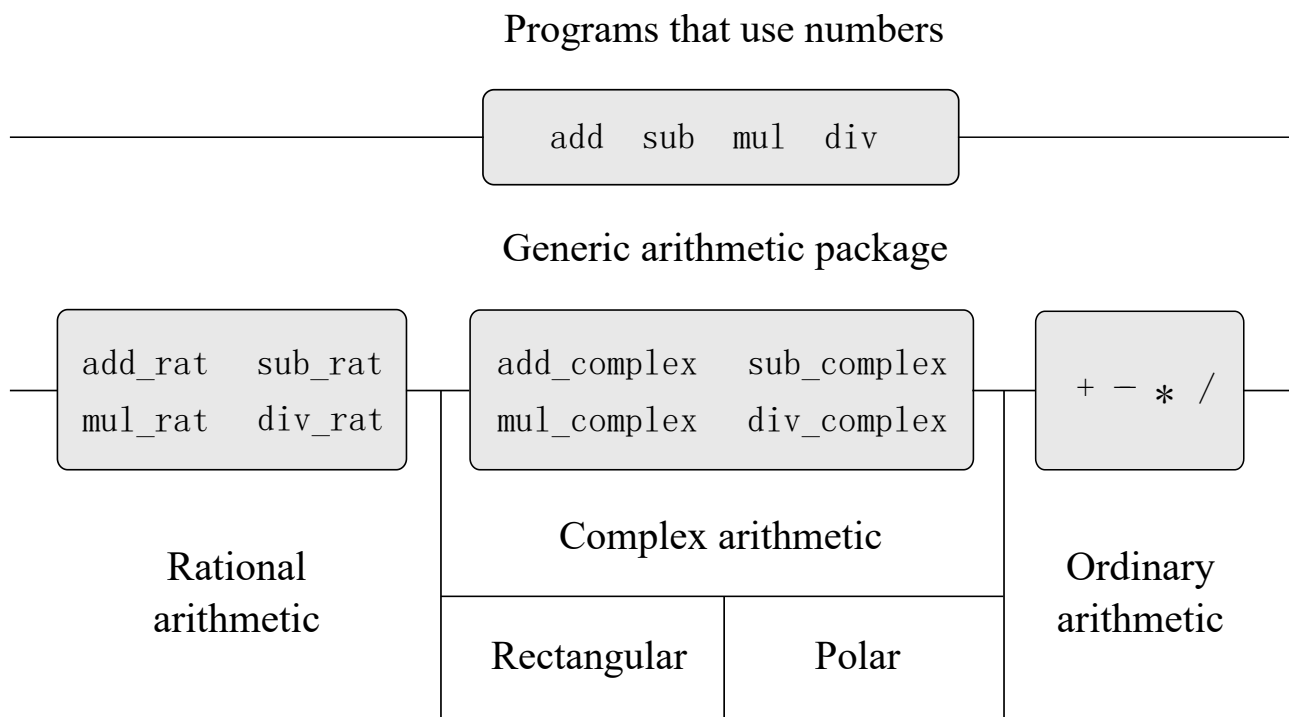
Exercise 2.76. As a large system with generic operations evolves, new types of data objects or new operations may be needed. For each of the three strategies—generic operations with explicit dispatch, data-directed style, and message-passing-style—describe the changes that must be made to a system in order to add new types or new operations. Which organization would be most appropriate for a system in which new types must often be added? Which would be most appropriate for a system in which new operations must often be added?

2.5 Systems with Generic Operations

In the previous section, we saw how to design systems in which data objects can be represented in more than one way. The key idea is to link the code that specifies the data operations to the several representations by means of generic interface functions. Now we will see how to use this same idea not only to define operations that are generic over different representations but also to define operations that are generic over different kinds of arguments. We have already seen several different packages of arithmetic operations: the primitive arithmetic (`+`, `-`, `*`, `/`) built into our language, the rational-number arithmetic (`add_rat`, `sub_rat`, `mul_rat`, `div_rat`) of section 2.1.1, and the complex-number arithmetic that we implemented in section 2.4.3. We will now use data-directed techniques to construct a package of arithmetic operations that incorporates all the arithmetic packages we have already constructed.

Figure 2.23 shows the structure of the system we shall build. Notice the abstraction barriers. From the perspective of someone using "numbers," there is a single function `add` that operates on whatever numbers are supplied. The function `add` is part of a generic interface that allows the separate ordinary-arithmetic, rational-arithmetic, and complex-arithmetic packages to be accessed uniformly by programs that use numbers. Any individual arithmetic package (such as the complex package) may itself be accessed through generic functions (such as `add_complex`) that combine packages designed for different representations (such as rectangular and polar). Moreover, the

structure of the system is additive, so that one can design the individual arithmetic packages separately and combine them to produce a generic arithmetic system.



List structure and primitive machine arithmetic

Figure 2.23. Generic arithmetic system.

2.5.1 Generic Arithmetic Operations

The task of designing generic arithmetic operations is analogous to that of designing the generic complex-number operations. We would like, for instance, to have a generic addition function `add` that acts like ordinary primitive addition `+` on ordinary numbers, like `add_rat` on rational numbers, and like `add_complex` on complex numbers. We can implement `add`, and the other generic arithmetic operations, by following the same strategy we used in section 2.4.3 to implement the generic selectors for complex numbers. We will attach a type tag to each kind of number and cause the generic function to dispatch to an appropriate package according to the data type of its arguments.

The generic arithmetic functions are defined as follows:

```
def add(x, y): return apply_generic("add", llist(x, y))

def sub(x, y): return apply_generic("sub", llist(x, y))

def mul(x, y): return apply_generic("mul", llist(x, y))

def div(x, y): return apply_generic("div", llist(x, y))
```

We begin by installing a package for handling *ordinary* numbers, that is, the primitive numbers of our language. We tag these with the string `"python_number"`. The arithmetic operations in this package are the primitive arithmetic functions (so there is no need to define extra functions to handle the untagged numbers). Since these operations each take two arguments, they are installed in the table keyed by the linked list `l1ist("python_number", "python_number")`:

```
def install_python_number_package():
    def tag(x):
        return attach_tag("python_number", x)
    put("add", l1ist("python_number", "python_number"),
        lambda x, y: tag(x + y))
    put("sub", l1ist("python_number", "python_number"),
        lambda x, y: tag(x - y))
    put("mul", l1ist("python_number", "python_number"),
        lambda x, y: tag(x * y))
    put("div", l1ist("python_number", "python_number"),
        lambda x, y: tag(x / y))
    put("make", "python_number",
        lambda x: tag(x))
    return "done"
```

Users of the Python-number package will create (tagged) ordinary numbers by means of the function:

```
def make_python_number(n):
    return get("make", "python_number")(n)
```

Now that the framework of the generic arithmetic system is in place, we can readily include new kinds of numbers. Here is a package that performs rational arithmetic. Notice that, as a benefit of additivity, we can use without modification the rational-number code from section 2.1.1 as the internal functions in the package:

```

def install_rational_package():
    # internal functions
    def numer(x): return head(x)
    def denom(x): return tail(x)
    def make_rat(n, d):
        g = gcd(n, d)
        return pair(n // g, d // g)
    def add_rat(x, y):
        return make_rat(numer(x) * denom(y) + numer(y) * denom(x),
                        denom(x) * denom(y))
    def sub_rat(x, y):
        return make_rat(numer(x) * denom(y) - numer(y) * denom(x),
                        denom(x) * denom(y))
    def mul_rat(x, y):
        return make_rat(numer(x) * numer(y),
                        denom(x) * denom(y))
    def div_rat(x, y):
        return make_rat(numer(x) * denom(y),
                        denom(x) * numer(y))
    # interface to rest of the system
    def tag(x):
        return attach_tag("rational", x)
    put("add", llist("rational", "rational"),
        lambda x, y: tag(add_rat(x, y)))
    put("sub", llist("rational", "rational"),
        lambda x, y: tag(sub_rat(x, y)))
    put("mul", llist("rational", "rational"),
        lambda x, y: tag(mul_rat(x, y)))
    put("div", llist("rational", "rational"),
        lambda x, y: tag(div_rat(x, y)))
    put("make", "rational",
        lambda n, d: tag(make_rat(n, d)))
    return "done"

def make_rational(n, d):
    return get("make", "rational")(n, d)

```

We can install a similar package to handle complex numbers, using the tag `"complex"`. In creating the package, we extract from the table the operations `make_from_real_imag` and `make_from_mag_ang` that were defined by the rectangular and polar packages. Additivity permits us to use, as the internal operations, the same `add_complex`, `sub_complex`, `mul_complex`, and `div_complex` functions from section 2.4.1.

```

def install_complex_package():
    # imported functions from rectangular and polar packages
    def make_from_real_imag(x, y):
        return get("make_from_real_imag", "rectangular")(x, y)
    def make_from_mag_ang(r, a):
        return get("make_from_mag_ang", "polar")(r, a)
    # internal functions
    def add_complex(z1, z2):
        return make_from_real_imag(real_part(z1) + real_part(z2),
                                   imag_part(z1) + imag_part(z2))
    def sub_complex(z1, z2):
        return make_from_real_imag(real_part(z1) - real_part(z2),
                                   imag_part(z1) - imag_part(z2))
    def mul_complex(z1, z2):
        return make_from_mag_ang(magnitude(z1) * magnitude(z2),
                                  angle(z1) + angle(z2))
    def div_complex(z1, z2):
        return make_from_mag_ang(magnitude(z1) / magnitude(z2),
                                  angle(z1) - angle(z2))
    # interface to rest of the system
    def tag(z): return attach_tag("complex", z)
    put("add", llist("complex", "complex"),
        lambda z1, z2: tag(add_complex(z1, z2)))
    put("sub", llist("complex", "complex"),
        lambda z1, z2: tag(sub_complex(z1, z2)))
    put("mul", llist("complex", "complex"),
        lambda z1, z2: tag(mul_complex(z1, z2)))
    put("div", llist("complex", "complex"),
        lambda z1, z2: tag(div_complex(z1, z2)))
    put("make_from_real_imag", "complex",
        lambda x, y: tag(make_from_real_imag(x, y)))
    put("make_from_mag_ang", "complex",
        lambda r, a: tag(make_from_mag_ang(r, a)))
    return "done"

```

Programs outside the complex-number package can construct complex numbers either from real and imaginary parts or from magnitudes and angles. Notice how the underlying functions, originally defined in the rectangular and polar packages, are exported to the complex package, and exported from there to the outside world.

```

def make_complex_from_real_imag(x, y):
    return get("make_from_real_imag", "complex")(x, y)
def make_complex_from_mag_ang(r, a):
    return get("make_from_mag_ang", "complex")(r, a)

```

What we have here is a two-level tag system. A typical complex number, such as $3+4i$ in rectangular form, would be represented as shown in figure 2.24. The outer tag ("complex") is used to direct the number to the complex package. Once within the complex package, the next tag ("rectangular") is used to direct the number to the rectangular package. In a large and complicated system there might be many levels, each interfaced with the next by means of generic

operations. As a data object is passed "downward," the outer tag that is used to direct it to the appropriate package is stripped off (by applying `contents`) and the next level of tag (if any) becomes visible to be used for further dispatching.

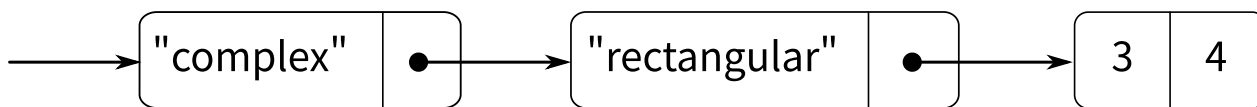


Figure 2.24. Representation of $3+4i$ in rectangular form.

In the above packages, we used `add_rat` , `add_complex` , and the other arithmetic functions exactly as originally written. Once these declarations are internal to different installation functions, however, they no longer need names that are distinct from each other: we could simply name them `add` , `sub` , `mul` , and `div` in both packages.

Exercise 2.77. Louis Reasoner tries to evaluate the expression `magnitude(z)` where `z` is the object shown in figure 2.24. To his surprise, instead of the answer `5` he gets an error message from `apply_generic` , saying there is no method for the operation `magnitude` on the types `l1list("complex")` . He shows this interaction to Alyssa P. Hacker, who says "The problem is that the complex-number selectors were never defined for `"complex"` numbers, just for `"polar"` and `"rectangular"` numbers. All you have to do to make this work is add the following to the `complex` package:"

```

put("real_part", l1list("complex"), real_part)
put("imag_part", l1list("complex"), imag_part)
put("magnitude", l1list("complex"), magnitude)
print(put("angle", l1list("complex"), angle))

```

Describe in detail why this works. As an example, trace through all the functions called in evaluating the expression `magnitude(z)` where `z` is the object shown in figure 2.24. In particular, how many times is `apply_generic` invoked? What function is dispatched to in each case?

Exercise 2.78. The internal functions in the `python_number` package are essentially nothing more than calls to the primitive functions `+` , `-` , etc. It was not possible to use the primitives of the language directly because our type-tag system requires that each data object have a type attached to it. In fact, however, all Python implementations do have a type system, which they use internally. Primitive predicates such as `is_string` and `is_number` determine whether data objects have particular types. Modify the definitions of `type_tag` , `contents` , and `attach_tag` from section 2.4.2 so that our generic system takes advantage of Python's internal type system. That is to say, the system should work as before except that ordinary numbers should be represented simply as Python numbers rather than as pairs whose `head` is the string `"python_number"` .

Exercise 2.79. Define a generic equality predicate `is_equal` that tests the equality of two numbers, and install it in the generic arithmetic package. This operation should work for ordinary numbers, rational numbers, and complex numbers.

Exercise 2.80. Define a generic predicate `is_equal_to_zero` that tests if its argument is zero, and install it in the generic arithmetic package. This operation should work for ordinary numbers, rational numbers, and complex numbers.

2.5.2 Combining Data of Different Types

We have seen how to define a unified arithmetic system that encompasses ordinary numbers, complex numbers, rational numbers, and any other type of number we might decide to invent, but we have ignored an important issue. The operations we have defined so far treat the different data types as being completely independent. Thus, there are separate packages for adding, say, two ordinary numbers, or two complex numbers. What we have not yet considered is the fact that it is meaningful to define operations that cross the type boundaries, such as the addition of a complex number to an ordinary number. We have gone to great pains to introduce barriers between parts of our programs so that they can be developed and understood separately. We would like to introduce the cross-type operations in some carefully controlled way, so that we can support them without seriously violating our module boundaries.

One way to handle cross-type operations is to design a different function for each possible combination of types for which the operation is valid. For example, we could extend the complex-number package so that it provides a function for adding complex numbers to ordinary numbers and installs this in the table using the tag `l1ist("complex", "python_number")` : [^43: We also have to supply an almost identical function to handle the types `l1ist("python_number", "complex")` .]

```
# to be included in the complex package
def add_complex_to_python_num(z, x):
    return make_complex_from_real_imag(real_part(z) + x, imag_part(z))
print(put("add", l1ist("complex", "python_number"),
    lambda z, x: tag(add_complex_to_python_num(z, x))))
```

This technique works, but it is cumbersome. With such a system, the cost of introducing a new type is not just the construction of the package of functions for that type but also the construction and installation of the functions that implement the cross-type operations. This can easily be much more code than is needed to define the operations on the type itself. The method also undermines our ability to combine separate packages additively, or least to limit the extent to which the implementors of the individual packages need to take account of other packages. For instance, in the example above, it seems reasonable that handling mixed operations on complex numbers and ordinary numbers should be the responsibility of the complex-number package. Combining rational numbers and complex numbers, however, might be done by the complex package, by the rational package, or by some third package that uses operations extracted from these two packages. Formulating coherent policies on the division of responsibility among packages can be an overwhelming task in designing systems with many packages and many cross-type operations.

Coercion

In the general situation of completely unrelated operations acting on completely unrelated types, implementing explicit cross-type operations, cumbersome though it may be, is the best that one can hope for. Fortunately, we can usually do better by taking advantage of additional structure that may be latent in our type system. Often the different data types are not completely independent, and there may be ways by which objects of one type may be viewed as being of another type. This process is called *coercion*. For example, if we are asked to arithmetically combine an ordinary number with a complex number, we can view the ordinary number as a complex number whose imaginary part is zero. This transforms the problem to that of combining two complex numbers, which can be handled in the ordinary way by the complex-arithmetic package.

In general, we can implement this idea by designing coercion functions that transform an object of one type into an equivalent object of another type. Here is a typical coercion function, which transforms a given ordinary number to a complex number with that real part and zero imaginary part:

```
def python_number_to_complex(n):  
    return make_complex_from_real_imag(contents(n), 0)
```

We install these coercion functions in a special coercion table, indexed under the names of the two types:

```
put_coercion("python_number", "complex",  
            python_number_to_complex)
```

(We assume that there are `put_coercion` and `get_coercion` functions available for manipulating this table.) Generally some of the slots in the table will be empty, because it is not generally possible to coerce an arbitrary data object of each type into all other types. For example, there is no way to coerce an arbitrary complex number to an ordinary number, so there will be no general `complex_to_python_number` function included in the table.

Once the coercion table has been set up, we can handle coercion in a uniform manner by modifying the `apply_generic` function of section 2.4.3. When asked to apply an operation, we first check whether the operation is defined for the arguments' types, just as before. If so, we dispatch to the function found in the operation-and-type table. Otherwise, we try coercion. For simplicity, we consider only the case where there are two arguments. [⁴⁴: See exercise 2.82 for generalizations.] We check the coercion table to see if objects of the first type can be coerced to the second type. If so, we coerce the first argument and try the operation again. If objects of the first type cannot in general be coerced to the second type, we try the coercion the other way around to see if there is a way to coerce the second argument to the type of the first argument.

Finally, if there is no known way to coerce either type to the other type, we give up. Here is the function:

```
def apply_generic(op, args):
    type_tags = map(type_tag, args)
    fun = get(op, type_tags)
    if not is_none(fun):
        return apply(fun, map(contents, args))
    else:
        if length(args) == 2:
            type1 = head(type_tags)
            type2 = head(tail(type_tags))
            a1 = head(args)
            a2 = head(tail(args))
            t1_to_t2 = get_coercion(type1, type2)
            t2_to_t1 = get_coercion(type2, type1)
            return (apply_generic(op, llist(t1_to_t2(a1), a2))
                    if not is_none(t1_to_t2)
                    else apply_generic(op, llist(a1, t2_to_t1(a2)))
                    if not is_none(t2_to_t1)
                    else error("no method for these types",
                               llist(op, type_tags)))
        else:
            return error("no method for these types",
                          llist(op, type_tags))
```

This coercion scheme has many advantages over the method of defining explicit cross-type operations, as outlined above. Although we still need to write coercion functions to relate the types (possibly n^2 functions for a system with n types), we need to write only one function for each pair of types rather than a different function for each collection of types and each generic operation. [45: If we are clever, we can usually get by with fewer than n^2 coercion functions. For instance, if we know how to convert from type 1 to type 2 and from type 2 to type 3, then we can use this knowledge to convert from type 1 to type 3. This can greatly decrease the number of coercion functions we need to supply explicitly when we add a new type to the system. If we are willing to build the required amount of sophistication into our system, we can have it search the "graph" of relations among types and automatically generate those coercion functions that can be inferred from the ones that are supplied explicitly.] What we are counting on here is the fact that the appropriate transformation between types depends only on the types themselves, not on the operation to be applied.

On the other hand, there may be applications for which our coercion scheme is not general enough. Even when neither of the objects to be combined can be converted to the type of the other it may still be possible to perform the operation by converting both objects to a third type. In order to deal with such complexity and still preserve modularity in our programs, it is usually necessary to build systems that take advantage of still further structure in the relations among types, as we discuss next.

Hierarchies of types

The coercion scheme presented above relied on the existence of natural relations between pairs of types. Often there is more "global" structure in how the different types relate to each other. For instance, suppose we are building a generic arithmetic system to handle integers, rational numbers, real numbers, and complex numbers. In such a system, it is quite natural to regard an integer as a special kind of rational number, which is in turn a special kind of real number, which is in turn a special kind of complex number. What we actually have is a so-called *hierarchy of types*, in which, for example, integers are a *subtype* of rational numbers (i.e., any operation that can be applied to a rational number can automatically be applied to an integer). Conversely, we say that rational numbers form a *supertype* of integers. The particular hierarchy we have here is of a very simple kind, in which each type has at most one supertype and at most one subtype. Such a structure, called a *tower*, is illustrated in figure 2.25.

complex



real



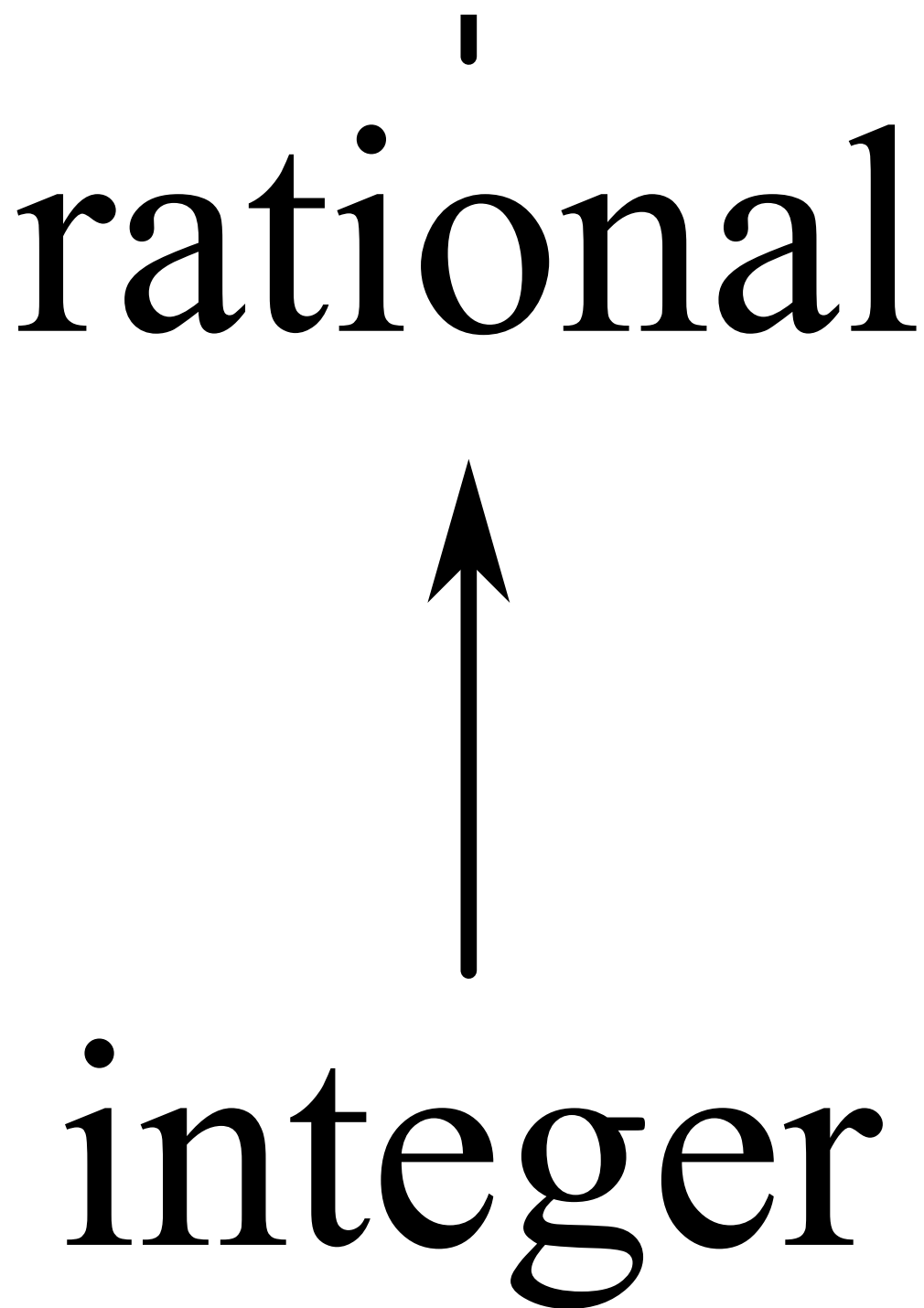


Figure 2.25. A tower of types.

If we have a tower structure, then we can greatly simplify the problem of adding a new type to the hierarchy, for we need only specify how the new type is embedded in the next supertype above it and how it is the supertype of the type below it. For example, if we want to add an integer to a complex number, we need not explicitly define a special coercion function `integer_to_complex`. Instead, we define how an integer can be transformed into a rational number, how a rational number is transformed into a real number, and how a real number is transformed into a complex

number. We then allow the system to transform the integer into a complex number through these steps and then add the two complex numbers.

We can redesign our `apply_generic` function in the following way: For each type, we need to supply a `promote` function, which "raises" objects of that type one level in the tower. Then when the system is required to operate on objects of different types it can successively raise the lower types until all the objects are at the same level in the tower. (Exercises 2.83 and 2.84 concern the details of implementing such a strategy.)

Another advantage of a tower is that we can easily implement the notion that every type "inherits" all operations defined on a supertype. For instance, if we do not supply a special function for finding the real part of an integer, we should nevertheless expect that `real_part` will be defined for integers by virtue of the fact that integers are a subtype of complex numbers. In a tower, we can arrange for this to happen in a uniform way by modifying `apply_generic`. If the required operation is not directly defined for the type of the object given, we raise the object to its supertype and try again. We thus crawl up the tower, transforming our argument as we go, until we either find a level at which the desired operation can be performed or hit the top (in which case we give up).

Yet another advantage of a tower over a more general hierarchy is that it gives us a simple way to "lower" a data object to the simplest representation. For example, if we add $2+3i$ to $4-3i$, it would be nice to obtain the answer as the integer 6 rather than as the complex number $6+0i$. Exercise 2.85 discusses a way to implement such a lowering operation. (The trick is that we need a general way to distinguish those objects that can be lowered, such as $6+0i$, from those that cannot, such as $6+2i$.)

Inadequacies of hierarchies

If the data types in our system can be naturally arranged in a tower, this greatly simplifies the problems of dealing with generic operations on different types, as we have seen. Unfortunately, this is usually not the case. Figure 2.26 illustrates a more complex arrangement of mixed types, this one showing relations among different types of geometric figures. We see that, in general, a type may have more than one subtype. Triangles and quadrilaterals, for instance, are both subtypes of polygons. In addition, a type may have more than one supertype. For example, an isosceles right triangle may be regarded either as an isosceles triangle or as a right triangle. This multiple-supertypes issue is particularly thorny, since it means that there is no unique way to "raise" a type in the hierarchy. Finding the "correct" supertype in which to apply an operation to an object may involve considerable searching through the entire type network on the part of a function such as `apply_generic`. Since there generally are multiple subtypes for a type, there is a similar problem in coercing a value "down" the type hierarchy. Dealing with large numbers of interrelated types while still preserving modularity in the design of large systems is very difficult, and is an area of much current research. [^46: This statement, which also appears in the first edition of this book, is just as true now as it was when we wrote it in 1984. Developing a useful, general framework for expressing the relations among different types of entities (what philosophers call "ontology") seems intractably difficult. The main difference between the confusion that existed in 1984 and the confusion that exists now is that now a variety of inadequate ontological theories have been embodied in a plethora of correspondingly inadequate programming languages. For example, much of the complexity of object-oriented programming languages—and the subtle and confusing differences among contemporary object-oriented languages—centers on the treatment of generic operations on interrelated types. Our own discussion of computational objects in chapter 3 avoids these issues entirely. Readers familiar with object-oriented programming will notice that we have much to say in chapter 3 about local state, but we do not even mention "classes" or "inheritance." In fact, we suspect that these problems cannot be adequately addressed in terms of computer-language design alone, without also drawing on work in knowledge representation and automated reasoning.]

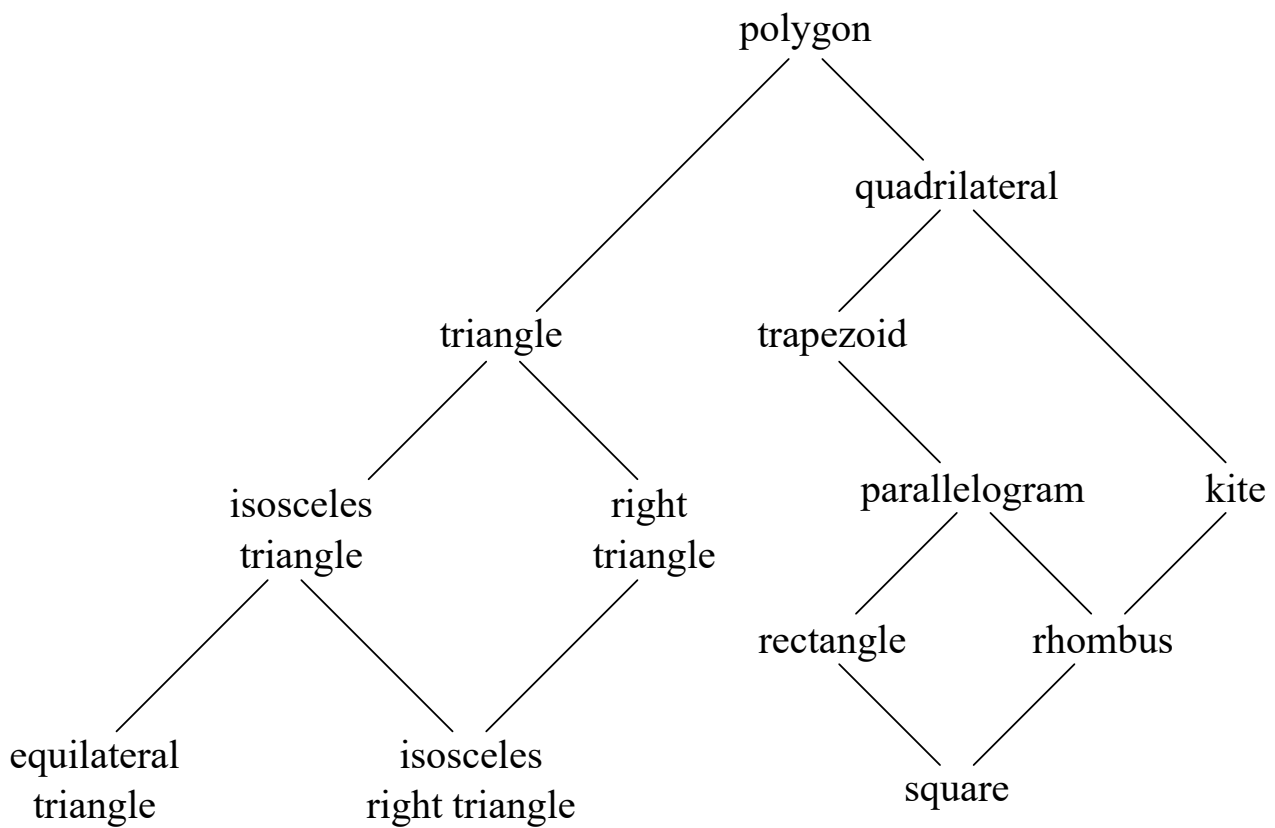


Figure 2.26. Relations among types of geometric figures.

Exercise 2.81. Louis Reasoner has noticed that `apply_generic` may try to coerce the arguments to each other's type even if they already have the same type. Therefore, he reasons, we need to put functions in the coercion table to "coerce" arguments of each type to their own type. For example, in addition to the `python_number_to_complex` coercion shown above, he would do:

```

def python_number_to_python_number(n): return n

def complex_to_complex(n): return n

put_coercion("python_number", "python_number",
             python_number_to_python_number)
put_coercion("complex", "complex", complex_to_complex)

```

a. With Louis's coercion functions installed, what happens if `apply_generic` is called with two arguments of type `"complex"` or two arguments of type `"python_number"` for an operation that is not found in the table for those types? For example, assume that we've defined a generic exponentiation operation:

```

def exp(x, y):
    return apply_generic("exp", llist(x, y))

```

and have put a function for exponentiation in the Python-number package but not in any other package:

```
# following added to Python-number package
put("exp", llist("python_number", "python_number"),
    lambda x, y: tag(math_pow(x, y))) # using primitive math\char`_pow
```

What happens if we call `exp` with two complex numbers as arguments? b. Is Louis correct that something had to be done about coercion with arguments of the same type, or does `apply_generic` work correctly as is? c. Modify `apply_generic` so that it doesn't try coercion if the two arguments have the same type.

Exercise 2.82. Show how to generalize `apply_generic` to handle coercion in the general case of multiple arguments. One strategy is to attempt to coerce all the arguments to the type of the first argument, then to the type of the second argument, and so on. Give an example of a situation where this strategy (and likewise the two-argument version given above) is not sufficiently general. (Hint: Consider the case where there are some suitable mixed-type operations present in the table that will not be tried.)

Exercise 2.83. Suppose you are designing a generic arithmetic system for dealing with the tower of types shown in figure 2.25: integer, rational, real, complex. For each type (except complex), design a function that raises objects of that type one level in the tower. Show how to install a generic `promote` operation that will work for each type (except complex).

Exercise 2.84. Using the `promote` operation of exercise 2.83, modify the `apply_generic` function so that it coerces its arguments to have the same type by the method of successive raising, as discussed in this section. You will need to devise a way to test which of two types is higher in the tower. Do this in a manner that is "compatible" with the rest of the system and will not lead to problems in adding new levels to the tower.

Exercise 2.85. This section mentioned a method for "simplifying" a data object by lowering it in the tower of types as far as possible. Design a function `drop` that accomplishes this for the tower described in exercise 2.83. The key is to decide, in some general way, whether an object can be lowered. For example, the complex number $1.5+0i$ can be lowered as far as `"real"`, the complex number $1+0i$ can be lowered as far as `"integer"`, and the complex number $2+3i$ cannot be lowered at all. Here is a plan for determining whether an object can be lowered: Begin by defining a generic operation `project` that "pushes" an object down in the tower. For example, projecting a complex number would involve throwing away the imaginary part. Then a number can be dropped if, when we `project` it and `promote` the result back to the type we started with, we end up with something equal to what we started with. Show how to implement this idea in detail, by writing a `drop` function that drops an object as far as possible. You will need to design the various projection operations [^47: A real number can be projected to an integer using the `math_round` primitive, which returns the closest integer to its argument.] and install `project` as a generic operation in the system. You will also need to make use of a generic equality predicate,

such as described in exercise 2.79. Finally, use `drop` to rewrite `apply_generic` from exercise 2.84 so that it "simplifies" its answers.

Exercise 2.86. Suppose we want to handle complex numbers whose real parts, imaginary parts, magnitudes, and angles can be either ordinary numbers, rational numbers, or other numbers we might wish to add to the system. Describe and implement the changes to the system needed to accommodate this. You will have to define operations such as `sine` and `cosine` that are generic over ordinary numbers and rational numbers.

2.5.3 Example: Symbolic Algebra

The manipulation of symbolic algebraic expressions is a complex process that illustrates many of the hardest problems that occur in the design of large-scale systems. An algebraic expression, in general, can be viewed as a hierarchical structure, a tree of operators applied to operands. We can construct algebraic expressions by starting with a set of primitive objects, such as constants and variables, and combining these by means of algebraic operators, such as addition and multiplication. As in other languages, we form abstractions that enable us to refer to compound objects in simple terms. Typical abstractions in symbolic algebra are ideas such as linear combination, polynomial, rational function, or trigonometric function. We can regard these as compound "types," which are often useful for directing the processing of expressions. For example, we could describe the expression

$x^2, \sin(y^2+1)+x, \cos 2y+\cos(y^3-2y^2)$

as a polynomial in x with coefficients that are trigonometric functions of polynomials in y whose coefficients are integers.

We will not attempt to develop a complete algebraic-manipulation system here. Such systems are exceedingly complex programs, embodying deep algebraic knowledge and elegant algorithms. What we will do is look at a simple but important part of algebraic manipulation: the arithmetic of polynomials. We will illustrate the kinds of decisions the designer of such a system faces, and how to apply the ideas of abstract data and generic operations to help organize this effort.

Arithmetic on polynomials

Our first task in designing a system for performing arithmetic on polynomials is to decide just what a polynomial is. Polynomials are normally defined relative to certain variables (the *indeterminates* of the polynomial). For simplicity, we will restrict ourselves to polynomials having just one indeterminate (*univariate polynomials*). [⁴⁸ On the other hand, we will allow polynomials whose coefficients are themselves polynomials in other variables. This will give us essentially the same representational power as a full multivariate system, although it does lead to coercion problems, as discussed below.] We will define a polynomial to be a sum of terms, each of which is either a coefficient, a power of the indeterminate, or a product of a coefficient and a power of the

indeterminate. A coefficient is defined as an algebraic expression that is not dependent upon the indeterminate of the polynomial. For example,

$$5x^2 + 3x + 7$$

is a simple polynomial in x , and

$$(y^2 + 1)x^3 + (2y)x + 1$$

is a polynomial in x whose coefficients are polynomials in y .

Already we are skirting some thorny issues. Is the first of these polynomials the same as the polynomial $5y^2 + 3y + 7$, or not? A reasonable answer might be "yes, if we are considering a polynomial purely as a mathematical function, but no, if we are considering a polynomial to be a syntactic form." The second polynomial is algebraically equivalent to a polynomial in y whose coefficients are polynomials in x . Should our system recognize this, or not? Furthermore, there are other ways to represent a polynomial—for example, as a product of factors, or (for a univariate polynomial) as the set of roots, or as a listing of the values of the polynomial at a specified set of points. [49: For univariate polynomials, giving the value of a polynomial at a given set of points can be a particularly good representation. This makes polynomial arithmetic extremely simple. To obtain, for example, the sum of two polynomials represented in this way, we need only add the values of the polynomials at corresponding points. To transform back to a more familiar representation, we can use the Lagrange interpolation formula, which shows how to recover the coefficients of a polynomial of degree n given the values of the polynomial at $n+1$ points.] We can finesse these questions by deciding that in our algebraic-manipulation system a "polynomial" will be a particular syntactic form, not its underlying mathematical meaning.

Now we must consider how to go about doing arithmetic on polynomials. In this simple system, we will consider only addition and multiplication. Moreover, we will insist that two polynomials to be combined must have the same indeterminate.

We will approach the design of our system by following the familiar discipline of data abstraction. We will represent polynomials using a data structure called a *poly*, which consists of a variable and a collection of terms. We assume that we have selectors `variable` and `term_list` that extract those parts from a poly and a constructor `make_poly` that assembles a poly from a given variable and a term list. A variable will be just a string, so we can use the `is_same_variable` function of section 2.3.2 to compare variables. The following functions define addition and multiplication of polys:

```

def add_poly(p1, p2):
    return (make_poly(variable(p1),
                        add_terms(term_list(p1), term_list(p2)))
            if is_same_variable(variable(p1), variable(p2))
            else error("polys not in same var -- add_poly", llist(p1, p2)))

def mul_poly(p1, p2):
    return (make_poly(variable(p1),
                        mul_terms(term_list(p1), term_list(p2)))
            if is_same_variable(variable(p1), variable(p2))
            else error("polys not in same var -- mul_poly", llist(p1, p2)))

```

To incorporate polynomials into our generic arithmetic system, we need to supply them with type tags. We'll use the tag "polynomial", and install appropriate operations on tagged polynomials in the operation table. We'll embed all our code in an installation function for the polynomial package, similar to the installation functions in section 2.5.1:

```

def install_polynomial_package():
    # internal functions
    # representation of poly
    def make_poly(variable, term_list):
        return pair(variable, term_list)
    def variable(p): return head(p)
    def term_list(p): return tail(p)
    functions is_same_variable and is_variable from section 2.3.2

    # representation of terms and term lists
    functions adjoin_term...coeff from text below

    def add_poly(p1, p2): ...
    functions used by add_poly
    def mul_poly(p1, p2): ...
    functions used by mul_poly

    # interface to rest of the system
    def tag(p): return attach_tag("polynomial", p)
    put("add", llist("polynomial", "polynomial"),
        lambda p1, p2: tag(add_poly(p1, p2)))
    put("mul", llist("polynomial", "polynomial"),
        lambda p1, p2: tag(mul_poly(p1, p2)))
    put("make", "polynomial",
        lambda variable, terms: tag(make_poly(variable, terms)))
    return "done"

```

Polynomial addition is performed termwise. Terms of the same order (i.e., with the same power of the indeterminate) must be combined. This is done by forming a new term of the same order whose coefficient is the sum of the coefficients of the addends. Terms in one addend for which there are no terms of the same order in the other addend are simply accumulated into the sum polynomial being constructed.

In order to manipulate term lists, we will assume that we have a constructor `the_empty_term` that returns an empty term list and a constructor `adjoin_term` that adjoins a new term to a term list. We will also assume that we have a predicate `is_empty_term` that tells if a given term list is empty, a selector `first_term` that extracts the highest-order term from a term list, and a selector `rest_terms` that returns all but the highest-order term. To manipulate terms, we will suppose that we have a constructor `make_term` that constructs a term with given order and coefficient, and selectors `order` and `coeff` that return, respectively, the order and the coefficient of the term. These operations allow us to consider both terms and term lists as data abstractions, whose concrete representations we can worry about separately.

Here is the function that constructs the term list for the sum of two polynomials: [^50: This operation is very much like the ordered `union_set` operation we developed in exercise 2.62. In fact, if we think of the terms of the polynomial as a set ordered according to the power of the indeterminate, then the program that produces the term list for a sum is almost identical to `union_set` .]

```
def add_terms(L1, L2):
    if is_empty_term(L1):
        return L2
    elif is_empty_term(L2):
        return L1
    else:
        t1 = first_term(L1)
        t2 = first_term(L2)
        return (adjoin_term(t1, add_terms(rest_terms(L1), L2))
                if order(t1) > order(t2)
                else adjoin_term(t2, add_terms(L1, rest_terms(L2)))
                if order(t1) < order(t2)
                else adjoin_term(make_term(order(t1),
                                           add(coeff(t1), coeff(t2))),
                                add_terms(rest_terms(L1),
                                           rest_terms(L2))))
```

The most important point to note here is that we used the generic addition function `add` to add together the coefficients of the terms being combined. This has powerful consequences, as we will see below.

In order to multiply two term lists, we multiply each term of the first linked list by all the terms of the other linked list, repeatedly using `mul_term_by_all_terms` , which multiplies a given term by all terms in a given term list. The resulting term lists (one for each term of the first list) are accumulated into a sum. Multiplying two terms forms a term whose order is the sum of the orders of the factors and whose coefficient is the product of the coefficients of the factors:

```

def mul_terms(L1, L2):
    return (the_empty_termlist
            if is_empty_termlist(L1)
            else add_terms(mul_term_by_all_terms(
                           first_term(L1), L2),
                           mul_terms(rest_terms(L1), L2)))
def mul_term_by_all_terms(t1, L):
    if is_empty_termlist(L):
        return the_empty_termlist
    else:
        t2 = first_term(L)
        return adjoin_term(
            make_term(order(t1) + order(t2),
                      mul(coeff(t1), coeff(t2))),
            mul_term_by_all_terms(t1, rest_terms(L)))

```

This is really all there is to polynomial addition and multiplication. Notice that, since we operate on terms using the generic functions `add` and `mul`, our polynomial package is automatically able to handle any type of coefficient that is known about by the generic arithmetic package. If we include a coercion mechanism such as one of those discussed in section 2.5.2, then we also are automatically able to handle operations on polynomials of different coefficient types, such as

$$\left(3x^2 + (2+3i)x + 7\right) \cdot \left(x^4 + \frac{2}{3}x^2 + (5+3i)\right)$$

Because we installed the polynomial addition and multiplication functions `add_poly` and `mul_poly` in the generic arithmetic system as the `add` and `mul` operations for type `polynomial`, our system is also automatically able to handle polynomial operations such as

$$\left((y+1)x^2 + (y^2 + 1)x + (y-1)\right) \cdot \left((y-2)x + (y^3 + 7)\right)$$

The reason is that when the system tries to combine coefficients, it will dispatch through `add` and `mul`. Since the coefficients are themselves polynomials (in y), these will be combined using `add_poly` and `mul_poly`. The result is a kind of "data-directed recursion" in which, for example, a call to `mul_poly` will result in recursive calls to `mul_poly` in order to multiply the coefficients. If the coefficients of the coefficients were themselves polynomials (as might be used to represent polynomials in three variables), the data direction would ensure that the system would follow through another level of recursive calls, and so on through as many levels as the structure of the data dictates. [51: To make this work completely smoothly, we should also add to our generic arithmetic system the ability to coerce a "number" to a polynomial by regarding it as a polynomial of degree zero whose coefficient is the number. This is necessary if we are going to perform operations such as $(x^2 + (y+1)x + 5) + (x^2 + 2x + 1)$ which requires adding the coefficient $y+1$ to the coefficient 2.]

Representing term lists

Finally, we must confront the job of implementing a good representation for term lists. A term list is, in effect, a set of coefficients keyed by the order of the term. Hence, any of the methods for representing sets, as discussed in section 2.3.3, can be applied to this task. On the other hand, our functions `add_terms` and `mul_terms` always access term lists sequentially from highest to lowest order. Thus, we will use some kind of ordered linked-list representation.

How should we structure the list that represents a term list? One consideration is the "density" of the polynomials we intend to manipulate. A polynomial is said to be *dense* if it has nonzero coefficients in terms of most orders. If it has many zero terms it is said to be *sparse*. For example,

$A: x^5 + 2x^4 + 3x^2 - 2x - 5$

is a dense polynomial, whereas

$B: x^{100} + 2x^2 + 1$

is sparse.

The term list of a dense polynomial is most efficiently represented as a linked list of the coefficients. For example, the polynomial A above would be nicely represented as `l1ist(1, 2, 0, 3, -2, -5)`. The order of a term in this representation is the length of the sublist beginning with that term's coefficient, decremented by 1. [⁵²: In these polynomial examples, we assume that we have implemented the generic arithmetic system using the type mechanism suggested in exercise 2.78. Thus, coefficients that are ordinary numbers will be represented as the numbers themselves rather than as pairs whose `head` is the string "python_number".] This would be a terrible representation for a sparse polynomial such as B : There would be a giant linked list of zeros punctuated by a few lonely nonzero terms. A more reasonable representation of the term list of a sparse polynomial is as a linked list of the nonzero terms, where each term is a linked list containing the order of the term and the coefficient for that order. In such a scheme, polynomial B is efficiently represented as `l1ist(l1ist(100, 1), l1ist(2, 2), l1ist(0, 1))`. As most polynomial manipulations are performed on sparse polynomials, we will use this method. We will assume that term lists are represented as linked lists of terms, arranged from highest-order to lowest-order term. Once we have made this decision, implementing the selectors and constructors for terms and term lists is straightforward: [⁵³: Although we are assuming that term lists are ordered, we have implemented `adjoin_term` to simply adjoin the new term to the front of the existing term list. We can get away with this so long as we guarantee that the functions (such as `add_terms`) that use `adjoin_term` always call it with a higher-order term than appears in the linked list. If we did not want to make such a guarantee, we could have implemented `adjoin_term` to be similar to the `adjoin_set` constructor for the ordered linked-list representation of sets (exercise 2.61).]

```

def adjoin_term(term, term_list):
    return (term_list
            if is_equal_to_zero(coeff(term))
            else pair(term, term_list))

the_empty_termlist = None

def first_term(term_list): return head(term_list)

def rest_terms(term_list): return tail(term_list)

def is_empty_termlist(term_list): return is_none(term_list)

def make_term(order, coeff): return llist(order, coeff)

def order(term): return head(term)

def coeff(term): return head(tail(term))

```

where `is_equal_to_zero` is as defined in exercise 2.80. (See also exercise 2.87 below.)

Users of the polynomial package will create (tagged) polynomials by means of the function:

```

def make_polynomial(variable, terms):
    return get("make", "polynomial")(variable, terms)

```

Exercise 2.87. Install `is_equal_to_zero` for polynomials in the generic arithmetic package. This will allow `adjoin_term` to work for polynomials with coefficients that are themselves polynomials.

Exercise 2.88. Extend the polynomial system to include subtraction of polynomials. (Hint: You may find it helpful to define a generic negation operation.)

Exercise 2.89. Declare functions that implement the term-list representation described above as appropriate for dense polynomials.

Exercise 2.90. Suppose we want to have a polynomial system that is efficient for both sparse and dense polynomials. One way to do this is to allow both kinds of term-list representations in our system. The situation is analogous to the complex-number example of section 2.4, where we allowed both rectangular and polar representations. To do this we must distinguish different types of term lists and make the operations on term lists generic. Redesign the polynomial system to implement this generalization. This is a major effort, not a local change.

Exercise 2.91. A univariate polynomial can be divided by another one to produce a polynomial quotient and a polynomial remainder. For example,

$$\frac{x^5 - 1}{x^2 - 1} = x^3 + x + \text{remainder } x - 1$$

Division can be performed via long division. That is, divide the highest-order term of the dividend by the highest-order term of the divisor. The result is the first term of the quotient. Next, multiply the result by the divisor, subtract that from the dividend, and produce the rest of the answer by recursively dividing the difference by the divisor. Stop when the order of the divisor exceeds the order of the dividend and declare the dividend to be the remainder. Also, if the dividend ever becomes zero, return zero as both quotient and remainder.

We can design a `div_poly` function on the model of `add_poly` and `mul_poly`. The function checks to see if the two polys have the same variable. If so, `div_poly` strips off the variable and passes the problem to `div_terms`, which performs the division operation on term lists. The function `div_poly` finally reattaches the variable to the result supplied by `div_terms`. It is convenient to design `div_terms` to compute both the quotient and the remainder of a division. The function `div_terms` can take two term lists as arguments and return a linked list of the quotient term list and the remainder term list.

Complete the following definition of `div_terms` by filling in the missing parts. Use this to implement `div_poly`, which takes two polys as arguments and returns a linked list of the quotient and remainder polys.

```
def div_terms(L1, L2):
    if is_empty_termlist(L1):
        return llist(the_empty_termlist, the_empty_termlist)
    else:
        t1 = first_term(L1)
        t2 = first_term(L2)
        if order(t2) > order(t1):
            return llist(the_empty_termlist, L1)
        else:
            new_c = div(coeff(t1), coeff(t2))
            new_o = order(t1) - order(t2)
            rest_of_result = compute rest of result recursively
            form and return complete result
```

Hierarchies of types in symbolic algebra

Our polynomial system illustrates how objects of one type (polynomials) may in fact be complex objects that have objects of many different types as parts. This poses no real difficulty in defining generic operations. We need only install appropriate generic operations for performing the necessary manipulations of the parts of the compound types. In fact, we saw that polynomials form a kind of "recursive data abstraction," in that parts of a polynomial may themselves be polynomials. Our generic operations and our data-directed programming style can handle this complication without much trouble.

On the other hand, polynomial algebra is a system for which the data types cannot be naturally arranged in a tower. For instance, it is possible to have polynomials in x whose coefficients are

polynomials in y . It is also possible to have polynomials in y whose coefficients are polynomials in x . Neither of these types is "above" the other in any natural way, yet it is often necessary to add together elements from each set. There are several ways to do this. One possibility is to convert one polynomial to the type of the other by expanding and rearranging terms so that both polynomials have the same principal variable. One can impose a towerlike structure on this by ordering the variables and thus always converting any polynomial to a "canonical form" with the highest-priority variable dominant and the lower-priority variables buried in the coefficients. This strategy works fairly well, except that the conversion may expand a polynomial unnecessarily, making it hard to read and perhaps less efficient to work with. The tower strategy is certainly not natural for this domain or for any domain where the user can invent new types dynamically using old types in various combining forms, such as trigonometric functions, power series, and integrals.

It should not be surprising that controlling coercion is a serious problem in the design of large-scale algebraic-manipulation systems. Much of the complexity of such systems is concerned with relationships among diverse types. Indeed, it is fair to say that we do not yet completely understand coercion. In fact, we do not yet completely understand the concept of a data type. Nevertheless, what we know provides us with powerful structuring and modularity principles to support the design of large systems.

Exercise 2.92. By imposing an ordering on variables, extend the polynomial package so that addition and multiplication of polynomials works for polynomials in different variables. (This is not easy!)

Extended exercise: Rational functions

We can extend our generic arithmetic system to include *rational functions*. These are "fractions" whose numerator and denominator are polynomials, such as

$$\frac{x+1}{x^3-1}$$

The system should be able to add, subtract, multiply, and divide rational functions, and to perform such computations as

$$\frac{x+1}{x^3-1} + \frac{x}{x^2-1} = \frac{x^3+2x^2+3x+1}{x^4+x^3-x-1}$$

(Here the sum has been simplified by removing common factors. Ordinary "cross multiplication" would have produced a fourth-degree polynomial over a fifth-degree polynomial.)

If we modify our rational-arithmetic package so that it uses generic operations, then it will do what we want, except for the problem of reducing fractions to lowest terms.

Exercise 2.93. Modify the rational-arithmetic package to use generic operations, but change `make_rat` so that it does not attempt to reduce fractions to lowest terms. Test your system by calling `make_rational` on two polynomials to produce a rational function

```
p1 = make_polynomial("x", llist(make_term(2, 1), make_term(0, 1)))
p2 = make_polynomial("x", llist(make_term(3, 1), make_term(0, 1)))
rf = make_rational(p2, p1)
```

Now add `rf` to itself, using `add`. You will observe that this addition function does not reduce fractions to lowest terms.

We can reduce polynomial fractions to lowest terms using the same idea we used with integers: modifying `make_rat` to divide both the numerator and the denominator by their greatest common divisor. The notion of "greatest common divisor" makes sense for polynomials. In fact, we can compute the GCD of two polynomials using essentially the same Euclid's Algorithm that works for integers. [^54: The fact that Euclid's Algorithm works for polynomials is formalized in algebra by saying that polynomials form a kind of algebraic domain called a *Euclidean ring*. A Euclidean ring is a domain that admits addition, subtraction, and commutative multiplication, together with a way of assigning to each element x of the ring a positive integer "measure" $m(x)$ with the properties that $m(xy) \geq m(x)$ for any nonzero x and y and that, given any x and y , there exists a q such that $y = qx + r$ and either $r = 0$ or $m(r) < m(x)$. From an abstract point of view, this is what is needed to prove that Euclid's Algorithm works. For the domain of integers, the measure m of an integer is the absolute value of the integer itself. For the domain of polynomials, the measure of a polynomial is its degree.] The integer version is

```
def gcd(a, b):
    return (a
            if b == 0
            else gcd(b, a % b))
```

Using this, we could make the obvious modification to define a GCD operation that works on term lists:

```
def gcd_terms(a, b):
    return (a
            if is_empty_termlist(b)
            else gcd_terms(b, remainder_terms(a, b)))
```

where `remainder_terms` picks out the remainder component of the list returned by the term-list division operation `div_terms` that was implemented in exercise 2.91.

Exercise 2.94. Using `div_terms`, implement the function `remainder_terms` and use this to define `gcd_terms` as above. Now write a function `gcd_poly` that computes the polynomial GCD of two

polys. (The function should signal an error if the two polys are not in the same variable.) Install in the system a generic operation `greatest_common_divisor` that reduces to `gcd_poly` for polynomials and to ordinary `gcd` for ordinary numbers. As a test, try

```
p1 = make_polynomial("x", llist(make_term(4, 1), make_term(3, -1),
                                make_term(2, -2), make_term(1, 2)))
p2 = make_polynomial("x", llist(make_term(3, 1), make_term(1, -1)))
greatest_common_divisor(p1, p2)
```

and check your result by hand.

Exercise 2.95. Define P_1 , P_2 , and P_3 to be the polynomials

$P_1: | x^2 - 2x + 1 |$ --- $P_2: | 11x^2 + 7 |$ $P_3: | 13x + 5 |$

Now define Q_1 to be the product of P_1 and P_2 and Q_2 to be the product of P_1 and P_3 , and use `greatest_common_divisor` (exercise 2.94) to compute the GCD of Q_1 and Q_2 . Note that the answer is not the same as P_1 . This example introduces noninteger operations into the computation, causing difficulties with the GCD algorithm. [55: In Python, division of integers can produce limited-precision decimal numbers, and thus we may fail to get a valid divisor.] To understand what is happening, try tracing `gcd_terms` while computing the GCD or try performing the division by hand.

We can solve the problem exhibited in exercise 2.95 if we use the following modification of the GCD algorithm (which really works only in the case of polynomials with integer coefficients). Before performing any polynomial division in the GCD computation, we multiply the dividend by an integer constant factor, chosen to guarantee that no fractions will arise during the division process. Our answer will thus differ from the actual GCD by an integer constant factor, but this does not matter in the case of reducing rational functions to lowest terms; the GCD will be used to divide both the numerator and denominator, so the integer constant factor will cancel out.

More precisely, if P and Q are polynomials, let O_1 be the order of P (i.e., the order of the largest term of P) and let O_2 be the order of Q . Let c be the leading coefficient of Q . Then it can be shown that, if we multiply P by the *integerizing factor* $c^{1+O_1-O_2}$, the resulting polynomial can be divided by Q by using the `div_terms` algorithm without introducing any fractions. The operation of multiplying the dividend by this constant and then dividing is sometimes called the *pseudodivision* of P by Q . The remainder of the division is called the *pseudoremainder*.

Exercise 2.96. a. Implement the function `pseudoremainder_terms`, which is just like `remainder_terms` except that it multiplies the dividend by the integerizing factor described above before calling `div_terms`. Modify `gcd_terms` to use `pseudoremainder_terms`, and verify that `greatest_common_divisor` now produces an answer with integer coefficients on the example in exercise 2.95. b. The GCD now has integer coefficients, but they are larger than those of P_1 .

Modify `gcd_terms` so that it removes common factors from the coefficients of the answer by dividing all the coefficients by their (integer) greatest common divisor.

Thus, here is how to reduce a rational function to lowest terms:

- Compute the GCD of the numerator and denominator, using the version of `gcd_terms` from exercise 2.96.
- When you obtain the GCD, multiply both numerator and denominator by the same integerizing factor before dividing through by the GCD, so that division by the GCD will not introduce any noninteger coefficients. As the factor you can use the leading coefficient of the GCD raised to the power $1 + O_1 - O_2$, where O_2 is the order of the GCD and O_1 is the maximum of the orders of the numerator and denominator. This will ensure that dividing the numerator and denominator by the GCD will not introduce any fractions.
- The result of this operation will be a numerator and denominator with integer coefficients. The coefficients will normally be very large because of all of the integerizing factors, so the last step is to remove the redundant factors by computing the (integer) greatest common divisor of all the coefficients of the numerator and the denominator and dividing through by this factor.

Exercise 2.97. a. Implement this algorithm as a function `reduce_terms` that takes two term lists `n` and `d` as arguments and returns a linked list `nn`, `dd`, which are `n` and `d` reduced to lowest terms via the algorithm given above. Also write a function `reduce_poly`, analogous to `add_poly`, that checks to see if the two polys have the same variable. If so, `reduce_poly` strips off the variable and passes the problem to `reduce_terms`, then reattaches the variable to the two term lists supplied by `reduce_terms`. b. Define a function analogous to `reduce_terms` that does what the original `make_rat` did for integers:

```
def reduce_integers(n, d):
    g = gcd(n, d)
    return llist(n // g, d // g)
```

and define `reduce` as a generic operation that calls `apply_generic` to dispatch either to `reduce_poly` (for `polynomial` arguments) or to `reduce_integers` (for `python_number` arguments). You can now easily make the rational-arithmetic package reduce fractions to lowest terms by having `make_rat` call `reduce` before combining the given numerator and denominator to form a rational number. The system now handles rational expressions in either integers or polynomials. To test your program, try the example at the beginning of this extended exercise:

```

p1 = make_polynomial("x", llist(make_term(1, 1), make_term(0, 1)))
p2 = make_polynomial("x", llist(make_term(3, 1), make_term(0, -1)))
p3 = make_polynomial("x", llist(make_term(1, 1)))
p4 = make_polynomial("x", llist(make_term(2, 1), make_term(0, -1)))

rf1 = make_rational(p1, p2)
rf2 = make_rational(p3, p4)

add(rf1, rf2)

```

See if you get the correct answer, correctly reduced to lowest terms.

The GCD computation is at the heart of any system that does operations on rational functions. The algorithm used above, although mathematically straightforward, is extremely slow. The slowness is due partly to the large number of division operations and partly to the enormous size of the intermediate coefficients generated by the pseudodivisions. One of the active areas in the development of algebraic-manipulation systems is the design of better algorithms for computing polynomial GCDs. [^56: One extremely efficient and elegant method for computing polynomial GCDs was discovered by Richard Zippel (1979). The method is a probabilistic algorithm, as is the fast test for primality that we discussed in chapter 1. Zippel's book (1993) describes this method, together with other ways to compute polynomial GCDs.]

3 Modularity, Objects, and State

α ϵ τ α β α λ λ ν
 α π α ϵ τ α ι (Even while it changes, it stands still.) — Heraclitus

Plus ça change, plus c'est la même chose. — Alphonse Karr

The preceding chapters introduced the basic elements from which programs are made. We saw how primitive functions and primitive data are combined to construct compound entities, and we learned that abstraction is vital in helping us to cope with the complexity of large systems. But these tools are not sufficient for designing programs. Effective program synthesis also requires organizational principles that can guide us in formulating the overall design of a program. In particular, we need strategies to help us structure large systems so that they will be *modular*, that is, so that they can be divided "naturally" into coherent parts that can be separately developed and maintained.

One powerful design strategy, which is particularly appropriate to the construction of programs for modeling physical systems, is to base the structure of our programs on the structure of the system being modeled. For each object in the system, we construct a corresponding computational object. For each system action, we define a symbolic operation in our computational model. Our hope in using this strategy is that extending the model to accommodate new objects or new actions will require no strategic changes to the program, only the addition of the new symbolic analogs of those objects or actions. If we have been successful in our system organization, then to add a new feature or debug an old one we will have to work on only a localized part of the system.

To a large extent, then, the way we organize a large program is dictated by our perception of the system to be modeled. In this chapter we will investigate two prominent organizational strategies arising from two rather different "world views" of the structure of systems. The first organizational strategy concentrates on *objects*, viewing a large system as a collection of distinct objects whose behaviors may change over time. An alternative organizational strategy concentrates on the *streams* of information that flow in the system, much as an electrical engineer views a signal-processing system.

Both the object-based approach and the stream-processing approach raise significant linguistic issues in programming. With objects, we must be concerned with how a computational object can change and yet maintain its identity. This will force us to abandon our old substitution model of computation (section 1.1.5) in favor of a more mechanistic but less theoretically tractable

environment model of computation. The difficulties of dealing with objects, change, and identity are a fundamental consequence of the need to grapple with time in our computational models. These difficulties become even greater when we allow the possibility of concurrent execution of programs. The stream approach can be most fully exploited when we decouple simulated time in our model from the order of the events that take place in the computer during evaluation. We will accomplish this using a technique known as *delayed evaluation*.

3.1 Assignment and Local State

We ordinarily view the world as populated by independent objects, each of which has a state that changes over time. An object is said to "have state" if its behavior is influenced by its history. A bank account, for example, has state in that the answer to the question "Can I withdraw \$100?" depends upon the history of deposit and withdrawal transactions. We can characterize an object's state by one or more *state variables*, which among them maintain enough information about history to determine the object's current behavior. In a simple banking system, we could characterize the state of an account by a current balance rather than by remembering the entire history of account transactions.

In a system composed of many objects, the objects are rarely completely independent. Each may influence the states of others through interactions, which serve to couple the state variables of one object to those of other objects. Indeed, the view that a system is composed of separate objects is most useful when the state variables of the system can be grouped into closely coupled subsystems that are only loosely coupled to other subsystems.

This view of a system can be a powerful framework for organizing computational models of the system. For such a model to be modular, it should be decomposed into computational objects that model the actual objects in the system. Each computational object must have its own *local state variables* describing the actual object's state. Since the states of objects in the system being modeled change over time, the state variables of the corresponding computational objects must also change. If we choose to model the flow of time in the system by the elapsed time in the computer, then we must have a way to construct computational objects whose behaviors change as our programs run. In particular, if we wish to model state variables by ordinary symbolic names in the programming language, then the language must provide an *reassignment operation* to enable us to change the value associated with a name.

3.1.1 Local State Variables

To illustrate what we mean by having a computational object with time-varying state, let us model the situation of withdrawing money from a bank account. We will do this using a function `withdraw`, which takes as argument an `amount` to be withdrawn. If there is enough money in the account to accommodate the withdrawal, then `withdraw` should return the balance

remaining after the withdrawal. Otherwise, `withdraw` should return the message *Insufficient funds*. For example, if we begin with \$100 in the account, we should obtain the following sequence of responses using `withdraw` :

```
print(withdraw(25))
```

75

```
print(withdraw(25))
```

50

```
print(withdraw(60))
```

"Insufficient funds"

```
print(withdraw(15))
```

35

Observe that the expression `withdraw(25)` , evaluated twice, yields different values. This is a new kind of behavior for a function. Until now, all our Python functions could be viewed as specifications for computing mathematical functions. A call to a function computed the value of the function applied to the given arguments, and two calls to the same function with the same arguments always produced the same result. [^1: Actually, this is not quite true. One exception was the random-number generator in section 1.2.6. Another exception involved the operation/type tables we introduced in section 2.4.3, where the values of two calls to `get` with the same arguments depended on intervening calls to `put` . On the other hand, until we introduce reassignment, we have no way to create such functions ourselves.]

To implement `withdraw` , we can use a variable `balance` to indicate the balance of money in the account and define `withdraw` as a function that accesses `balance` . The `withdraw` function checks to see if `balance` is at least as large as the requested `amount` . If so, `withdraw` decrements `balance` by `amount` and returns the new value of `balance` . Otherwise, `withdraw` returns the *Insufficient funds* message. Here are the declarations of `balance` and `withdraw` :


```

balance = 100

def withdraw(amount):
    global balance
    if balance >= amount:
        balance = balance - amount
        return balance
    else:
        return "Insufficient funds"

```

Decrementing `balance` is accomplished by the reassignment

```
balance = balance - amount
```

This statement looks like a declaration assignment, but it does not declare a new variable. The global declaration

```
global balance
```

ensures that the name `balance` is already declared in the body of the function `withdraw` and refers to the globally declared variable `balance`. An assignment of the form

```
*name* = *new-value*
```

where the *name* is already declared, is called a *reassignment*. The reassignment changes *name* so that its value is the result obtained by evaluating *new-value*. In the case at hand, we are changing `balance` so that its new value will be the result of subtracting `amount` from the previous value of `balance`. [²: Reassignments and declaration assignments look similar to and should not be confused with expressions of the form which evaluate to `True` if *expression*₁ evaluates to the same value as *expression*₂ and to `False` otherwise.]

```
*expression_1* == *expression_2*
```

The function `withdraw` also uses a *sequence of statements* to cause two statements to be evaluated in the case where the `if` test is true: first decrementing `balance` and then returning the value of `balance`. In general, executing a sequence

*stmt*₁ *stmt*₂ ... *stmt*_n

causes the statements *stmt*₁ through *stmt*_n to be evaluated in sequence. [³: We have already used sequences implicitly in our programs, because in Python the body block of a function can contain a sequence of function definitions followed by a return statement, not just a single return statement, as discussed in section .]

Although `withdraw` works as desired, the variable `balance` presents a problem. As specified above, `balance` is a name defined in the global environment and is freely accessible to be examined or modified by any function. It would be much better if we could somehow make `balance` internal to `withdraw`, so that `withdraw` would be the only function that could access `balance` directly and any other function could access `balance` only indirectly (through calls to `withdraw`). This would more accurately model the notion that `balance` is a local state variable used by `withdraw` to keep track of the state of the account.

We can make `balance` internal to `withdraw` by rewriting the definition as follows:

```
def make_withdraw_balance_100():
    balance = 100
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    return withdraw

new_withdraw = make_withdraw_balance_100()
```

What we have done here is use a declaration assignment to establish an environment with a local variable `balance`, bound to the initial value 100. Within this local environment, we use a function declaration to create a function `withdraw` that takes `amount` as an argument and—returned as the result of evaluating the body of the `make_withdraw_balance_100` function—behaves in precisely the same way as our previous `withdraw` function, but its variable `balance` is not accessible by any other function. [^4: In programming-language jargon, the variable `balance` is said to be *encapsulated* within the `new_withdraw` function. Encapsulation reflects the general system-design principle known as the *hiding principle*: One can make a system more modular and robust by protecting parts of the system from each other; that is, by providing information access only to those parts of the system that have a "need to know."]

The nested `withdraw` function faces a similar problem as the previous global `withdraw` function: The reassignment

```
balance = balance - amount
```

should refer to the variable `balance` declared outside of the `withdraw` function, and should not redeclare the name `balance`. However, now the name `balance` is not a global name but a name that is declared in the surrounding function `make_withdraw_balance_100` and thus a global declaration is not appropriate. Python uses `nonlocal` declarations in this situation. The declaration `nonlocal balance` in the nested `withdraw` function indicates that any reassignment to the name

`balance` inside the `withdraw` function refers to the variable `balance` declared outside the function but not globally. [⁵: Similar to global declarations, a nonlocal declaration does not extend to functions that are nested inside the function in which the nonlocal declaration appears. For these nested functions to reassign the nonlocal variable, they also need to declare the variable as nonlocal.]

Combining reassignments with local variables is the general programming technique we will use for constructing computational objects with local state. Unfortunately, using this technique raises a serious problem: When we first introduced functions, we also introduced the substitution model of evaluation (section 1.1.5) to provide an interpretation of what function application means. We said that applying a function whose body is a return statement should be interpreted as evaluating the return expression of the function with the parameters replaced by their values. For functions with more complex bodies, we need to evaluate the whole body with the parameters replaced by their values. The trouble is that, as soon as we introduce reassignment into our language, substitution is no longer an adequate model of function application. (We will see why this is so in section 3.1.3.) As a consequence, we technically have at this point no way to understand why the `new_withdraw` function behaves as claimed above. In order to really understand a function such as `new_withdraw`, we will need to develop a new model of function application. In section 3.2 we will introduce such a model, together with an explanation of reassignments and local variables. First, however, we examine some variations on the theme established by `new_withdraw`.

The following function, `make_withdraw`, creates "withdrawal processors." The parameter `balance` in `make_withdraw` specifies the initial amount of money in the account. [⁶: In contrast with `make_withdraw_balance_100` above, we do not have to use a declaration assignment to make `balance` a local variable, since parameters are already local. This will be clearer after the discussion of the environment model of evaluation in section 3.2. (See also exercise 3.10.)]

```
def make_withdraw(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    return withdraw
```

The function `make_withdraw` can be used as follows to create two objects `w1` and `w2`:

```
w1 = make_withdraw(100)
w2 = make_withdraw(100)
```

```
print(w1(50))
```

50

```
print(w2(70))
```

30

```
print(w2(40))
```

"Insufficient funds"

```
print(w1(40))
```

10

Observe that `w1` and `w2` are completely independent objects, each with its own local state variable `balance`. Withdrawals from one do not affect the other.

We can also create objects that handle deposits as well as withdrawals, and thus we can represent simple bank accounts. Here is a function that returns a "bank-account object" with a specified initial balance:

```
def make_account(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    def dispatch(m):
        return (withdraw if m == "withdraw"
                else deposit if m == "deposit"
                else error("unknown request -- make_account", m))
    return dispatch
```

Each call to `make_account` sets up an environment with a local state variable `balance`. Within this environment, `make_account` defines functions `deposit` and `withdraw` that access `balance` and an additional function `dispatch` that takes a "message" as input and returns one of the two local functions. The `dispatch` function itself is returned as the value that represents the bank-account

object. This is precisely the *message-passing* style of programming that we saw in section 2.4.3, although here we are using it in conjunction with the ability to modify local variables.

The function `make_account` can be used as follows:

```
acc = make_account(100)
```

```
print(acc("withdraw")(50))
```

```
50
```

```
print(acc("withdraw")(60))
```

```
"Insufficient funds"
```

```
print(acc("deposit")(40))
```

```
90
```

```
print(acc("withdraw")(60))
```

```
30
```

Each call to `acc` returns the locally defined `deposit` or `withdraw` function, which is then applied to the specified `amount`. As was the case with `make_withdraw`, another call to `make_account`

```
acc2 = make_account(100)
```

will produce a completely separate account object, which maintains its own local `balance`.

Exercise 3.1. An *accumulator* is a function that is called repeatedly with a single numeric argument and accumulates its arguments into a sum. Each time it is called, it returns the currently accumulated sum. Write a function `make_accumulator` that generates accumulators, each maintaining an independent sum. The input to `make_accumulator` should specify the initial value of the sum; for example

```
a = make_accumulator(5)
```

```
print(a(10))
```

```
15
```

```
print(a(10))
```

```
25
```

Exercise 3.2. In software-testing applications, it is useful to be able to count the number of times a given function is called during the course of a computation. Write a function `make_monitored` that takes as input a function, `f`, that itself takes one input. The result returned by `make_monitored` is a third function, say `mf`, that keeps track of the number of times it has been called by maintaining an internal counter. If the input to `mf` is the string `"how many calls"`, then `mf` returns the value of the counter. If the input is the string `"reset count"`, then `mf` resets the counter to zero. For any other input, `mf` returns the result of calling `f` on that input and increments the counter. For instance, we could make a monitored version of the `sqrt` function:

```
s = make_monitored(math_sqrt)
```

```
print(s(100))
```

```
10
```

```
print(s("how many calls"))
```

```
1
```

Exercise 3.3. Modify the `make_account` function so that it creates password-protected accounts. That is, `make_account` should take a string as an additional argument, as in

```
acc = make_account(100, "secret password")
```

The resulting account object should process a request only if it is accompanied by the password with which the account was created, and should otherwise return a complaint:

```
print(acc("secret password", "withdraw")(40))
```

```
60
```

```
print(acc("some other password", "deposit")(40))
```

```
"Incorrect password"
```

Exercise 3.4. Modify the `make_account` function of exercise 3.3 by adding another local state variable so that, if an account is accessed more than seven consecutive times with an incorrect password, it invokes the function `call_the_cops`.

3.1.2 The Benefits of Introducing Reassignment

As we shall see, introducing reassignment into our programming language leads us into a thicket of difficult conceptual issues. Nevertheless, viewing systems as collections of objects with local state is a powerful technique for maintaining a modular design. As a simple example, consider the design of a function `rand` that, whenever it is called, returns an integer chosen at random.

It is not at all clear what is meant by "chosen at random." What we presumably want is for successive calls to `rand` to produce a sequence of numbers that has statistical properties of uniform distribution. We will not discuss methods for generating suitable sequences here. Rather, let us assume that we have a function `rand_update` that has the property that if we start with a given number x_1 and form

$$x_2 = \text{rand_update}(x_1) \quad x_3 = \text{rand_update}(x_2)$$

then the sequence of values $x_1, x_2, x_3, \dots$, will have the desired statistical properties. [^7: One common way to implement `rand_update` is to use the rule that x is updated to $ax + b$ modulo m , where a , b , and m are appropriately chosen integers. Chapter 3 of Knuth 1997b includes an extensive discussion of techniques for generating sequences of random numbers and establishing their statistical properties. Notice that the `rand_update` function computes a mathematical function: Given the same input twice, it produces the same output. Therefore, the number sequence produced by `rand_update` certainly is not "random," if by "random" we insist that each number in the sequence is unrelated to the preceding number. The relation between "real randomness" and so-called *pseudo-random* sequences, which are produced by well-determined computations and yet have suitable statistical properties, is a complex question involving difficult issues in mathematics and philosophy. Kolmogorov, Solomonoff, and Chaitin have made great progress in clarifying these issues; a discussion can be found in Chaitin 1975.]

We can implement `rand` as a function with a local state variable `x` that is initialized to some fixed value `random_init`. Each call to `rand` computes `rand_update` of the current value of `x`, returns this as the random number, and also stores this as the new value of `x`.

```
def make_rand():
    x = random_init
    def rand():
        nonlocal x
        x = rand_update(x)
        return x
    return rand

rand = make_rand()
```

Of course, we could generate the same sequence of random numbers without using reassignment by simply calling `rand_update` directly. However, this would mean that any part of our program that used random numbers would have to explicitly remember the current value of `x` to be passed as an argument to `rand_update`. To realize what an annoyance this would be, consider using random numbers to implement a technique called *Monte Carlo simulation*.

The Monte Carlo method consists of choosing sample experiments at random from a large set and then making deductions on the basis of the probabilities estimated from tabulating the results of those experiments. For example, we can approximate π using the fact that $6/\pi^2$ is the probability that two integers chosen at random will have no factors in common; that is, that their greatest common divisor will be 1. [^8: This theorem is due to G. Lejeune Dirichlet. See section 4.5.2 of Knuth 1997b for a discussion and a proof.] To obtain the approximation to π , we perform a large number of experiments. In each experiment we choose two integers at random and perform a test to see if their GCD is 1. The fraction of times that the test is passed gives us our estimate of $6/\pi^2$, and from this we obtain our approximation to π .

The heart of our program is a function `monte_carlo`, which takes as arguments the number of times to try an experiment, together with the experiment, represented as a no-argument function that will return either true or false each time it is run. The function `monte_carlo` runs the experiment for the designated number of trials and returns a number telling the fraction of the trials in which the experiment was found to be true.

```
def estimate_pi(trials):
    return math_sqrt(6 / monte_carlo(trials, dirichlet_test))

def dirichlet_test():
    return gcd(rand(), rand()) == 1

def monte_carlo(trials, experiment):
    def iter(trials_remaining, trials_passed):
        return (trials_passed / trials
                if trials_remaining == 0
                else iter(trials_remaining - 1, trials_passed + 1)
                if experiment()
                else iter(trials_remaining - 1, trials_passed))
    return iter(trials, 0)
```


Now let us try the same computation using `rand_update` directly rather than `rand`, the way we would be forced to proceed if we did not use reassignment to model local state:

```
def estimate_pi(trials):
    return math_sqrt(6 / random_gcd_test(trials, random_init))

def random_gcd_test(trials, initial_x):
    def iter(trials_remaining, trials_passed, x):
        x1 = rand_update(x)
        x2 = rand_update(x1)
        return (trials_passed / trials
                if trials_remaining == 0
                else iter(trials_remaining - 1, trials_passed + 1, x2)
                if gcd(x1, x2) == 1
                else iter(trials_remaining - 1, trials_passed, x2))
    return iter(trials, 0, initial_x)
```

While the program is still simple, it betrays some painful breaches of modularity. In our first version of the program, using `rand`, we can express the Monte Carlo method directly as a general `monte_carlo` function that takes as an argument an arbitrary `experiment` function. In our second version of the program, with no local state for the random-number generator, `random_gcd_test` must explicitly manipulate the random numbers `x1` and `x2` and recycle `x2` through the iterative loop as the new input to `rand_update`. This explicit handling of the random numbers intertwines the structure of accumulating test results with the fact that our particular experiment uses two random numbers, whereas other Monte Carlo experiments might use one random number or three. Even the top-level function `estimate_pi` has to be concerned with supplying an initial random number. The fact that the random-number generator's insides are leaking out into other parts of the program makes it difficult for us to isolate the Monte Carlo idea so that it can be applied to other tasks. In the first version of the program, reassignment encapsulates the state of the random-number generator within the `rand` function, so that the details of random-number generation remain independent of the rest of the program.

The general phenomenon illustrated by the Monte Carlo example is this: From the point of view of one part of a complex process, the other parts appear to change with time. They have hidden time-varying local state. If we wish to write computer programs whose structure reflects this decomposition, we make computational objects (such as bank accounts and random-number generators) whose behavior changes with time. We model state with local state variables, and we model the changes of state with reassignments to those variables.

It is tempting to conclude this discussion by saying that, by introducing reassignment and the technique of hiding state in local variables, we are able to structure systems in a more modular fashion than if all state had to be manipulated explicitly, by passing additional parameters. Unfortunately, as we shall see, the story is not so simple.

Exercise 3.5. *Monte Carlo integration* is a method of estimating definite integrals by means of Monte Carlo simulation. Consider computing the area of a region of space described by a predicate $P(x, y)$ that is true for points (x, y) in the region and false for points not in the region. For example, the region contained within a circle of radius 3 centered at $(5, 7)$ is described by the predicate that tests whether $(x-5)^2 + (y-7)^2 \leq 3^2$. To estimate the area of the region described by such a predicate, begin by choosing a rectangle that contains the region. For example, a rectangle with diagonally opposite corners at $(2, 4)$ and $(8, 10)$ contains the circle above. The desired integral is the area of that portion of the rectangle that lies in the region. We can estimate the integral by picking, at random, points (x, y) that lie in the rectangle, and testing $P(x, y)$ for each point to determine whether the point lies in the region. If we try this with many points, then the fraction of points that fall in the region should give an estimate of the proportion of the rectangle that lies in the region. Hence, multiplying this fraction by the area of the entire rectangle should produce an estimate of the integral.

Implement Monte Carlo integration as a function `estimate_integral` that takes as arguments a predicate `p`, upper and lower bounds `x1`, `x2`, `y1`, and `y2` for the rectangle, and the number of trials to perform in order to produce the estimate. Your function should use the same `monte_carlo` function that was used above to estimate π . Use your `estimate_integral` to produce an estimate of π by measuring the area of a unit circle.

You will find it useful to have a function that returns a number chosen at random from a given range. The following `random_in_range` function implements this in terms of the `math_random` function used in section 1.2.6, which returns a nonnegative number less than 1.

```
def random_in_range(low, high):
    range = high - low
    return low + math_random() * range
```

Exercise 3.6. It is useful to be able to reset a random-number generator to produce a sequence starting from a given value. Design a new `rand` function that is called with an argument that is either the string "generate" or the string "reset" and behaves as follows: `rand("generate")` produces a new random number; `rand("reset")(new-value)` resets the internal state variable to the designated *new-value*. Thus, by resetting the state, one can generate repeatable sequences. These are very handy to have when testing and debugging programs that use random numbers.

3.1.3 The Costs of Introducing Reassignment

As we have seen, reassignment enables us to model objects that have local state. However, this advantage comes at a price. Our programming language can no longer be interpreted in terms of the substitution model of function application that we introduced in section 1.1.5. Moreover, no simple model with "nice" mathematical properties can be an adequate framework for dealing with objects and reassignment in programming languages.

So long as we do not use reassignments, two evaluations of the same function with the same arguments will produce the same result, so that functions can be viewed as computing mathematical functions. Programming without any use of reassignments, as we did throughout the first two chapters of this book, is accordingly known as *functional programming*.

To understand how reassignment complicates matters, consider a simplified version of the `make_withdraw` function of section 3.1.1 that does not bother to check for an insufficient amount:

```
def make_simplified_withdraw(balance):
    def withdraw(amount):
        nonlocal balance
        balance = balance - amount
        return balance
    return withdraw
```

```
W = make_simplified_withdraw(25)
```

```
print(W(20))
```

```
5
```

```
print(W(10))
```

```
-5
```

Compare this function with the following `make_decrementer` function, which does not use reassignment:

```
def make_decrementer(balance):
    return lambda amount: balance - amount
```

The function `make_decrementer` returns a function that subtracts its input from a designated amount `balance`, but there is no accumulated effect over successive calls, as with `make_simplified_withdraw`:

```
D = make_decrementer(25)
```

```
print(D(20))
```

```
5
```

```
print(D(10))
```

```
15
```

We can use the substitution model to explain how `make_decrements` works. For instance, let us analyze the evaluation of the expression

```
print(make_decrements(25)(20))
```

We first simplify the function expression of the application by substituting `25` for `balance` in the body of `make_decrements`. This reduces the expression to

```
(lambda amount: 25 - amount)(20)
```

Now we apply the function by substituting `20` for `amount` in the body of the lambda expression:

```
print(25 - 20)
```

The final answer is 5.

Observe, however, what happens if we attempt a similar substitution analysis with `make_simplified_withdraw`:

```
make_simplified_withdraw(25)(20)
```

We first simplify the function expression by substituting `25` for `balance` in the body of `make_simplified_withdraw`. This reduces the expression to `25 - amount`. We don't substitute for the occurrence of `balance` in the reassignment because the *name* in a reassignment is not evaluated. If we did substitute for it, we would get `25 = 25 - amount`, which makes no sense.]

```
def withdraw(amount):  
    balance = 25 - amount  
    return 25  
  
withdraw(20)
```

Now we apply the function by substituting `20` for `amount` in the body of the function definition:

```
balance = 25 - 20  
return 25
```

If we adhered to the substitution model, we would have to say that the meaning of the function application is to first set `balance` to 5 and then return 25 as the value of the expression. This gets the wrong answer. In order to get the correct answer, we would have to somehow distinguish the first occurrence of `balance` (before the effect of the reassignment) from the second occurrence of `balance` (after the effect of the reassignment), and the substitution model cannot do this.

The trouble here is that substitution is based ultimately on the notion that the names in our language are essentially symbols for values. This worked well for constants. But a variable, whose value can change with reassignment, cannot simply be a name for a value. A variable somehow refers to a place where a value can be stored, and the value stored at this place can change. In section 3.2 we will see how environments play this role of "place" in our computational model.

Sameness and change

The issue surfacing here is more profound than the mere breakdown of a particular model of computation. As soon as we introduce change into our computational models, many notions that were previously straightforward become problematical. Consider the concept of two things being "the same."

Suppose we call `make_decrementer` twice with the same argument to create two functions:

```
D1 = make_decrementer(25)

D2 = make_decrementer(25)
```

Are `D1` and `D2` the same? An acceptable answer is yes, because `D1` and `D2` have the same computational behavior—each is a function that subtracts its input from 25. In fact, `D1` could be substituted for `D2` in any computation without changing the result.

Contrast this with making two calls to `make_simplified_withdraw`:

```
W1 = make_simplified_withdraw(25)

W2 = make_simplified_withdraw(25)
```

Are `W1` and `W2` the same? Surely not, because calls to `W1` and `W2` have distinct effects, as shown by the following sequence of interactions:

```
print(W1(20))
```

```
5
```

```
print(w1(20))
```

```
-15
```

```
print(w2(20))
```

```
5
```

Even though `w1` and `w2` are "equal" in the sense that they are both created by evaluating the same expression, `make_simplified_withdraw(25)`, it is not true that `w1` could be substituted for `w2` in any expression without changing the result of evaluating the expression.

A language that supports the concept that "equals can be substituted for equals" in an expression without changing the value of the expression is said to be *referentially transparent*. Referential transparency is violated when we include reassignment in our computer language. This makes it tricky to determine when we can simplify expressions by substituting equivalent expressions. Consequently, reasoning about programs that use reassignment becomes drastically more difficult.

Once we forgo referential transparency, the notion of what it means for computational objects to be "the same" becomes difficult to capture in a formal way. Indeed, the meaning of "same" in the real world that our programs model is hardly clear in itself. In general, we can determine that two apparently identical objects are indeed "the same one" only by modifying one object and then observing whether the other object has changed in the same way. But how can we tell if an object has "changed" other than by observing the "same" object twice and seeing whether some property of the object differs from one observation to the next? Thus, we cannot determine "change" without some a priori notion of "sameness," and we cannot determine sameness without observing the effects of change.

As an example of how this issue arises in programming, consider the situation where Peter and Paul have a bank account with \$100 in it. There is a substantial difference between modeling this as

```
peter_acc = make_account(100)
paul_acc = make_account(100)
```

and modeling it as

```
peter_acc = make_account(100)
paul_acc = peter_acc
```

In the first situation, the two bank accounts are distinct. Transactions made by Peter will not affect Paul's account, and vice versa. In the second situation, however, we have defined `paul_acc` to be

the same thing as `peter_acc`. In effect, Peter and Paul now have a joint bank account, and if Peter makes a withdrawal from `peter_acc` Paul will observe less money in `paul_acc`. These two similar but distinct situations can cause confusion in building computational models. With the shared account, in particular, it can be especially confusing that there is one object (the bank account) that has two different names (`peter_acc` and `paul_acc`) if we are searching for all the places in our program where `paul_acc` can be changed, we must remember to look also at things that change `peter_acc`. [^10: The phenomenon of a single computational object being accessed by more than one name is known as *aliasing*. The joint bank account situation illustrates a very simple example of an alias. In section 3.3 we will see much more complex examples, such as "distinct" compound data structures that share parts. Bugs can occur in our programs if we forget that a change to an object may also, as a "side effect," change a "different" object because the two "different" objects are actually a single object appearing under different aliases. These so-called *side-effect bugs* are so difficult to locate and to analyze that some people have proposed that programming languages be designed in such a way as to not allow side effects or aliasing (Lampson et al. 1981; Morris, Schmidt, and Wadler 1980).]

With reference to the above remarks on "sameness" and "change," observe that if Peter and Paul could only examine their bank balances, and could not perform operations that changed the balance, then the issue of whether the two accounts are distinct would be moot. In general, so long as we never modify data objects, we can regard a compound data object to be precisely the totality of its pieces. For example, a rational number is determined by giving its numerator and its denominator. But this view is no longer valid in the presence of change, where a compound data object has an "identity" that is something different from the pieces of which it is composed. A bank account is still "the same" bank account even if we change the balance by making a withdrawal; conversely, we could have two different bank accounts with the same state information. This complication is a consequence, not of our programming language, but of our perception of a bank account as an object. We do not, for example, ordinarily regard a rational number as a changeable object with identity, such that we could change the numerator and still have "the same" rational number.

Pitfalls of imperative programming

In contrast to functional programming, programming that makes extensive use of reassignment is known as *imperative programming*. In addition to raising complications about computational models, programs written in imperative style are susceptible to bugs that cannot occur in functional programs. For example, recall the iterative factorial program from section 1.2.1 (here using a conditional statement instead of a conditional expression):

```
def factorial(n):
    def iter(product, counter):
        if counter > n:
            return product
        else:
            return iter(counter * product,
                        counter + 1)
    return iter(1, 1)
```

Instead of passing arguments in the internal iterative loop, we could adopt a more imperative style by using explicit reassignment to update the values of the variables `product` and `counter` :

```
def factorial(n):
    product = 1
    counter = 1
    def iter():
        nonlocal product, counter
        if counter > n:
            return product
        else:
            product = counter * product
            counter = counter + 1
            return iter()
    return iter()
```

This does not change the results produced by the program, but it does introduce a subtle trap. How do we decide the order of the reassignments? As it happens, the program is correct as written. But writing the reassignments in the opposite order

```
counter = counter + 1
product = counter * product
```

would have produced a different, incorrect result. In general, programming with reassignment forces us to carefully consider the relative orders of the reassignments to make sure that each statement is using the correct version of the variables that have been changed. This issue simply does not arise in functional programs. [^11: In view of this, it is ironic that introductory programming is most often taught in a highly imperative style. This may be a vestige of a belief, common throughout the 1960s and 1970s, that programs that call functions must inherently be less efficient than programs that perform reassignments. (Steele (1977) debunks this argument.) Alternatively it may reflect a view that step-by-step reassignment is easier for beginners to visualize than function call. Whatever the reason, it often saddles beginning programmers with "should I set this variable before or after that one" concerns that can complicate programming and obscure the important ideas.]

The complexity of imperative programs becomes even worse if we consider applications in which several processes execute concurrently. We will return to this in section 3.4. First, however, we will

address the issue of providing a computational model for expressions that involve reassignment, and explore the uses of objects with local state in designing simulations.

Exercise 3.7. Consider the bank account objects created by `make_account`, with the password modification described in exercise 3.3. Suppose that our banking system requires the ability to make joint accounts. Define a function `make_joint` that accomplishes this. The function `make_joint` should take three arguments. The first is a password-protected account. The second argument must match the password with which the account was defined in order for the `make_joint` operation to proceed. The third argument is a new password. The function `make_joint` is to create an additional access to the original account using the new password. For example, if `peter_acc` is a bank account with password `"open sesame"`, then

```
paul_acc = make_joint(peter_acc, "open sesame", "rosebud")
```

will allow one to make transactions on `peter_acc` using the name `paul_acc` and the password `"rosebud"`. You may wish to modify your solution to exercise 3.3 to accommodate this new feature.

Exercise 3.8. When we defined the evaluation model in section 1.1.3, we said that the first step in evaluating an expression is to evaluate its subexpressions. But we never specified the order in which the subexpressions should be evaluated (e.g., left to right or right to left). When we introduce reassignment, the order in which the operands of an operator combination are evaluated can make a difference to the result. Define a simple function `f` such that evaluating `f(0) + f(1)` will return 0 if the operands of `+` are evaluated from left to right but will return 1 if the operands are evaluated from right to left.

3.2 The Environment Model of Evaluation

When we introduced compound functions in chapter 1, we used the substitution model of evaluation (section 1.1.5) to define what is meant by applying a function to arguments:

- To apply a compound function to arguments, evaluate the return expression of the function (more generally, the body) with each parameter replaced by the corresponding argument.

Once we admit reassignment into our programming language, such a definition is no longer adequate. In particular, section 3.1.3 argued that, in the presence of reassignment, a variable cannot be considered to be merely a name for a value. Rather, a variable must somehow designate a "place" in which values can be stored. In our new model of evaluation, these places will be maintained in structures called *environments*.

An environment is a sequence of *frames*. Each frame is a table (possibly empty) of *bindings*, which associate variable names with their corresponding values. (A single frame may contain at most one

binding for any variable.) Each frame also has a pointer to its *enclosing environment*, unless the frame is the *built-in frame*, reserved for Python's built-in functions and values. The *value of a variable* with respect to an environment is the value given by the binding of the variable in the first frame in the environment that contains a binding for that variable. If no frame in the sequence specifies a binding for the variable, then the variable is said to be *unbound* in the environment.

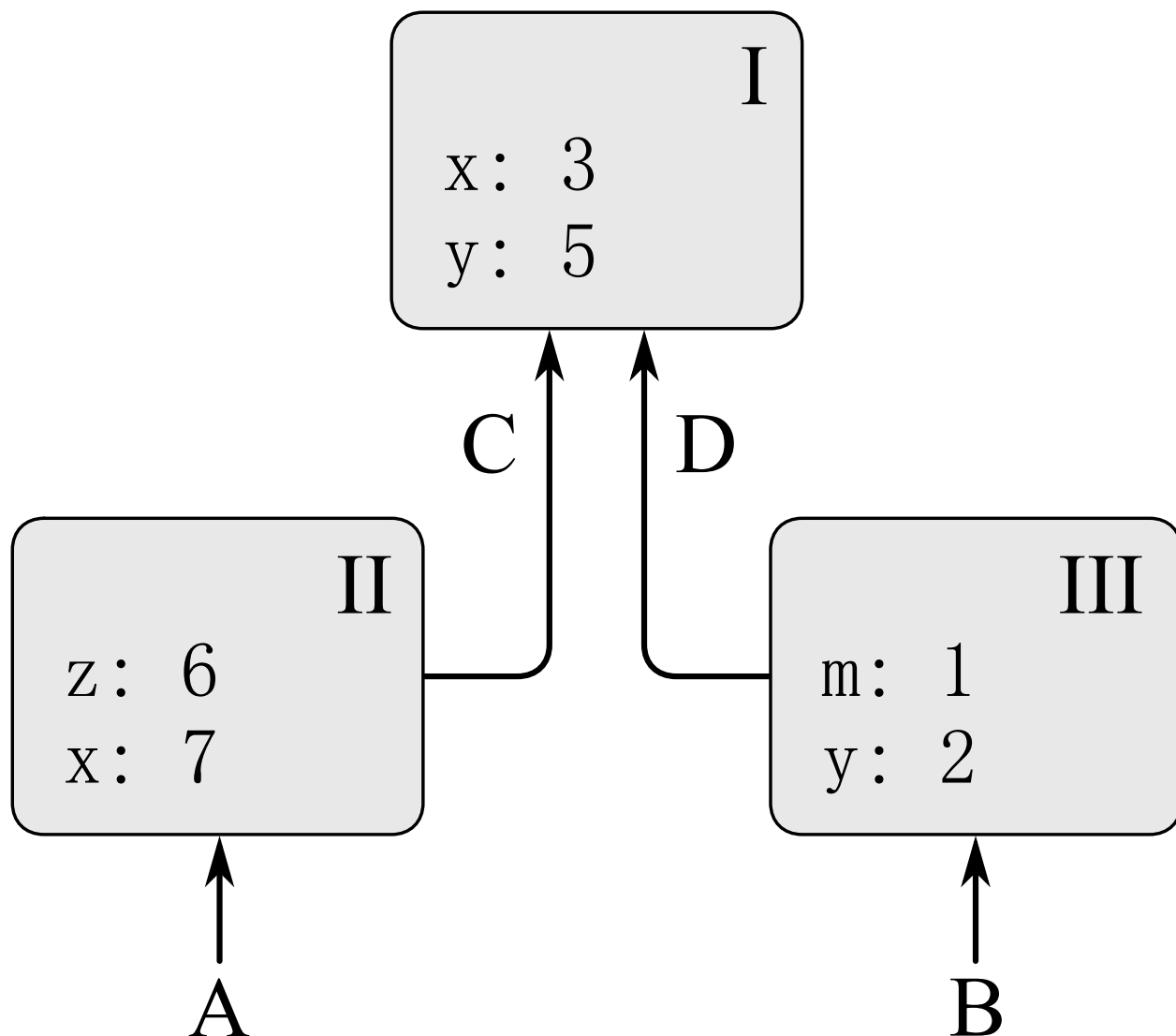


Figure 3.1. A simple environment structure.

Figure 3.1 shows a simple environment structure consisting of three frames, labeled I, II, and III. In the diagram, A, B, C, and D are pointers to environments. C and D point to the same environment. The variables `z` and `x` are bound in frame II, while `y` and `x` are bound in frame I. The value of `x` in environment D is 3. The value of `x` with respect to environment B is also 3. This is determined as follows: We examine the first frame in the sequence (frame III) and do not find a binding for `x`, so we proceed to the enclosing environment D and find the binding in frame I. On the other hand, the value of `x` in environment A is 7, because the first frame in the sequence (frame II) contains a binding of `x` to 7. With respect to environment A, the binding of `x` to 7 in frame II is said to *shadow* the binding of `x` to 3 in frame I.

The environment is crucial to the evaluation process, because it determines the context in which an expression should be evaluated. Indeed, one could say that expressions in a programming language do not, in themselves, have any meaning. Rather, an expression acquires a meaning only with respect to some environment in which it is evaluated. Even the interpretation of an expression as straightforward as `print(1)` depends on an understanding that one is operating in a context in which the name `print` refers to the built-in function that prints a value. Thus, in our model of evaluation we will always speak of evaluating an expression with respect to some environment. To describe interactions with the interpreter, we will suppose that there is a built-in environment, consisting of a single frame (with no enclosing environment) that includes values for the names associated with the built-in functions. For example, the idea that `print` is the name for the built-in print function is captured by saying that the name `print` is bound in the built-in environment to the built-in print function.

3.2.1 The Rules for Evaluation

The overall specification of how the interpreter evaluates a function application remains the same as when we first introduced it in section 1.1.4:

- To evaluate an application:
- Evaluate the subexpressions of the application. [¹²: Reassignment introduces a subtlety into step 1 of the evaluation rule. As shown in exercise , the presence of reassignment allows us to write expressions that will produce different values depending on the order in which the subexpressions in a combination are evaluated. To eliminate such ambiguities, Python specifies left-to-right evaluation of the subexpressions of combinations and of the argument expressions of applications.]
- Apply the value of the function subexpression to the values of the argument subexpressions.

The environment model of evaluation replaces the substitution model in specifying what it means to apply a compound function to arguments.

Before applying these rules, we need to say in which environment the statements of a program are evaluated. It is not the built-in environment of section 3.2: that frame is reserved for Python's built-in functions and values, and a program never adds bindings to it. Instead, the interpreter evaluates a program in a *global environment*, which consists of a single *global frame* whose enclosing environment is the built-in environment. Every definition and reassignment at the top level of a program adds or changes a binding in the global frame, whereas the built-in frame supplies the names that the program does not define itself. Looking up `print` in the global environment therefore finds no binding in the global frame and proceeds to the built-in frame, while a name defined by the program is found in the global frame itself.

In the environment model of evaluation, a function is always a pair consisting of some code and a pointer to an environment. Functions are created in two ways: by evaluating a function definition

or a lambda expression. In both cases, the result is a function whose code is obtained from the program text and whose environment is the environment in which the function definition or lambda expression was evaluated to produce the function. For example, consider the function definition

```
def square(x):  
    return x * x
```

evaluated in the global environment. As far as the environment model is concerned, the function definition statement behaves like an assignment of a lambda expression (section 1.3.2). It would have been equivalent to have used

```
square = lambda x: x * x
```

which evaluates `lambda x: x * x` and binds `square` to the resulting function value, all in the global environment.

Figure 3.2 shows the result of evaluating this function definition. The function object is a pair whose code specifies that the function has one parameter, namely `x`, and a function body `return x * x`. The environment part of the function is a pointer to the global environment, since that is the environment in which the lambda expression was evaluated to produce the function. A new binding, which associates the function object with the name `square`, has been added to the global frame.

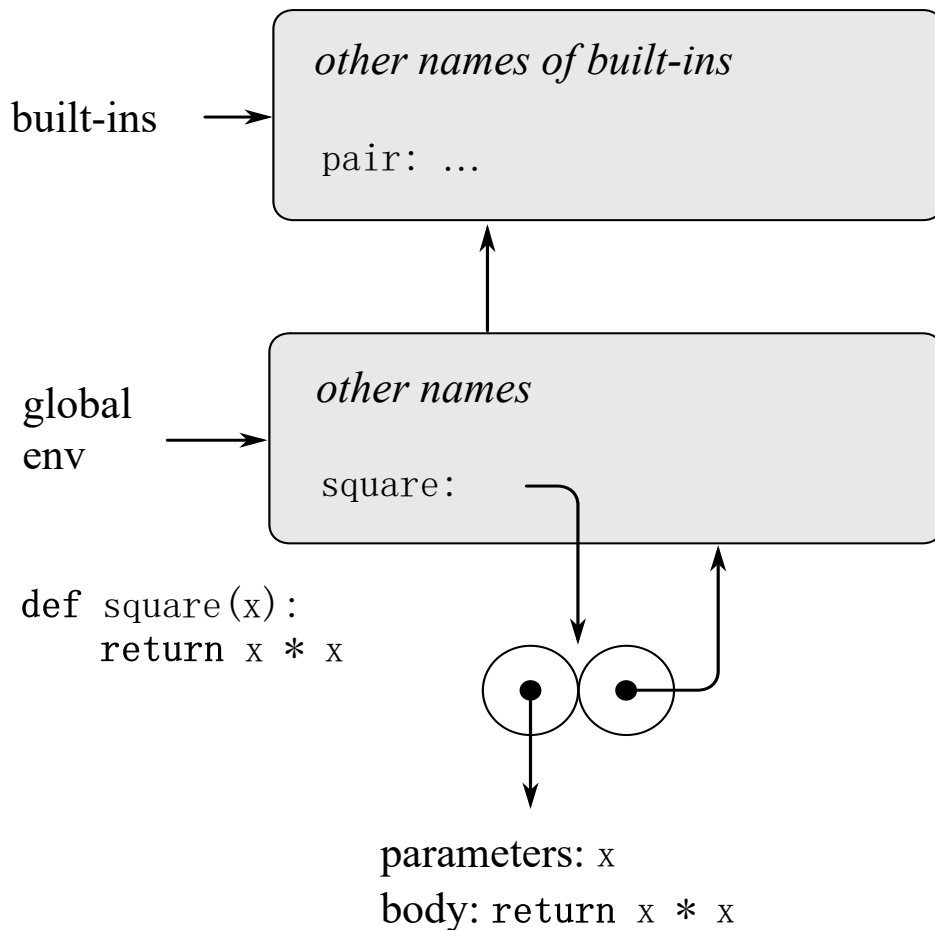


Figure 3.2. Environment structure produced by evaluating `def square(x): return x * x` in the global environment.

Now that we have seen how functions are created, we can describe how functions are applied. The environment model specifies: To apply a function to arguments, create a new environment containing a frame that binds the parameters to the values of the arguments. The enclosing environment of this frame is the environment specified by the function. Now, within this new environment, evaluate the function body.

To show how this rule is followed, figure 3.3 illustrates the environment structure created by evaluating the expression `square(5)` in the global environment, where `square` is the function generated in figure 3.2. Applying the function results in the creation of a new environment, labeled E1 in the figure, that begins with a frame in which `x`, the parameter for the function, is bound to the argument 5. The pointer leading upward from this frame shows that the frame's enclosing environment is the global environment. The global environment is chosen here, because this is the environment that is indicated as part of the `square` function object. Within E1, we evaluate the body of the function, `return x * x`. Since the value of `x` in E1 is 5, the result is `5 * 5`, or 25.

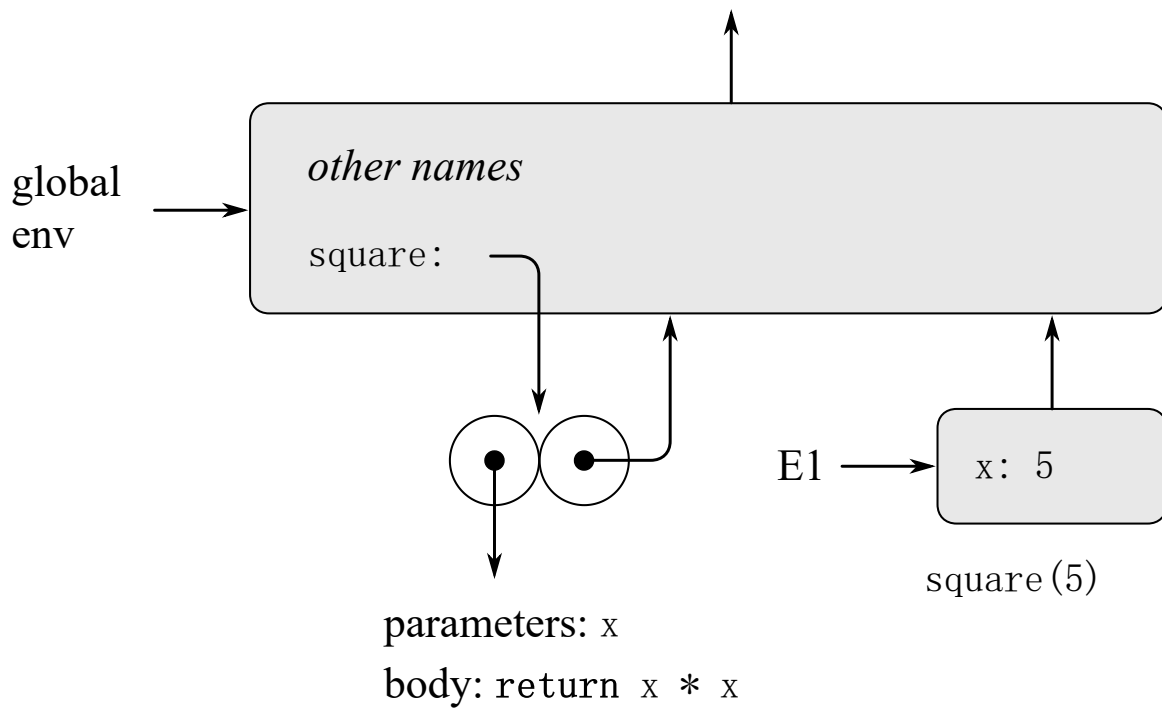


Figure 3.3. Environment created by evaluating `square(5)` in the global environment.

The environment model of function application can be summarized by two rules:

- A function object is applied to a set of arguments by constructing a frame, binding the parameters of the function to the arguments of the call, and then evaluating the body of the function in the context of the new environment constructed. The new frame has as its enclosing environment the environment part of the function object being applied. The result of the application is the result of evaluating the return expression of the first return statement encountered while evaluating the function body.
- A function is created by evaluating a function definition or lambda expression relative to a given environment. The resulting function object is a pair consisting of the text of the function body and a pointer to the environment in which the function was created.

Finally, we specify the behavior of assignment (declaration assignment or reassignment). It was reassignment that forced us to introduce the environment model in the first place. Evaluating the assignment `name = value` in some environment locates the binding of the variable in the environment. For this, one finds the first frame in the environment that contains a binding for the variable and modifies that frame. If the variable is unbound in the environment, then the assignment signals an error.

These evaluation rules, though considerably more complex than the substitution model, are still reasonably straightforward. Moreover, the evaluation model, though abstract, provides a correct description of how the interpreter evaluates expressions. In chapter we shall see how this model can serve as a blueprint for implementing a working interpreter. The following sections elaborate the details of the model by analyzing some illustrative programs.

3.2.2 Applying Simple Functions

When we introduced the substitution model in section 1.1.5 we showed how the application `f(5)` evaluates to 136, given the following function definitions:

```
def square(x):  
    return x * x  
  
def sum_of_squares(x, y):  
    return square(x) + square(y)  
  
def f(a):  
    return sum_of_squares(a + 1, a * 2)
```

We can analyze the same example using the environment model. Figure 3.4 shows the three function objects created by evaluating the definitions of `f`, `square`, and `sum_of_squares` in the global environment. Each function object consists of some code, together with a pointer to the global environment.

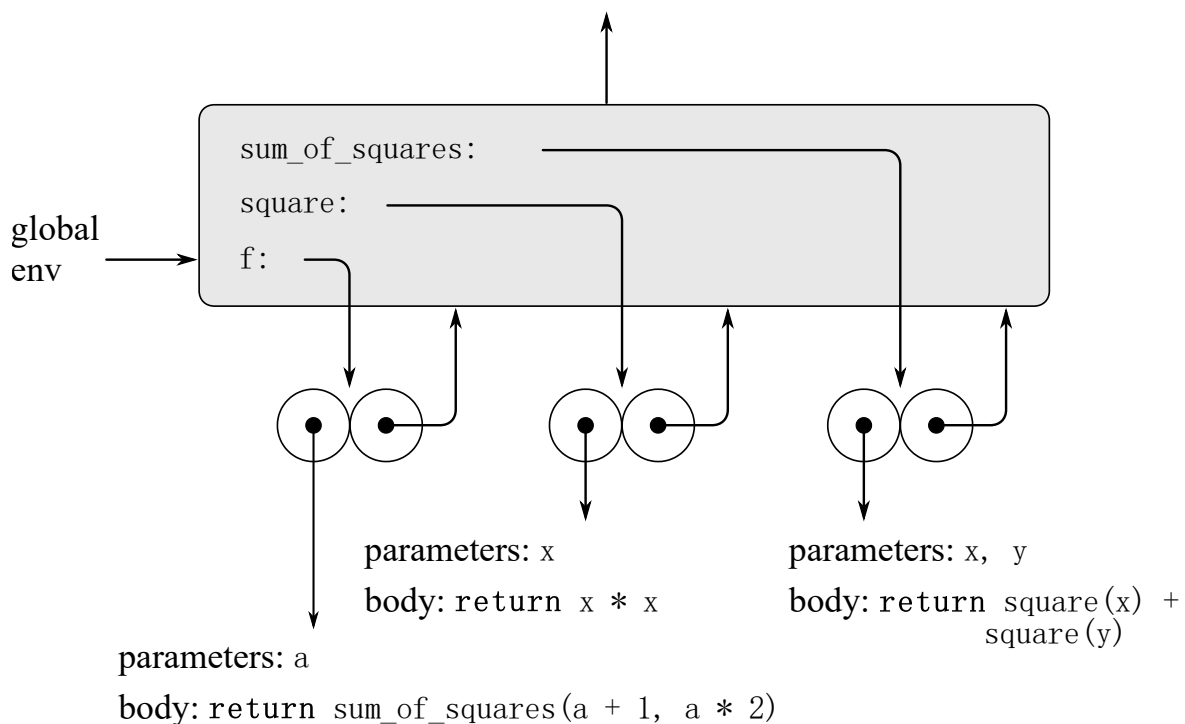


Figure 3.4. Function objects in the global frame.

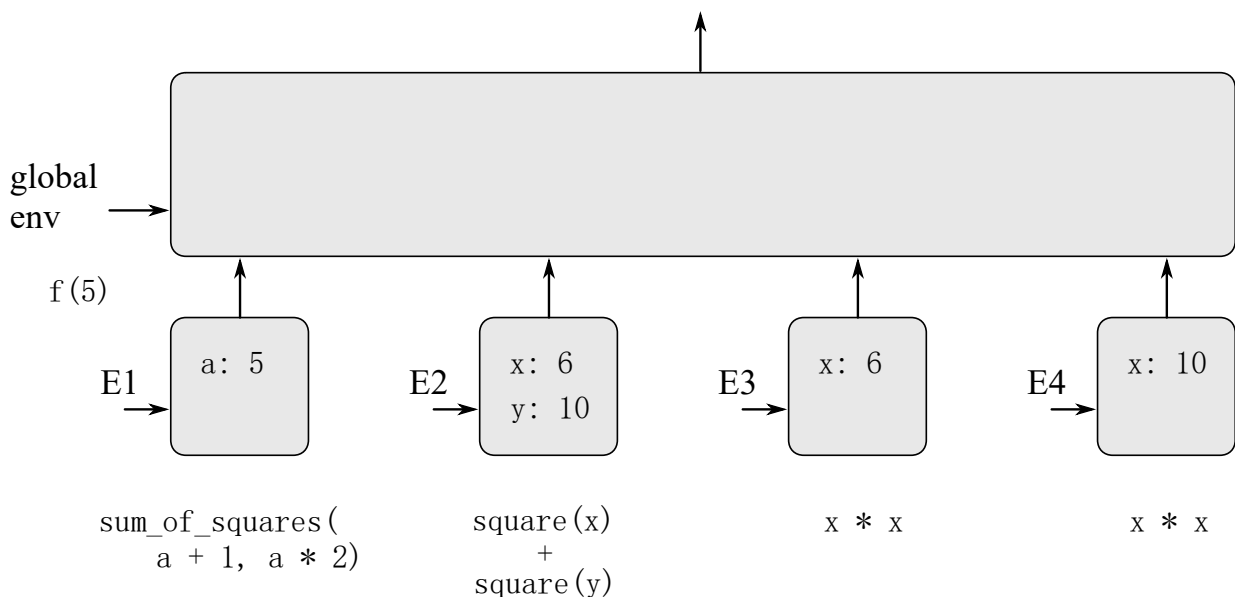


Figure 3.5. Environments created by evaluating `f(5)` using the functions in figure 3.4.

In figure 3.5 we see the environment structure created by evaluating the expression `f(5)`. The call to `f` creates a new environment, E1, beginning with a frame in which `a`, the parameter of `f`, is bound to the argument 5. In E1, we evaluate the body of `f`:

```
return sum_of_squares(a + 1, a * 2)
```

To evaluate the return statement, we first evaluate the subexpressions of the return expression. The first subexpression, `sum_of_squares`, has a value that is a function object. (Notice how this value is found: We first look in the first frame of E1, which contains no binding for `sum_of_squares`. Then we proceed to the enclosing environment, i.e., the global environment, and find the binding shown in figure 3.4.) The other two subexpressions are evaluated by applying the primitive operations `+` and `*` to evaluate the two combinations `a + 1` and `a * 2` to obtain 6 and 10, respectively.

Now we apply the function object `sum_of_squares` to the arguments 6 and 10. This results in a new environment, E2, in which the parameters `x` and `y` are bound to the arguments. Within E2 we evaluate the statement

```
return square(x) + square(y)
```

This leads us to evaluate `square(x)`, where `square` is found in the global frame and `x` is 6. Once again, we set up a new environment, E3, in which `x` is bound to 6, and within this we evaluate the body of `square`, which is `return x * x`. Also as part of applying `sum_of_squares`, we must evaluate the subexpression `square(y)`, where `y` is 10. This second call to `square` creates another environment, E4, in which `x`, the parameter of `square`, is bound to 10. And within E4 we must evaluate `return x * x`.

The important point to observe is that each call to `square` creates a new environment containing a binding for `x`. We can see here how the different frames serve to keep separate the different local variables all named `x`. Notice that each frame created by `square` points to the global environment, since this is the environment indicated by the `square` function object.

After the subexpressions are evaluated, the results are returned. The values generated by the two calls to `square` are added by `sum_of_squares`, and this result is returned by `f`. Since our focus here is on the environment structures, we will not dwell on how these returned values are passed from call to call; however, this is also an important aspect of the evaluation process, and we will return to it in detail in chapter .

Exercise 3.9. In section 1.2.1 we used the substitution model to analyze two functions for computing factorials, a recursive version

```
def factorial(n):
    return (1
            if n == 1
            else n * factorial(n - 1))
```

and an iterative version

```
def factorial(n):
    return fact_iter(1, 1, n)

def fact_iter(product, counter, max_count):
    return (product
            if counter > max_count
            else fact_iter(counter * product,
                           counter + 1,
                           max_count))
```

Show the environment structures created by evaluating `factorial(6)` using each version of the `factorial` function. [^13: The environment model will not clarify our claim in section 1.2.1 that the interpreter can execute a function such as `fact_iter` in a constant amount of space using tail recursion. We will discuss tail recursion when we deal with the control structure of the interpreter in section .]

3.2.3 Functions with Local Declaration Assignments

To handle functions with local variables, we extend the environment model in a straightforward manner. Consider for example the function `f_3` from section 1.3.2 and its helper function `square`.

```

def square(z):
    return z * z
def f_3(x, y):
    a = 1 + x * y
    b = 1 - y
    return x * square(a) + y * b + a * b

```

As with simple functions, a call `f_3(3, 4)` creates a new environment, E1, beginning with a frame in which `x` and `y`, the parameters of `f_3`, are bound to the arguments `3` and `4`, respectively. However, `f_3` has declaration assignments for the local variables `a` and `b`, and subsequent references to the local variables need to refer to their values. To prepare for the evaluation of declaration assignments and their variables, the frame has bindings for `a` and `b` in addition to `x` and `y`. Before the body of `f_3` is evaluated, `a` and `b` do not refer to any values yet; we say they are *unassigned*. The environment structure is shown in figure 3.6.

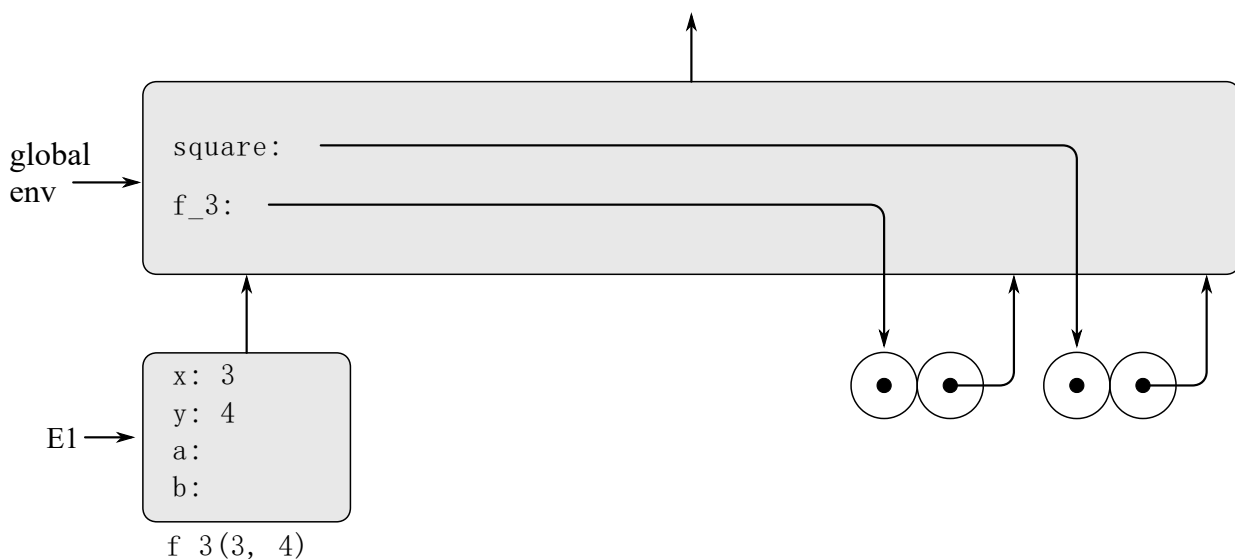


Figure 3.6. Environment created by `f_3(3, 4)` just before the body of `f_3` gets evaluated.

The evaluation of the assignments to `a` and `b` follows the rule for assignment in the previous section and thus modifies the frame so that `a` refers to 13, the result of evaluating `1 + x * y`, and `b` refers to -3, the result of evaluating `1 - y`. The remaining statement in the function body, `return x * square(a) + y * b + a * b`, contains the call `square(a)`, which evaluates the body of `square`, `return z * z`, in a new environment, E2, beginning with a frame in which the parameter `z` refers to 13. The evaluation of the return expression of `f_3` uses the result 169 and the values of the other variables in the environment E1 and computes 456 as the result of the function call `f_3(3, 4)`. The resulting environment structure is shown in figure 3.7.

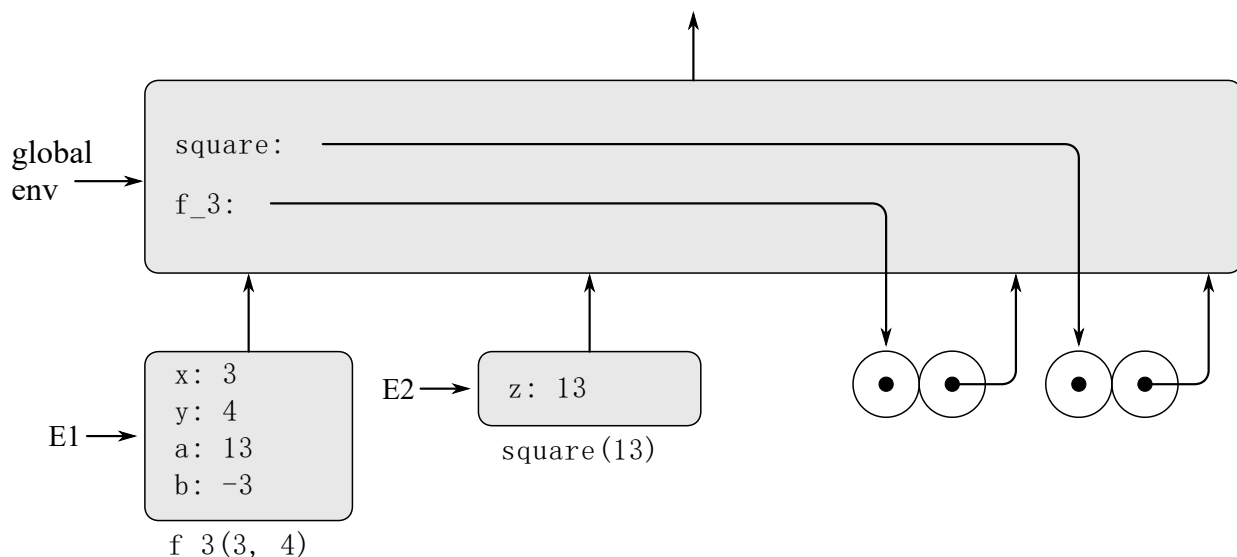


Figure 3.7. Environments created by `f_3(3, 4)`.

3.2.4 Internal Function Definitions

In this section we handle the evaluation of function bodies that contain function definitions. Each function opens a new scope for names declared in the function. In order to evaluate a function body when the function is applied, we extend the function's environment by a new frame that contains the parameters (bound to the arguments) and all names declared in the function body and then evaluate the body in the newly constructed environment.

Section 1.1.8 introduced the idea that functions can have internal function definitions, thus leading to a block structure as in the following function to compute square roots:

```
def sqrt(x):
    def is_good_enough(guess):
        return abs(square(guess) - x) < 0.001
    def improve(guess):
        return average(guess, x / guess)
    def sqrt_iter(guess):
        return (guess
                if is_good_enough(guess)
                else sqrt_iter(improve(guess)))
    return sqrt_iter(1)
```

Now we can use the environment model to see why these internal definitions behave as desired. Figure 3.8 shows the point in the evaluation of the expression `sqrt(2)` where the internal function `is_good_enough` has been called for the first time with `guess` equal to 1.

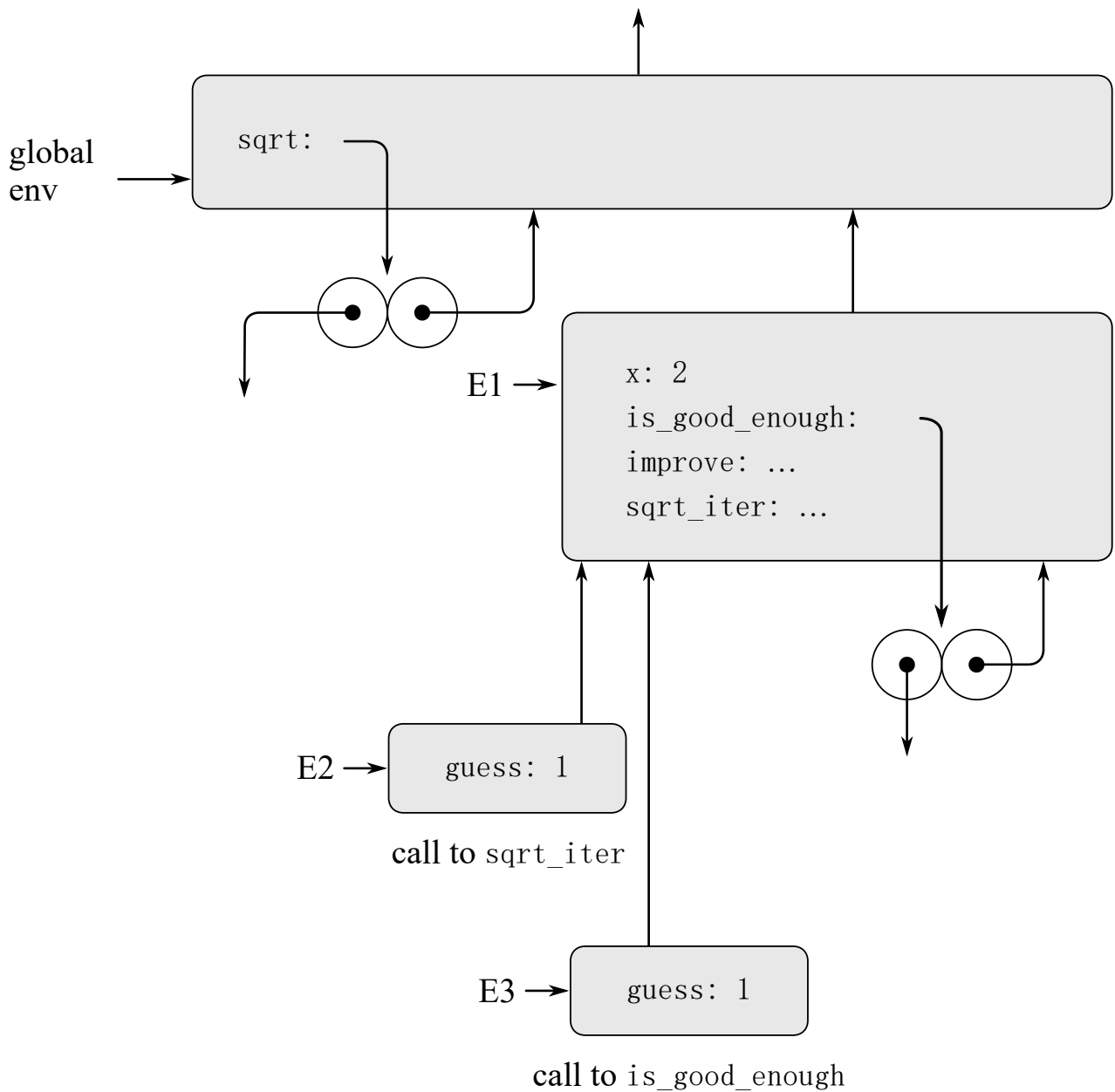


Figure 3.8. The `sqrt` function with internal definitions.

Observe the structure of the environment. The name `sqrt` is bound in the global environment to a function object whose associated environment is the global environment. When `sqrt` was called, a new environment, E1, was formed, subordinate to the global environment, in which the parameter `x` is bound to 2. The body of `sqrt` was then evaluated in E1. Since the first statement in the body is

```
def is_good_enough(guess):
    return abs(square(guess) - x) < 0.001
```

evaluating this definition created the function `is_good_enough` in the environment E1. To be more precise, the name `is_good_enough` in the first frame of E1 was bound to a function object whose

associated environment is E1. Similarly, `improve` and `sqrt_iter` were defined as functions in E1. For conciseness, figure 3.8 shows only the function object for `is_good_enough`.

After the local functions were defined, the expression `sqrt_iter(1)` was evaluated, still in environment E1. So the function object bound to `sqrt_iter` in E1 was called with 1 as an argument. This created an environment E2 in which `guess`, the parameter of `sqrt_iter`, is bound to 1. The function `sqrt_iter` in turn called `is_good_enough` with the value of `guess` (from E2) as the argument for `is_good_enough`. This set up another environment, E3, in which `guess` (the parameter of `is_good_enough`) is bound to 1. Although `sqrt_iter` and `is_good_enough` both have a parameter named `guess`, these are two distinct local variables located in different frames. Also, E2 and E3 both have E1 as their enclosing environment, because the `sqrt_iter` and `is_good_enough` functions both have E1 as their environment part. One consequence of this is that the name `x` that appears in the body of `is_good_enough` will reference the binding of `x` that appears in E1, namely the value of `x` with which the original `sqrt` function was called.

The environment model thus explains the two key properties that make local function definitions a useful technique for modularizing programs:

- The names of the local functions do not interfere with names external to the enclosing function, because the local function names will be bound in the frame that the function creates when it is run, rather than being bound in the global environment.
- The local functions can access the arguments of the enclosing function, simply by using parameter names as free names. This is because the body of the local function is evaluated in an environment that is subordinate to the evaluation environment for the enclosing function.

Mutual recursion

As we saw, the scope of the names declared in `sqrt` is the whole body of `sqrt`. This explains why *mutual recursion* works, as in this (quite wasteful) way of checking whether a nonnegative integer is even.

```
def f(x):
    def is_even(n):
        return (True if n == 0
                else is_odd(n - 1))
    def is_odd(n):
        return (False if n == 0
                else is_even(n - 1))
    return is_even(x)
```

At the time when `is_even` is called during a call to `f`, the environment diagram looks like the one in figure 3.8 when `sqrt_iter` is called. The functions `is_even` and `is_odd` are bound in E1 to function objects that point to E1 as the environment in which to evaluate calls to those

functions. Thus `is_odd` in the body of `is_even` refers to the right function. Although `is_odd` is defined after `is_even`, this is no different from how in the body of `sqrt_iter` the name `improve` and the name `sqrt_iter` itself refer to the right functions.

We said that a function's body is evaluated in an environment that contains all names declared in the body of the function. A locally declared name is put into the environment when the function body is entered, but without an associated value. The evaluation of its declaration assignment during evaluation of the function body then assigns to the name the result of evaluating the expression to the right of the `=`. Since the addition of the name to the environment is separate from the evaluation of the declaration, and the whole function body is in the scope of the name, an erroneous program could attempt to access the value of a name before its declaration is evaluated; the evaluation of an unassigned name signals an error. [^14: This explains why the program in footnote of chapter 1 goes wrong.]

3.2.5 Frames as the Repository of Local State

We can turn to the environment model to see how functions and reassignment can be used to represent objects with local state. As an example, consider the "withdraw processor" from section 3.1.1 created by calling the function

```
def make_withdraw(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    return withdraw
```

Let us describe the evaluation of

```
W1 = make_withdraw(100)
```

followed by

```
W1(50)
```

Figure 3.9 shows the result of declaring the `make_withdraw` function in the global environment. This produces a function object that contains a pointer to the global environment. So far, this is no different from the examples we have already seen, except that the body of the function defines another function, which is returned.

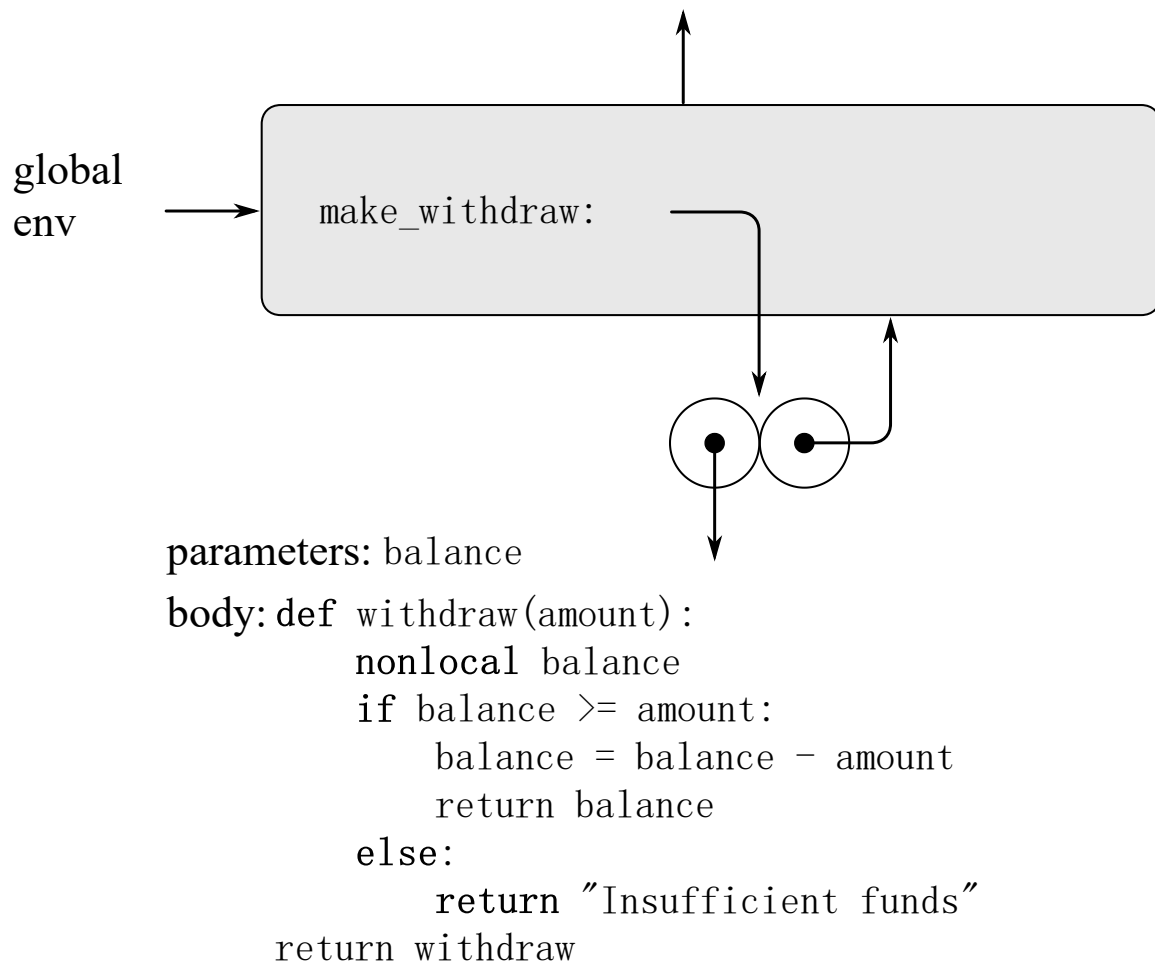


Figure 3.9. Result of defining `make_withdraw` in the global environment.

The interesting part of the computation happens when we apply the function `make_withdraw` to an argument:

```
W1 = make_withdraw(100)
```

We begin by setting up an environment `E1` in which the parameter `balance` is bound to the argument 100 and the locally declared variable `withdraw` is unassigned. Within this environment, we evaluate the body of `make_withdraw`, namely a function definition and a return statement. The function definition constructs a function object which is bound to `withdraw` in `E1`. The function object is the value returned by the call to `make_withdraw`. This is bound to `W1` in the global environment, since the declaration assignment itself is being evaluated in the global environment. Figure 3.10 shows the resulting environment structure.

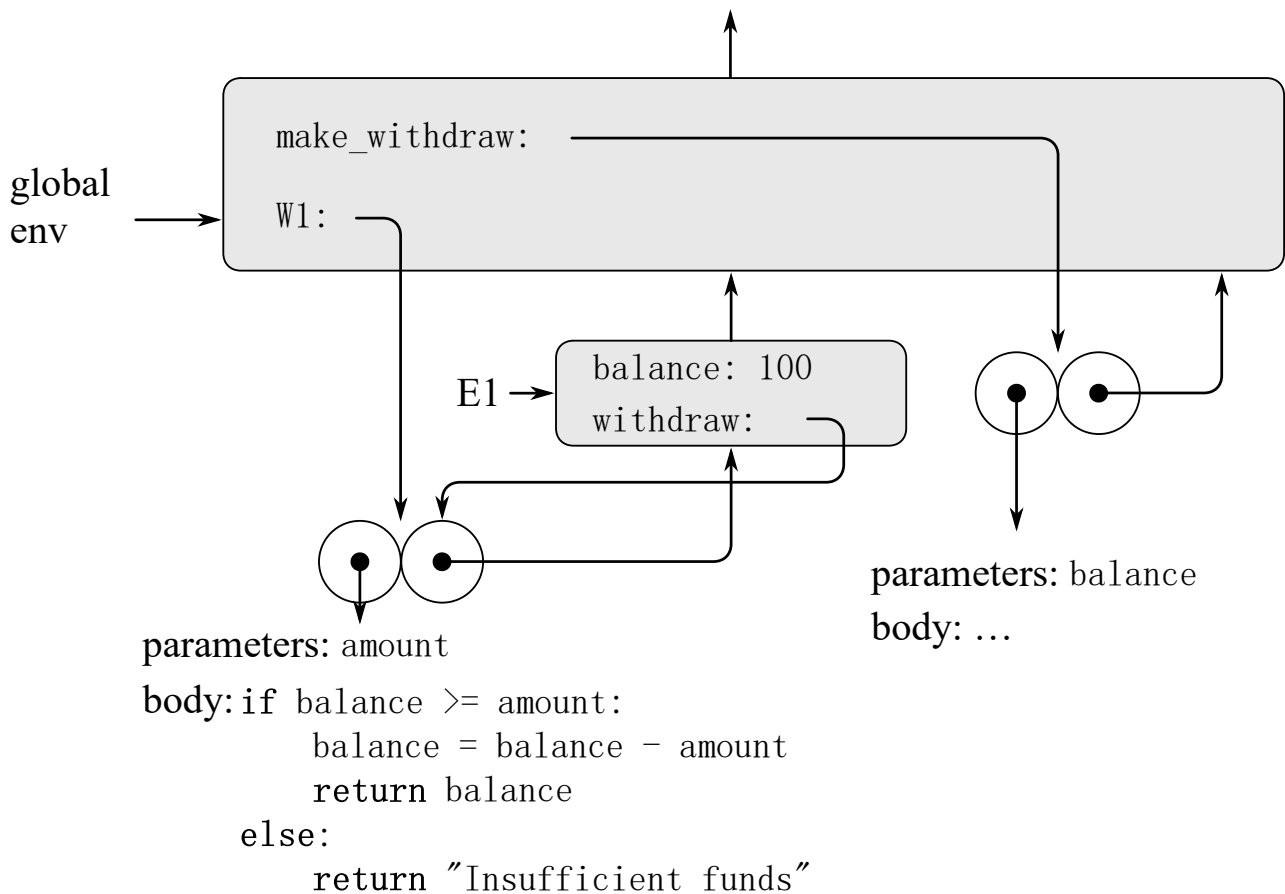


Figure 3.10. Result of evaluating `W1 = make_withdraw(100)`.

Now we can analyze what happens when `W1` is applied to an argument:

`W1(50)`

We begin by constructing a frame in which `amount`, the parameter of `W1`, is bound to the argument 50. The crucial point to observe is that this frame has as its enclosing environment not the global environment, but rather the environment `E1`, because this is the environment that is specified by the `W1` function object. Within this new environment, we evaluate the body of the function:

```
if balance >= amount:
    balance = balance - amount
    return balance
else:
    return "Insufficient funds"
```

The resulting environment structure is shown in figure 3.11. The expression being evaluated references both `amount` and `balance`. The variable `amount` will be found in the first frame in the environment, and `balance` will be found by following the enclosing-environment pointer to `E1`.

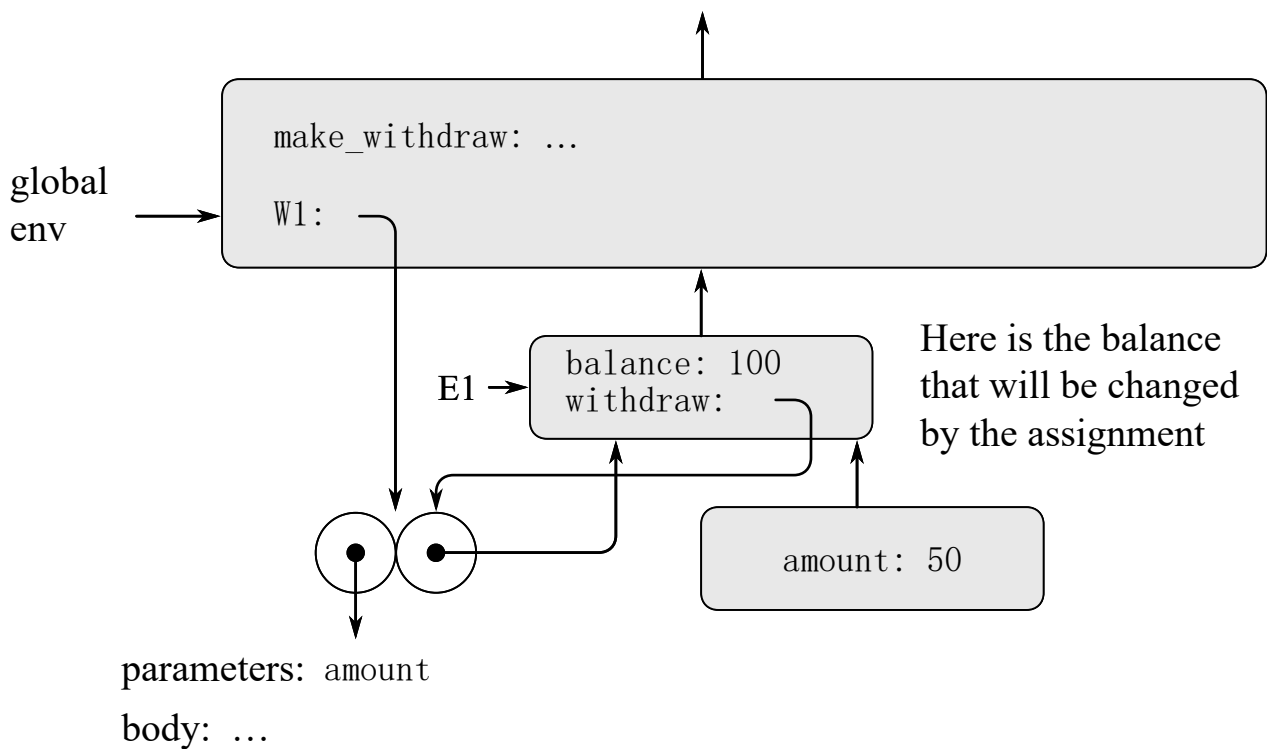


Figure 3.11. Environments created by applying the function object `W1`.

When the reassignment is executed, the binding of `balance` in `E1` is changed. At the completion of the call to `W1`, `balance` is 50, and the frame that contains `balance` is still pointed to by the function object `W1`. The frame that binds `amount` (in which we executed the code that changed `balance`) is no longer relevant, since the function call that constructed it has terminated, and there are no pointers to that frame from other parts of the environment. The next time `W1` is called, this will build a new frame that binds `amount` and whose enclosing environment is `E1`. We see that `E1` serves as the "place" that holds the local state variable for the function object `W1`. Figure 3.12 shows the situation after the call to `W1`.

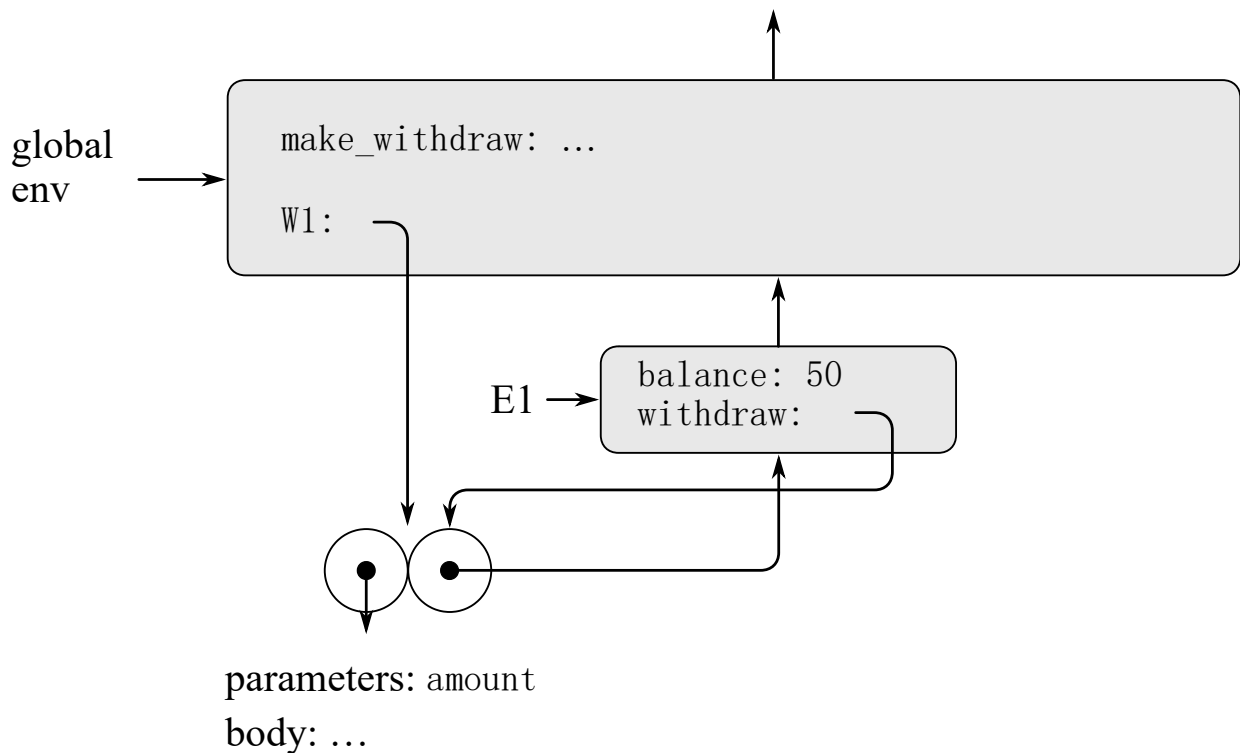


Figure 3.12. Environments after the call to `W1`.

Observe what happens when we create a second "withdraw" object by making another call to `make_withdraw`:

```
W2 = make_withdraw(100)
```

This produces the environment structure of figure 3.13, which shows that `W2` is a function object, that is, a pair with some code and an environment. The environment `E2` for `W2` was created by the call to `make_withdraw`. It contains a frame with its own local binding for `balance`. On the other hand, `W1` and `W2` have the same code: the code specified by the function definition of `withdraw` in the body of `make_withdraw`. [¹⁵ Whether `W1` and `W2` share the same physical code stored in the computer, or whether they each keep a copy of the code, is a detail of the implementation. For the interpreter we implement in chapter , the code is in fact shared.] We see here why `W1` and `W2` behave as independent objects. Calls to `W1` reference the state variable `balance` stored in `E1`, whereas calls to `W2` reference the `balance` stored in `E2`. Thus, changes to the local state of one object do not affect the other object.

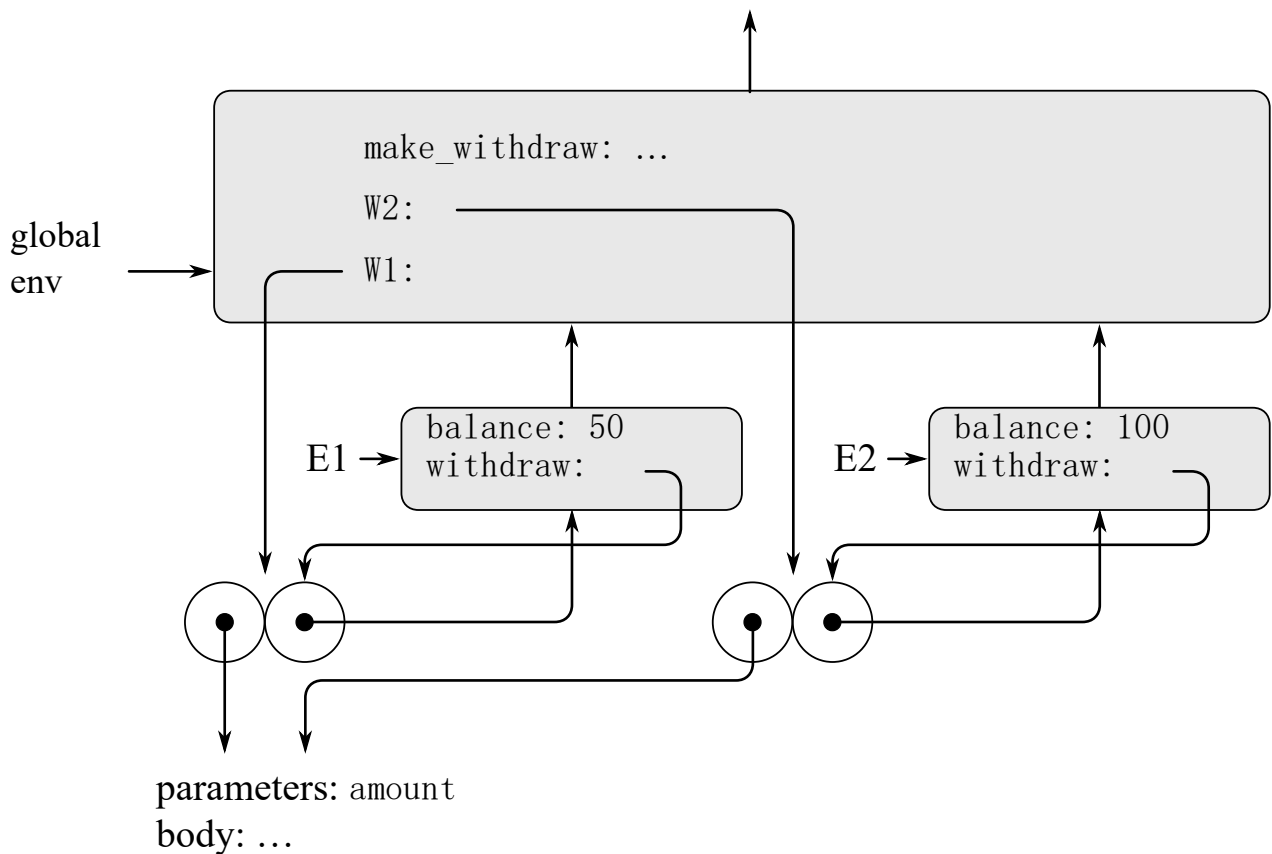


Figure 3.13. Using `W2 = make_withdraw(100)` to create a second object.

Exercise 3.10. In the `make_withdraw` function the local variable `balance` is created as a parameter of `make_withdraw`. We could also create the local state variable separately, using what we might call an *immediately invoked function definition* as follows:

```
def make_withdraw(initial_amount):
    def init(balance):
        def withdraw(amount):
            nonlocal balance
            if balance >= amount:
                balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
        return withdraw
    return init(initial_amount)
```

The `init` function is invoked immediately after it is evaluated. Its only purpose is to create a local variable `balance` and initialize it to `initial_amount`. Use the environment model to analyze this alternate version of `make_withdraw`, drawing figures like the ones above to illustrate the interactions

```
W1 = make_withdraw(100)

W1(50)

W2 = make_withdraw(100)
```

Show that the two versions of `make_withdraw` create objects with the same behavior. How do the environment structures differ for the two versions?

Exercise 3.11. In section 3.2.4, we saw how internal declarations work. Now we have seen how the environment model described the behavior of functions with local state. A typical message-passing function contains both of these aspects. Consider the bank account function of section 3.1.1:

```
def make_account(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    def dispatch(m):
        return (withdraw if m == "withdraw"
                else deposit if m == "deposit"
                else error("Unknown request: make_account", m))
    return dispatch
```

Show the environment structure generated by the sequence of interactions

```
acc = make_account(50)
```

```
print(acc("deposit")(40))
```

```
90
```

```
print(acc("withdraw")(60))
```

```
30
```

Where is the local state for `acc` kept? Suppose we define another account

```
acc2 = make_account(100)
```

How are the local states for the two accounts kept distinct? Which parts of the environment structure are shared between `acc` and `acc2` ?

3.3 Modeling with Mutable Data

Chapter 2 dealt with compound data as a means for constructing computational objects that have several parts, in order to model real-world objects that have several aspects. In that chapter we introduced the discipline of data abstraction, according to which data structures are specified in terms of constructors, which create data objects, and selectors, which access the parts of compound data objects. But we now know that there is another aspect of data that chapter 2 did not address. The desire to model systems composed of objects that have changing state leads us to the need to modify compound data objects, as well as to construct and select from them. In order to model compound objects with changing state, we will design data abstractions to include, in addition to selectors and constructors, operations called *mutators*, which modify data objects. For instance, modeling a banking system requires us to change account balances. Thus, a data structure for representing bank accounts might admit an operation

```
set_balance(*account*, *new-value*)
```

that changes the balance of the designated account to the designated new value. Data objects for which mutators are defined are known as *mutable data objects*.

Chapter 2 introduced pairs as a general-purpose "glue" for synthesizing compound data. We begin this section by defining basic mutators for pairs, so that pairs can serve as building blocks for constructing mutable data objects. These mutators greatly enhance the representational power of pairs, enabling us to build data structures other than the sequences and trees that we worked with in section 2.2. We also present some examples of simulations in which complex systems are modeled as collections of objects with local state.

3.3.1 Mutable List Structure

The basic operations on pairs—`pair` , `head` , and `tail` —can be used to construct list structure and to select parts from list structure, but they are incapable of modifying list structure. The same is true of the list operations we have used so far, such as `append` and `list` , since these can be defined in terms of `pair` , `head` , and `tail` . To modify list structures we need new operations.

The primitive mutators for pairs are `set_head` and `set_tail` .The function `set_head` takes two arguments, the first of which must be a pair. It modifies this pair, replacing the `head` pointer by a

pointer to the second argument of `set_head` . [^16: The functions `set_head` and `set_tail` return the value `None` . They should be used only for their effect.]

As an example, suppose that `x` is bound to `l1ist(l1ist("a", "b"), "c", "d")` and `y` to `l1ist("e", "f")` as illustrated in figure 3.14. Evaluating the expression `set_head(x, y)` modifies the pair to which `x` is bound, replacing its `head` by the value of `y` . The result of the operation is shown in figure 3.15. The structure `x` has been modified and is now equivalent to `l1ist(l1ist("e", "f"), "c", "d")` . The pairs representing the list `l1ist("a", "b")` , identified by the pointer that was replaced, are now detached from the original structure. [^17: We see from this that mutation operations on lists can create "garbage" that is not part of any accessible structure. We will see in section that Python memory-management systems include a *garbage collector*, which identifies and recycles the memory space used by unneeded pairs.]

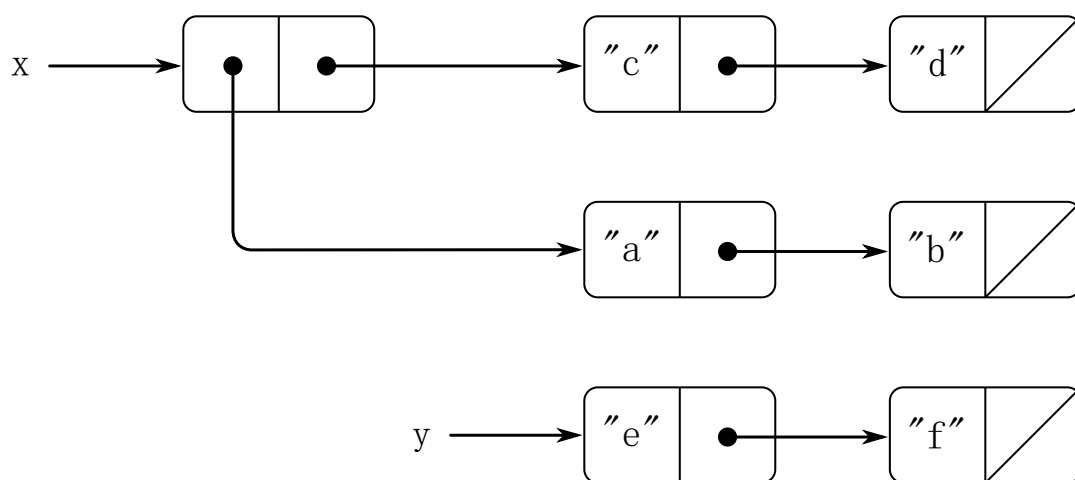


Figure 3.14. Lists `x : l1ist(l1ist("a", "b"), "c", "d")` and `y : l1ist("e", "f")` .

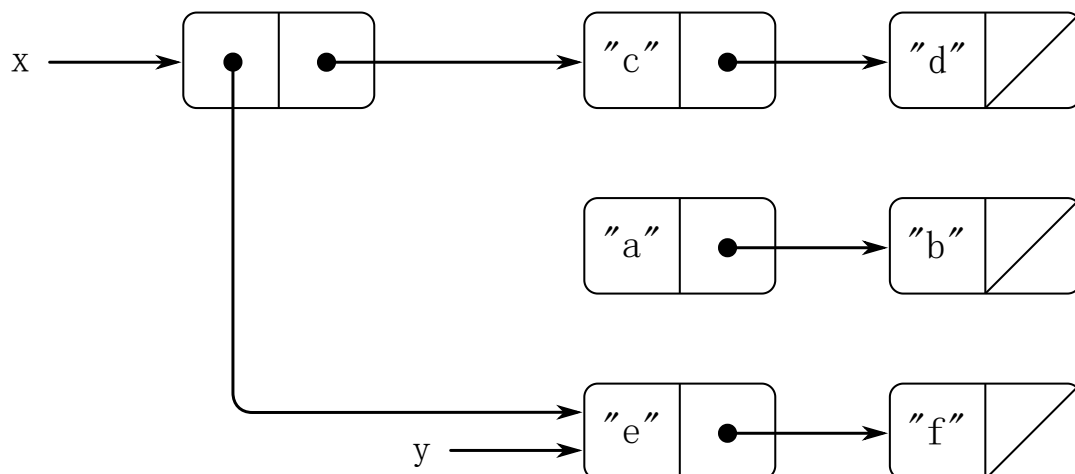


Figure 3.15. Effect of `set_head(x, y)` on the lists in figure 3.14.

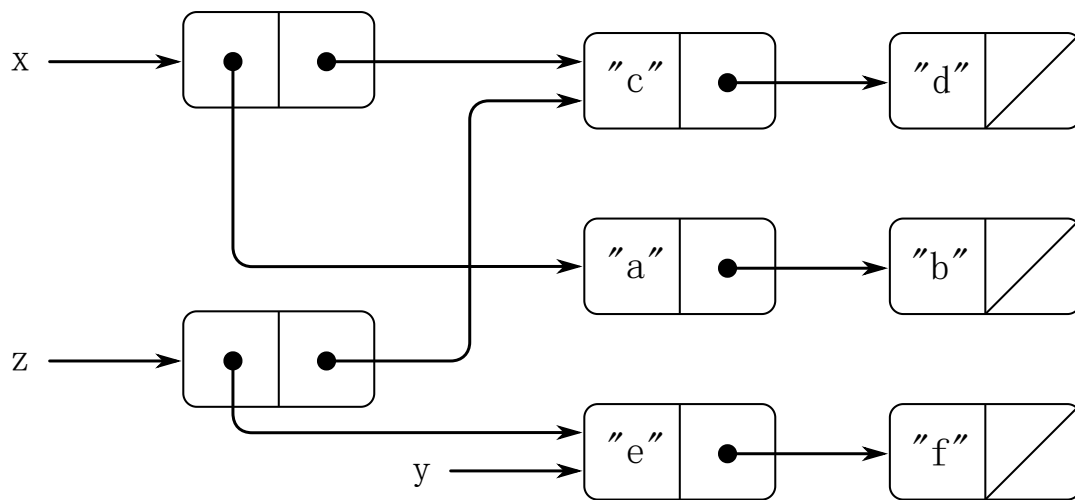


Figure 3.16. Effect of `z = pair(y, tail(x))` on the lists in figure 3.14.

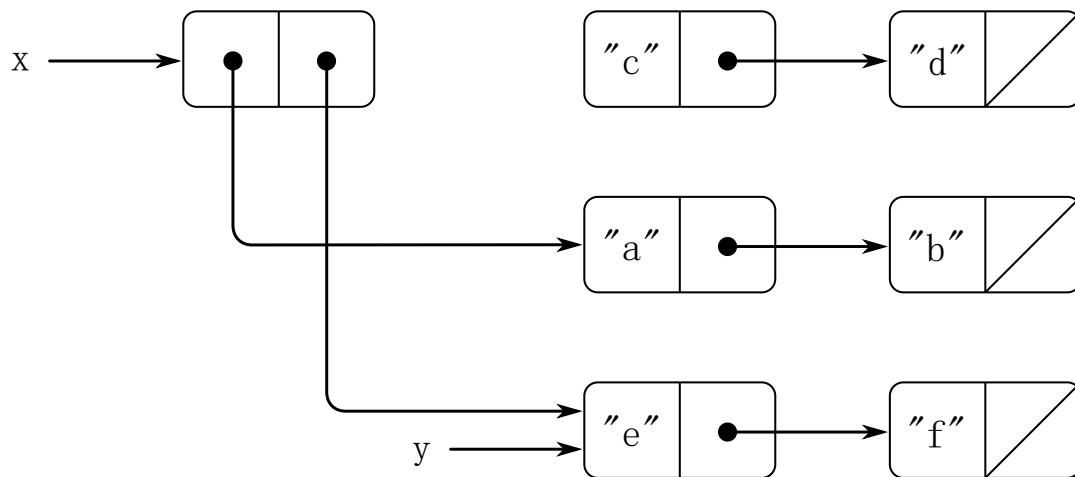


Figure 3.17. Effect of `set_tail(x, y)` on the lists in figure 3.14.

Compare figure 3.15 with figure 3.16, which illustrates the result of executing

```
z = pair(y, tail(x))
```

with `x` and `y` bound to the original lists of figure 3.14. The name `z` is now bound to a new pair created by the `pair` operation; the list to which `x` is bound is unchanged.

The `set_tail` operation is similar to `set_head`. The only difference is that the `tail` pointer of the pair, rather than the `head` pointer, is replaced. The effect of executing `set_tail(x, y)` on the lists of figure 3.14 is shown in figure 3.17. Here the `tail` pointer of `x` has been replaced by the pointer to `llist("e", "f")`. Also, the list `llist("c", "d")`, which used to be the `tail` of `x`, is now detached from the structure.

The function `pair` builds new list structure by creating new pairs, whereas `set_head` and `set_tail` modify existing pairs. Indeed, we could implement `pair` in terms of the two mutators, together with a function `get_new_pair`, which returns a new pair that is not part of any existing list structure. We obtain the new pair, set its `head` and `tail` pointers to the designated objects, and return the new pair as the result of the `pair`. [^18: Section will show how a memory-management system can implement `get_new_pair`.]

```
def pair(x, y):
    fresh = get_new_pair()
    set_head(fresh, x)
    set_tail(fresh, y)
    return fresh
```

Exercise 3.12. The following function for appending lists was introduced in section 2.2.1:

```
def append(x, y):
    return (y if is_none(x)
            else pair(head(x), append(tail(x), y)))
```

The function `append` forms a new list by successively adjoining the elements of `x` to the front of `y`. The function `append_mutator` is similar to `append`, but it is a mutator rather than a constructor. It appends the lists by splicing them together, modifying the final pair of `x` so that its `tail` is now `y`. (It is an error to call `append_mutator` with an empty `x`.)

```
def append_mutator(x, y):
    set_tail(last_pair(x), y)
    return x
```

Here `last_pair` is a function that returns the last pair in its argument:

```
def last_pair(x):
    return (x if is_none(tail(x))
            else last_pair(tail(x)))
```

Consider the interaction

```
x = llist("a", "b")
```

```
y = llist("c", "d")
```

```
z = append(x, y)
```



```
print(z)
```

```
["a", ["b", ["c", ["d", None]]]]
```

```
print(tail(x))
```

```
*response*
```

```
w = append_mutator(x, y)
```

```
print(w)
```

```
["a", ["b", ["c", ["d", None]]]]
```

```
print(tail(x))
```

```
*response*
```

What are the missing *responses*? Draw box-and-pointer diagrams to explain your answer.

Exercise 3.13. Consider the following `make_cycle` function, which uses the `last_pair` function defined in exercise 3.12:

```
def make_cycle(x):  
    set_tail(last_pair(x), x)  
    return x
```

Draw a box-and-pointer diagram that shows the structure `z` created by

```
z = make_cycle(list("a", "b", "c"))
```

What happens if we try to compute `last_pair(z)` ?

Exercise 3.14. The following function is quite useful, although obscure:

```
def mystery(x):
    def loop(x, y):
        if is_none(x):
            return y
        else:
            temp = tail(x)
            set_tail(x, y)
            return loop(temp, x)
    return loop(x, None)
```

The function `loop` uses the "temporary" name `temp` to hold the old value of the `tail` of `x`, since the `set_tail` on the next line destroys the `tail`. Explain what `mystery` does in general. Suppose `v` is defined by

```
v = llist("a", "b", "c", "d")
```

Draw the box-and-pointer diagram that represents the list to which `v` is bound. Suppose that we now evaluate

```
w = mystery(v)
```

Draw box-and-pointer diagrams that show the structures `v` and `w` after evaluating this program. What would be printed as the values of `v` and `w`?

Sharing and identity

We mentioned in section 3.1.3 the theoretical issues of "sameness" and "change" raised by the introduction of reassignment. These issues arise in practice when individual pairs are *shared* among different data objects. For example, consider the structure formed by

```
x = llist("a", "b")
z1 = pair(x, x)
```

As shown in figure 3.18, `z1` is a pair whose `head` and `tail` both point to the same pair `x`. This sharing of `x` by the `head` and `tail` of `z1` is a consequence of the straightforward way in which `pair` is implemented. In general, using `pair` to construct lists will result in an interlinked structure of pairs in which many individual pairs are shared by many different structures.

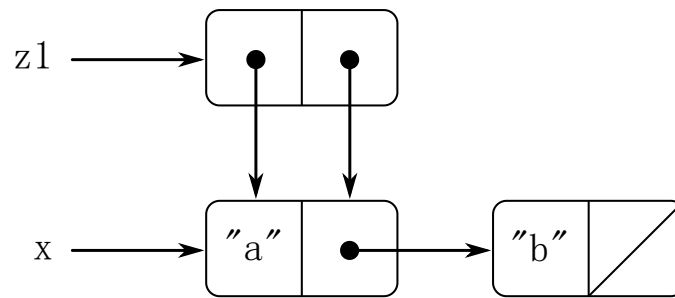


Figure 3.18. The list `z1` formed by `pair(x, x)`.

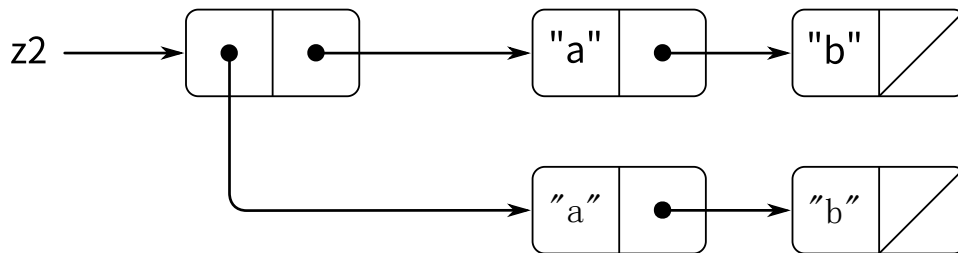


Figure 3.19. The list `z2` formed by `pair(llist("a", "b"), llist("a", "b"))`.

In contrast to figure 3.18, figure 3.19 shows the structure created by

```
z2 = pair(llist("a", "b"), llist("a", "b"))
```

In this structure, the pairs in the two `llist("a", "b")` lists are distinct, although they contain the same strings. [^19: The two pairs are distinct because each call to `pair` returns a new pair. The strings are "the same" in the sense that they are primitive data (just like numbers) that are composed of the same characters in the same order. Since Python provides no way to mutate a string, any sharing that the designers of a Python interpreter might decide to implement for strings is undetectable. We consider primitive data such as numbers, booleans, and strings to be *identical* if and only if they are *indistinguishable*.]

When thought of as a list, `z1` and `z2` both represent "the same" list:

```
print(llist(llist("a", "b"), "a", "b"))
```

In general, sharing is completely undetectable if we operate on lists using only `pair`, `head`, and `tail`. However, if we allow mutators on list structure, sharing becomes significant. As an example of the difference that sharing can make, consider the following function, which modifies the `head` of the structure to which it is applied:

```
def set_to_wow(x):
    set_head(head(x), "wow")
    return x
```

Even though `z1` and `z2` are "the same" structure, applying `set_to_wow` to them yields different results. With `z1`, altering the `head` also changes the `tail`, because in `z1` the `head` and the `tail` are the same pair. With `z2`, the `head` and `tail` are distinct, so `set_to_wow` modifies only the `head`:

```
print(z1)
```

```
[["a", ["b", None]], ["a", ["b", None]]]
```

```
print(set_to_wow(z1))
```

```
[["wow", ["b", None]], ["wow", ["b", None]]]
```

```
print(z2)
```

```
[["a", ["b", None]], ["a", ["b", None]]]
```

```
print(set_to_wow(z2))
```

```
[["wow", ["b", None]], ["a", ["b", None]]]
```

One way to detect sharing in list structures is to use the built-in predicate `is`. When applied to two nonprimitive values, `x is y` tests whether `x` and `y` are the same object (that is, whether `x` and `y` are equal as pointers). Thus, with `z1` and `z2` as defined in figure 3.18 and 3.19, `head(z1) is tail(z1)` is true and `head(z2) is tail(z2)` is false.

As will be seen in the following sections, we can exploit sharing to greatly extend the repertoire of data structures that can be represented by pairs. On the other hand, sharing can also be dangerous, since modifications made to structures will also affect other structures that happen to share the modified parts. The mutation operations `set_head` and `set_tail` should be used with care; unless we have a good understanding of how our data objects are shared, mutation can have unanticipated results. [^20: The subtleties of dealing with sharing of mutable data objects reflect the underlying issues of "sameness" and "change" that were raised in section 3.1.3. We mentioned there that admitting change to our language requires that a compound object must have an "identity" that is something different from the pieces from which it is composed. In Python, we

consider this "identity" to be the quality that is tested by `is`, i.e., by equality of pointers. Since in most Python implementations a pointer is essentially a memory address, we are "solving the problem" of defining the identity of objects by stipulating that a data object "itself" is the information stored in some particular set of memory locations in the computer. This suffices for simple Python programs, but is hardly a general way to resolve the issue of "sameness" in computational models.]

Exercise 3.15. Draw box-and-pointer diagrams to explain the effect of `set_to_wow` on the structures `z1` and `z2` above.

Exercise 3.16. Ben Bitdiddle decides to write a function to count the number of pairs in any list structure. "It's easy," he reasons. "The number of pairs in any structure is the number in the `head` plus the number in the `tail` plus one more to count the current pair." So Ben writes the following function

```
def count_pairs(x):
    return (0 if not is_pair(x)
            else count_pairs(head(x)) +
                count_pairs(tail(x)) +
                1)
```

Show that this function is not correct. In particular, draw box-and-pointer diagrams representing list structures made up of exactly three pairs for which Ben's function would return 3; return 4; return 7; never return at all.

Exercise 3.17. Devise a correct version of the `count_pairs` function of exercise 3.16 that returns the number of distinct pairs in any structure. (Hint: Traverse the structure, maintaining an auxiliary data structure that is used to keep track of which pairs have already been counted.)

Exercise 3.18. Write a function that examines a list and determines whether it contains a cycle, that is, whether a program that tried to find the end of the list by taking successive `tail`s would go into an infinite loop. Exercise 3.13 constructed such lists.

Exercise 3.19. Redo exercise 3.18 using an algorithm that takes only a constant amount of space. (This requires a very clever idea.)

Mutation is just reassignment

When we introduced compound data, we observed in section 2.1.3 that pairs can be represented purely in terms of functions:

```

def pair(x, y):
    def dispatch(m):
        return (x if m == "head"
                else y if m == "tail"
                else error("undefined operation -- pair", m))
    return dispatch

def head(z):
    return z("head")

def tail(z):
    return z("tail")

```

The same observation is true for mutable data. We can implement mutable data objects as functions using reassignment and local state. For instance, we can extend the above pair implementation to handle `set_head` and `set_tail` in a manner analogous to the way we implemented bank accounts using `make_account` in section 3.1.1:

```

def pair(x, y):
    def set_x(v):
        nonlocal x
        x = v
    def set_y(v):
        nonlocal y
        y = v
    return lambda m: (x if m == "head"
                      else y if m == "tail"
                      else set_x if m == "set_head"
                      else set_y if m == "set_tail"
                      else error("undefined operation -- pair", m))

def head(z):
    return z("head")

def tail(z):
    return z("tail")

def set_head(z, new_value):
    z("set_head")(new_value)
    return z

def set_tail(z, new_value):
    z("set_tail")(new_value)
    return z

```

Reassignment is all that is needed, theoretically, to account for the behavior of mutable data. As soon as we admit reassignment to our language, we raise all the issues, not only of reassignment, but of mutable data in general. [^21: On the other hand, from the viewpoint of implementation, reassignment requires us to modify the environment, which is itself a mutable data structure. Thus, reassignment and mutation are equipotent: Each can be implemented in terms of the other.]

Exercise 3.20. Draw environment diagrams to illustrate the evaluation of the sequence of statements

```
x = pair(1, 2)
z = pair(x, x)
set_head(tail(z), 17)
```

```
print(head(x))
```

```
17
```

using the functional implementation of pairs given above. (Compare exercise 3.11.)

3.3.2 Representing Queues

The mutators `set_head` and `set_tail` enable us to use pairs to construct data structures that cannot be built with `pair`, `head`, and `tail` alone. This section shows how to use pairs to represent a data structure called a queue. Section 3.3.3 will show how to represent data structures called tables.

A *queue* is a sequence in which items are inserted at one end (called the *rear* of the queue) and deleted from the other end (the *front*). Figure 3.20 shows an initially empty queue in which the items `a` and `b` are inserted. Then `a` is removed, `c` and `d` are inserted, and `b` is removed. Because items are always removed in the order in which they are inserted, a queue is sometimes called a *FIFO* (first in, first out) buffer.

Figure 3.20. Queue operations.

Operation	Resulting Queue
<code>q = make_queue()</code>	---
<code>insert_queue(q, "a")</code>	a
<code>insert_queue(q, "b")</code>	a b
<code>delete_queue(q)</code>	b
<code>insert_queue(q, "c")</code>	b c
<code>insert_queue(q, "d")</code>	b c d
<code>delete_queue(q)</code>	c d

In terms of data abstraction, we can regard a queue as defined by the following set of operations:

- a constructor: `make_queue()` returns an empty queue (a queue containing no items).
- a predicate: `is_empty_queue(queue)` tests if the queue is empty.
- a selector: `front_queue(queue)` returns the object at the front of the queue, signaling an error if the queue is empty; it does not modify the queue.
- two mutators: `insert_queue(queue, item)` inserts the item at the rear of the queue and returns the modified queue as its value. `delete_queue(queue)` removes the item at the front of the queue and returns the modified queue as its value, signaling an error if the queue is empty before the deletion.

Because a queue is a sequence of items, we could certainly represent it as an ordinary list; the front of the queue would be the `head` of the list, inserting an item in the queue would amount to appending a new element at the end of the list, and deleting an item from the queue would just be taking the `tail` of the list. However, this representation is inefficient, because in order to insert an item we must scan the list until we reach the end. Since the only method we have for scanning a list is by successive `tail` operations, this scanning requires $\Theta(n)$ steps for a list of n items. A simple modification to the list representation overcomes this disadvantage by allowing the queue operations to be implemented so that they require $\Theta(1)$ steps; that is, so that the number of steps needed is independent of the length of the queue.

The difficulty with the list representation arises from the need to scan to find the end of the list. The reason we need to scan is that, although the standard way of representing a list as a chain of pairs readily provides us with a pointer to the beginning of the list, it gives us no easily accessible pointer to the end. The modification that avoids the drawback is to represent the queue as a list, together with an additional pointer that indicates the final pair in the list. That way, when we go to insert an item, we can consult the rear pointer and so avoid scanning the list.

A queue is represented, then, as a pair of pointers, `front_ptr` and `rear_ptr`, which indicate, respectively, the first and last pairs in an ordinary list. Since we would like the queue to be an identifiable object, we can use `pair` to combine the two pointers. Thus, the queue itself will be the `pair` of the two pointers. Figure 3.21 illustrates this representation.

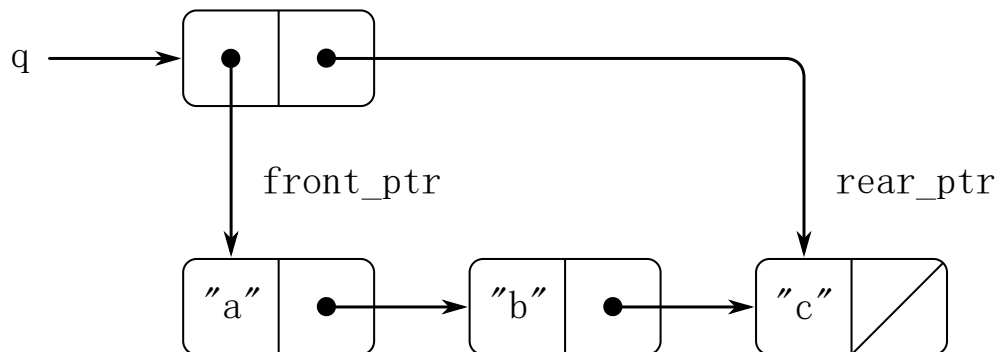


Figure 3.21. Implementation of a queue as a list with front and rear pointers.

To define the queue operations we use the following functions, which enable us to select and to modify the front and rear pointers of a queue:


```
def front_ptr(queue):
    return head(queue)

def rear_ptr(queue):
    return tail(queue)

def set_front_ptr(queue, item):
    set_head(queue, item)

def set_rear_ptr(queue, item):
    set_tail(queue, item)
```

Now we can implement the actual queue operations. We will consider a queue to be empty if its front pointer is the empty list:

```
def is_empty_queue(queue):
    return is_none(front_ptr(queue))
```

The `make_queue` constructor returns, as an initially empty queue, a pair whose `head` and `tail` are both the empty list:

```
def make_queue():
    return pair(None, None)
```

To select the item at the front of the queue, we return the `head` of the pair indicated by the front pointer:

```
def front_queue(queue):
    return (error("front_queue called with an empty queue", queue)
            if is_empty_queue(queue)
            else head(front_ptr(queue)))
```

To insert an item in a queue, we follow the method whose result is indicated in figure 3.22. We first create a new pair whose `head` is the item to be inserted and whose `tail` is the empty list. If the queue was initially empty, we set the front and rear pointers of the queue to this new pair. Otherwise, we modify the final pair in the queue to point to the new pair, and also set the rear pointer to the new pair.

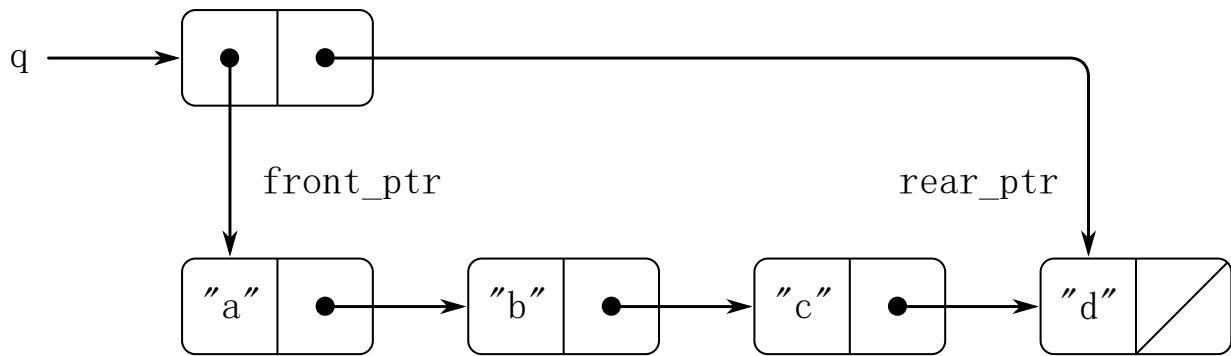


Figure 3.22. Result of using `insert_queue(q, "d")` on the queue of figure 3.21.

```
def insert_queue(queue, item):
    new_pair = pair(item, None)
    if is_empty_queue(queue):
        set_front_ptr(queue, new_pair)
        set_rear_ptr(queue, new_pair)
    else:
        set_tail(rear_ptr(queue), new_pair)
        set_rear_ptr(queue, new_pair)
    return queue
```

To delete the item at the front of the queue, we merely modify the front pointer so that it now points at the second item in the queue, which can be found by following the `tail` pointer of the first item (see figure 3.23): [^22: If the first item is the final item in the queue, the front pointer will be the empty list after the deletion, which will mark the queue as empty; we needn't worry about updating the rear pointer, which will still point to the deleted item, because `is_empty_queue` looks only at the front pointer.]

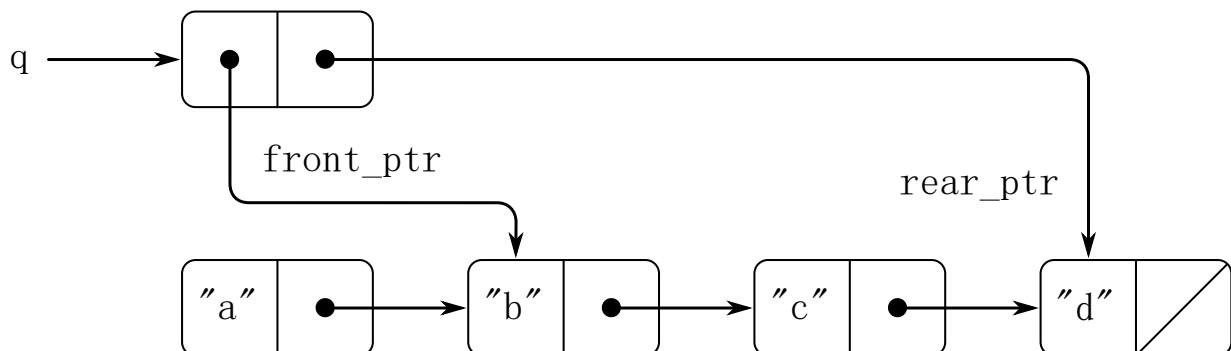


Figure 3.23. Result of using `delete_queue(q)` on the queue of figure 3.22.

```
def delete_queue(queue):
    if is_empty_queue(queue):
        error("delete_queue called with an empty queue", queue)
    else:
        set_front_ptr(queue, tail(front_ptr(queue)))
        return queue
```

Exercise 3.21. Ben Bitdiddle decides to test the queue implementation described above. He types in the functions to the Python interpreter and proceeds to try them out:

```
q1 = make_queue()
```

```
print(insert_queue(q1, "a"))
```

```
["a", None], ["a", None]
```

```
print(insert_queue(q1, "b"))
```

```
["a", ["b", None]], ["b", None]
```

```
print(delete_queue(q1))
```

```
["b", None], ["b", None]
```

```
print(delete_queue(q1))
```

```
[None, ["b", None]]
```

"It's all wrong!" he complains. "The interpreter's response shows that the last item is inserted into the queue twice. And when I delete both items, the second `b` is still there, so the queue isn't empty, even though it's supposed to be." Eva Lu Ator suggests that Ben has misunderstood what is happening. "It's not that the items are going into the queue twice," she explains. "It's just that the standard Python printer doesn't know how to make sense of the queue representation. If you want to see the queue printed correctly, you'll have to define your own print function for queues." Explain what Eva Lu is talking about. In particular, show why Ben's examples produce the printed results that they do. Define a function `print_queue` that takes a queue as input and prints the sequence of items in the queue.

Exercise 3.22. Instead of representing a queue as a pair of pointers, we can build a queue as a function with local state. The local state will consist of pointers to the beginning and the end of an ordinary list. Thus, the `make_queue` function will have the form

```
def make_queue(): front_ptr =  $\dots$  rear_ptr =  $\dots$  declarations of internal functions
def dispatch(m):  $\dots$  return dispatch
```

Complete the definition of `make_queue` and provide implementations of the queue operations using this representation.

Exercise 3.23. A *deque* ("double-ended queue") is a sequence in which items can be inserted and deleted either at the front or at the rear. Operations on deques are the constructor `make_deque`, the predicate `is_empty_deque`, selectors `front_deque` and `rear_deque`, and mutators `front_insert_deque`, `front_delete_deque`, `rear_insert_deque`, and `rear_delete_deque`. Show how to represent deques using pairs, and give implementations of the operations. [²³: Be careful not to make the interpreter try to print a structure that contains cycles. (See exercise 3.13.)] All operations should be accomplished in $\Theta(1)$ steps.

3.3.3 Representing Tables

When we studied various ways of representing sets in chapter 2, we mentioned in section 2.3.3 the task of maintaining a table of records indexed by identifying keys. In the implementation of data-directed programming in section 2.4.3, we made extensive use of two-dimensional tables, in which information is stored and retrieved using two keys. Here we see how to build tables as mutable list structures.

We first consider a one-dimensional table, in which each value is stored under a single key. We implement the table as a list of records, each of which is implemented as a pair consisting of a key and the associated value. The records are glued together to form a list by pairs whose `head`s point to successive records. These gluing pairs are called the *backbone* of the table. In order to have a place that we can change when we add a new record to the table, we build the table as a *headed list*. A headed list has a special backbone pair at the beginning, which holds a dummy "record"—in this case the arbitrarily chosen string `"*table*"`. Figure 3.24 shows the box-and-pointer diagram for the table

```
a: 1
b: 2
c: 3
```

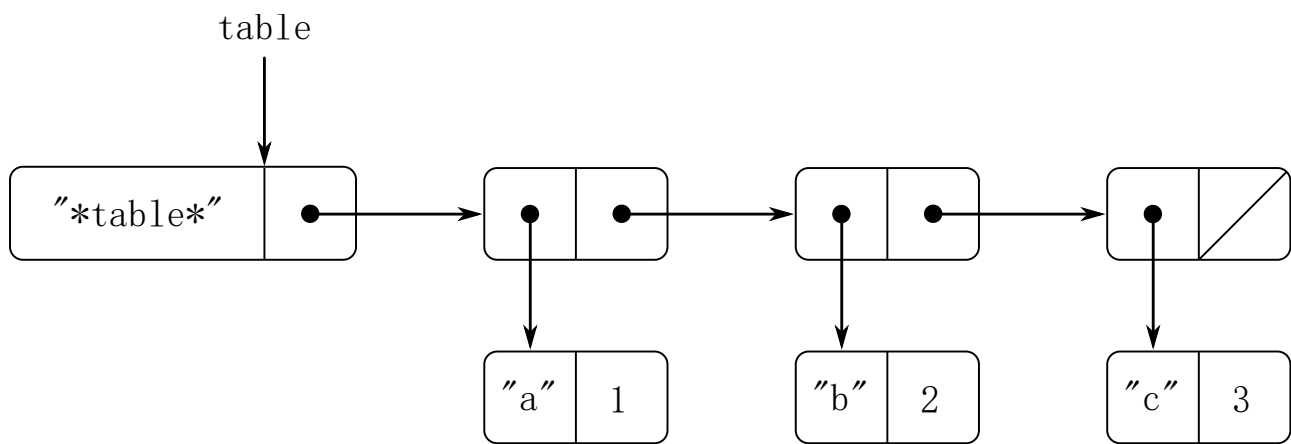


Figure 3.24. A table represented as a headed list.

To extract information from a table we use the `lookup` function, which takes a key as argument and returns the associated value (or `None` if there is no value stored under that key). The function `lookup` is defined in terms of the `assoc` operation, which expects a key and a list of records as arguments. Note that `assoc` never sees the dummy record. The function `assoc` returns the record that has the given key as its `head`. [²⁴: Because `assoc` uses `equal`, it can recognize keys that are strings, numbers, or list structure.] The function `lookup` then checks to see that the resulting record returned by `assoc` is not `None`, and returns the value (the `tail`) of the record.

```

def lookup(key, table):
    record = assoc(key, tail(table))
    return (None
            if is_none(record)
            else tail(record))

def assoc(key, records):
    return (None
            if is_none(records)
            else head(records)
            if key == head(head(records))
            else assoc(key, tail(records)))
  
```

To insert a value in a table under a specified key, we first use `assoc` to see if there is already a record in the table with this key. If not, we form a new record by `pair` ing the key with the value, and insert this at the head of the table's list of records, after the dummy record. If there already is a record with this key, we set the `tail` of this record to the designated new value. The header of the table provides us with a fixed location to modify in order to insert the new record. [²⁵: Thus, the first backbone pair is the object that represents the table "itself"; that is, a pointer to the table is a pointer to this pair. This same backbone pair always starts the table. If we did not arrange things in this way, `insert` would have to return a new value for the start of the table when it added a new record.]

```
def insert(key, value, table):
    record = assoc(key, tail(table))
    if is_none(record):
        set_tail(table,
                  pair(pair(key, value), tail(table)))
    else:
        set_tail(record, value)
    return "ok"
```

To construct a new table, we simply create a list containing just the string `"*table*"` :

```
def make_table():
    return llist("*table*")
```

Two-dimensional tables

In a two-dimensional table, each value is indexed by two keys. We can construct such a table as a one-dimensional table in which each key identifies a subtable. Figure 3.25 shows the box-and-pointer diagram for the table

```
"math":
  "+": 43
  "-": 45
  "*": 42
"letters":
  "a": 97
  "b": 98
```

which has two subtables. (The subtables don't need a special header string, since the key that identifies the subtable serves this purpose.)

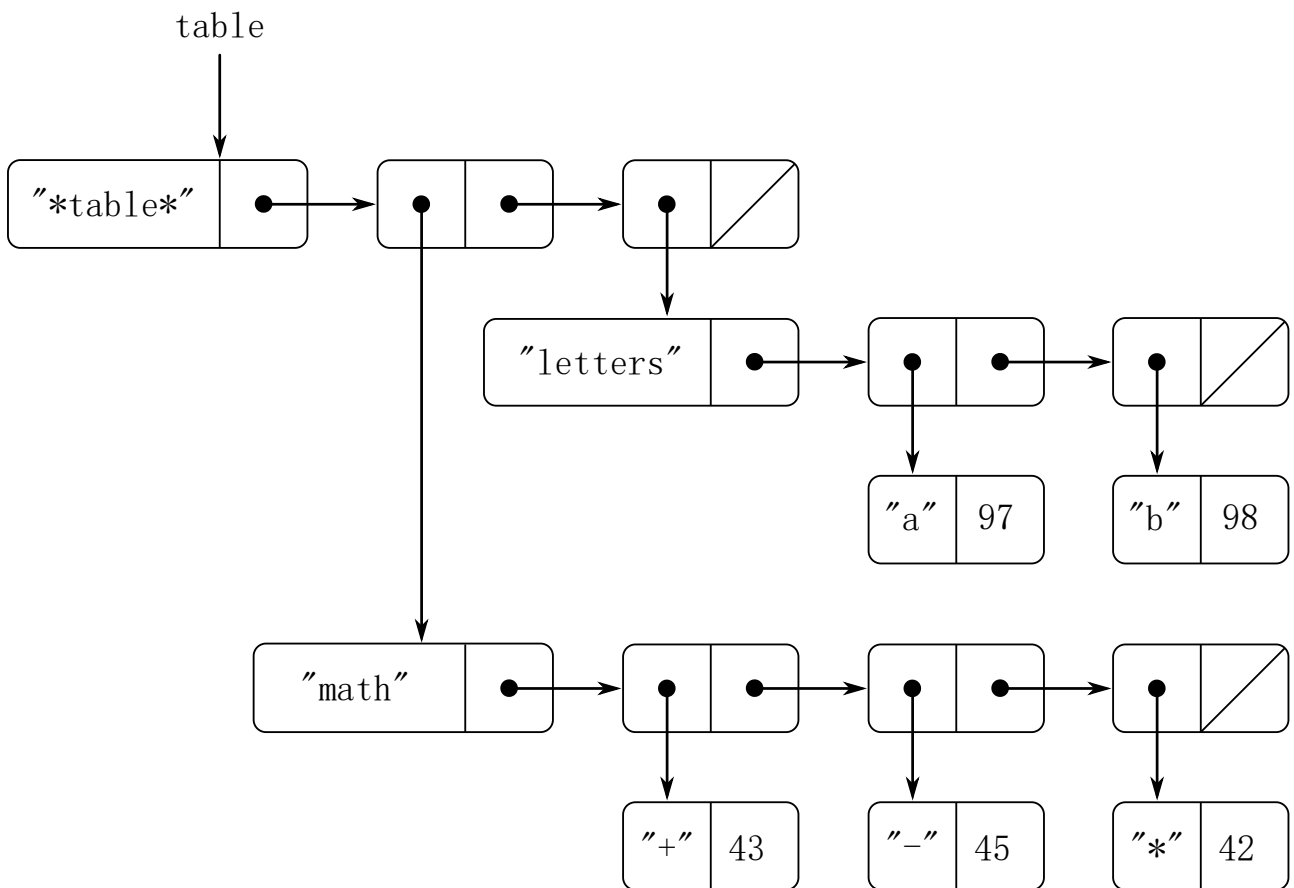


Figure 3.25. A two-dimensional table.

When we look up an item, we use the first key to identify the correct subtable. Then we use the second key to identify the record within the subtable.

```

def lookup(key_1, key_2, table):
    subtable = assoc(key_1, tail(table))
    if is_none(subtable):
        return None
    else:
        record = assoc(key_2, tail(subtable))
        return (None
                if is_none(record)
                else tail(record))
  
```

To insert a new item under a pair of keys, we use `assoc` to see if there is a subtable stored under the first key. If not, we build a new subtable containing the single record (`key_2` , `value`) and insert it into the table under the first key. If a subtable already exists for the first key, we insert the new record into this subtable, using the insertion method for one-dimensional tables described above:

```

def insert(key_1, key_2, value, table):
    subtable = assoc(key_1, tail(table))
    if is_none(subtable):
        set_tail(table,
                  pair(1list(key_1, pair(key_2, value)), tail(table)))
    else:
        record = assoc(key_2, tail(subtable))
        if is_none(record):
            set_tail(subtable,
                    pair(pair(key_2, value), tail(subtable)))
        else:
            set_tail(record, value)
    return "ok"

```

Creating local tables

The `lookup` and `insert` operations defined above take the table as an argument. This enables us to use programs that access more than one table. Another way to deal with multiple tables is to have separate `lookup` and `insert` functions for each table. We can do this by representing a table procedurally, as an object that maintains an internal table as part of its local state. When sent an appropriate message, this "table object" supplies the function with which to operate on the internal table. Here is a generator for two-dimensional tables represented in this fashion:


```

def make_table():
    local_table = llist("*table*")
    def lookup(key_1, key_2):
        subtable = assoc(key_1, tail(local_table))
        if is_none(subtable):
            return None
        else:
            record = assoc(key_2, tail(subtable))
            return (None
                    if is_none(record)
                    else tail(record))
    def insert(key_1, key_2, value):
        subtable = assoc(key_1, tail(local_table))
        if is_none(subtable):
            set_tail(local_table,
                    pair(llist(key_1, pair(key_2, value)),
                        tail(local_table)))
        else:
            record = assoc(key_2, tail(subtable))
            if is_none(record):
                set_tail(subtable,
                        pair(pair(key_2, value), tail(subtable)))
            else:
                set_tail(record, value)
    def dispatch(m):
        return (lookup if m == "lookup"
                else insert if m == "insert"
                else error("unknown operation -- table", m))
    return dispatch

```

Using `make_table`, we could implement the `get` and `put` operations used in section 2.4.3 for data-directed programming, as follows:

```

operation_table = make_table()
get = operation_table("lookup")
put = operation_table("insert")

```

The function `get` takes as arguments two keys, and `put` takes as arguments two keys and a value. Both operations access the same local table, which is encapsulated within the object created by the call to `make_table`.

Exercise 3.24. In the table implementations above, the keys are tested for equality using `equal` (called by `assoc`). This is not always the appropriate test. For instance, we might have a table with numeric keys in which we don't need an exact match to the number we're looking up, but only a number within some tolerance of it. Design a table constructor `make_table` that takes as an argument a `same_key` function that will be used to test "equality" of keys. The function `make_table` should return a `dispatch` function that can be used to access appropriate `lookup` and `insert` functions for a local table.

Exercise 3.25. Generalizing one- and two-dimensional tables, show how to implement a table in which values are stored under an arbitrary number of keys and different values may be stored under different numbers of keys. The `lookup` and `insert` functions should take as input a list of keys used to access the table.

Exercise 3.26. To search a table as implemented above, one needs to scan through the list of records. This is basically the unordered list representation of section 2.3.3. For large tables, it may be more efficient to structure the table in a different manner. Describe a table implementation where the (key, value) records are organized using a binary tree, assuming that keys can be ordered in some way (e.g., numerically or alphabetically). (Compare exercise 2.66 of chapter 2.)

Exercise 3.27. *Memoization* (also called *tabulation*) is a technique that enables a function to record, in a local table, values that have previously been computed. This technique can make a vast difference in the performance of a program. A memoized function maintains a table in which values of previous calls are stored using as keys the arguments that produced the values. When the memoized function is asked to compute a value, it first checks the table to see if the value is already there and, if so, just returns that value. Otherwise, it computes the new value in the ordinary way and stores this in the table. As an example of memoization, recall from section 1.2.2 the exponential process for computing Fibonacci numbers:

```
def fib(n):  
    return (0 if n == 0  
           else 1 if n == 1  
           else fib(n - 1) + fib(n - 2))
```

The memoized version of the same function is

```
memo_fib = memoize(lambda n: (0  
                             if n == 0  
                             else 1  
                             if n == 1  
                             else memo_fib(n - 1) +  
                             memo_fib(n - 2)))
```

where the memoizer is defined as

```

def memoize(f):
    table = make_table()
    def memoized(x):
        previously_computed_result = lookup(x, table)
        if is_none(previously_computed_result):
            result = f(x)
            insert(x, result, table)
            return result
        else:
            return previously_computed_result
    return memoized

```

Draw an environment diagram to analyze the computation of `memo_fib(3)`. Explain why `memo_fib` computes the n th Fibonacci number in a number of steps proportional to n . Would the scheme still work if we had simply defined `memo_fib` to be `memoize(fib)`?

3.3.4 A Simulator for Digital Circuits

Designing complex digital systems, such as computers, is an important engineering activity. Digital systems are constructed by interconnecting simple elements. Although the behavior of these individual elements is simple, networks of them can have very complex behavior. Computer simulation of proposed circuit designs is an important tool used by digital systems engineers. In this section we design a system for performing digital logic simulations. This system typifies a kind of program called an *event-driven simulation*, in which actions ("events") trigger further events that happen at a later time, which in turn trigger more events, and so on.

Our computational model of a circuit will be composed of objects that correspond to the elementary components from which the circuit is constructed. There are *wires*, which carry *digital signals*. A digital signal may at any moment have only one of two possible values, 0 and 1. There are also various types of digital *function boxes*, which connect wires carrying input signals to other output wires. Such boxes produce output signals computed from their input signals. The output signal is delayed by a time that depends on the type of the function box. For example, an *inverter* is a primitive function box that inverts its input. If the input signal to an inverter changes to 0, then one *inverter-delay* later the inverter will change its output signal to 1. If the input signal to an inverter changes to 1, then one *inverter-delay* later the inverter will change its output signal to 0. We draw an inverter symbolically as in figure 3.26. An *and-gate*, also shown in figure 3.26, is a primitive function box with two inputs and one output. It drives its output signal to a value that is the *logical and* of the inputs. That is, if both of its input signals become 1, then one *and-gate-delay* time later the and-gate will force its output signal to be 1; otherwise the output will be 0. An *or-gate* is a similar two-input primitive function box that drives its output signal to a value that is the *logical or* of the inputs. That is, the output will become 1 if at least one of the input signals is 1; otherwise the output will become 0.

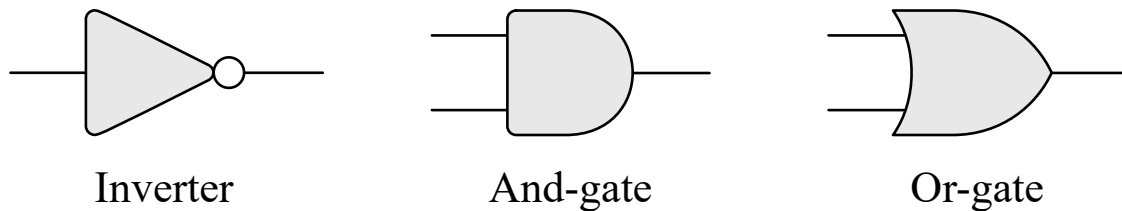


Figure 3.26. Primitive functions in the digital logic simulator.

We can connect primitive functions together to construct more complex functions. To accomplish this we wire the outputs of some function boxes to the inputs of other function boxes. For example, the *half-adder* circuit shown in figure 3.27 consists of an or-gate, two and-gates, and an inverter. It takes two input signals, A and B , and has two output signals, S and C . S will become 1 whenever precisely one of A and B is 1, and C will become 1 whenever A and B are both 1. We can see from the figure that, because of the delays involved, the outputs may be generated at different times. Many of the difficulties in the design of digital circuits arise from this fact.

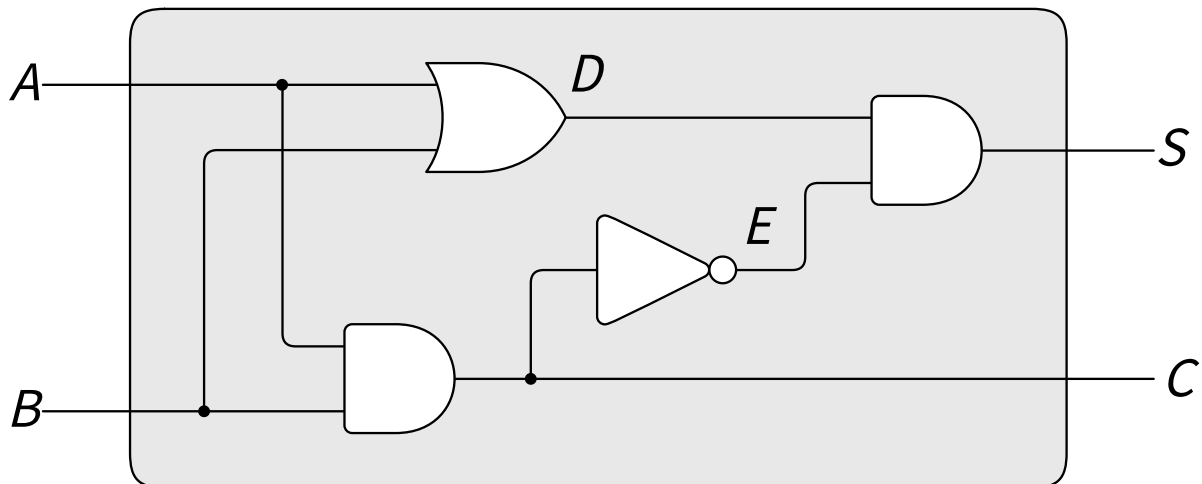


Figure 3.27. A half-adder circuit.

We will now build a program for modeling the digital logic circuits we wish to study. The program will construct computational objects modeling the wires, which will "hold" the signals. Function boxes will be modeled by functions that enforce the correct relationships among the signals.

One basic element of our simulation will be a function `make_wire`, which constructs wires. For example, we can construct six wires as follows:

```
a = make_wire()
b = make_wire()
c = make_wire()
d = make_wire()
e = make_wire()
s = make_wire()
```

We attach a function box to a set of wires by calling a function that constructs that kind of box. The arguments to the constructor function are the wires to be attached to the box. For example, given that we can construct and-gates, or-gates, and inverters, we can wire together the half-adder shown in figure 3.27:

```
print(or_gate(a, b, d))
```

```
"ok"
```

```
print(and_gate(a, b, c))
```

```
"ok"
```

```
print(inverter(c, e))
```

```
"ok"
```

```
print(and_gate(d, e, s))
```

```
"ok"
```

Better yet, we can explicitly name this operation by defining a function `half_adder` that constructs this circuit, given the four external wires to be attached to the half-adder:

```
def half_adder(a, b, s, c):  
    d = make_wire()  
    e = make_wire()  
    or_gate(a, b, d)  
    and_gate(a, b, c)  
    inverter(c, e)  
    and_gate(d, e, s)  
    return "ok"
```

The advantage of making this definition is that we can use `half_adder` itself as a building block in creating more complex circuits. Figure 3.28, for example, shows a *full-adder* composed of two half-adders and an or-gate. [^26: A full-adder is a basic circuit element used in adding two binary numbers. Here A and B are the bits at corresponding positions in the two numbers to be added, and C_{in} is the carry bit from the addition one place to the right. The circuit generates SUM , which is the sum bit in the corresponding position, and

C_{out} , which is the carry bit to be propagated to the left.] We can construct a full-adder as follows:

```
def full_adder(a, b, c_in, sum, c_out):
    s = make_wire()
    c1 = make_wire()
    c2 = make_wire()
    half_adder(b, c_in, s, c1)
    half_adder(a, s, sum, c2)
    or_gate(c1, c2, c_out)
    return "ok"
```

Having defined `full_adder` as a function, we can now use it as a building block for creating still more complex circuits. (For example, see exercise 3.30.)

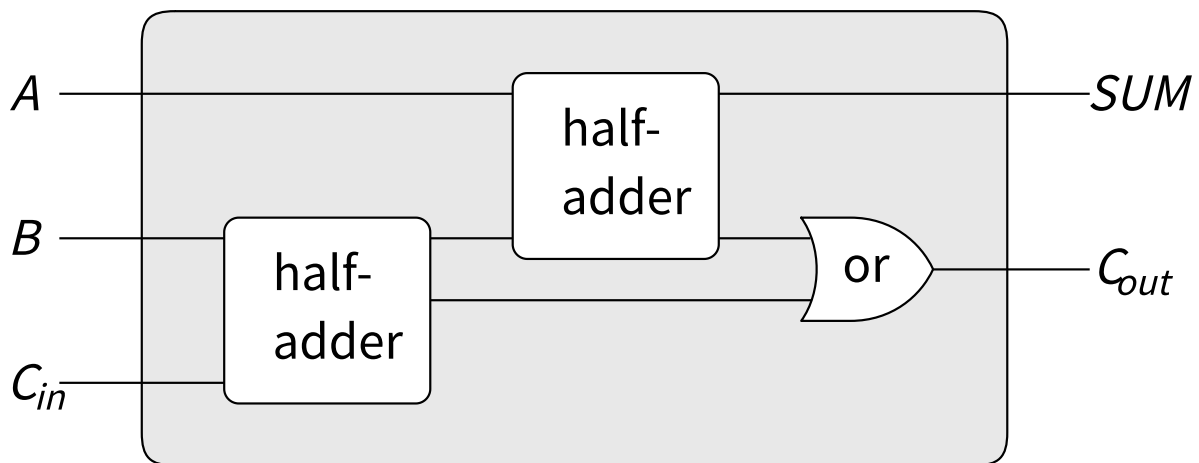


Figure 3.28. A full-adder circuit.

In essence, our simulator provides us with the tools to construct a language of circuits. If we adopt the general perspective on languages with which we approached the study of Python in section 1.1, we can say that the primitive function boxes form the primitive elements of the language, that wiring boxes together provides a means of combination, and that specifying wiring patterns as functions serves as a means of abstraction.

Primitive function boxes

The primitive function boxes implement the "forces" by which a change in the signal on one wire influences the signals on other wires. To build function boxes, we use the following operations on wires:

- `get_signal( wire )` returns the current value of the signal on the wire.
- `set_signal( wire , new-value )` : changes the value of the signal on the wire to the new value.

- `add_action( wire , function-of-no-arguments )` : asserts that the designated function should be run whenever the signal on the wire changes value. Such functions are the vehicles by which changes in the signal value on the wire are communicated to other wires. In addition, we will make use of a function `after_delay` that takes a time delay and a function to be run and executes the given function after the given delay.

Using these functions, we can define the primitive digital logic functions. To connect an input to an output through an inverter, we use `add_action` to associate with the input wire a function that will be run whenever the signal on the input wire changes value. The function computes the `logical_not` of the input signal, and then, after one `inverter_delay` , sets the output signal to be this new value:

```
def inverter(input, output):
    def invert_input():
        new_value = logical_not(get_signal(input))
        after_delay(inverter_delay,
                    lambda: set_signal(output, new_value))
    add_action(input, invert_input)
    return "ok"

def logical_not(s):
    return (1 if s == 0
            else 0 if s == 1
            else error("invalid signal", s))
```

An and-gate is a little more complex. The action function must be run if either of the inputs to the gate changes. It computes the `logical_and` (using a function analogous to `logical_not`) of the values of the signals on the input wires and sets up a change to the new value to occur on the output wire after one `and_gate_delay` .

```
def and_gate(a1, a2, output):
    def and_action_function():
        new_value = logical_and(get_signal(a1),
                                get_signal(a2))
        after_delay(and_gate_delay,
                    lambda: set_signal(output, new_value))
    add_action(a1, and_action_function)
    add_action(a2, and_action_function)
    return "ok"
```

Exercise 3.28. Define an or-gate as a primitive function box. Your `or_gate` constructor should be similar to `and_gate` .

Exercise 3.29. Another way to construct an or-gate is as a compound digital logic device, built from and-gates and inverters. Define a function `or_gate` that accomplishes this. What is the delay time of the or-gate in terms of `and_gate_delay` and `inverter_delay` ?

Exercise 3.30. Figure 3.29 shows a *ripple-carry adder* formed by stringing together n full-adders. This is the simplest form of parallel adder for adding two n -bit binary numbers. The inputs $A_1, A_2, A_3, \dots, A_n$ and $B_1, B_2, B_3, \dots, B_n$ are the two binary numbers to be added (each A_k and B_k is a 0 or a 1). The circuit generates $S_1, S_2, S_3, \dots, S_n$, the n bits of the sum, and C , the carry from the addition. Write a function `ripple_carry_adder` that generates this circuit. The function should take as arguments three lists of n wires each—the A_k , the B_k , and the S_k —and also another wire C . The major drawback of the ripple-carry adder is the need to wait for the carry signals to propagate. What is the delay needed to obtain the complete output from an n -bit ripple-carry adder, expressed in terms of the delays for and-gates, or-gates, and inverters?

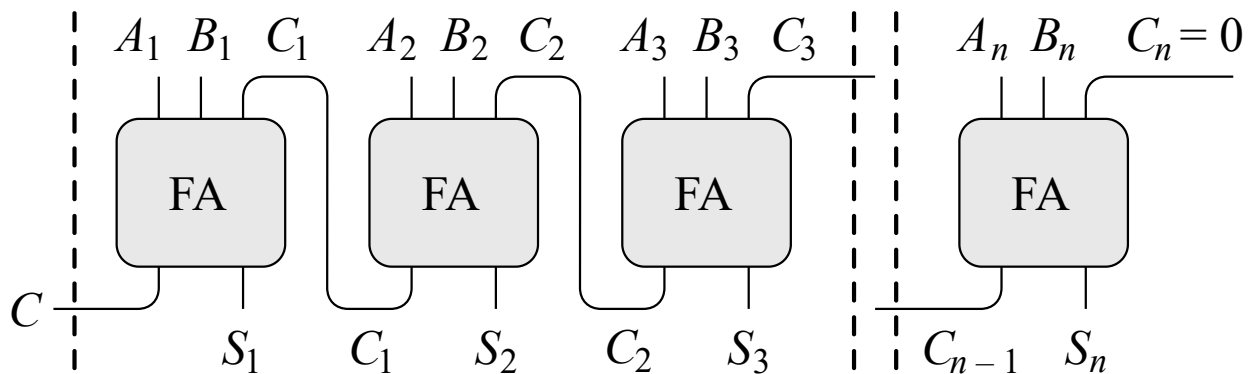


Figure 3.29. A ripple-carry adder for n -bit numbers.

Representing wires

A wire in our simulation will be a computational object with two local state variables: a `signal_value` (initially taken to be 0) and a collection of `action_functions` to be run when the signal changes value. We implement the wire, using message-passing style, as a collection of local functions together with a `dispatch` function that selects the appropriate local operation, just as we did with the simple bank-account object in section 3.1.1:


```

def make_wire():
    signal_value = 0
    action_functions = None
    def set_my_signal(new_value):
        nonlocal signal_value
        if signal_value != new_value:
            signal_value = new_value
            return call_each(action_functions)
        else:
            return "done"
    def accept_action_function(fun):
        nonlocal action_functions
        action_functions = pair(fun, action_functions)
        fun()
    def dispatch(m):
        return (signal_value if m == "get_signal"
                else set_my_signal if m == "set_signal"
                else accept_action_function if m == "add_action"
                else error("unknown operation -- wire", m))
    return dispatch

```

The local function `set_my_signal` tests whether the new signal value changes the signal on the wire. If so, it runs each of the action functions, using the following function `call_each`, which calls each of the items in a list of no-argument functions:

```

def call_each(functions):
    if is_none(functions):
        return "done"
    else:
        head(functions)()
        return call_each(tail(functions))

```

The local function `accept_action_function` adds the given function to the list of functions to be run, and then runs the new function once. (See exercise 3.31.)

With the local `dispatch` function set up as specified, we can provide the following functions to access the local operations on wires: [^27: These functions are simply syntactic sugar that allow us to use ordinary functional syntax to access the local functions of objects. It is striking that we can interchange the role of "functions" and "data" in such a simple way. For example, if we write `wire("get_signal")` we think of `wire` as a function that is called with the message `"get_signal"` as input. Alternatively, writing `get_signal(wire)` encourages us to think of `wire` as a data object that is the input to a function `get_signal`. The truth of the matter is that, in a language in which we can deal with functions as objects, there is no fundamental difference between "functions" and "data," and we can choose our syntactic sugar to allow us to program in whatever style we choose.]

```
def get_signal(wire):
    return wire("get_signal")

def set_signal(wire, new_value):
    return wire("set_signal")(new_value)

def add_action(wire, action_function):
    return wire("add_action")(action_function)
```

Wires, which have time-varying signals and may be incrementally attached to devices, are typical of mutable objects. We have modeled them as functions with local state variables that are modified by reassignment. When a new wire is created, a new set of state variables is allocated (by the `let` statements in `make_wire`) and a new `dispatch` function is constructed and returned, capturing the environment with the new state variables.

The wires are shared among the various devices that have been connected to them. Thus, a change made by an interaction with one device will affect all the other devices attached to the wire. The wire communicates the change to its neighbors by calling the action functions provided to it when the connections were established.

The agenda

The only thing needed to complete the simulator is `after_delay`. The idea here is that we maintain a data structure, called an *agenda*, that contains a schedule of things to do. The following operations are defined for agendas:

- `make_agenda()` : returns a new empty agenda.
- `is_empty_agenda( agenda )` is true if the specified agenda is empty.
- `first_agenda_item( agenda )` returns the first item on the agenda.
- `remove_first_agenda_item( agenda )` modifies the agenda by removing the first item.
- `add_to_agenda( time , action , agenda )` modifies the agenda by adding the given action function to be run at the specified time.
- `current_time( agenda )` returns the current simulation time.

The particular agenda that we use is denoted by `the_agenda`. The function `after_delay` adds new elements to `the_agenda` :

```
def after_delay(delay, action):
    add_to_agenda(delay + current_time(the_agenda),
                  action,
                  the_agenda)
```

The simulation is driven by the function `propagate`, which executes each function on `the_agenda` in sequence. In general, as the simulation runs, new items will be added to the agenda, and `propagate` will continue the simulation as long as there are items on the agenda:

```
def propagate():
    if is_empty_agenda(the_agenda):
        return "done"
    else:
        first_item = first_agenda_item(the_agenda)
        first_item()
        remove_first_agenda_item(the_agenda)
        return propagate()
```

A sample simulation

The following function, which places a "probe" on a wire, shows the simulator in action. The probe tells the wire that, whenever its signal changes value, it should print the new signal value, together with the current time and a name that identifies the wire.

```
def probe(name, wire):
    add_action(wire,
        lambda: print(name + " " +
                      str(current_time(the_agenda)) +
                      ", new value = " +
                      str(get_signal(wire))))
```

We begin by initializing the agenda and specifying delays for the primitive function boxes:

```
the_agenda = make_agenda()
inverter_delay = 2
and_gate_delay = 3
or_gate_delay = 5
```

Now we define four wires, placing probes on two of them:

```
input_1 = make_wire()
input_2 = make_wire()
sum = make_wire()
carry = make_wire()

probe("sum", sum)
```

```
"sum 0, new value = 0"
```

```
probe("carry", carry)
```

```
"carry 0, new value = 0"
```

Next we connect the wires in a half-adder circuit (as in figure 3.27), set the signal on `input_1` to 1, and run the simulation:

```
print(half_adder(input_1, input_2, sum, carry))
```

```
"ok"
```

```
print(set_signal(input_1, 1))
```

```
"done"
```

```
print(propagate())
```

```
"sum 8, new value = 1"  
"done"
```

The `sum` signal changes to 1 at time 8. We are now eight time units from the beginning of the simulation. At this point, we can set the signal on `input_2` to 1 and allow the values to propagate:

```
print(set_signal(input_2, 1))
```

```
"done"
```

```
print(propagate())
```

```
"carry 11, new value = 1"  
"sum 16, new value = 0"  
"done"
```

The `carry` changes to 1 at time 11 and the `sum` changes to 0 at time 16.

Exercise 3.31. The internal function `accept_action_function` defined in `make_wire` specifies that when a new action function is added to a wire, the function is immediately run. Explain why this initialization is necessary. In particular, trace through the half-adder example in the paragraphs above and say how the system's response would differ if we had defined `accept_action_function` as

```
def accept_action_function(fun):
    nonlocal action_functions
    action_functions = pair(fun, action_functions)
```

Implementing the agenda

Finally, we give details of the agenda data structure, which holds the functions that are scheduled for future execution.

The agenda is made up of *time segments*. Each time segment is a pair consisting of a number (the time) and a queue (see exercise 3.32) that holds the functions that are scheduled to be run during that time segment.

```
def make_time_segment(time, queue):
    return pair(time, queue)

def segment_time(s):
    return head(s)

def segment_queue(s):
    return tail(s)
```

We will operate on the time-segment queues using the queue operations described in section 3.3.2.

The agenda itself is a one-dimensional table of time segments. It differs from the tables described in section 3.3.3 in that the segments will be sorted in order of increasing time. In addition, we store the *current time* (i.e., the time of the last action that was processed) at the head of the agenda. A newly constructed agenda has no time segments and has a current time of 0: [^28: The agenda is a headed list, like the tables in section 3.3.3, but since the list is headed by the time, we do not need an additional dummy header (such as the `"*table*"` string used with tables).]

```

def make_agenda():
    return llist(0)

def current_time(agenda):
    return head(agenda)

def set_current_time(agenda, time):
    set_head(agenda, time)

def segments(agenda):
    return tail(agenda)

def set_segments(agenda, segs):
    set_tail(agenda, segs)

def first_segment(agenda):
    return head(segments(agenda))

def rest_segments(agenda):
    return tail(segments(agenda))

```

An agenda is empty if it has no time segments:

```

def is_empty_agenda(agenda):
    return is_none(segments(agenda))

```

To add an action to an agenda, we first check if the agenda is empty. If so, we create a time segment for the action and install this in the agenda. Otherwise, we scan the agenda, examining the time of each segment. If we find a segment for our appointed time, we add the action to the associated queue. If we reach a time later than the one to which we are appointed, we insert a new time segment into the agenda just before it. If we reach the end of the agenda, we must create a new time segment at the end.

```

def add_to_agenda(time, action, agenda):
    def belongs_before(segs):
        return is_none(segs) or time < segment_time(head(segs))
    def make_new_time_segment(time, action):
        q = make_queue()
        insert_queue(q, action)
        return make_time_segment(time, q)
    def add_to_segments(segs):
        if segment_time(head(segs)) == time:
            insert_queue(segment_queue(head(segs)), action)
        else:
            rest = tail(segs)
            if belongs_before(rest):
                set_tail(segs, pair(make_new_time_segment(time, action),
                                    tail(segs)))
            else:
                add_to_segments(rest)
    segs = segments(agenda)
    if belongs_before(segs):
        set_segments(agenda,
                     pair(make_new_time_segment(time, action), segs))
    else:
        add_to_segments(segs)

```

The function that removes the first item from the agenda deletes the item at the front of the queue in the first time segment. If this deletion makes the time segment empty, we remove it from the list of segments: [^29: Observe that the conditional statement in this function has a `pass` statement as its alternative statement that does nothing.]

```

def remove_first_agenda_item(agenda):
    q = segment_queue(first_segment(agenda))
    delete_queue(q)
    if is_empty_queue(q):
        set_segments(agenda, rest_segments(agenda))
    else:
        pass

```

The first agenda item is found at the head of the queue in the first time segment. Whenever we extract an item, we also update the current time: [^30: In this way, the current time will always be the time of the action most recently processed. Storing this time at the head of the agenda ensures that it will still be available even if the associated time segment has been deleted.]

```

def first_agenda_item(agenda):
    if is_empty_agenda(agenda):
        error("agenda is empty -- first_agenda_item")
    else:
        first_seg = first_segment(agenda)
        set_current_time(agenda, segment_time(first_seg))
        return front_queue(segment_queue(first_seg))

```

Exercise 3.32. The functions to be run during each time segment of the agenda are kept in a queue. Thus, the functions for each segment are called in the order in which they were added to the agenda (first in, first out). Explain why this order must be used. In particular, trace the behavior of an and-gate whose inputs change from 0,1 to 1,0 in the same segment and say how the behavior would differ if we stored a segment's functions in an ordinary list, adding and removing functions only at the front (last in, first out).

3.3.5 Propagation of Constraints

Computer programs are traditionally organized as one-directional computations, which perform operations on prespecified arguments to produce desired outputs. On the other hand, we often model systems in terms of relations among quantities. For example, a mathematical model of a mechanical structure might include the information that the deflection d of a metal rod is related to the force F on the rod, the length L of the rod, the cross-sectional area A , and the elastic modulus E via the equation

$$\begin{array}{l} d A E \\ \& = \& F L \end{array}$$

Such an equation is not one-directional. Given any four of the quantities, we can use it to compute the fifth. Yet translating the equation into a traditional computer language would force us to choose one of the quantities to be computed in terms of the other four. Thus, a function for computing the area A could not be used to compute the deflection d , even though the computations of A and d arise from the same equation. [^31: Constraint propagation first appeared in the incredibly forward-looking SKETCHPAD system of Ivan Sutherland (1963). A beautiful constraint-propagation system based on the Smalltalk language was developed by Alan Borning (1977) at Xerox Palo Alto Research Center. Sussman, Stallman, and Steele applied constraint propagation to electrical circuit analysis (Sussman and Stallman 1975; Sussman and Steele 1980). TK!Solver (Konopasek and Jayaraman 1984) is an extensive modeling environment based on constraints.]

In this section, we sketch the design of a language that enables us to work in terms of relations themselves. The primitive elements of the language are *primitive constraints*, which state that certain relations hold between quantities. For example, `adder(a, b, c)` specifies that the quantities a , b , and c must be related by the equation $a+b=c$, `multiplier(x, y, z)` expresses the constraint $xy = z$, and `constant(3.14, x)` says that the value of x must be 3.14.

Our language provides a means of combining primitive constraints in order to express more complex relations. We combine constraints by constructing *constraint networks*, in which constraints are joined by *connectors*. A connector is an object that "holds" a value that may participate in one or more constraints. For example, we know that the relationship between Fahrenheit and Celsius temperatures is

$$9C = 5(F - 32)$$

Such a constraint can be thought of as a network consisting of primitive adder, multiplier, and constant constraints (figure 3.30). In the figure, we see on the left a multiplier box with three terminals, labeled m_1 , m_2 , and p . These connect the multiplier to the rest of the network as follows: The m_1 terminal is linked to a connector C , which will hold the Celsius temperature. The m_2 terminal is linked to a connector w , which is also linked to a constant box that holds 9. The p terminal, which the multiplier box constrains to be the product of m_1 and m_2 , is linked to the p terminal of another multiplier box, whose m_2 is connected to a constant 5 and whose m_1 is connected to one of the terms in a sum.

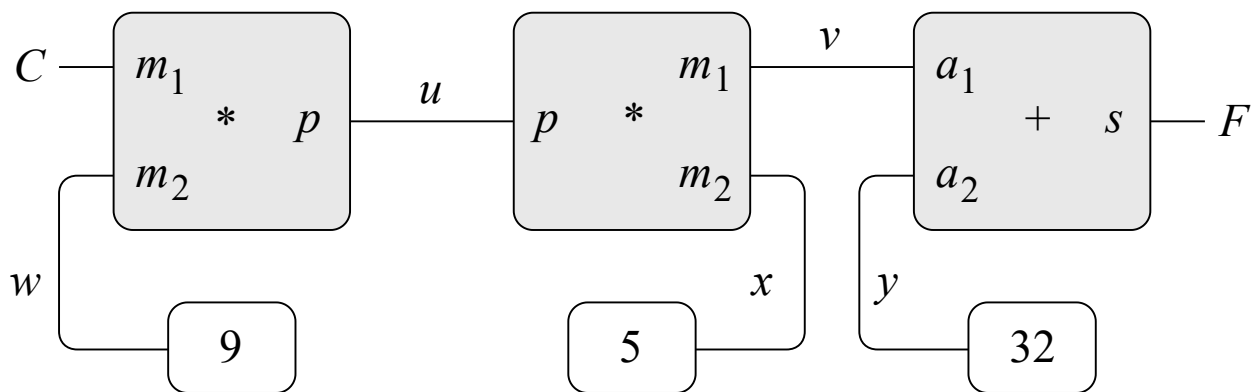


Figure 3.30. The relation $9C = 5(F - 32)$ expressed as a constraint network.

Computation by such a network proceeds as follows: When a connector is given a value (by the user or by a constraint box to which it is linked), it awakens all of its associated constraints (except for the constraint that just awakened it) to inform them that it has a value. Each awakened constraint box then polls its connectors to see if there is enough information to determine a value for a connector. If so, the box sets that connector, which then awakens all of its associated constraints, and so on. For instance, in conversion between Celsius and Fahrenheit, w , x , and y are immediately set by the constant boxes to 9, 5, and 32, respectively. The connectors awaken the multipliers and the adder, which determine that there is not enough information to proceed. If the user (or some other part of the network) sets C to a value (say 25), the leftmost multiplier will be awakened, and it will set u to $25 \cdot 9 = 225$. Then u awakens the second multiplier, which sets v to 45, and v awakens the adder, which sets F to 77.

Using the constraint system

To use the constraint system to carry out the temperature computation outlined above, we first call the constructor `make_connector` to create two connectors, `C` and `F`, and then link them in an appropriate network:

```
C = make_connector()
F = make_connector()
print(celsius_fahrenheit_converter(C, F))
```

```
"ok"
```

The function that creates the network is defined as follows:

```
def celsius_fahrenheit_converter(c, f):
    u = make_connector()
    v = make_connector()
    w = make_connector()
    x = make_connector()
    y = make_connector()
    multiplier(c, w, u)
    multiplier(v, x, u)
    adder(v, y, f)
    constant(9, w)
    constant(5, x)
    constant(32, y)
    return "ok"
```

This function creates the internal connectors `u`, `v`, `w`, `x`, and `y`, and links them as shown in figure 3.30 using the primitive constraint constructors `adder`, `multiplier`, and `constant`. Just as with the digital-circuit simulator of section 3.3.4, expressing these combinations of primitive elements in terms of functions automatically provides our language with a means of abstraction for compound objects.

To watch the network in action, we can place probes on the connectors `C` and `F`, using a `probe` function similar to the one we used to monitor wires in section 3.3.4. Placing a probe on a connector will cause a message to be printed whenever the connector is given a value:

```
probe("Celsius temp", C)
probe("Fahrenheit temp", F)
```

Next we set the value of `C` to 25. (The third argument to `set_value` tells `C` that this directive comes from the `user`.)

```
print(set_value(C, 25, "user"))
```

```
"Probe: Celsius temp = 25"
"Probe: Fahrenheit temp = 77"
"done"
```

The probe on `C` awakens and reports the value. `C` also propagates its value through the network as described above. This sets `F` to 77, which is reported by the probe on `F`.

Now we can try to set `F` to a new value, say 212:

```
set_value(F, 212, "user")
```

```
"Error! Contradiction: (77, 212)"
```

The connector complains that it has sensed a contradiction: Its value is 77, and someone is trying to set it to 212. If we really want to reuse the network with new values, we can tell `C` to forget its old value:

```
print(forget_value(C, "user"))
```

```
"Probe: Celsius temp = ?"  
"Probe: Fahrenheit temp = ?"  
"done"
```

`C` finds that the "user", who set its value originally, is now retracting that value, so `C` agrees to lose its value, as shown by the probe, and informs the rest of the network of this fact. This information eventually propagates to `F`, which now finds that it has no reason for continuing to believe that its own value is 77. Thus, `F` also gives up its value, as shown by the probe.

Now that `F` has no value, we are free to set it to 212:

```
print(set_value(F, 212, "user"))
```

```
"Probe: Fahrenheit temp = 212"  
"Probe: Celsius temp = 100"  
"done"
```

This new value, when propagated through the network, forces `C` to have a value of 100, and this is registered by the probe on `C`. Notice that the very same network is being used to compute `C` given `F` and to compute `F` given `C`. This nondirectionality of computation is the distinguishing feature of constraint-based systems.

Implementing the constraint system

The constraint system is implemented via procedural objects with local state, in a manner very similar to the digital-circuit simulator of section 3.3.4. Although the primitive objects of the

constraint system are somewhat more complex, the overall system is simpler, since there is no concern about agendas and logic delays.

The basic operations on connectors are the following:

- `has_value( connector )` tells whether the connector has a value.
- `get_value( connector )` returns the connector's current value.
- `set_value( connector , new-value , informant )` indicates that the informant is requesting the connector to set its value to the new value.
- `forget_value( connector , retractor )` tells the connector that the retractor is requesting it to forget its value.
- `connect( connector , new-constraint )` tells the connector to participate in the new constraint. The connectors communicate with the constraints by means of the functions `inform_about_value`, which tells the given constraint that the connector has a value, and `inform_about_no_value`, which tells the constraint that the connector has lost its value.

`Adder` constructs an adder constraint among summand connectors `a1` and `a2` and a `sum` connector. An adder is implemented as a function with local state (the function `me` below):

```
def adder(a1, a2, sum):
    def process_new_value():
        if has_value(a1) and has_value(a2):
            set_value(sum, get_value(a1) + get_value(a2), me)
        elif has_value(a1) and has_value(sum):
            set_value(a2, get_value(sum) - get_value(a1), me)
        elif has_value(a2) and has_value(sum):
            set_value(a1, get_value(sum) - get_value(a2), me)
        else:
            pass
    def process_forget_value():
        forget_value(sum, me)
        forget_value(a1, me)
        forget_value(a2, me)
        process_new_value()
    def me(request):
        if request == "I have a value.":
            process_new_value()
        elif request == "I lost my value.":
            process_forget_value()
        else:
            error("unknown request -- adder", request)
    connect(a1, me)
    connect(a2, me)
    connect(sum, me)
    return me
```

The function `adder` connects the new adder to the designated connectors and returns it as its value. The function `me`, which represents the adder, acts as a dispatch to the local functions. The following "syntax interfaces" (see footnote in section 3.3.4) are used in conjunction with the dispatch:

```
def inform_about_value(constraint):  
    return constraint("I have a value.")  
  
def inform_about_no_value(constraint):  
    return constraint("I lost my value.")
```

The adder's local function `process_new_value` is called when the adder is informed that one of its connectors has a value. The adder first checks to see if both `a1` and `a2` have values. If so, it tells `sum` to set its value to the sum of the two addends. The `informant` argument to `set_value` is `me`, which is the adder object itself. If `a1` and `a2` do not both have values, then the adder checks to see if perhaps `a1` and `sum` have values. If so, it sets `a2` to the difference of these two. Finally, if `a2` and `sum` have values, this gives the adder enough information to set `a1`. If the adder is told that one of its connectors has lost a value, it requests that all of its connectors now lose their values. (Only those values that were set by this adder are actually lost.) Then it runs `process_new_value`. The reason for this last step is that one or more connectors may still have a value (that is, a connector may have had a value that was not originally set by the adder), and these values may need to be propagated back through the adder.

A multiplier is very similar to an adder. It will set its `product` to 0 if either of the factors is 0, even if the other factor is not known.

```

def multiplier(m1, m2, product):
    def process_new_value():
        if ((has_value(m1) and get_value(m1) == 0)
            or (has_value(m2) and get_value(m2) == 0)):
            set_value(product, 0, me)
        elif has_value(m1) and has_value(m2):
            set_value(product, get_value(m1) * get_value(m2), me)
        elif has_value(product) and has_value(m1):
            set_value(m2, get_value(product) / get_value(m1), me)
        elif has_value(product) and has_value(m2):
            set_value(m1, get_value(product) / get_value(m2), me)
        else:
            pass
    def process_forget_value():
        forget_value(product, me)
        forget_value(m1, me)
        forget_value(m2, me)
        process_new_value()
    def me(request):
        if request == "I have a value.":
            process_new_value()
        elif request == "I lost my value.":
            process_forget_value()
        else:
            error("unknown request -- multiplier", request)
    connect(m1, me)
    connect(m2, me)
    connect(product, me)
    return me

```

A `constant` constructor simply sets the value of the designated connector. Any `"I have a value."` or `"I lost my value."` message sent to the constant box will produce an error.

```

def constant(value, connector):
    def me(request):
        error("unknown request -- constant", request)
    connect(connector, me)
    set_value(connector, value, me)
    return me

```

Finally, a probe prints a message about the setting or unsetting of the designated connector:

```

def probe(name, connector):
    def print_probe(value):
        print("Probe: " + name + " = " + str(value))
    def process_new_value():
        print_probe(get_value(connector))
    def process_forget_value():
        print_probe("?")
    def me(request):
        return (process_new_value() if request == "I have a value."
                else process_forget_value() if request == "I lost my value."
                else error("unknown request -- probe", request))
    connect(connector, me)
    return me

```

Representing connectors

A connector is represented as a procedural object with local state variables `value`, the current value of the connector; `informant`, the object that set the connector's value; and `constraints`, a list of the constraints in which the connector participates.

```

def make_connector():
    value = False
    informant = False
    constraints = None
    def set_my_value(newval, setter):
        nonlocal value, informant
        if not has_value(me):
            value = newval
            informant = setter
            return for_each_except(setter,
                                   inform_about_value,
                                   constraints)

        elif value != newval:
            error("contradiction", llist(value, newval))
        else:
            return "ignored"
    def forget_my_value(retractor):
        nonlocal informant
        if retractor is informant:
            informant = False
            return for_each_except(retractor,
                                   inform_about_no_value,
                                   constraints)

        else:
            return "ignored"
    def connect(new_constraint):
        nonlocal constraints
        if is_none(member(new_constraint, constraints)):
            constraints = pair(new_constraint, constraints)
        else:
            pass
        if has_value(me):
            inform_about_value(new_constraint)
        else:
            pass
        return "done"
    def me(request):
        if request == "has_value":
            return informant is not False
        elif request == "value":
            return value
        elif request == "set_value":
            return set_my_value
        elif request == "forget":
            return forget_my_value
        elif request == "connect":
            return connect
        else:
            error("unknown operation -- connector", request)
    return me

```

The connector's local function `set_my_value` is called when there is a request to set the connector's value. If the connector does not currently have a value, it will set its value and

remember as `informant` the constraint that requested the value to be set. [^32: The `setter` might not be a constraint. In our temperature example, we used `"user"` as the `setter` .] Then the connector will notify all of its participating constraints except the constraint that requested the value to be set. This is accomplished using the following iterator, which applies a designated function to all items in a list except a given one:

```
def for_each_except(exception, fun, list):
    def loop(items):
        if is_none(items):
            return "done"
        elif head(items) is exception:
            return loop(tail(items))
        else:
            fun(head(items))
            return loop(tail(items))
    return loop(list)
```

If a connector is asked to forget its value, it runs `forget_my_value` , a local function that first checks to make sure that the request is coming from the same object that set the value originally. If so, the connector informs its associated constraints about the loss of the value.

The local function `connect` adds the designated new constraint to the list of constraints if it is not already in that list. Then, if the connector has a value, it informs the new constraint of this fact.

The connector's function `me` serves as a dispatch to the other internal functions and also represents the connector as an object. The following functions provide a syntax interface for the dispatch:

```
def has_value(connector):
    return connector("has_value")

def get_value(connector):
    return connector("value")

def set_value(connector, new_value, informant):
    return connector("set_value")(new_value, informant)

def forget_value(connector, retractor):
    return connector("forget")(retractor)

def connect(connector, new_constraint):
    return connector("connect")(new_constraint)
```

Exercise 3.33. Using primitive multiplier, adder, and constant constraints, define a function `averager` that takes three connectors `a` , `b` , and `c` as inputs and establishes the constraint that the value of `c` is the average of the values of `a` and `b` .

Exercise 3.34. Louis Reasoner wants to build a squarer, a constraint device with two terminals such that the value of connector `b` on the second terminal will always be the square of the value `a` on the first terminal. He proposes the following simple device made from a multiplier:

```
def squarer(a, b):  
    return multiplier(a, a, b)
```

There is a serious flaw in this idea. Explain.

Exercise 3.35. Ben Bitdiddle tells Louis that one way to avoid the trouble in exercise 3.34 is to define a squarer as a new primitive constraint. Fill in the missing portions in Ben's outline for a function to implement such a constraint:

```
def squarer(a, b):  
    def process_new_value():  
        if has_value(b):  
            if get_value(b) < 0:  
                error("square less than 0 -- squarer", get_value(b))  
            else:  
                *alternative_1*  
        else:  
            *alternative_2*  
    def process_forget_value():  
        *body_1*  
    def me(request):  
        *body_2*  
    *statements*  
    return me
```

Exercise 3.36. Suppose we evaluate the following sequence of statements in the program environment:

```
a = make_connector()  
b = make_connector()  
set_value(a, 10, "user")
```

At some time during evaluation of the `set_value`, the following expression from the connector's local function is evaluated:

```
for_each_except(setter, inform_about_value, constraints)
```

Draw an environment diagram showing the environment in which the above expression is evaluated.

Exercise 3.37. The `celsius_fahrenheit_converter` function is cumbersome when compared with a more expression-oriented style of definition, such as

```
def celsius_fahrenheit_converter(x):
    return cplus(cmul(cdiv(cv(9), cv(5)), x), cv(32))

C = make_connector()
F = celsius_fahrenheit_converter(C)
```

Here `cplus`, `cmul`, etc. are the "constraint" versions of the arithmetic operations. For example, `cplus` takes two connectors as arguments and returns a connector that is related to these by an adder constraint:

```
def cplus(x, y):
    z = make_connector()
    adder(x, y, z)
    return z
```

Define analogous functions `cminus`, `cmul`, `cdiv`, and `cv` (constant value) that enable us to define compound constraints as in the converter example above. [^33: The expression-oriented format is convenient because it avoids the need to name the intermediate expressions in a computation. Our original formulation of the constraint language is cumbersome in the same way that many languages are cumbersome when dealing with operations on compound data. For example, if we wanted to compute the product $\{(a+b)\} \cdot \{(c+d)\}$, where the variables represent vectors, we could work in "imperative style," using functions that set the values of designated vector arguments but do not themselves return vectors as values: Alternatively, we could deal with expressions, using functions that return vectors as values, and thus avoid explicitly mentioning `temp1` and `temp2`: Since Python allows us to return compound objects as values of functions, we can transform our imperative-style constraint language into an expression-oriented style as shown in this exercise. Given the advantage of the expression-oriented format, one might ask if there is any reason to have implemented the system in imperative style, as we did in this section. One reason is that the non-expression-oriented constraint language provides a handle on constraint objects (e.g., the value of the `adder` function) as well as on connector objects. This is useful if we wish to extend the system with new operations that communicate with constraints directly rather than only indirectly via operations on connectors. Although it is easy to implement the expression-oriented style in terms of the imperative implementation, it is very difficult to do the converse.]

```
v_sum("a", "b", temp1)
v_sum("c", "d", temp2)
v_prod(temp1, temp2, answer)
```

```
answer = v_prod(v_sum("a", "b"), v_sum("c", "d"))
```

3.4 Concurrency: Time Is of the Essence

We've seen the power of computational objects with local state as tools for modeling. Yet, as section 3.1.3 warned, this power extracts a price: the loss of referential transparency, giving rise to a thicket of questions about sameness and change, and the need to abandon the substitution model of evaluation in favor of the more intricate environment model.

The central issue lurking beneath the complexity of state, sameness, and change is that by introducing reassignment we are forced to admit *time* into our computational models. Before we introduced reassignment, all our programs were timeless, in the sense that any expression that has a value always has the same value. In contrast, recall the example of modeling withdrawals from a bank account and returning the resulting balance, introduced at the beginning of section 3.1.1:

```
print(withdraw(25))
```

75

```
print(withdraw(25))
```

50

Here successive evaluations of the same expression yield different values. This behavior arises from the fact that the execution of reassignments (in this case, reassignments to the variable `balance`) delineates *moments in time* when values change. The result of evaluating an expression depends not only on the expression itself, but also on whether the evaluation occurs before or after these moments. Building models in terms of computational objects with local state forces us to confront time as an essential concept in programming.

We can go further in structuring computational models to match our perception of the physical world. Objects in the world do not change one at a time in sequence. Rather we perceive them as acting *concurrently*—all at once. So it is often natural to model systems as collections of *threads* (sequences of computational steps) that execute concurrently. [³⁴: Such sequential threads are called "processes" throughout the book, but in this section we use the term "thread" to emphasize their access to shared memory.] Just as we can make our programs modular by organizing models in terms of objects with separate local state, it is often appropriate to divide computational models into parts that evolve separately and concurrently. Even if the programs are to be executed on a sequential computer, the practice of writing programs as if they were to be executed concurrently forces the programmer to avoid inessential timing constraints and thus makes programs more modular.

In addition to making programs more modular, concurrent computation can provide a speed advantage over sequential computation. Sequential computers execute only one operation at a time, so the amount of time it takes to perform a task is proportional to the total number of operations performed. [³⁵: Most real processors actually execute a few operations at a time, following a strategy called *pipelining*. Although this technique greatly improves the effective utilization of the hardware, it is used only to speed up the execution of a sequential instruction stream, while retaining the behavior of the sequential program.] However, if it is possible to decompose a problem into pieces that are relatively independent and need to communicate only rarely, it may be possible to allocate pieces to separate computing processors, producing a speed advantage proportional to the number of processors available.

Unfortunately, the complexities introduced by reassignment become even more problematic in the presence of concurrency. The fact of concurrent execution, either because the world operates in parallel or because our computers do, entails additional complexity in our understanding of time.

3.4.1 The Nature of Time in Concurrent Systems

On the surface, time seems straightforward. It is an ordering imposed on events. [³⁶: To quote some graffiti seen on a building wall in Cambridge, Massachusetts: "Time is a device that was invented to keep everything from happening at once."] For any events A and B , either A occurs before B , A and B are simultaneous, or A occurs after B . For instance, returning to the bank account example, suppose that Peter withdraws \$10 and Paul withdraws \$25 from a joint account that initially contains \$100, leaving \$65 in the account. Depending on the order of the two withdrawals, the sequence of balances in the account is either $\$100 \rightarrow \$90 \rightarrow \$65$ or $\$100 \rightarrow \$75 \rightarrow \$65$. In a computer implementation of the banking system, this changing sequence of balances could be modeled by successive reassignments to a variable `balance`.

In complex situations, however, such a view can be problematic. Suppose that Peter and Paul, and other people besides, are accessing the same bank account through a network of banking machines distributed all over the world. The actual sequence of balances in the account will depend critically on the detailed timing of the accesses and the details of the communication among the machines.

This indeterminacy in the order of events can pose serious problems in the design of concurrent systems. For instance, suppose that the withdrawals made by Peter and Paul are implemented as two separate threads sharing a common variable `balance`, each thread specified by the function given in section 3.1.1:

```
def withdraw(amount):
    global balance
    if balance >= amount:
        balance = balance - amount
        return balance
    else:
        return "Insufficient funds"
```

If the two threads operate independently, then Peter might test the balance and attempt to withdraw a legitimate amount. However, Paul might withdraw some funds in between the time that Peter checks the balance and the time Peter completes the withdrawal, thus invalidating Peter's test.

Things can be worse still. Consider the statement

```
balance = balance - amount
```

executed as part of each withdrawal process. This consists of three steps: (1) accessing the value of the `balance` variable; (2) computing the new balance; (3) setting `balance` to this new value. If Peter and Paul's withdrawals execute this statement concurrently, then the two withdrawals might interleave the order in which they access `balance` and set it to the new value.

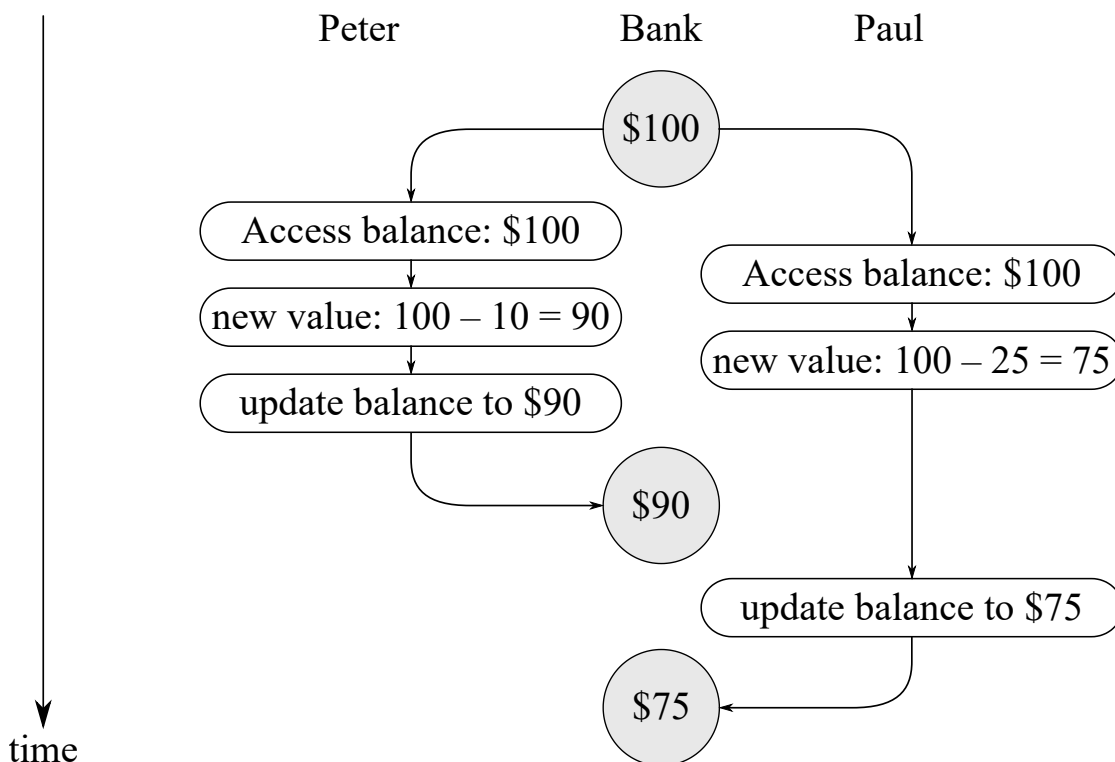


Figure 3.31. Timing diagram showing how interleaving the order of events in two banking withdrawals can lead to an incorrect final balance.

The timing diagram in figure 3.31 depicts an order of events where `balance` starts at 100, Peter withdraws 10, Paul withdraws 25, and yet the final value of `balance` is 75. As shown in the diagram, the reason for this anomaly is that Paul's reassignment of 75 to `balance` is made under the assumption that the value of `balance` to be decremented is 100. That assumption, however, became invalid when Peter changed `balance` to 90. This is a catastrophic failure for the banking system, because the total amount of money in the system is not conserved. Before the transactions, the total amount of money was \$100. Afterwards, Peter has \$10, Paul has \$25, and the bank has \$75. [^37: An even worse failure for this system could occur if the two reassignments attempt to change the balance simultaneously, in which case the actual data appearing in memory might end up being a random combination of the information being written by the two threads. Most computers have interlocks on the primitive memory-write operations, which protect against such simultaneous access. Even this seemingly simple kind of protection, however, raises implementation challenges in the design of multiprocessing computers, where elaborate *cache-coherence* protocols are required to ensure that the various processors will maintain a consistent view of memory contents, despite the fact that data may be replicated ("cached") among the different processors to increase the speed of memory access.]

The general phenomenon illustrated here is that several threads may share a common state variable. What makes this complicated is that more than one thread may be trying to manipulate the shared state at the same time. For the bank account example, during each transaction, each customer should be able to act as if the other customers did not exist. When customers change the balance in a way that depends on the balance, they must be able to assume that, just before the moment of change, the balance is still what they thought it was.

Correct behavior of concurrent programs

The above example typifies the subtle bugs that can creep into concurrent programs. The root of this complexity lies in the reassignments to variables that are shared among the different threads. We already know that we must be careful in writing programs that use reassignment, because the results of a computation depend on the order in which the reassignments occur. [^38: The factorial program in section 3.1.3 illustrates this for a single sequential thread.] With concurrent threads we must be especially careful about reassignments, because we may not be able to control the order of the reassignments made by the different threads. If several such changes might be made concurrently (as with two depositors accessing a joint account) we need some way to ensure that our system behaves correctly. For example, in the case of withdrawals from a joint bank account, we must ensure that money is conserved. To make concurrent programs behave correctly, we may have to place some restrictions on concurrent execution.

One possible restriction on concurrency would stipulate that no two operations that change any shared state variables can occur at the same time. This is an extremely stringent requirement. For distributed banking, it would require the system designer to ensure that only one transaction could proceed at a time. This would be both inefficient and overly conservative. Figure 3.32 shows Peter

and Paul sharing a bank account, where Paul has a private account as well. The diagram illustrates two withdrawals from the shared account (one by Peter and one by Paul) and a deposit to Paul's private account. [^39: The columns show the contents of Peter's wallet, the joint account (in Bank1), Paul's wallet, and Paul's private account (in Bank2), before and after each withdrawal (W) and deposit (D). Peter withdraws \$10 from Bank1; Paul deposits \$5 in Bank2, then withdraws \$25 from Bank1.] The two withdrawals from the shared account must not be concurrent (since both access and update the same account), and Paul's deposit and withdrawal must not be concurrent (since both access and update the amount in Paul's wallet). But there should be no problem permitting Paul's deposit to his private account to proceed concurrently with Peter's withdrawal from the shared account.

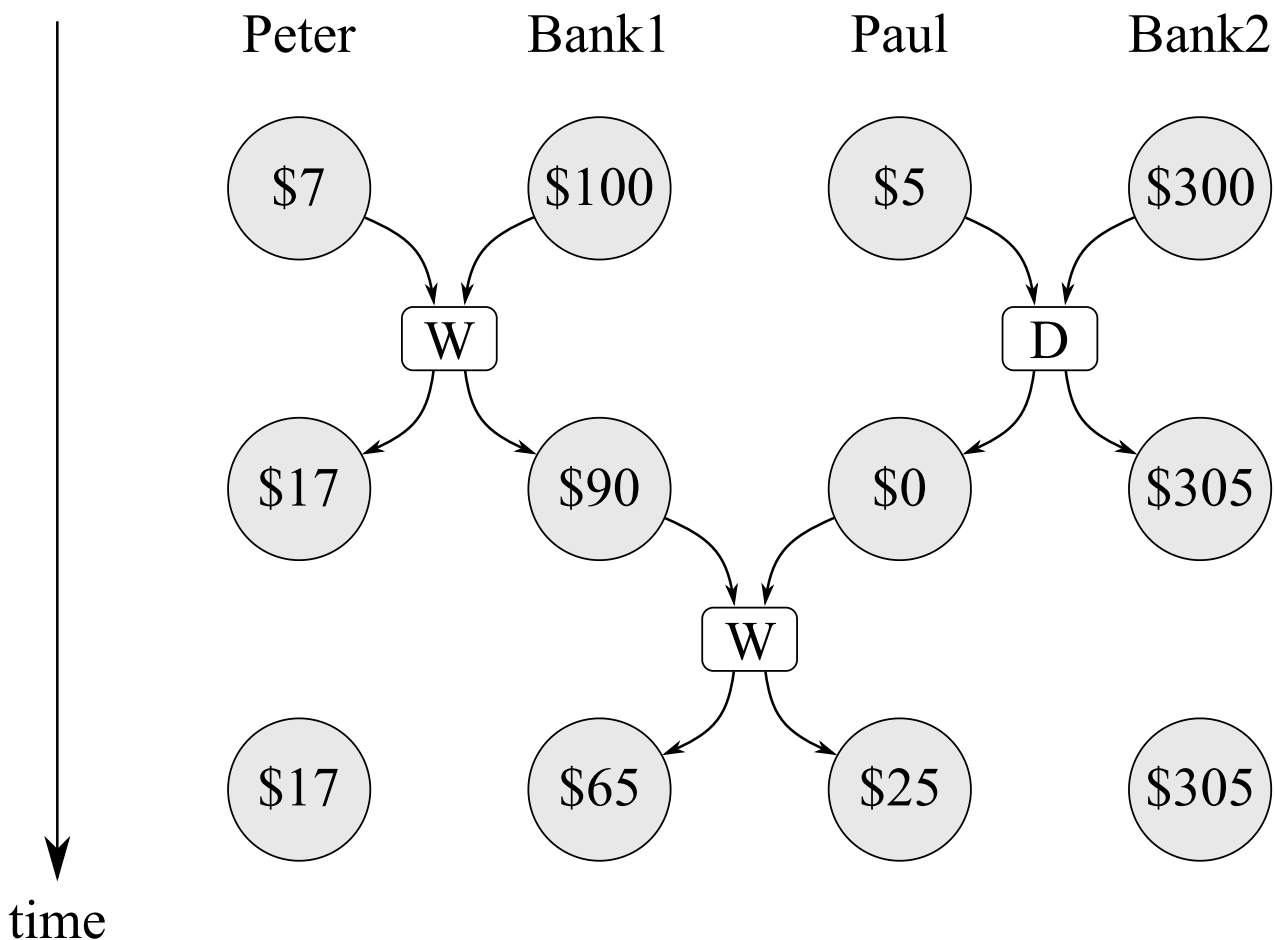


Figure 3.32. Concurrent deposits and withdrawals from a joint account in Bank1 and a private account in Bank2.

A less stringent restriction on concurrency would ensure that a concurrent system produces the same result as if the threads had run sequentially in some order. There are two important aspects to this requirement. First, it does not require the threads to actually run sequentially, but only to produce results that are the same *as if* they had run sequentially. For the example in figure 3.32, the designer of the bank account system can safely allow Paul's deposit and Peter's withdrawal to happen concurrently, because the net result will be the same as if the two operations had happened sequentially. Second, there may be more than one possible "correct" result produced by

a concurrent program, because we require only that the result be the same as for *some* sequential order. For example, suppose that Peter and Paul's joint account starts out with \$100, and Peter deposits \$40 while Paul concurrently withdraws half the money in the account. Then sequential execution could result in the account balance being either \$70 or \$90 (see exercise 3.38). [^40: A more formal way to express this idea is to say that concurrent programs are inherently *nondeterministic*. That is, they are described not by single-valued functions, but by functions whose results are sets of possible values. In section we will study a language for expressing nondeterministic computations.]

There are still weaker requirements for correct execution of concurrent programs. A program for simulating diffusion (say, the flow of heat in an object) might consist of a large number of threads, each one representing a small volume of space, that update their values concurrently. Each thread repeatedly changes its value to the average of its own value and its neighbors' values. This algorithm converges to the right answer independent of the order in which the operations are done; there is no need for any restrictions on concurrent use of the shared values.

Exercise 3.38. Suppose that Peter, Paul, and Mary share a joint bank account that initially contains \$100. Concurrently, Peter deposits \$10, Paul withdraws \$20, and Mary withdraws half the money in the account, by executing the following commands:

```
| Peter: | balance = balance + 10 | | --- | --- | | Paul: | balance = balance - 20 | | Mary: |
balance = balance - (balance / 2) |
```

- List all the different possible values for `balance` after these three transactions have been completed, assuming that the banking system forces the three threads to run sequentially in some order.
- What are some other values that could be produced if the system allows the threads to be interleaved? Draw timing diagrams like the one in figure 3.31 to explain how these values can occur.

3.4.2 Mechanisms for Controlling Concurrency

We've seen that the difficulty in dealing with concurrent threads is rooted in the need to consider the interleaving of the order of events in the different threads. For example, suppose we have two threads, one with three ordered events (a,b,c) and one with three ordered events (x,y,z) . If the two threads run concurrently, with no constraints on how their execution is interleaved, then there are 20 different possible orderings for the events that are consistent with the individual orderings for the two threads:

```

\begin{array}{l}
(a,b,c,x,y,z) \ \& \ (a,x,b,y,c,z) \ \& \ (x,a,b,c,y,z) \ \& \ (x,a,y,z,b,c) \ \backslash \ (a,b,x,c,y,z) \ \& \ (a,x,b,y,z,c) \\
\ \& \ (x,a,b,y,c,z) \ \& \ (x,y,a,b,c,z) \ \backslash \ (a,b,x,y,c,z) \ \& \ (a,x,y,b,c,z) \ \& \ (x,a,b,y,z,c) \ \& \ (x,y,a,b,z,c) \ \backslash \ (a,b,x,y,z,c) \ \& \\
(a,x,y,b,z,c) \ \& \ (x,a,y,b,c,z) \ \& \ (x,y,a,z,b,c) \ \backslash \ (a,x,b,c,y,z) \ \& \ (a,x,y,z,b,c) \ \& \ (x,a,y,b,z,c) \ \& \ (x,y,z,a,b,c) \\
\end{array}

```

As programmers designing this system, we would have to consider the effects of each of these 20 orderings and check that each behavior is acceptable. Such an approach rapidly becomes unwieldy as the numbers of threads and events increase.

A more practical approach to the design of concurrent systems is to devise general mechanisms that allow us to constrain the interleaving of concurrent threads so that we can be sure that the program behavior is correct. Many mechanisms have been developed for this purpose. In this section, we describe one of them, the *serializer*.

Serializing access to shared state

Serialization implements the following idea: Threads will execute concurrently, but there will be certain collections of functions that cannot be executed concurrently. More precisely, serialization creates distinguished sets of functions such that only one execution of a function in each serialized set is permitted to happen at a time. If some function in the set is being executed, then a thread that attempts to execute any function in the set will be forced to wait until the first execution has finished.

We can use serialization to control access to shared variables. For example, if we want to update a shared variable based on the previous value of that variable, we put the access to the previous value of the variable and the reassignment of the new value to the variable in the same function. We then ensure that no other function that assigns to the variable can run concurrently with this function by serializing all of these functions with the same serializer. This guarantees that the value of the variable cannot be changed between an access and the corresponding reassignment.

Serializers

To make the above mechanism more concrete, suppose that we have extended Python to include a function called `concurrent_execute` :

```
concurrent_execute( $f_1$ ,  $f_2$ , ...,  $f_k$ )
```

Each f_i must be a function of no arguments. The function `concurrent_execute` creates a separate thread for each f_i , which applies f_i (to no arguments). These threads all run concurrently. [^41: The function `concurrent_execute` is not part of the Python standard, but the examples in this section can be implemented in ECMAScript 2020.]

As an example of how this is used, consider

```

x = 10

def t1():
    global x
    x = x * x

def t2():
    global x
    x = x + 1

concurrent_execute(t1, t2)

```

This creates two concurrent threads—`T_1`, which sets `x` to `x` times `x`, and `T_2`, which increments `x`. After execution is complete, `x` will be left with one of five possible values, depending on the interleaving of the events of `T_1` and `T_2`:

| 101: | `T_1` sets `x` to 100 and then `T_2` increments `x` to 101. | | --- | --- | | 121: | `T_2` increments `x` to 11 and then `T_1` sets `x` to `x` times `x`. | | 110: | `T_2` changes `x` from 10 to 11 between the two times that `T_1` accesses the value of `x` during the evaluation of `x * x`. | | 11: | `T_2` accesses `x`, then `T_1` sets `x` to 100, then `T_2` sets `x`. | | 100: | `T_1` accesses `x` (twice), then `T_2` sets `x` to 11, then `T_1` sets `x`.

We can constrain the concurrency by using serialized functions, which are created by *serializers*. Serializers are constructed by `make_serializer`, whose implementation is given below. A serializer takes a function as argument and returns a serialized function that behaves like the original function. All calls to a given serializer return serialized functions in the same set.

Thus, in contrast to the example above, executing

```

x = 10

s = make_serializer()

def t1():
    global x
    x = x * x

def t2():
    global x
    x = x + 1

concurrent_execute(s(t1), s(t2))

```

can produce only two possible values for `x`, 101 or 121. The other possibilities are eliminated, because the execution of `T_1` and `T_2` cannot be interleaved.

Here is a version of the `make_account` function from section 3.1.1, where the deposits and withdrawals have been serialized:

```

def make_account(balance):
    def withdraw(amount):
        nonlocal balance
        if balance > amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    protect = make_serializer()
    def dispatch(m):
        return (protect(withdraw) if m == "withdraw"
                else protect(deposit) if m == "deposit"
                else balance if m == "balance"
                else error("unknown request -- make_account", m))
    return dispatch

```

With this implementation, two threads cannot be withdrawing from or depositing into a single account concurrently. This eliminates the source of the error illustrated in figure 3.31, where Peter changes the account balance between the times when Paul accesses the balance to compute the new value and when Paul actually performs the reassignment. On the other hand, each account has its own serializer, so that deposits and withdrawals for different accounts can proceed concurrently.

Exercise 3.39. Which of the five possibilities in the concurrent execution shown above remain if we instead serialize execution as follows:

```

x = 10

s = make_serializer()

def t1():
    global x
    x = s(lambda: x * x)()

def t2():
    global x
    x = x + 1

concurrent_execute(t1, s(t2))

```

Exercise 3.40. Give all possible values of `x` that can result from executing

```

x = 10

def t1():
    global x
    x = x * x

def t2():
    global x
    x = x * x * x

concurrent_execute(t1, t2)

```

Which of these possibilities remain if we instead use serialized functions:

```

x = 10

s = make_serializer()

def t1():
    global x
    x = x * x

def t2():
    global x
    x = x * x * x

concurrent_execute(s(t1), s(t2))

```

Exercise 3.41. Ben Bitdiddle worries that it would be better to implement the bank account as follows (where the commented line has been changed):

```

def make_account(balance):
    def withdraw(amount):
        nonlocal balance
        if balance > amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    protect = make_serializer()
    def dispatch(m):
        return (protect(withdraw) if m == "withdraw"
                else protect(deposit) if m == "deposit"
                else protect(lambda: balance)() if m == "balance" # serialized
                else error("unknown request -- make_account", m))
    return dispatch

```

because allowing unserialized access to the bank balance can result in anomalous behavior. Do you agree? Is there any scenario that demonstrates Ben's concern?

Exercise 3.42. Ben Bitdiddle suggests that it's a waste of time to create a new serialized function in response to every `withdraw` and `deposit` message. He says that `make_account` could be changed so that the calls to `protect` are done outside the `dispatch` function. That is, an account would return the same serialized function (which was created at the same time as the account) each time it is asked for a withdrawal function.

```
def make_account(balance):
    def withdraw(amount):
        nonlocal balance
        if balance > amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    protect = make_serializer()
    protect_withdraw = protect(withdraw)
    protect_deposit = protect(deposit)
    def dispatch(m):
        return (protect_withdraw if m == "withdraw"
                else protect_deposit if m == "deposit"
                else balance if m == "balance"
                else error("unknown request -- make_account", m))
    return dispatch
```

Is this a safe change to make? In particular, is there any difference in what concurrency is allowed by these two versions of `make_account` ?

Complexity of using multiple shared resources

Serializers provide a powerful abstraction that helps isolate the complexities of concurrent programs so that they can be dealt with carefully and (hopefully) correctly. However, while using serializers is relatively straightforward when there is only a single shared resource (such as a single bank account), concurrent programming can be treacherously difficult when there are multiple shared resources.

To illustrate one of the difficulties that can arise, suppose we wish to swap the balances in two bank accounts. We access each account to find the balance, compute the difference between the balances, withdraw this difference from one account, and deposit it in the other account. We could implement this as follows: [^42: We have simplified `exchange` by exploiting the fact that our `deposit` message accepts negative amounts. (This is a serious bug in our banking system!)]

```
def exchange(account1, account2):
    difference = account1("balance") - account2("balance")
    account1("withdraw")(difference)
    account2("deposit")(difference)
```

This function works well when only a single thread is trying to do the exchange. Suppose, however, that Peter and Paul both have access to accounts \$a_1\$, \$a_2\$, and \$a_3\$, and that Peter exchanges \$a_1\$ and \$a_2\$ while Paul concurrently exchanges \$a_1\$ and \$a_3\$. Even with account deposits and withdrawals serialized for individual accounts (as in the `make_account` function shown above in this section), `exchange` can still produce incorrect results. For example, Peter might compute the difference in the balances for \$a_1\$ and \$a_2\$, but then Paul might change the balance in \$a_1\$ before Peter is able to complete the exchange. [43: If the account balances start out as \$10, \$20, and \$30, then after any number of concurrent exchanges, the balances should still be \$10, \$20, and \$30 in some order. Serializing the deposits to individual accounts is not sufficient to guarantee this. See exercise 3.43.] For correct behavior, we must arrange for the `exchange` function to lock out any other concurrent accesses to the accounts during the entire time of the exchange.

One way we can accomplish this is by using both accounts' serializers to serialize the entire `exchange` function. To do this, we will arrange for access to an account's serializer. Note that we are deliberately breaking the modularity of the bank-account object by exposing the serializer. The following version of `make_account` is identical to the original version given in section 3.1.1, except that a serializer is provided to protect the balance variable, and the serializer is exported via message passing:

```
def make_account_and_serializer(balance):
    def withdraw(amount):
        nonlocal balance
        if balance > amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    balance_serializer = make_serializer()
    return lambda m: (withdraw if m == "withdraw"
                      else deposit if m == "deposit"
                      else balance if m == "balance"
                      else balance_serializer if m == "serializer"
                      else error("unknown request -- make_account", m))
```

We can use this to do serialized deposits and withdrawals. However, unlike our earlier serialized account, it is now the responsibility of each user of bank-account objects to explicitly manage the

serialization, for example as follows: [^44: Exercise 3.45 investigates why deposits and withdrawals are no longer automatically serialized by the account.]

```
def deposit(account, amount):
    s = account("serializer")
    d = account("deposit")
    s(d(amount))
```

Exporting the serializer in this way gives us enough flexibility to implement a serialized exchange program. We simply serialize the original `exchange` function with the serializers for both accounts:

```
def serialized_exchange(account1, account2):
    serializer1 = account1("serializer")
    serializer2 = account2("serializer")
    serializer1(serializer2(exchange))(account1, account2)
```

Exercise 3.43. Suppose that the balances in three accounts start out as \$10, \$20, and \$30, and that multiple threads run, exchanging the balances in the accounts. Argue that if the threads are run sequentially, after any number of concurrent exchanges, the account balances should be \$10, \$20, and \$30 in some order. Draw a timing diagram like the one in figure 3.31 to show how this condition can be violated if the exchanges are implemented using the first version of the account-exchange program in this section. On the other hand, argue that even with this `exchange` program, the sum of the balances in the accounts will be preserved. Draw a timing diagram to show how even this condition would be violated if we did not serialize the transactions on individual accounts.

Exercise 3.44. Consider the problem of transferring an amount from one account to another. Ben Bitdiddle claims that this can be accomplished with the following function, even if there are multiple people concurrently transferring money among multiple accounts, using any account mechanism that serializes deposit and withdrawal transactions, for example, the version of `make_account` in the text above.

```
def transfer(from_account, to_account, amount):
    from_account("withdraw")(amount)
    to_account("deposit")(amount)
```

Louis Reasoner claims that there is a problem here, and that we need to use a more sophisticated method, such as the one required for dealing with the exchange problem. Is Louis right? If not, what is the essential difference between the transfer problem and the exchange problem? (You should assume that the balance in `from_account` is at least `amount`.)

Exercise 3.45. Louis Reasoner thinks our bank-account system is unnecessarily complex and error-prone now that deposits and withdrawals aren't automatically serialized. He suggests that

`make_account_and_serializer` should have exported the serializer (for use by such functions as `serialized_exchange`) in addition to (rather than instead of) using it to serialize accounts and deposits as `make_account` did. He proposes to redefine accounts as follows:

```
def make_account_and_serializer(balance):
    def withdraw(amount):
        nonlocal balance
        if balance > amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    balance_serializer = make_serializer()
    return lambda m: (balance_serializer(withdraw) if m == "withdraw"
                      else balance_serializer(deposit) if m == "deposit"
                      else balance if m == "balance"
                      else balance_serializer if m == "serializer"
                      else error("unknown request -- make_account", m))
```

Then deposits are handled as with the original `make_account` :

```
def deposit(account, amount):
    account("deposit")(amount)
```

Explain what is wrong with Louis's reasoning. In particular, consider what happens when `serialized_exchange` is called.

Implementing serializers

We implement serializers in terms of a more primitive synchronization mechanism called a *mutex*. A mutex is an object that supports two operations—the mutex can be *acquired*, and the mutex can be *released*. Once a mutex has been acquired, no other acquire operations on that mutex may proceed until the mutex is released. [⁴⁵: The term "mutex" is an abbreviation for *mutual exclusion*. The general problem of arranging a mechanism that permits concurrent threads to safely share resources is called the mutual exclusion problem. Our mutex is a simple variant of the *semaphore* mechanism (see exercise 3.47), which was introduced in the "THE" Multiprogramming System developed at the Technological University of Eindhoven and named for the university's initials in Dutch (Dijkstra 1968a). The acquire and release operations were originally called P and V, from the Dutch words *passeren* (to pass) and *vrijgeven* (to release), in reference to the semaphores used on railroad systems. Dijkstra's classic exposition (1968b) was one of the first to clearly present the issues of concurrency control, and showed how to use semaphores to handle a variety of concurrency problems.] In our implementation, each serializer has an associated mutex. Given a

function `f`, the serializer returns a function that acquires the mutex, runs `f`, and then releases the mutex. This ensures that only one of the functions produced by the serializer can be running at once, which is precisely the serialization property that we need to guarantee. To apply serializers to functions that take an arbitrary number of arguments, we use Python's *rest* parameter and *spread* syntax. The `...` in front of the parameter `args` collects the rest (here all) of the arguments of any call of the function into a *vector* data structure. The `...` in front of `args` in the application `f(...args)` spreads the elements of `args` so that they become separate arguments of `f`.

```
def make_serializer():
    mutex = make_mutex()
    def serializer(f):
        def serialized_f(*args):
            mutex("acquire")
            val = f(*args)
            mutex("release")
            return val
        return serialized_f
    return serializer
```

The mutex is a mutable object (here we'll use a one-element list, which we'll refer to as a *cell*) that can hold the value true or false. When the value is false, the mutex is available to be acquired. When the value is true, the mutex is unavailable, and any thread that attempts to acquire the mutex must wait.

Our mutex constructor `make_mutex` begins by initializing the cell contents to false. To acquire the mutex, we test the cell. If the mutex is available, we set the cell contents to true and proceed. Otherwise, we wait in a loop, attempting to acquire over and over again, until we find that the mutex is available. [^46: In most time-shared operating systems, threads that are blocked by a mutex do not waste time "busy-waiting" as above. Instead, the system schedules another thread to run while the first is waiting, and the blocked thread is awakened when the mutex becomes available.] To release the mutex, we set the cell contents to false.

```
def make_mutex():
    cell = llist(False)
    def the_mutex(m):
        return ((the_mutex("acquire") # retry
                if test_and_set(cell)
                else True)
                if m == "acquire"
                else clear(cell) if m == "release"
                else error("unknown request -- mutex", m))
    return the_mutex

def clear(cell):
    set_head(cell, False)
```

The function `test_and_set` tests the cell and returns the result of the test. In addition, if the test was false, `test_and_set` sets the cell contents to true before returning false. We can express this behavior as the following function:

```
def test_and_set(cell):
    if head(cell):
        return True
    else:
        set_head(cell, True)
        return False
```

However, this implementation of `test_and_set` does not suffice as it stands. There is a crucial subtlety here, which is the essential place where concurrency control enters the system: The `test_and_set` operation must be performed *atomically*. That is, we must guarantee that, once a thread has tested the cell and found it to be false, the cell contents will actually be set to true before any other thread can test the cell. If we do not make this guarantee, then the mutex can fail in a way similar to the bank-account failure in figure 3.31. (See exercise 3.46.)

The actual implementation of `test_and_set` depends on the details of how our system runs concurrent threads. For example, we might be executing concurrent threads on a sequential processor using a time-slicing mechanism that cycles through the threads, permitting each thread to run for a short time before interrupting it and moving on to the next thread. In that case, `test_and_set` can work by disabling time slicing during the testing and setting. Alternatively, multiprocessing computers provide instructions that support atomic operations directly in hardware. [^47: There are many variants of such instructions—including test-and-set, test-and-clear, swap, compare-and-exchange, load-reserve, and store-conditional—whose design must be carefully matched to the machine's processor-memory interface. One issue that arises here is to determine what happens if two threads attempt to acquire the same resource at exactly the same time by using such an instruction. This requires some mechanism for making a decision about which thread gets control. Such a mechanism is called an *arbiter*. Arbiters usually boil down to some sort of hardware device. Unfortunately, it is possible to prove that one cannot physically construct a fair arbiter that works 100% of the time unless one allows the arbiter an arbitrarily long time to make its decision. The fundamental phenomenon here was originally observed by the fourteenth-century French philosopher Jean Buridan in his commentary on Aristotle's *De caelo*. Buridan argued that a perfectly rational dog placed between two equally attractive sources of food will starve to death, because it is incapable of deciding which to go to first.]

Exercise 3.46. Suppose that we implement `test_and_set` using an ordinary function as shown in the text, without attempting to make the operation atomic. Draw a timing diagram like the one in figure 3.31 to demonstrate how the mutex implementation can fail by allowing two threads to acquire the mutex at the same time.

Exercise 3.47. A semaphore (of size n) is a generalization of a mutex. Like a mutex, a semaphore supports acquire and release operations, but it is more general in that up to n threads can acquire it concurrently. Additional threads that attempt to acquire the semaphore must wait for release operations. Give implementations of semaphores a. in terms of mutexes b. in terms of atomic `test_and_set` operations.

Deadlock

Now that we have seen how to implement serializers, we can see that account exchanging still has a problem, even with the `serialized_exchange` function above. Imagine that Peter attempts to exchange a_1 with a_2 while Paul concurrently attempts to exchange a_2 with a_1 . Suppose that Peter's thread reaches the point where it has entered a serialized function protecting a_1 and, just after that, Paul's thread enters a serialized function protecting a_2 . Now Peter cannot proceed (to enter a serialized function protecting a_2) until Paul exits the serialized function protecting a_2 . Similarly, Paul cannot proceed until Peter exits the serialized function protecting a_1 . Each thread is stalled forever, waiting for the other. This situation is called a *deadlock*. Deadlock is always a danger in systems that provide concurrent access to multiple shared resources.

One way to avoid the deadlock in this situation is to give each account a unique identification number and rewrite `serialized_exchange` so that a thread will always attempt to enter a function protecting the lowest-numbered account first. Although this method works well for the exchange problem, there are other situations that require more sophisticated deadlock-avoidance techniques, or where deadlock cannot be avoided at all. (See exercises 3.48 and 3.49.) [^48: The general technique for avoiding deadlock by numbering the shared resources and acquiring them in order is due to Havender (1968). Situations where deadlock cannot be avoided require *deadlock-recovery* methods, which entail having threads "back out" of the deadlocked state and try again. Deadlock-recovery mechanisms are widely used in data-base-management systems, a topic that is treated in detail in Gray and Reuter 1993.]

Exercise 3.48. Explain in detail why the deadlock-avoidance method described above, (i.e., the accounts are numbered, and each thread attempts to acquire the smaller-numbered account first) avoids deadlock in the exchange problem. Rewrite `serialized_exchange` to incorporate this idea. (You will also need to modify `make_account` so that each account is created with a number, which can be accessed by sending an appropriate message.)

Exercise 3.49. Give a scenario where the deadlock-avoidance mechanism described above does not work. (Hint: In the exchange problem, each thread knows in advance which accounts it will need to get access to. Consider a situation where a thread must get access to some shared resources before it can know which additional shared resources it will require.)

Concurrency, time, and communication

We've seen how programming concurrent systems requires controlling the ordering of events when different threads access shared state, and we've seen how to achieve this control through judicious use of serializers. But the problems of concurrency lie deeper than this, because, from a fundamental point of view, it's not always clear what is meant by "shared state."

Mechanisms such as `test_and_set` require threads to examine a global shared flag at arbitrary times. This is problematic and inefficient to implement in modern high-speed processors, where due to optimization techniques such as pipelining and cached memory, the contents of memory may not be in a consistent state at every instant. In some multiprocessing systems, therefore, the serializer paradigm is being supplanted by other approaches to concurrency control. [^49: One such alternative to serialization is called *barrier synchronization*. The programmer permits concurrent threads to execute as they please, but establishes certain synchronization points ("barriers") through which no thread can proceed until all the threads have reached the barrier. Some processors provide machine instructions that permit programmers to establish synchronization points at places where consistency is required. The PowerPCTM, for example, includes for this purpose two instructions called SYNC and EIEIO (Enforced In-order Execution of Input/Output).]

The problematic aspects of shared state also arise in large, distributed systems. For instance, imagine a distributed banking system where individual branch banks maintain local values for bank balances and periodically compare these with values maintained by other branches. In such a system the value of "the account balance" would be undetermined, except right after synchronization. If Peter deposits money in an account he holds jointly with Paul, when should we say that the account balance has changed—when the balance in the local branch changes, or not until after the synchronization? And if Paul accesses the account from a different branch, what are the reasonable constraints to place on the banking system such that the behavior is "correct"? The only thing that might matter for correctness is the behavior observed by Peter and Paul individually and the "state" of the account immediately after synchronization. Questions about the "real" account balance or the order of events between synchronizations may be irrelevant or meaningless. [^50: This may seem like a strange point of view, but there are systems that work this way. International charges to credit-card accounts, for example, are normally cleared on a per-country basis, and the charges made in different countries are periodically reconciled. Thus the account balance may be different in different countries.]

The basic phenomenon here is that synchronizing different threads, establishing shared state, or imposing an order on events requires communication among the threads. In essence, any notion of time in concurrency control must be intimately tied to communication. [^51: For distributed systems, this perspective was pursued by Lamport (1978), who showed how to use communication to establish "global clocks" that can be used to establish orderings on events in distributed systems.] It is intriguing that a similar connection between time and communication also arises in

the Theory of Relativity, where the speed of light (the fastest signal that can be used to synchronize events) is a fundamental constant relating time and space. The complexities we encounter in dealing with time and state in our computational models may in fact mirror a fundamental complexity of the physical universe.

3.5 Streams

We've gained a good understanding of reassignment as a tool in modeling, as well as an appreciation of the complex problems that reassignment raises. It is time to ask whether we could have gone about things in a different way, so as to avoid some of these problems. In this section, we explore an alternative approach to modeling state, based on data structures called *streams*. As we shall see, streams can mitigate some of the complexity of modeling state.

Let's step back and review where this complexity comes from. In an attempt to model real-world phenomena, we made some apparently reasonable decisions: We modeled real-world objects with local state by computational objects with local variables. We identified time variation in the real world with time variation in the computer. We implemented the time variation of the states of the model objects in the computer with reassignments to the local variables of the model objects.

Is there another approach? Can we avoid identifying time in the computer with time in the modeled world? Must we make the model change with time in order to model phenomena in a changing world? Think about the issue in terms of mathematical functions. We can describe the time-varying behavior of a quantity x as a function of time $x(t)$. If we concentrate on x instant by instant, we think of it as a changing quantity. Yet if we concentrate on the entire time history of values, we do not emphasize change—the function itself does not change. [⁵²: Physicists sometimes adopt this view by introducing the "world lines" of particles as a device for reasoning about motion. We've also already mentioned (section 2.2.3) that this is the natural way to think about signal-processing systems. We will explore applications of streams to signal processing in section 3.5.3.]

If time is measured in discrete steps, then we can model a time function as a (possibly infinite) sequence. In this section, we will see how to model change in terms of sequences that represent the time histories of the systems being modeled. To accomplish this, we introduce new data structures called *streams*. From an abstract point of view, a stream is simply a sequence. However, we will find that the straightforward implementation of streams as lists (as in section 2.2.1) doesn't fully reveal the power of stream processing. As an alternative, we introduce the technique of *delayed evaluation*, which enables us to represent very large (even infinite) sequences as streams.

Stream processing lets us model systems that have state without ever using reassignment or mutable data. This has important implications, both theoretical and practical, because we can build models that avoid the drawbacks inherent in introducing reassignment. On the other hand, the

stream framework raises difficulties of its own, and the question of which modeling technique leads to more modular and more easily maintained systems remains open.

3.5.1 Streams Are Delayed Lists

As we saw in section 2.2.3, sequences can serve as standard interfaces for combining program modules. We formulated powerful abstractions for manipulating sequences, such as `map`, `filter`, and `accumulate`, that capture a wide variety of operations in a manner that is both succinct and elegant.

Unfortunately, if we represent sequences as lists, this elegance is bought at the price of severe inefficiency with respect to both the time and space required by our computations. When we represent manipulations on sequences as transformations of lists, our programs must construct and copy data structures (which may be huge) at every step of a process.

To see why this is true, let us compare two programs for computing the sum of all the prime numbers in an interval. The first program is written in standard iterative style: [^53: Assume that we have a predicate `is_prime` (e.g., as in section 1.2.6) that tests for primality.]

```
def sum_primes(a, b):
    def iter(count, accum):
        return (accum if count > b
                else iter(count + 1, count + accum) if is_prime(count)
                else iter(count + 1, accum))
    return iter(a, 0)
```

The second program performs the same computation using the sequence operations of section 2.2.3:

```
def sum_primes(a, b):
    return reduce(lambda x, y: x + y,
                 0,
                 filter(is_prime,
                       enumerate_interval(a, b)))
```

In carrying out the computation, the first program needs to store only the sum being accumulated. In contrast, the filter in the second program cannot do any testing until `enumerate_interval` has constructed a complete list of the numbers in the interval. The filter generates another list, which in turn is passed to `accumulate` before being collapsed to form a sum. Such large intermediate storage is not needed by the first program, which we can think of as enumerating the interval incrementally, adding each prime to the sum as it is generated.

The inefficiency in using lists becomes painfully apparent if we use the sequence paradigm to compute the second prime in the interval from 10,000 to 1,000,000 by evaluating the expression


```
head(tail(filter(is_prime,
                 enumerate_interval(10000, 1000000))))
```

This expression does find the second prime, but the computational overhead is outrageous. We construct a list of almost a million integers, filter this list by testing each element for primality, and then ignore almost all of the result. In a more traditional programming style, we would interleave the enumeration and the filtering, and stop when we reached the second prime.

Streams are a clever idea that allows one to use sequence manipulations without incurring the costs of manipulating sequences as lists. With streams we can achieve the best of both worlds: We can formulate programs elegantly as sequence manipulations, while attaining the efficiency of incremental computation. The basic idea is to arrange to construct a stream only partially, and to pass the partial construction to the program that consumes the stream. If the consumer attempts to access a part of the stream that has not yet been constructed, the stream will automatically construct just enough more of itself to produce the required part, thus preserving the illusion that the entire stream exists. In other words, although we will write programs as if we were processing complete sequences, we design our stream implementation to automatically and transparently interleave the construction of the stream with its use.

To accomplish this, we will construct streams using pairs, with the first item of the stream in the head of the pair. However, rather than placing the value of the rest of the stream into the tail of the pair, we will put there a "promise" to compute the rest if it is ever requested. If we have a data item `h` and a stream `t`, we construct a stream whose head is `h` and whose tail is `t` by evaluating `pair(h, lambda: t)`—the tail `t` of a stream is "wrapped" in a function of no arguments, so that its evaluation will be *delayed*. The empty stream is `null`, the same as the empty list.

To access the first data item of a nonempty stream, we simply select the `head` of the pair, as with a list. But to access the tail of a stream, we need to evaluate the delayed expression. For convenience, we define

```
def stream_tail(stream):
    return tail(stream)()
```

This selects the tail of the pair and applies the function found there to obtain the next pair of the stream (or `null` if the tail of the stream is empty)—in effect, *forcing* the function in the tail of the pair to fulfill its promise.

We can make and use streams, in just the same way as we can make and use lists, to represent aggregate data arranged in a sequence. In particular, we can build stream analogs of the list operations from chapter 2, such as `list_ref`, `map`, and `for_each`:^[54] This should bother you. The fact that we are defining such similar functions for streams and lists indicates that we are missing some underlying abstraction. Unfortunately, in order to exploit this abstraction, we will

need to exert finer control over the process of evaluation than we can at present. We will discuss this point further at the end of section 3.5.4. In section , we'll develop a framework that unifies lists and streams.]

```
def stream_ref(s, n):
    return (head(s) if n == 0
            else stream_ref(stream_tail(s), n - 1))

def stream_map(f, s):
    return (None if is_none(s)
            else pair(f(head(s)),
                      lambda: stream_map(f, stream_tail(s))))

def stream_for_each(fun, s):
    if is_none(s):
        return True
    else:
        fun(head(s))
        return stream_for_each(fun, stream_tail(s))
```

The function `stream_for_each` is useful for viewing streams:

```
def display_stream(s):
    return stream_for_each(print, s)
```

To make the stream implementation automatically and transparently interleave the construction of a stream with its use, we have arranged for the tail of a stream to be evaluated when it is accessed by the `stream_tail` function rather than when the stream is constructed by `pair`. This implementation choice is reminiscent of our discussion of rational numbers in section 2.1.2, where we saw that we can choose to implement rational numbers so that the reduction of numerator and denominator to lowest terms is performed either at construction time or at selection time. The two rational-number implementations produce the same data abstraction, but the choice has an effect on efficiency. There is a similar relationship between streams and ordinary lists. As a data abstraction, streams are the same as lists. The difference is the time at which the elements are evaluated. With ordinary lists, both the `head` and the `tail` are evaluated at construction time. With streams, the `tail` is evaluated at selection time.

Streams in action

To see how this data structure behaves, let us analyze the "outrageous" prime computation we saw above, reformulated in terms of streams:

```
print(head(stream_tail(stream_filter(
    is_prime,
    stream_enumerate_interval(10000, 1000000)))))
```

We will see that it does indeed work efficiently.

We begin by calling `stream_enumerate_interval` with the arguments 10,000 and 1,000,000. The function `stream_enumerate_interval` is the stream analog of `enumerate_interval` (section 2.2.3):

```
def stream_enumerate_interval(low, high):
    return (None if low > high
            else pair(low,
                      lambda: stream_enumerate_interval(low + 1, high)))
```

and thus the result returned by `stream_enumerate_interval`, formed by the `pair`, is [^55: The numbers shown here do not really appear in the delayed expression. What actually appears is the original expression, in an environment in which the variables are bound to the appropriate numbers. For example, `low + 1` with `low` bound to 10,000 actually appears where `10001` is shown.]

```
print(pair(10000, lambda: stream_enumerate_interval(10001, 1000000)))
```

That is, `stream_enumerate_interval` returns a stream represented as a pair whose `head` is 10,000 and whose `tail` is a promise to enumerate more of the interval if so requested. This stream is now filtered for primes, using the stream analog of the `filter` function (section 2.2.3):

```
def stream_filter(pred, stream):
    return (None if is_none(stream)
            else pair(head(stream),
                      lambda: stream_filter(pred, stream_tail(stream)))
            if pred(head(stream))
            else stream_filter(pred, stream_tail(stream)))
```

The function `stream_filter` tests the `head` of the stream (which is 10,000). Since this is not prime, `stream_filter` examines the tail of its input stream. The call to `stream_tail` forces evaluation of the delayed `stream_enumerate_interval`, which now returns

```
print(pair(10001, lambda: stream_enumerate_interval(10002, 1000000)))
```

The function `stream_filter` now looks at the `head` of this stream, 10,001, sees that this is not prime either, forces another `stream_tail`, and so on, until `stream_enumerate_interval` yields the prime 10,007, whereupon `stream_filter`, according to its definition, returns

```
pair(head(stream),
     lambda: stream_filter(pred, stream_tail(stream)))
```

which in this case is

```
print(pair(10007,
  lambda: stream_filter(
    is_prime,
    pair(10008,
      lambda: stream_enumerate_interval(10009, 1000000))))))
```

This result is now passed to `stream_tail` in our original expression. This forces the delayed `stream_filter`, which in turn keeps forcing the delayed `stream_enumerate_interval` until it finds the next prime, which is 10,009. Finally, the result passed to `head` in our original expression is

```
print(pair(10009,
  lambda: stream_filter(
    is_prime,
    pair(10010,
      lambda: stream_enumerate_interval(10011, 1000000))))))
```

The function `head` returns 10,009, and the computation is complete. Only as many integers were tested for primality as were necessary to find the second prime, and the interval was enumerated only as far as was necessary to feed the prime filter.

In general, we can think of delayed evaluation as "demand-driven" programming, whereby each stage in the stream process is activated only enough to satisfy the next stage. What we have done is to decouple the actual order of events in the computation from the apparent structure of our functions. We write functions as if the streams existed "all at once" when, in reality, the computation is performed incrementally, as in traditional programming styles.

An optimization

When we construct stream pairs, we delay the evaluation of their tail expressions by wrapping these expressions in a function. We force their evaluation when needed, by applying the function.

This implementation suffices for streams to work as advertised, but there is an important optimization that we shall consider where needed. In many applications, we end up forcing the same delayed object many times. This can lead to serious inefficiency in recursive programs involving streams. (See exercise 3.57.) The solution is to build delayed objects so that the first time they are forced, they store the value that is computed. Subsequent forcings will simply return the stored value without repeating the computation. In other words, we implement the construction of stream pairs as a memoized function similar to the one described in exercise 3.27. One way to accomplish this is to use the following function, which takes as argument a function (of no arguments) and returns a memoized version of the function. The first time the memoized function is run, it saves the computed result. On subsequent evaluations, it simply returns the result. [^56:

There are many possible implementations of streams other than the one described in this section. Delayed evaluation, which is the key to making streams practical, was inherent in Algol 60's *call-by-name* parameter-passing method. The use of this mechanism to implement streams was first described by Landin (1965). Delayed evaluation for streams was introduced into Lisp by Friedman and Wise (1976). In their implementation, `cons` (the Lisp equivalent of our `pair` function) always delays evaluating its arguments, so that lists automatically behave as streams. The memoizing optimization is also known as *call-by-need*. The Algol community would refer to our original delayed objects as *call-by-name thunks* and to the optimized versions as *call-by-need thunks*.]

```
def memo(fun):
    already_run = False
    result = None
    def memoized():
        nonlocal already_run, result
        if not already_run:
            result = fun()
            already_run = True
            return result
        else:
            return result
    return memoized
```

We can make use of `memo` whenever we construct a stream pair. For example, instead of

```
def stream_map(f, s):
    return (None if is_none(s)
            else pair(f(head(s)),
                     lambda: stream_map(f, stream_tail(s))))
```

we can define an optimized function `stream_map` as follows:

```
def stream_map_optimized(f, s):
    return (None if is_none(s)
            else pair(f(head(s)),
                     memo(lambda:
                          stream_map_optimized(f, stream_tail(s)))))
```

Exercise 3.50. Declare a function `stream_map_2` that takes a binary function and two streams as arguments and returns a stream whose elements are the results of applying the function pairwise to the corresponding elements of the argument streams.

```
def stream_map_2(f, s1, s2):
    ...
```

Similar to `stream_map_optimized`, declare a function `stream_map_2_optimized` by modifying your `stream_map_2` such that the result stream employs memoization.

Exercise 3.51. What does the interpreter print in response to evaluating each statement in the following sequence? [^57: Exercises such as 3.51 and 3.52 are valuable for testing our understanding of how delayed evaluation works. On the other hand, intermixing delayed evaluation with printing—and, even worse, with reassignment—is extremely confusing, and instructors of courses on computer languages have traditionally tormented their students with examination questions such as the ones in this section. Needless to say, writing programs that depend on such subtleties is odious programming style. Part of the power of stream processing is that it lets us ignore the order in which events actually happen in our programs. Unfortunately, this is precisely what we cannot afford to do in the presence of reassignment, which forces us to be concerned with time and change.]

```
x = stream_map(print, stream_enumerate_interval(0, 10))

stream_ref(x, 5)

stream_ref(x, 7)
```

What does the interpreter print if `stream_map_optimized` is used instead of `stream_map`?

```
x = stream_map_optimized(print, stream_enumerate_interval(0, 10))

stream_ref(x, 5)

stream_ref(x, 7)
```

Exercise 3.52. Consider the sequence of statements

```
sum = 0

def accum(x):
    global sum
    sum = x + sum
    return sum

seq = stream_map(accum, stream_enumerate_interval(1, 20))

y = stream_filter(is_even, seq)

z = stream_filter(lambda x: x % 5 == 0, seq)

stream_ref(y, 7)

display_stream(z)
```

What is the value of `sum` after each of the above statements is evaluated? What is the printed response to evaluating the `stream_ref` and `display_stream` expressions? Would these responses differ if we had applied the function `memo` on every tail of every constructed stream pair, as suggested in the optimization above? Explain.

3.5.2 Infinite Streams

We have seen how to support the illusion of manipulating streams as complete entities even though, in actuality, we compute only as much of the stream as we need to access. We can exploit this technique to represent sequences efficiently as streams, even if the sequences are very long. What is more striking, we can use streams to represent sequences that are infinitely long. For instance, consider the following definition of the stream of positive integers:

```
def integers_starting_from(n):  
    return pair(n, lambda: integers_starting_from(n + 1))
```

```
integers = integers_starting_from(1)
```

This makes sense because `integers` will be a pair whose `head` is 1 and whose `tail` is a promise to produce the integers beginning with 2. This is an infinitely long stream, but in any given time we can examine only a finite portion of it. Thus, our programs will never know that the entire infinite stream is not there.

Using `integers` we can define other infinite streams, such as the stream of integers that are not divisible by 7:

```
def is_divisible(x, y):  
    return x % y == 0
```

```
no_sevens = stream_filter(lambda x: not is_divisible(x, 7),  
                           integers)
```

Then we can find integers not divisible by 7 simply by accessing elements of this stream:

```
print(stream_ref(no_sevens, 100))
```

```
117
```

In analogy with `integers`, we can define the infinite stream of Fibonacci numbers:

```
def fibgen(a, b):
    return pair(a, lambda: fibgen(b, a + b))

fibs = fibgen(0, 1)
```

The constant `fibs` is a pair whose `head` is 0 and whose `tail` is a promise to evaluate `fibgen(1, 1)`. When we evaluate this delayed `fibgen(1, 1)`, it will produce a pair whose `head` is 1 and whose `tail` is a promise to evaluate `fibgen(1, 2)`, and so on.

For a look at a more exciting infinite stream, we can generalize the `no_sevens` example to construct the infinite stream of prime numbers, using a *sieving* method. [^58: This method is reminiscent of the ancient *sieve of Eratosthenes*, named for Eratosthenes, a third-century BCE Alexandrian Greek mathematician. His sieve finds the primes by repeatedly crossing off the multiples of each prime in turn; although ancient, it has formed the basis for special-purpose hardware "sieves" that, until the 1970s, were the most powerful tools in existence for locating large primes. Since then, however, these methods have been superseded by outgrowths of the probabilistic techniques discussed in section 1.2.6. But the stream-based method given here is not actually Eratosthenes's sieve: as Melissa E. O'Neill shows in O'Neill 2009, it tests each candidate for divisibility by the primes found so far—a form of *trial division*—rather than crossing off multiples, and is substantially less efficient than the genuine sieve.] We start with the integers beginning with 2, which is the first prime. To get the rest of the primes, we start by filtering the multiples of 2 from the rest of the integers. This leaves a stream beginning with 3, which is the next prime. Now we filter the multiples of 3 from the rest of this stream. This leaves a stream beginning with 5, which is the next prime, and so on. In other words, we construct the primes by a sieving process, described as follows: To sieve a stream *S*, form a stream whose first element is the first element of *S* and the rest of which is obtained by filtering all multiples of the first element of *S* out of the rest of *S* and sieving the result. This process is readily described in terms of stream operations:

```
def sieve(stream):
    return pair(head(stream),
               lambda: sieve(stream_filter(
                           lambda x: not is_divisible(x, head(stream)),
                           stream_tail(stream))))

primes = sieve(integers_starting_from(2))
```

Now to find a particular prime we need only ask for it:

```
print(stream_ref(primes, 50))
```

233

It is interesting to contemplate the signal-processing system set up by `sieve`, shown in the "Henderson diagram" in figure 3.33. [^59: We have named these figures after Peter Henderson, who was the first person to show us diagrams of this sort as a way of thinking about stream processing.] The input stream feeds into an "unpairer" that separates the first element of the stream from the rest of the stream. The first element is used to construct a divisibility filter, through which the rest is passed, and the output of the filter is fed to another sieve box. Then the original first element is adjoined to the output of the internal sieve to form the output stream. Thus, not only is the stream infinite, but the signal processor is also infinite, because the sieve contains a sieve within it.

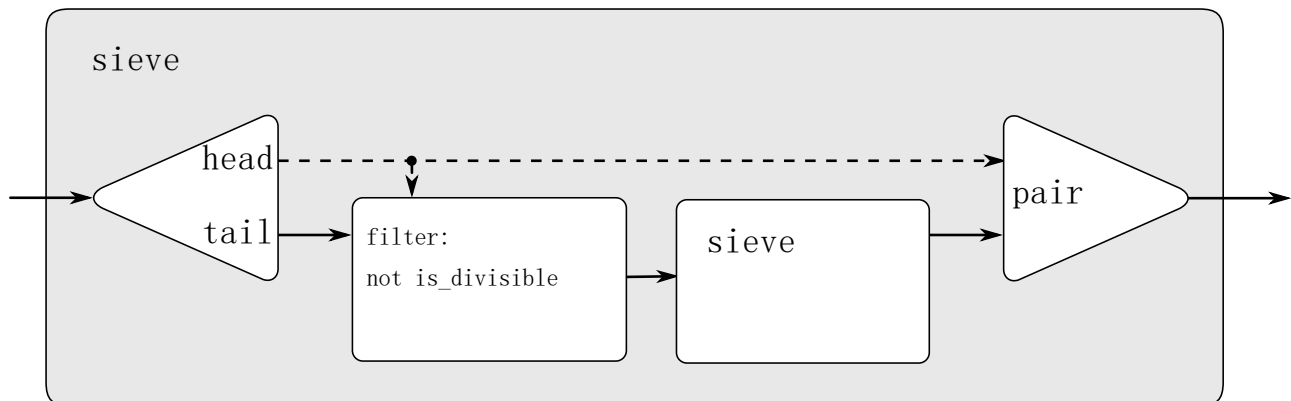


Figure 3.33. The prime sieve viewed as a signal-processing system. Each solid line represents a stream of values being transmitted. The dashed line from the `head` to the `pair` and the `filter` indicates that this is a single value rather than a stream.

Defining streams implicitly

The `integers` and `fibs` streams above were defined by specifying "generating" functions that explicitly compute the stream elements one by one. An alternative way to specify streams is to take advantage of delayed evaluation to define streams implicitly. For example, the following statement defines the stream `ones` to be an infinite stream of ones:

```
ones = pair(1, lambda: ones)
```

This works much like the declaration of a recursive function: `ones` is a pair whose `head` is 1 and whose `tail` is a promise to evaluate `ones`. Evaluating the `tail` gives us again a 1 and a promise to evaluate `ones`, and so on.

We can do more interesting things by manipulating streams with operations such as `add_streams`, which produces the elementwise sum of two given streams: [^60: This uses the function `stream_map_2` from exercise 3.50.]

```
def add_streams(s1, s2):
    return stream_map_2(lambda x1, x2: x1 + x2, s1, s2)
```


Now we can define the integers as follows:

```
integers = pair(1, lambda: add_streams(ones, integers))
```

This defines `integers` to be a stream whose first element is 1 and the rest of which is the sum of `ones` and `integers`. Thus, the second element of `integers` is 1 plus the first element of `integers`, or 2; the third element of `integers` is 1 plus the second element of `integers`, or 3; and so on. This definition works because, at any point, enough of the `integers` stream has been generated so that we can feed it back into the definition to produce the next integer.

We can define the Fibonacci numbers in the same style:

```
fibs = pair(0,
            lambda: pair(1,
                          lambda: add_streams(stream_tail(fibs),
                                                fibs)))
```

This definition says that `fibs` is a stream beginning with 0 and 1, such that the rest of the stream can be generated by adding `fibs` to itself shifted by one place:

$$\begin{array}{l} \text{stream} \\ \text{fibs} \end{array} \begin{array}{l} \text{tail}(\text{fibs}) \\ \hline 0 \end{array} \begin{array}{l} 1 \\ 1 \end{array} \begin{array}{l} 1 \\ 2 \end{array} \begin{array}{l} 2 \\ 3 \end{array} \begin{array}{l} 3 \\ 5 \end{array} \begin{array}{l} 5 \\ 8 \end{array} \begin{array}{l} 8 \\ 13 \end{array} \begin{array}{l} 13 \\ 21 \end{array} \dots$$

The function `scale_stream` is also useful in formulating such stream definitions. This multiplies each item in a stream by a given constant:

```
def scale_stream(stream, factor):
    return stream_map(lambda x: x * factor,
                      stream)
```

For example,

```
double = pair(1, lambda: scale_stream(double, 2))
```

produces the stream of powers of 2: 1, 2, 4, 8, 16, 32,....

An alternate definition of the stream of primes can be given by starting with the integers and filtering them by testing for primality. We will need the first prime, 2, to get started:

```
primes = pair(2,
              lambda: stream_filter(is_prime,
                                    integers_starting_from(3)))
```

This definition is not so straightforward as it appears, because we will test whether a number n is prime by checking whether n is divisible by a prime (not by just any integer) less than or equal to $\sqrt{n}$:

```
def is_prime(n):
    def iter(ps):
        return (True if square(head(ps)) > n
                else False if is_divisible(n, head(ps))
                else iter(stream_tail(ps)))
    return iter(primes)
```

This is a recursive definition, since `primes` is defined in terms of the `is_prime` predicate, which itself uses the `primes` stream. The reason this function works is that, at any point, enough of the `primes` stream has been generated to test the primality of the numbers we need to check next. That is, for every n we test for primality, either n is not prime (in which case there is a prime already generated that divides it) or n is prime (in which case there is a prime already generated—i.e., a prime less than n —that is greater than $\sqrt{n}$). [^61: This last point is very subtle and relies on the fact that $p_{n+1} \leq p_n^2$. (Here, p_k denotes the k th prime.) Estimates such as these are very difficult to establish. The ancient proof by Euclid that there are an infinite number of primes shows that $p_{n+1} \leq p_1 p_2 \cdots p_n + 1$, and no substantially better result was proved until 1851, when the Russian mathematician P. L. Chebyshev established that $p_{n+1} \leq 2p_n$ for all n . This result, originally conjectured in 1845, is known as *Bertrand's hypothesis*. A proof can be found in section 22.3 of Hardy and Wright 1960.]

Exercise 3.53. Without running the program, describe the elements of the stream defined by

```
s = pair(1, lambda: add_streams(s, s))
```

Exercise 3.54. Define a function `mul_streams`, analogous to `add_streams`, that produces the elementwise product of its two input streams. Use this together with the stream of `integers` to complete the following definition of the stream whose n th element (counting from 0) is $n+1$ factorial:

```
factorials = pair(1, lambda: mul_streams(??, ??))
```

Exercise 3.55. Define a function `partial_sums` that takes as argument a stream `SS` and returns the stream whose elements are $S_0, S_0+S_1, S_0+S_1+S_2, \dots$. For example, `partial_sums(integers)` should be the stream `1, 3, 6, 10, 15, \dots`.

Exercise 3.56. A famous problem, first raised by R. Hamming, is to enumerate, in ascending order with no repetitions, all positive integers with no prime factors other than 2, 3, or 5. One obvious way to do this is to simply test each integer in turn to see whether it has any factors other than 2, 3, and 5. But this is very inefficient, since, as the integers get larger, fewer and fewer of them fit the requirement. As an alternative, let us call the required stream of numbers `S` and notice the following facts about it.

- `S` begins with 1.
- The elements of `scale_stream(S, 2)` are also elements of `S`.
- The same is true for `scale_stream(S, 3)` and `scale_stream(S, 5)`.
- These are all the elements of `S`. Now all we have to do is combine elements from these sources. For this we define a function `merge` that combines two ordered streams into one ordered result stream, eliminating repetitions:

```
def merge(s1, s2):
    if is_none(s1):
        return s2
    elif is_none(s2):
        return s1
    else:
        s1head = head(s1)
        s2head = head(s2)
        return (pair(s1head, lambda: merge(stream_tail(s1), s2))
                if s1head < s2head
                else pair(s2head, lambda: merge(s1, stream_tail(s2)))
                if s1head > s2head
                else pair(s1head, lambda: merge(stream_tail(s1), stream_tail(s2))))
```

Then the required stream may be constructed with `merge`, as follows:

```
S = pair(1, lambda: merge(??, ??))
```

Fill in the missing expressions in the places marked above.

Exercise 3.57. How many additions are performed when we compute the n th Fibonacci number using the declaration of `fibs` based on the `add_streams` function? Show that this number is exponentially greater than the number of additions performed if `add_streams` had used the function `stream_map_2_optimized` described in exercise 3.50, and if we had declared `fibs` as follows:

```
fibs = pair(0,
            memo(lambda: pair(1,
                              memo(lambda: add_streams(stream_tail(fibs),
                                                            fibs))))))
```

Exercise 3.58. Give an interpretation of the stream computed by the function

```
def expand(num, den, radix):
    return pair(math_trunc((num * radix) / den),
                lambda: expand((num * radix) % den, den, radix))
```

where `math_trunc` discards the fractional part of its argument, here the remainder of the division. What are the successive elements produced by `expand(1, 7, 10)`? What is produced by `expand(3, 8, 10)`?

Exercise 3.59. In section 2.5.3 we saw how to implement a polynomial arithmetic system representing polynomials as lists of terms. In a similar way, we can work with *power series*, such as

$$\begin{array}{l} e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots, \\ \cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots, \\ \sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots, \end{array}$$

represented as infinite streams. We will represent the series $a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots$ as the stream whose elements are the coefficients $a_0, a_1, a_2, a_3, \dots$. The integral of the series $a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \cdots$ is the series

$$c + a_0 x + \frac{1}{2}a_1 x^2 + \frac{1}{3}a_2 x^3 + \frac{1}{4}a_3 x^4 + \cdots$$

where c is any constant. Define a function `integrate_series` that takes as input a stream $a_0, a_1, a_2, \dots$ representing a power series and returns the stream $a_0, \frac{1}{2}a_1, \frac{1}{3}a_2, \dots$ of coefficients of the nonconstant terms of the integral of the series. (Since the result has no constant term, it doesn't represent a power series; when we use `integrate_series`, we will use `pair` to adjoin the appropriate constant to the beginning of the stream.) b. The function $x \mapsto e^x$ is its own derivative. This implies that e^x and the integral of e^x are the same series, except for the constant term, which is $e^0 = 1$. Accordingly, we can generate the series for e^x as

```
exp_series = pair(1, lambda: integrate_series(exp_series))
```

Show how to generate the series for sine and cosine, starting from the facts that the derivative of sine is cosine and the derivative of cosine is the negative of sine:

```
cosine_series = pair(1, ??)
sine_series = pair(0, ??)
```

Exercise 3.60. With power series represented as streams of coefficients as in exercise 3.59, adding series is implemented by `add_streams`. Complete the declaration of the following function for

multiplying series:

```
def mul_series(s1, s2):  
    pair(??, lambda: add_streams(??, ??))
```

You can test your function by verifying that $\sin^2 x + \cos^2 x = 1$, using the series from exercise 3.59.

Exercise 3.61. Let S be a power series (exercise 3.59) whose constant term is 1. Suppose we want to find the power series $1/S$, that is, the series X such that $S \cdot X = 1$. Write $S = 1 + S_R$ where S_R is the part of S after the constant term. Then we can solve for X as follows:
$$S \cdot X = 1 \implies (1 + S_R) \cdot X = 1 \implies X + S_R \cdot X = 1 \implies X = 1 - S_R \cdot X$$
 In other words, X is the power series whose constant term is 1 and whose higher-order terms are given by the negative of S_R times X . Use this idea to write a function `invert_unit_series` that computes $1/S$ for a power series S with constant term 1. You will need to use `mul_series` from exercise 3.60.

Exercise 3.62. Use the results of exercises 3.60 and 3.61 to define a function `div_series` that divides two power series. The function `div_series` should work for any two series, provided that the denominator series begins with a nonzero constant term. (If the denominator has a zero constant term, then `div_series` should signal an error.) Show how to use `div_series` together with the result of exercise 3.59 to generate the power series for tangent.

3.5.3 Exploiting the Stream Paradigm

Streams with delayed evaluation can be a powerful modeling tool, providing many of the benefits of local state and reassignment. Moreover, they avoid some of the theoretical tangles that accompany the introduction of reassignment into a programming language.

The stream approach can be illuminating because it allows us to build systems with different module boundaries than systems organized around reassignment to state variables. For example, we can think of an entire time series (or signal) as a focus of interest, rather than the values of the state variables at individual moments. This makes it convenient to combine and compare components of state from different moments.

Formulating iterations as stream processes

In section 1.2.1, we introduced iterative processes, which proceed by updating state variables. We know now that we can represent state as a "timeless" stream of values rather than as a set of variables to be updated. Let's adopt this perspective in revisiting the square-root function from section 1.1.7. Recall that the idea is to generate a sequence of better and better guesses for the square root of x by applying over and over again the function that improves guesses:

```
def sqrt_improve(guess, x):
    return average(guess, x / guess)
```

In our original `sqrt` function, we made these guesses be the successive values of a state variable. Instead we can generate the infinite stream of guesses, starting with an initial guess of 1:

```
def sqrt_stream(x):
    return pair(1, lambda: stream_map(lambda guess: sqrt_improve(guess, x),
                                      sqrt_stream(x)))
```

```
print(display_stream(sqrt_stream(2)))
```

```
1
1.5
1.4166666666666665
1.4142156862745097
1.4142135623746899
...
```

We can generate more and more terms of the stream to get better and better guesses. If we like, we can write a function that keeps generating terms until the answer is good enough. (See exercise 3.64.)

Another iteration that we can treat in the same way is to generate an approximation to π , based upon the alternating series that we saw in section 1.3.1:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$$

We first generate the stream of summands of the series (the reciprocals of the odd integers, with alternating signs). Then we take the stream of sums of more and more terms (using the `partial_sums` function of exercise 3.55) and scale the result by 4:

```
def pi_summands(n):
    return pair(1 / n, lambda: stream_map(lambda x: -x, pi_summands(n + 2)))

pi_stream = scale_stream(partial_sums(pi_summands(1)), 4)
```

```
print(display_stream(pi_stream))
```

```

4
2.666666666666667
3.466666666666667
2.8952380952380956
3.3396825396825403
2.9760461760461765
3.2837384837384844
3.017071817071818
...

```

This gives us a stream of better and better approximations to π , although the approximations converge rather slowly. Eight terms of the sequence bound the value of π between 3.284 and 3.017.

So far, our use of the stream of states approach is not much different from updating state variables. But streams give us an opportunity to do some interesting tricks. For example, we can transform a stream with a *sequence accelerator* that converts a sequence of approximations to a new sequence that converges to the same value as the original, only faster.

One such accelerator, due to the eighteenth-century Swiss mathematician Leonhard Euler, works well with sequences that are partial sums of alternating series (series of terms with alternating signs). In Euler's technique, if S_n is the n th term of the original sum sequence, then the accelerated sequence has terms

$$S_{n+1} - \frac{(S_{n+1} - S_n)^2}{S_{n-1} - 2S_n + S_{n+1}}$$

Thus, if the original sequence is represented as a stream of values, the transformed sequence is given by

```

def euler_transform(s): s0 = stream_ref(s, 0) # S_{n-1}
s1 = stream_ref(s, 1) # S_n
s2 = stream_ref(s, 2) # S_{n+1}
return pair(s2 - square(s2 - s1) / (s0 + (-2) * s1 + s2), memo(lambda:
euler_transform(stream_tail(s))))

```

Note that we make use of the memoization optimization of section 3.5.1, because in the following we will rely on repeated evaluation of the resulting stream.

We can demonstrate Euler acceleration with our sequence of approximations to π :

```

print(display_stream(euler_transform(pi_stream)))

```

```

3.166666666666667
3.133333333333337
3.1452380952380956
3.13968253968254
3.1427128427128435
3.1408813408813416
3.142071817071818
3.1412548236077655
...

```

Even better, we can accelerate the accelerated sequence, and recursively accelerate that, and so on. Namely, we create a stream of streams (a structure we'll call a *tableau*) in which each stream is the transform of the preceding one:

```

def make_tableau(transform, s):
    return pair(s, lambda: make_tableau(transform, transform(s)))

```

The tableau has the form

$$\begin{array}{cccccccccccccccccccc} s_{00} & s_{01} & s_{02} & s_{03} & s_{04} & \dots & s_{10} & s_{11} & s_{12} \\ & s_{13} & \dots & & s_{20} & s_{21} & s_{22} & \dots & & & \dots & \end{array}$$

Finally, we form a sequence by taking the first term in each row of the tableau:

```

def accelerated_sequence(transform, s):
    return stream_map(head, make_tableau(transform, s))

```

We can demonstrate this kind of "super-acceleration" of the π sequence:

```

print(display_stream(accelerated_sequence(euler_transform, pi_stream)))

```

```

4
3.166666666666667
3.142105263157895
3.141599357319005
3.1415927140337785
3.1415926539752927
3.1415926535911765
3.141592653589778
...

```

The result is impressive. Taking eight terms of the sequence yields the correct value of π to 14 decimal places. If we had used only the original π sequence, we would need to compute on the order of 10^{13} terms (i.e., expanding the series far enough so that the individual terms are less than 10^{-13}) to get that much accuracy!

We could have implemented these acceleration techniques without using streams. But the stream formulation is particularly elegant and convenient because the entire sequence of states is available to us as a data structure that can be manipulated with a uniform set of operations.

Exercise 3.63. Louis Reasoner is not happy with the performance of the stream produced by the `sqrt_stream` function and tries to optimize it using memoization:

```
def sqrt_stream_optimized(x):
    return pair(1,
               memo(lambda: stream_map(lambda guess:
                                       sqrt_improve(guess, x),
                                       sqrt_stream_optimized(x))))
```

Alyssa P. Hacker instead proposes

```
def sqrt_stream_optimized_2(x):
    guesses = pair(1,
                  memo(lambda: stream_map(lambda guess:
                                          sqrt_improve(guess, x),
                                          guesses)))
    return guesses
```

and claims that Louis's version is considerably less efficient than hers, because it performs redundant computation. Explain Alyssa's answer. Would Alyssa's approach without memoization be more efficient than the original `sqrt_stream`?

Exercise 3.64. Write a function `stream_limit` that takes as arguments a stream and a number (the tolerance). It should examine the stream until it finds two successive elements that differ in absolute value by less than the tolerance, and return the second of the two elements. Using this, we could compute square roots up to a given tolerance by

```
def sqrt(x, tolerance):
    return stream_limit(sqrt_stream(x), tolerance)
```

Exercise 3.65. Use the series

$$\ln 2 = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \cdots$$

to compute three sequences of approximations to the natural logarithm of 2, in the same way we did above for π . How rapidly do these sequences converge?

Infinite streams of pairs

In section , we saw how the sequence paradigm handles traditional nested loops as processes defined on sequences of pairs. If we generalize this technique to infinite streams, then we can write

programs that are not easily represented as loops, because the "looping" must range over an infinite set.

For example, suppose we want to generalize the `prime_sum_pairs` function of section to produce the stream of pairs of *all* integers (i,j) with $i \leq j$ such that $i+j$ is prime. If `int_pairs` is the sequence of all pairs of integers (i,j) with $i \leq j$, then our required stream is simply [^62: As in section 2.2.3, we represent a pair of integers as a list rather than a pair.]

```
print(stream_filter(lambda pair: is_prime(head(pair) + head(tail(pair))),
                    int_pairs))
```

Our problem, then, is to produce the stream `int_pairs`. More generally, suppose we have two streams $S = (S_i)$ and $T = (T_j)$, and imagine the infinite rectangular array

$$\begin{array}{cccc} (S_0, T_0) & (S_0, T_1) & (S_0, T_2) & \dots \\ & (S_1, T_0) & (S_1, T_1) & (S_1, T_2) \\ & \dots & (S_2, T_0) & (S_2, T_1) & (S_2, T_2) & \dots \end{array}$$

We wish to generate a stream that contains all the pairs in the array that lie on or above the diagonal, i.e., the pairs

$$\begin{array}{cccc} (S_0, T_0) & (S_0, T_1) & (S_0, T_2) & \dots \\ & (S_1, T_1) & (S_1, T_2) & \dots \\ & & (S_2, T_2) & \dots \end{array}$$

(If we take both S and T to be the stream of integers, then this will be our desired stream `int_pairs`.)

Call the general stream of pairs `pairs(S, T)`, and consider it to be composed of three parts: the pair (S_0, T_0) , the rest of the pairs in the first row, and the remaining pairs: [^63: See exercise 3.68 for some insight into why we chose this decomposition.]

$$\begin{array}{c|cccc} (S_0, T_0) & (S_0, T_1) & (S_0, T_2) & \dots & \\ \hline & (S_1, T_1) & (S_1, T_2) & \dots & \\ & & (S_2, T_2) & \dots & \end{array}$$

Observe that the third piece in this decomposition (pairs that are not in the first row) is (recursively) the pairs formed from `stream_tail(S)` and `stream_tail(T)`. Also note that the second piece (the rest of the first row) is

```
stream_map(lambda x: llist(head(s), x),
            stream_tail(t))
```

Thus we can form our stream of pairs as follows:

```
def pairs(s, t):
    return pair(llist(head(s), head(t)),
               lambda: *combine-in-some-way*(
                   stream_map(lambda x: llist(head(s), x),
                               stream_tail(t)),
                   pairs(stream_tail(s), stream_tail(t))))
```

In order to complete the function, we must choose some way to combine the two inner streams. One idea is to use the stream analog of the `append` function from section 2.2.1:

```
def stream_append(s1, s2):
    return (s2 if is_none(s1)
            else pair(head(s1),
                      lambda: stream_append(stream_tail(s1), s2)))
```

This is unsuitable for infinite streams, however, because it takes all the elements from the first stream before incorporating the second stream. In particular, if we try to generate all pairs of positive integers using

```
print(pairs(integers, integers))
```

our stream of results will first try to run through all pairs with the first integer equal to 1, and hence will never produce pairs with any other value of the first integer.

To handle infinite streams, we need to devise an order of combination that ensures that every element will eventually be reached if we let our program run long enough. An elegant way to accomplish this is with the following `interleave` function: [^64: The precise statement of the required property on the order of combination is as follows: There should be a function f of two arguments such that the pair corresponding to element i of the first stream and element j of the second stream will appear as element number $f(i,j)$ of the output stream. The trick of using `interleave` to accomplish this was shown to us by David Turner, who employed it in the language KRC (Turner 1981).]

```
def interleave(s1, s2):
    return (s2 if is_none(s1)
            else pair(head(s1),
                      lambda: interleave(s2, stream_tail(s1))))
```

Since `interleave` takes elements alternately from the two streams, every element of the second stream will eventually find its way into the interleaved stream, even if the first stream is infinite.

We can thus generate the required stream of pairs as

```
def pairs(s, t):
    return pair(l1st(head(s), head(t)),
               lambda: interleave(stream_map(lambda x: l1st(head(s), x),
                                             stream_tail(t)),
                                   pairs(stream_tail(s),
                                         stream_tail(t))))
```

Exercise 3.66. Examine the stream `pairs(integers, integers)`. Can you make any general comments about the order in which the pairs are placed into the stream? For example, approximately how many pairs precede the pair (1,100)? the pair (99,100)? the pair (100,100)? (If you can make precise mathematical statements here, all the better. But feel free to give more qualitative answers if you find yourself getting bogged down.)

Exercise 3.67. Modify the `pairs` function so that `pairs(integers, integers)` will produce the stream of *all* pairs of integers (i,j) (without the condition $i \leq j$). Hint: You will need to mix in an additional stream.

Exercise 3.68. Louis Reasoner thinks that building a stream of pairs from three parts is unnecessarily complicated. Instead of separating the pair (S_0, T_0) from the rest of the pairs in the first row, he proposes to work with the whole first row, as follows:

```
def pairs(s, t):
    return interleave(stream_map(lambda x: l1st(head(s), x),
                                t),
                    pair(stream_tail(s), stream_tail(t)))
```

Does this work? Consider what happens if we evaluate `pairs(integers, integers)` using Louis's definition of `pairs`.

Exercise 3.69. Write a function `triples` that takes three infinite streams, S , T , and U , and produces the stream of triples (S_i, T_j, U_k) such that $i \leq j \leq k$. Use `triples` to generate the stream of all Pythagorean triples of positive integers, i.e., the triples (i,j,k) such that $i \leq j$ and $i^2 + j^2 = k^2$.

Exercise 3.70. It would be nice to be able to generate streams in which the pairs appear in some useful order, rather than in the order that results from an *ad hoc* interleaving process. We can use a technique similar to the `merge` function of exercise 3.56, if we define a way to say that one pair of integers is "less than" another. One way to do this is to define a "weighting function" $W(i,j)$ and stipulate that (i_1, j_1) is less than (i_2, j_2) if $W(i_1, j_1) < W(i_2, j_2)$. Write a function `merge_weighted` that is like `merge`, except that `merge_weighted` takes an additional argument `weight`, which is a function that computes the weight of a pair, and is used to determine the order in which elements should appear in the resulting merged stream. [^65: We will require that the weighting function be such that the weight of a pair increases as we move out along a row or down along a column of the array of pairs.] Using this, generalize `pairs` to a function

`weighted_pairs` that takes two streams, together with a function that computes a weighting function, and generates the stream of pairs, ordered according to weight. Use your function to generate a. the stream of all pairs of positive integers (i,j) with $i \leq j$ ordered according to the sum $i + j$ b. the stream of all pairs of positive integers (i,j) with $i \leq j$, where neither i nor j is divisible by 2, 3, or 5, and the pairs are ordered according to the sum $2i + 3j + 5ij$.

Exercise 3.71. Numbers that can be expressed as the sum of two cubes in more than one way are sometimes called *Ramanujan numbers*, in honor of the mathematician Srinivasa Ramanujan. [^66: To quote from G. H. Hardy's obituary of Ramanujan (Hardy 1921): "It was Mr. Littlewood (I believe) who remarked that "every positive integer was one of his friends." I remember once going to see him when he was lying ill at Putney. I had ridden in taxi-cab No. 1729, and remarked that the number seemed to me a rather dull one, and that I hoped it was not an unfavorable omen. "No," he replied, "it is a very interesting number; it is the smallest number expressible as the sum of two cubes in two different ways.""] The trick of using weighted pairs to generate the Ramanujan numbers was shown to us by Charles Leiserson.] Ordered streams of pairs provide an elegant solution to the problem of computing these numbers. To find a number that can be written as the sum of two cubes in two different ways, we need only generate the stream of pairs of integers (i,j) weighted according to the sum $i^3 + j^3$ (see exercise 3.70), then search the stream for two consecutive pairs with the same weight. Write a function to generate the Ramanujan numbers. The first such number is 1,729. What are the next five?

Exercise 3.72. In a similar way to exercise 3.71 generate a stream of all numbers that can be written as the sum of two squares in three different ways (showing how they can be so written).

Streams as signals

We began our discussion of streams by describing them as computational analogs of the "signals" in signal-processing systems. In fact, we can use streams to model signal-processing systems in a very direct way, representing the values of a signal at successive time intervals as consecutive elements of a stream. For instance, we can implement an *integrator* or *summer* that, for an input stream $x=(x_i)$, an initial value C , and a small increment dt , accumulates the sum

$$S_i = C + \sum_{j=1}^i x_j, \quad dt$$

and returns the stream of values $S=(S_i)$. The following `integral` function is reminiscent of the "implicit style" definition of the stream of integers (section 3.5.2):

```
def integral(integrand, initial_value, dt):
    integ = pair(initial_value,
                  lambda: add_streams(scale_stream(integrand, dt),
                                      integ))
    return integ
```

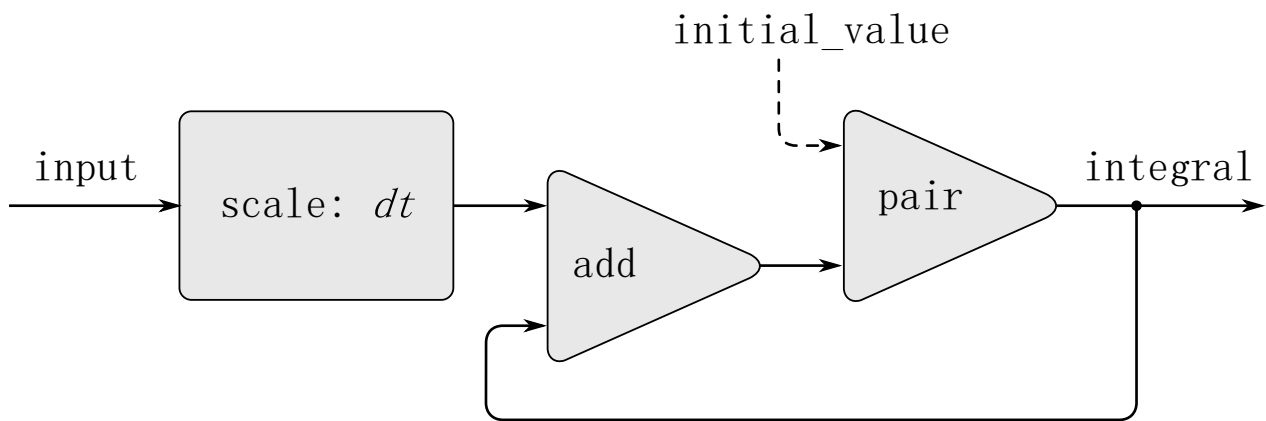


Figure 3.34. The `integral` function viewed as a signal-processing system.

Figure 3.34 is a picture of a signal-processing system that corresponds to the `integral` function. The input stream is scaled by dt and passed through an adder, whose output is passed back through the same adder. The self-reference in the definition of `integ` is reflected in the figure by the feedback loop that connects the output of the adder to one of the inputs.

Exercise 3.73. We can model electrical circuits using streams to represent the values of currents or voltages at a sequence of times. For instance, suppose we have an *RC circuit* consisting of a resistor of resistance R and a capacitor of capacitance C in series. The voltage response v of the circuit to an injected current i is determined by the formula in figure 3.35, whose structure is shown by the accompanying signal-flow diagram.

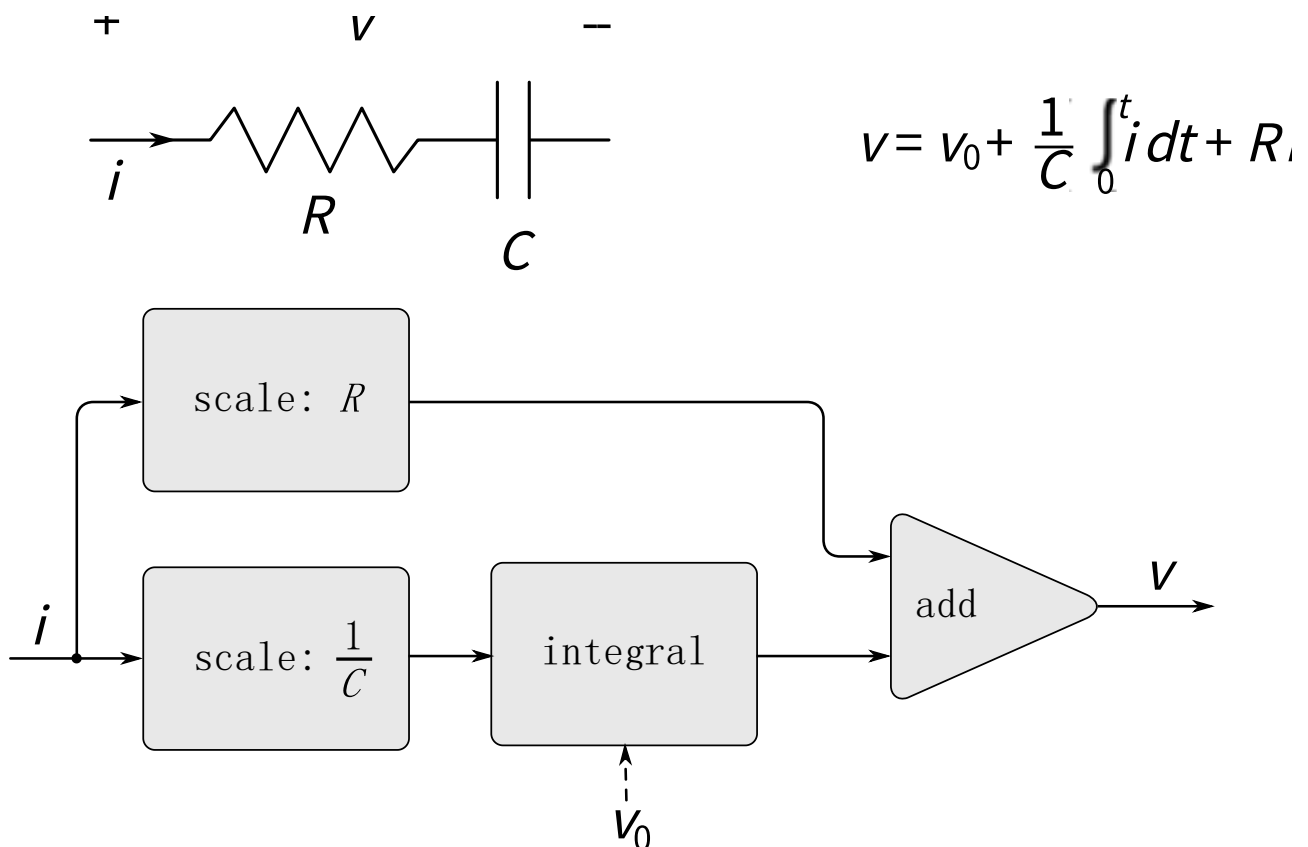


Figure 3.35. An RC circuit and the associated signal-flow diagram.

Write a function `RC` that models this circuit. `RC` should take as inputs the values of R , C , and Δt and should return a function that takes as inputs a stream representing the current i and an initial value for the capacitor voltage $v_{\{0\}}$ and produces as output the stream of voltages v . For example, you should be able to use `RC` to model an RC circuit with $R = 5$ ohms, $C = 1$ farad, and a 0.5-second time step by evaluating `const RC1 = RC(5, 1, 0.5)`. This defines `RC1` as a function that takes a stream representing the time sequence of currents and an initial capacitor voltage and produces the output stream of voltages.

Exercise 3.74. Alyssa P. Hacker is designing a system to process signals coming from physical sensors. One important feature she wishes to produce is a signal that describes the *zero crossings* of the input signal. That is, the resulting signal should be $+1$ whenever the input signal changes from negative to positive, -1 whenever the input signal changes from positive to negative, and 0 otherwise. (Assume that the sign of a 0 input is positive.) For example, a typical input signal with its associated zero-crossing signal would be

$\dots \ 1 \ 2 \ 1.5 \ 1 \ 0.5 \ -0.1 \ -2 \ -3 \ -2 \ -0.5 \ 0.2 \ 3 \ 4 \ \dots$ $\dots \ 0 \ 0 \ 0 \ 0 \ 0 \ -1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ \dots$

In Alyssa's system, the signal from the sensor is represented as a stream `sense_data` and the stream `zero_crossings` is the corresponding stream of zero crossings. Alyssa first writes a function `sign_change_detector` that takes two values as arguments and compares the signs of the values to produce an appropriate 0 , 1 , or -1 . She then constructs her zero-crossing stream as follows:

```
def make_zero_crossings(input_stream, last_value):
    return pair(sign_change_detector(head(input_stream), last_value),
               lambda: make_zero_crossings(stream_tail(input_stream),
                                             head(input_stream)))

zero_crossings = make_zero_crossings(sense_data, 0)
```

Alyssa's boss, Eva Lu Ator, walks by and suggests that this program is approximately equivalent to the following one, which uses the function `stream_map_2` from exercise 3.50:

```
zero_crossings = stream_map_2(sign_change_detector,
                             sense_data,
                             *expression*)
```

Complete the program by supplying the indicated *expression*.

Exercise 3.75. Unfortunately, Alyssa's zero-crossing detector in exercise 3.74 proves to be insufficient, because the noisy signal from the sensor leads to spurious zero crossings. Lem E. Tweakit, a hardware specialist, suggests that Alyssa smooth the signal to filter out the noise before extracting the zero crossings. Alyssa takes his advice and decides to extract the zero crossings from the signal constructed by averaging each value of the sense data with the previous value. She

explains the problem to her assistant, Louis Reasoner, who attempts to implement the idea, altering Alyssa's program as follows:

```
def make_zero_crossings(input_stream, last_value):
    avpt = (head(input_stream) + last_value) / 2
    return pair(sign_change_detector(avpt, last_value),
                lambda: make_zero_crossings(stream_tail(input_stream),
                                                avpt))
```

This does not correctly implement Alyssa's plan. Find the bug that Louis has installed and fix it without changing the structure of the program. (Hint: You will need to increase the number of arguments to `make_zero_crossings`.)

Exercise 3.76. Eva Lu Ator has a criticism of Louis's approach in exercise 3.75. The program he wrote is not modular, because it intermixes the operation of smoothing with the zero-crossing extraction. For example, the extractor should not have to be changed if Alyssa finds a better way to condition her input signal. Help Louis by writing a function `smooth` that takes a stream as input and produces a stream in which each element is the average of two successive input stream elements. Then use `smooth` as a component to implement the zero-crossing detector in a more modular style.

3.5.4 Streams and Delayed Evaluation

The `integral` function at the end of the preceding section shows how we can use streams to model signal-processing systems that contain feedback loops. The feedback loop for the adder shown in figure 3.34 is modeled by the fact that `integral`'s internal stream `integ` is defined in terms of itself:

```
integ = pair(initial_value,
             lambda: add_streams(scale_stream(integrand, dt),
                                 integ))
```

The interpreter's ability to deal with such an implicit definition depends on the delay resulting from wrapping the call to `add_streams` in a lambda expression. Without this delay, the interpreter could not construct `integ` before evaluating the call to `add_streams`, which would require that `integ` already be defined. In general, such a delay is crucial for using streams to model signal-processing systems that contain loops. Without a delay, our models would have to be formulated so that the inputs to any signal-processing component would be fully evaluated before the output could be produced. This would outlaw loops.

Unfortunately, stream models of systems with loops may require uses of a delay beyond the stream programming pattern seen so far. For instance, figure 3.36 shows a signal-processing system for solving the differential equation $dy/dt=f(y)$ where f is a given function. The figure

shows a mapping component, which applies f to its input signal, linked in a feedback loop to an integrator in a manner very similar to that of the analog computer circuits that are actually used to solve such equations.

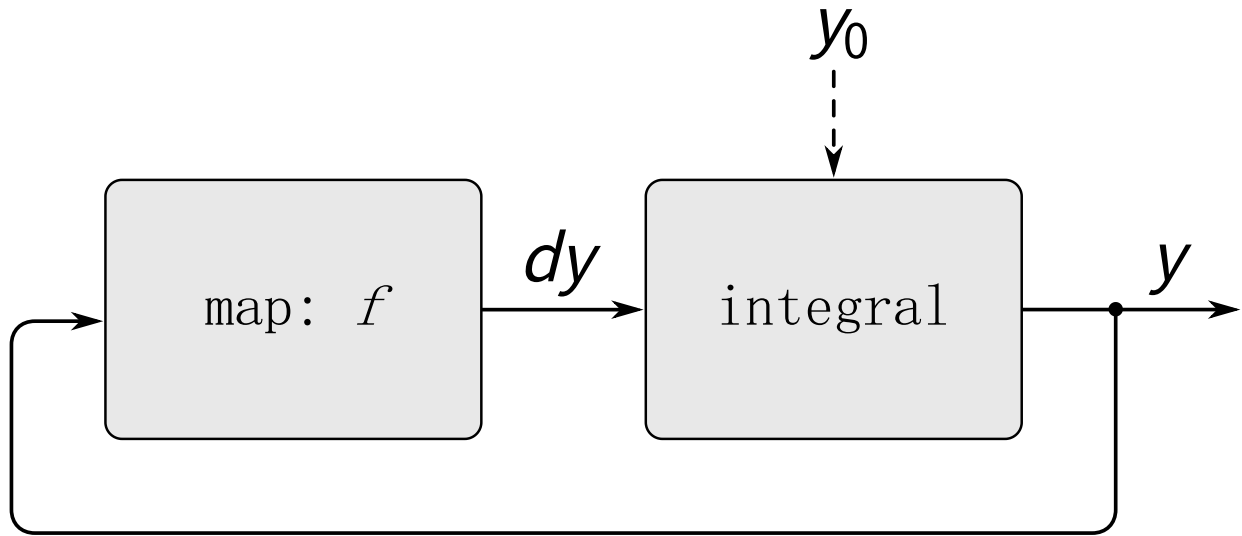


Figure 3.36. An "analog computer circuit" that solves the equation $dy/dt = f(y)$.

Assuming we are given an initial value y_0 for y , we could try to model this system using the function

```
def solve(f, y0, dt):
    y = integral(dy, y0, dt)
    dy = stream_map(f, y)
    return y
```

This function does not work, because in the first line of `solve` the call to `integral` requires that the input `dy` be defined, which does not happen until the second line of `solve`.

On the other hand, the intent of our definition does make sense, because we can, in principle, begin to generate the `y` stream without knowing `dy`. Indeed, `integral` and many other stream operations can generate part of the answer given only partial information about the arguments. For `integral`, the first element of the output stream is the specified `initial_value`. Thus, we can generate the first element of the output stream without evaluating the integrand `dy`. Once we know the first element of `y`, the `stream_map` in the second line of `solve` can begin working to generate the first element of `dy`, which will produce the next element of `y`, and so on.

To take advantage of this idea, we will redefine `integral` to expect the integrand stream to be a *delayed argument*. The function `integral` will force the integrand to be evaluated only when it is required to generate more than the first element of the output stream:

```
def integral(delayed_integrand, initial_value, dt):
    integ = pair(initial_value,
                  lambda: add_streams(
                      scale_stream(delayed_integrand(), dt),
                      integ))
    return integ
```

Now we can implement our `solve` function by delaying the evaluation of `dy` in the declaration of `y`:

```
def solve(f, y0, dt):
    y = integral(lambda: dy, y0, dt)
    dy = stream_map(f, y)
    return y
```

In general, every caller of `integral` must now delay the integrand argument. We can demonstrate that the `solve` function works by approximating $e \approx 2.718$ by computing the value at $y=1$ of the solution to the differential equation $dy/dt=y$ with initial condition $y(0)=1$: [^67: To complete in reasonable time, this calculation requires the use of the memoization optimization from section 3.5.1 in `integral` and in the function `add_streams` used in `integral` (using the function `stream_map_2_optimized` as suggested in exercise 3.57).]

```
print(stream_ref(solve(lambda y: y, 1, 0.001), 1000))
```

```
2.716923932235896
```

Exercise 3.77. The `integral` function used above was analogous to the "implicit" definition of the infinite stream of integers in section 3.5.2. Alternatively, we can give a definition of `integral` that is more like `integers-starting-from` (also in section 3.5.2):

```
def integral(integrand, initial_value, dt):
    return pair(initial_value,
                None if is_none(integrand)
                else integral(stream_tail(integrand),
                              dt * head(integrand) + initial_value,
                              dt))
```

When used in systems with loops, this function has the same problem as does our original version of `integral`. Modify the function so that it expects the `integrand` as a delayed argument and hence can be used in the `solve` function shown above.

Exercise 3.78. Consider the problem of designing a signal-processing system to study the homogeneous second-order linear differential equation

$$\frac{d^2 y}{dt^2} - a \frac{dy}{dt} - by = 0$$

The output stream, modeling y , is generated by a network that contains a loop. This is because the value of $d^2 y/dt^2$ depends upon the values of y and dy/dt and both of these are determined by integrating $d^2 y/dt^2$. The diagram we would like to encode is shown in figure 3.37. Write a function `solve_2nd` that takes as arguments the constants a , b , and dt and the initial values y_0 and dy_0 for y and dy/dt and generates the stream of successive values of y .

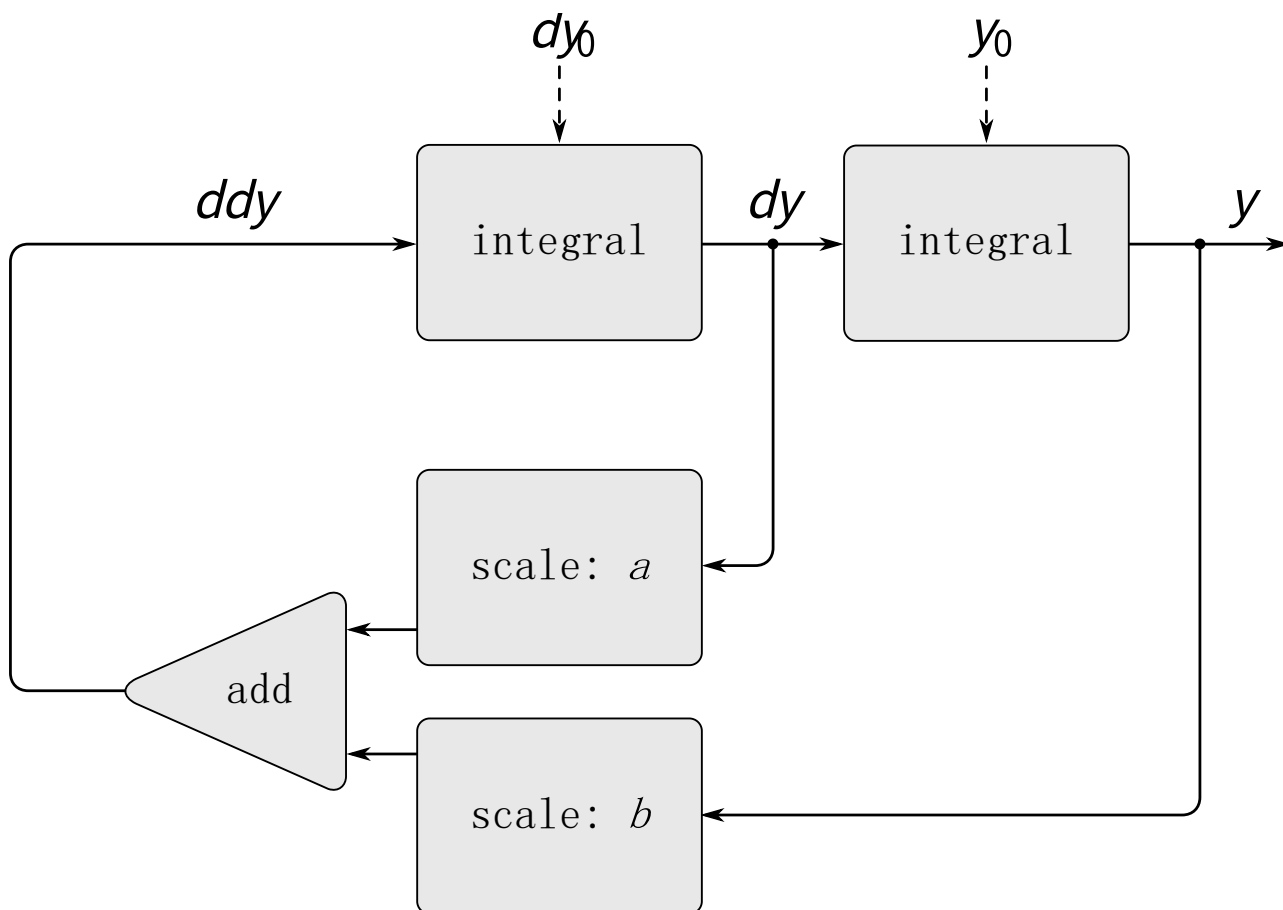


Figure 3.37. Signal-flow diagram for the solution to a second-order linear differential equation.

Exercise 3.79. Generalize the `solve_2nd` function of exercise 3.78 so that it can be used to solve general second-order differential equations $d^2 y/dt^2 = f(dy/dt, y)$.

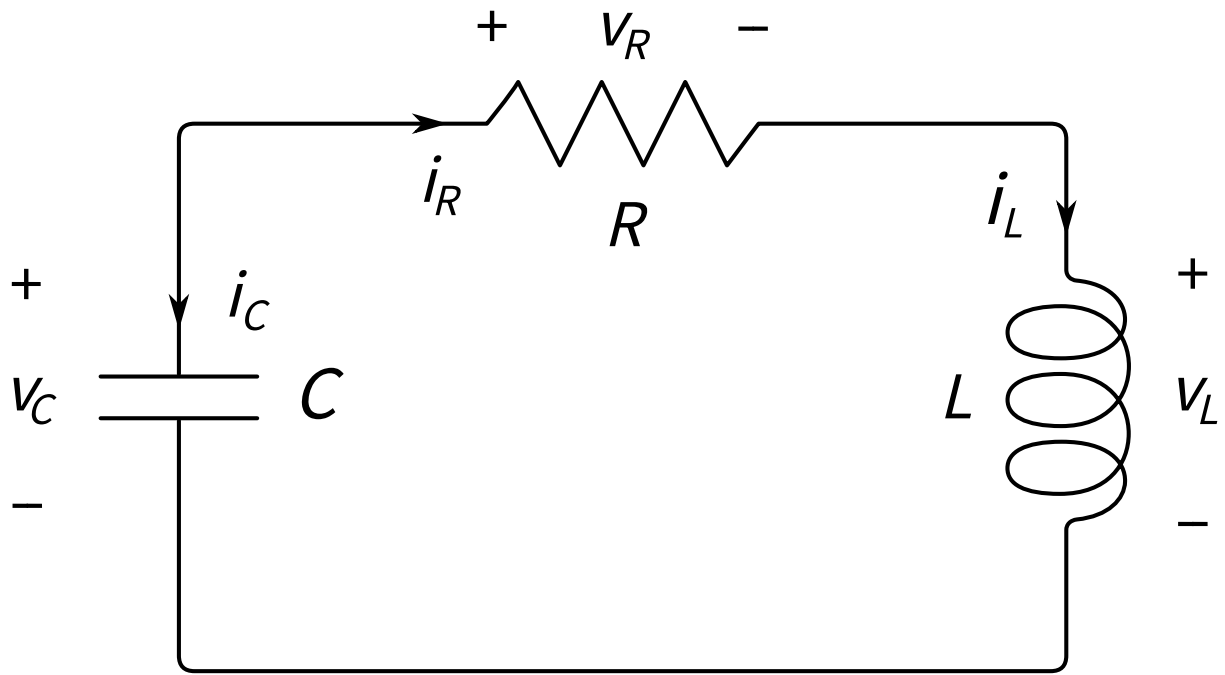


Figure 3.38. A series RLC circuit.

Exercise 3.80. A *series RLC circuit* consists of a resistor, a capacitor, and an inductor connected in series, as shown in figure 3.38. If R , L , and C are the resistance, inductance, and capacitance, then the relations between voltage (v) and current (i) for the three components are described by the equations

$$\begin{array}{lll} v_R & = & i_R R \\ v_L & = & L \frac{di_L}{dt} \\ i_C & = & C \frac{dv_C}{dt} \end{array}$$

and the circuit connections dictate the relations

$$\begin{array}{lll} i_R & = & i_L = -i_C \\ v_C & = & v_L + v_R \end{array}$$

Combining these equations shows that the state of the circuit (summarized by v_C , the voltage across the capacitor, and i_L , the current in the inductor) is described by the pair of differential equations

$$\begin{array}{lll} \frac{dv_C}{dt} & = & -\frac{i_L}{C} \\ \frac{di_L}{dt} & = & \frac{1}{L} v_C - \frac{R}{L} i_L \end{array}$$

The signal-flow diagram representing this system of differential equations is shown in figure 3.39.

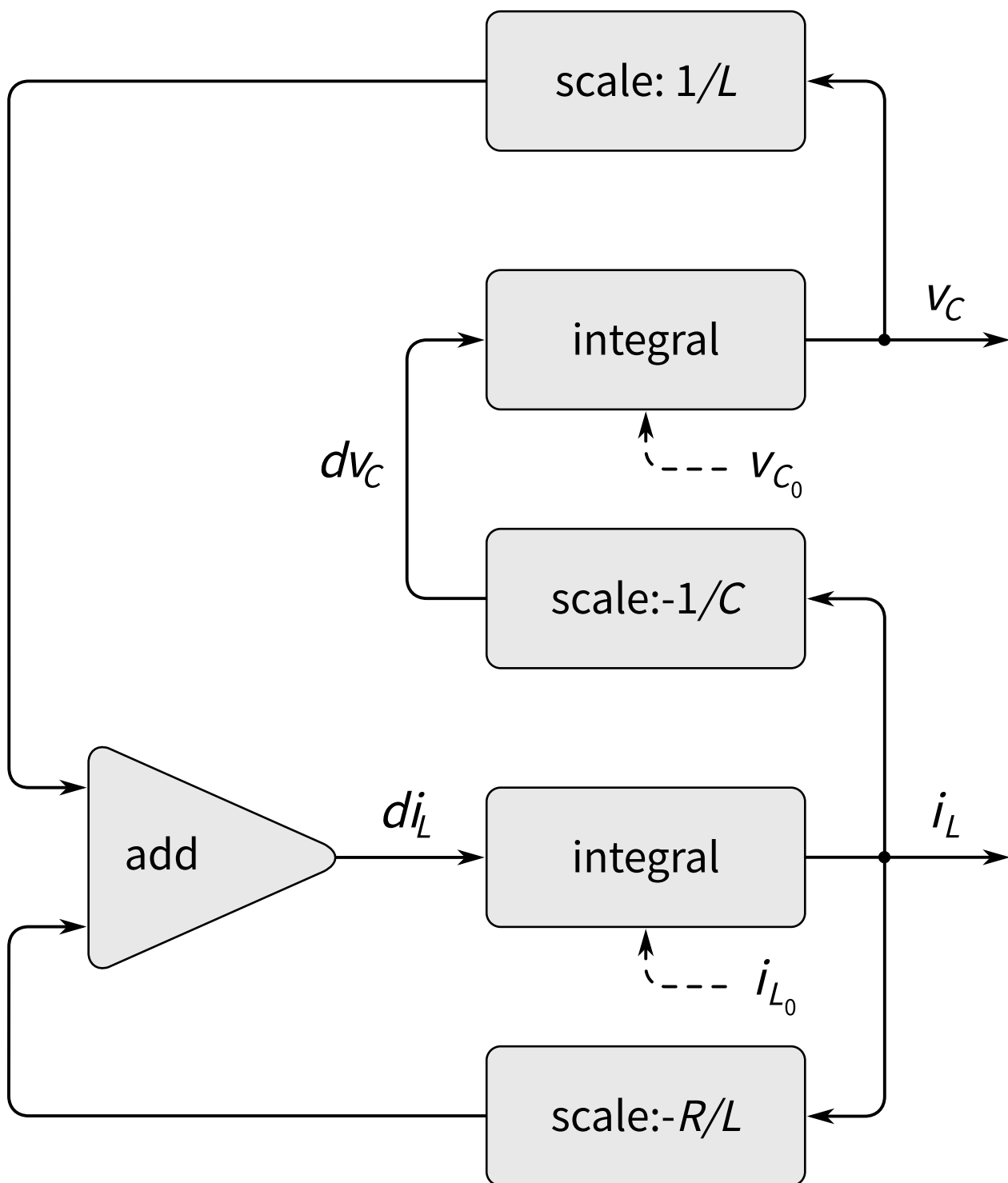


Figure 3.39. A signal-flow diagram for the solution to a series RLC circuit.

Write a function `RLC` that takes as arguments the parameters R , L , and C of the circuit and the time increment Δt . In a manner similar to that of the `RC` function of exercise 3.73, `RLC` should produce a function that takes the initial values of the state variables, v_{C_0} and i_{L_0} , and produces a pair (using `pair`) of the streams of states v_C and i_L . Using `RLC`, generate the pair of streams that models the behavior of a series RLC circuit with $R = 1$ ohm, $C = 0.2$ farad, $L = 1$ henry, $\Delta t = 0.1$ second, and initial values $i_{L_0} = 0$ amps and $v_{C_0} = 10$ volts.

Normal-order evaluation

The examples in this section illustrate how delayed evaluation provides great programming flexibility, but the same examples also show how this can make our programs more complex. Our new `integral` function, for instance, gives us the power to model systems with loops, but we must now remember that `integral` should be called with a delayed integrand, and every function that uses `integral` must be aware of this. In effect, we have created two classes of functions: ordinary functions and functions that take delayed arguments. In general, creating separate classes of functions forces us to create separate classes of higher-order functions as well. [^68: This is a small reflection, in Python, of the difficulties that early statically typed languages such as Pascal had in coping with higher-order functions. In these languages, the programmer had to specify the data types of the arguments and the result of each function: number, logical value, sequence, and so on. Consequently, we could not express an abstraction such as "map a given function `fun` over all the elements in a sequence" by a single higher-order function such as `stream_map`. Rather, we would need a different mapping function for each different combination of argument and result data types that might be specified for a `fun`. Maintaining a practical notion of "data type" in the presence of higher-order functions raises many difficult issues. One way of dealing with this problem is illustrated by the language ML (Gordon, Milner, and Wadsworth 1979), whose "parametrically polymorphic data types" include templates for higher-order transformations between data types. Moreover, data types for most functions in ML are never explicitly declared by the programmer. Instead, ML includes a *type-inferencing* mechanism that uses information in the environment to deduce the data types for newly defined functions. Today, statically typed programming languages have evolved to typically support some form of type inference as well as parametric polymorphism, with varying degrees of power. Haskell couples an expressive type system with powerful type inference.]

One way to avoid the need for two different classes of functions is to make all functions take delayed arguments. We could adopt a model of evaluation in which all arguments to functions are automatically delayed and arguments are forced only when they are actually needed (for example, when they are required by a primitive operation). This would transform our language to use normal-order evaluation, which we first described when we introduced the substitution model for evaluation in section 1.1.5. Converting to normal-order evaluation provides a uniform and elegant way to simplify the use of delayed evaluation, and this would be a natural strategy to adopt if we were concerned only with stream processing. In section , after we have studied the evaluator, we will see how to transform our language in just this way. Unfortunately, including delays in function calls wreaks havoc with our ability to design programs that depend on the order of events, such as programs that use reassignment, mutate data, or perform input or output. Even a single delay in the tail of a pair can cause great confusion, as illustrated by exercises 3.51 and 3.52. As far as anyone knows, mutability and delayed evaluation do not mix well in programming languages.

3.5.5 Modularity of Functional Programs and Modularity of Objects

As we saw in section 3.1.2, one of the major benefits of introducing reassignment is that we can increase the modularity of our systems by encapsulating, or "hiding," parts of the state of a large system within local variables. Stream models can provide an equivalent modularity without the use of reassignment. As an illustration, we can reimplement the Monte Carlo estimation of π , which we examined in section 3.1.2, from a stream-processing point of view.

The key modularity issue was that we wished to hide the internal state of a random-number generator from programs that used random numbers. We began with a function `rand_update`, whose successive values furnished our supply of random numbers, and used this to produce a random-number generator:

```
def make_rand():
    x = random_init
    def rand():
        nonlocal x
        x = rand_update(x)
        return x
    return rand

rand = make_rand()
```

In the stream formulation there is no random-number generator *per se*, just a stream of random numbers produced by successive calls to `rand_update`:

```
random_numbers = pair(random_init,
                      lambda: stream_map(rand_update, random_numbers))
```

We use this to construct the stream of outcomes of the Cesàro experiment performed on consecutive pairs in the `random_numbers` stream:

```
def map_successive_pairs(f, s):
    return pair(f(head(s), head(stream_tail(s))),
               lambda: map_successive_pairs(
                   f,
                   stream_tail(stream_tail(s))))

dirichlet_stream = map_successive_pairs(lambda r1, r2: gcd(r1, r2) == 1,
                                       random_numbers)
```

The `dirichlet_stream` is now fed to a `monte_carlo` function, which produces a stream of estimates of probabilities. The results are then converted into a stream of estimates of π . This version of the program doesn't need a parameter telling how many trials to perform. Better

estimates of π (from performing more experiments) are obtained by looking farther into the `pi` stream:

```
def monte_carlo(experiment_stream, passed, failed):
    def next(passed, failed):
        return pair(passed / (passed + failed),
                    lambda: monte_carlo(stream_tail(experiment_stream),
                                         passed, failed))
    return (next(passed + 1, failed)
            if head(experiment_stream)
            else next(passed, failed + 1))

pi = stream_map(lambda p: math_sqrt(6 / p),
               monte_carlo(dirichlet_stream, 0, 0))
```

There is considerable modularity in this approach, because we still can formulate a general `monte_carlo` function that can deal with arbitrary experiments. Yet there is no reassignment or local state.

Exercise 3.81. Exercise 3.6 discussed generalizing the random-number generator to allow one to reset the random-number sequence so as to produce repeatable sequences of "random" numbers. Produce a stream formulation of this same generator that operates on an input stream of requests to "generate" a new random number or to "reset" the sequence to a specified value and that produces the desired stream of random numbers. Don't use reassignment in your solution.

Exercise 3.82. Redo exercise 3.5 on Monte Carlo integration in terms of streams. The stream version of `estimate_integral` will not have an argument telling how many trials to perform. Instead, it will produce a stream of estimates based on successively more trials.

A functional-programming view of time

Let us now return to the issues of objects and state that were raised at the beginning of this chapter and examine them in a new light. We introduced reassignment and mutable objects to provide a mechanism for modular construction of programs that model systems with state. We constructed computational objects with local state variables and used reassignment to modify these variables. We modeled the temporal behavior of the objects in the world by the temporal behavior of the corresponding computational objects.

Now we have seen that streams provide an alternative way to model objects with local state. We can model a changing quantity, such as the local state of some object, using a stream that represents the time history of successive states. In essence, we represent time explicitly, using streams, so that we decouple time in our simulated world from the sequence of events that take place during evaluation. Indeed, because of the presence of delayed evaluation there may be little relation between simulated time in the model and the order of events during the evaluation.

In order to contrast these two approaches to modeling, let us reconsider the implementation of a "withdrawal processor" that monitors the balance in a bank account. In section 3.1.3 we implemented a simplified version of such a processor:

```
def make_simplified_withdraw(balance):
    def withdraw(amount):
        nonlocal balance
        balance = balance - amount
        return balance
    return withdraw
```

Calls to `make_simplified_withdraw` produce computational objects, each with a local state variable `balance` that is decremented by successive calls to the object. The object takes an `amount` as an argument and returns the new balance. We can imagine the user of a bank account typing a sequence of inputs to such an object and observing the sequence of returned values shown on a display screen.

Alternatively, we can model a withdrawal processor as a function that takes as input a balance and a stream of amounts to withdraw and produces the stream of successive balances in the account:

```
def stream_withdraw(balance, amount_stream):
    return pair(balance,
                lambda: stream_withdraw(balance - head(amount_stream),
                                         stream_tail(amount_stream)))
```

The function `stream_withdraw` implements a well-defined mathematical function whose output is fully determined by its input. Suppose, however, that the input `amount_stream` is the stream of successive values typed by the user and that the resulting stream of balances is displayed. Then, from the perspective of the user who is typing values and watching results, the stream process has the same behavior as the object created by `make_simplified_withdraw`. However, with the stream version, there is no reassignment, no local state variable, and consequently none of the theoretical difficulties that we encountered in section 3.1.3. Yet the system has state!

This is really remarkable. Even though `stream_withdraw` implements a well-defined mathematical function whose behavior does not change, the user's perception here is one of interacting with a system that has a changing state. One way to resolve this paradox is to realize that it is the user's temporal existence that imposes state on the system. If the user could step back from the interaction and think in terms of streams of balances rather than individual transactions, the system would appear stateless. [^69: Similarly in physics, when we observe a moving particle, we say that the position (state) of the particle is changing. However, from the perspective of the particle's world line in space-time there is no change involved.]

From the point of view of one part of a complex process, the other parts appear to change with time. They have hidden time-varying local state. If we wish to write programs that model this kind

of natural decomposition in our world (as we see it from our viewpoint as a part of that world) with structures in our computer, we make computational objects that are not functional—they must change with time. We model state with local state variables, and we model the changes of state with reassignments to those variables. By doing this we make the time of execution of a computation model time in the world that we are part of, and thus we get "objects" in our computer.

Modeling with objects is powerful and intuitive, largely because this matches the perception of interacting with a world of which we are part. However, as we've seen repeatedly throughout this chapter, these models raise thorny problems of constraining the order of events and of synchronizing multiple processes. The possibility of avoiding these problems has stimulated the development of *functional programming languages*, which do not include any provision for reassignment or mutable data. In such a language, all functions implement well-defined mathematical functions of their arguments, whose behavior does not change. The functional approach is extremely attractive for dealing with concurrent systems. [^70: John Backus, the inventor of Fortran, gave high visibility to functional programming when he was awarded the ACM Turing award in 1978. His acceptance speech (Backus 1978) strongly advocated the functional approach. A good overview of functional programming is given in Henderson 1980 and in Darlington, Henderson, and Turner 1982.]

On the other hand, if we look closely, we can see time-related problems creeping into functional models as well. One particularly troublesome area arises when we wish to design interactive systems, especially ones that model interactions between independent entities. For instance, consider once more the implementation of a banking system that permits joint bank accounts. In a conventional system using reassignment and objects, we would model the fact that Peter and Paul share an account by having both Peter and Paul send their transaction requests to the same bank-account object, as we saw in section 3.1.3. From the stream point of view, where there are no "objects"*per se*, we have already indicated that a bank account can be modeled as a process that operates on a stream of transaction requests to produce a stream of responses. Accordingly, we could model the fact that Peter and Paul have a joint bank account by merging Peter's stream of transaction requests with Paul's stream of requests and feeding the result to the bank-account stream process, as shown in figure 3.40.

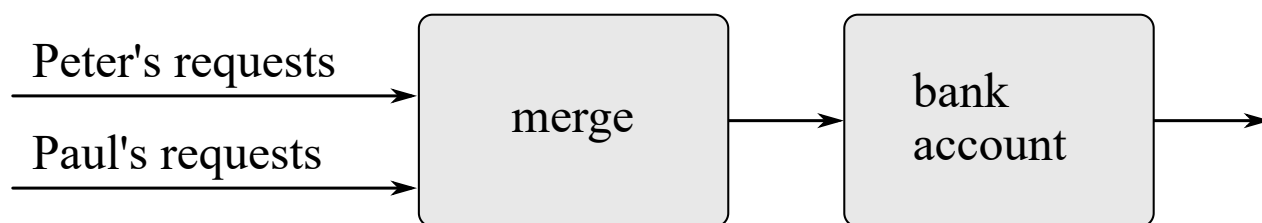


Figure 3.40. A joint bank account, modeled by merging two streams of transaction requests.

The trouble with this formulation is in the notion of *merge*. It will not do to merge the two streams by simply taking alternately one request from Peter and one request from Paul. Suppose Paul

accesses the account only very rarely. We could hardly force Peter to wait for Paul to access the account before he could issue a second transaction. However such a merge is implemented, it must interleave the two transaction streams in some way that is constrained by "real time" as perceived by Peter and Paul, in the sense that, if Peter and Paul meet, they can agree that certain transactions were processed before the meeting, and other transactions were processed after the meeting. [^71: Observe that, for any two streams, there is in general more than one acceptable order of interleaving. Thus, technically, "merge" is a relation rather than a function—the answer is not a deterministic function of the inputs. We already mentioned (footnote) that nondeterminism is essential when dealing with concurrency. The merge relation illustrates the same essential nondeterminism, from the functional perspective. In section , we will look at nondeterminism from yet another point of view.] This is precisely the same constraint that we had to deal with in section 3.4.1, where we found the need to introduce explicit synchronization to ensure a "correct" order of events in concurrent processing of objects with state. Thus, in an attempt to support the functional style, the need to merge inputs from different agents reintroduces the same problems that the functional style was meant to eliminate.

We began this chapter with the goal of building computational models whose structure matches our perception of the real world we are trying to model. We can model the world as a collection of separate, time-bound, interacting objects with state, or we can model the world as a single, timeless, stateless unity. Each view has powerful advantages, but neither view alone is completely satisfactory. A grand unification has yet to emerge. [^72: The object model approximates the world by dividing it into separate pieces. The functional model does not modularize along object boundaries. The object model is useful when the unshared state of the "objects" is much larger than the state that they share. An example of a place where the object viewpoint fails is quantum mechanics, where thinking of things as individual particles leads to paradoxes and confusions. Unifying the object view with the functional view may have little to do with programming, but rather with fundamental epistemological issues.]

References

- Abelson, Harold, Andrew Berlin, Jacob Katzenelson, William McAllister, Guillermo Rozas, Gerald Jay Sussman, and Jack Wisdom. 1992. The Supercomputer Toolkit: A general framework for special-purpose computing. *International Journal of High-Speed Electronics* 3(3):337–361.
- Allen, John. 1978. *Anatomy of Lisp*. New York: McGraw-Hill.
- Appel, Andrew W. 1987. Garbage collection can be faster than stack allocation. *Information Processing Letters* 25(4):275–279.
- Backus, John. 1978. Can programming be liberated from the von Neumann style? *Communications of the ACM* 21(8):613–641.
- Baker, Henry G., Jr. 1978. List processing in real time on a serial computer. *Communications of the ACM* 21(4):280–293.
- Batali, John, Neil Mayle, Howard Shrobe, Gerald Jay Sussman, and Daniel Weise. 1982. The Scheme-81 architecture—System and chip. In *Proceedings of the MIT Conference on Advanced Research in VLSI*, edited by Paul Penfield, Jr. Dedham, MA: Artech House.
- Borning, Alan. 1977. ThingLab—An object-oriented system for building simulations using constraints. In *Proceedings of the 5th International Joint Conference on Artificial Intelligence*.
- Borodin, Alan, and Ian Munro. 1975. *The Computational Complexity of Algebraic and Numeric Problems*. New York: American Elsevier.
- Chaitin, Gregory J. 1975. Randomness and mathematical proof. *Scientific American* 232(5):47–52.
- Church, Alonzo. 1941. *The Calculi of Lambda-Conversion*. Princeton, N.J.: Princeton University Press.
- Clark, Keith L. 1978. Negation as failure. In *Logic and Data Bases*. New York: Plenum Press, pp. 293–322.
- Clinger, William. 1982. Nondeterministic call by need is neither lazy nor by name. In *Proceedings of the ACM Symposium on Lisp and Functional Programming*, pp. 226–234.
- Colmerauer A., H. Kanoui, R. Pasero, and P. Roussel. 1973. Un système de communication homme-machine en français. Technical report, Groupe d'Intelligence Artificielle, Université

d'Aix-Marseille II, Luminy.

- Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms*. 4th edition. Cambridge, MA: MIT Press.
- Crockford, Douglas. 2008. *Python: The Good Parts*. Sebastopol, CA: O'Reilly Media.
- Darlington, John, Peter Henderson, and David Turner. 1982. *Functional Programming and Its Applications*. New York: Cambridge University Press.
- Dijkstra, Edsger W. 1968a. The structure of the "THE" multiprogramming system. *Communications of the ACM* 11(5):341–346.
- Dijkstra, Edsger W. 1968b. Cooperating sequential processes. In *Programming Languages*, edited by F. Genuys. New York: Academic Press, pp. 43–112.
- Dinesman, Howard P. 1968. *Superior Mathematical Puzzles*. New York: Simon and Schuster.
- de Kleer, Johan, Jon Doyle, Guy Steele, and Gerald J. Sussman. 1977. AMORD: Explicit control of reasoning. In *Proceedings of the ACM Symposium on Artificial Intelligence and Programming Languages*, pp. 116–125.
- Doyle, Jon. 1979. A truth maintenance system. *Artificial Intelligence* 12:231–272.
- ECMA. 1997. ECMAScript: A general purpose, cross-platform programming language. 1st edition, edited by Guy L. Steele Jr. *Ecma International*.
- ECMA. 2015. ECMAScript: A general purpose, cross-platform programming language. 6th edition, edited by Allen Wirfs-Brock. *Ecma International*.
- ECMA. 2020. ECMAScript: A general purpose, cross-platform programming language. 11th edition, edited by Jordan Harband. *Ecma International*.
- Edwards, A. W. F. 2019. *Pascal's Arithmetical Triangle*. Mineola, New York: Dover Publications.
- Feeley, Marc. 1986. Deux approches à l'implantation du langage Scheme. Masters thesis, Université de Montréal.
- Feeley, Marc and Guy Lapalme. 1987. Using closures for code generation. *Journal of Computer Languages* 12(1):47–66.
- Feigenbaum, Edward, and Howard Shrobe. 1993. The Japanese National Fifth Generation Project: Introduction, survey, and evaluation. In *Future Generation Computer Systems*, vol. 9, pp. 105–117.
- Feller, William. 1957. *An Introduction to Probability Theory and Its Applications*, volume 1. New York: John Wiley & Sons.

- Fenichel, R., and J. Yochelson. 1969. A Lisp garbage collector for virtual memory computer systems. *Communications of the ACM* 12(11):611–612.
- Floyd, Robert. 1967. Nondeterministic algorithms. *JACM*, 14(4):636–644.
- Forbus, Kenneth D., and Johan de Kleer. 1993. *Building Problem Solvers*. Cambridge, MA: MIT Press.
- Friedman, Daniel P., and David S. Wise. 1976. CONS should not evaluate its arguments. In *Automata, Languages, and Programming: Third International Colloquium*, edited by S. Michaelson and R. Milner, pp. 257–284.
- Friedman, Daniel P., Mitchell Wand, and Christopher T. Haynes. 1992. *Essentials of Programming Languages*. Cambridge, MA: MIT Press/McGraw-Hill.
- Gabriel, Richard P. 1988. The Why of Y. *Lisp Pointers* 2(2):15–25.
- Goldberg, Adele, and David Robson. 1983. *Smalltalk-80: The Language and Its Implementation*. Reading, MA: Addison-Wesley.
- Gordon, Michael, Robin Milner, and Christopher Wadsworth. 1979. *Edinburgh LCF*. Lecture Notes in Computer Science, volume 78. New York: Springer-Verlag.
- Gray, Jim, and Andreas Reuter. 1993. *Transaction Processing: Concepts and Models*. San Mateo, CA: Morgan-Kaufman.
- Green, Cordell. 1969. Application of theorem proving to problem solving. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 219–240.
- Green, Cordell, and Bertram Raphael. 1968. The use of theorem-proving techniques in question-answering systems. In *Proceedings of the ACM National Conference*, pp. 169–181.
- Guttag, John V. 1977. Abstract data types and the development of data structures. *Communications of the ACM* 20(6):397–404.
- Hamming, Richard W. 1980. *Coding and Information Theory*. Englewood Cliffs, N.J.: Prentice-Hall.
- Hanson, Christopher P. 1990. Efficient stack allocation for tail-recursive languages. In *Proceedings of ACM Conference on Lisp and Functional Programming*, pp. 106–118.
- Hanson, Christopher P. 1991. A syntactic closures macro facility. *Lisp Pointers*, 4(4):9–16.
- Hardy, Godfrey H. 1921. Srinivasa Ramanujan. *Proceedings of the London Mathematical Society* XIX(2).

- Hardy, Godfrey H., and E. M. Wright. 1960. *An Introduction to the Theory of Numbers*. 4th edition. New York: Oxford University Press.
- Havender, J. 1968. Avoiding deadlocks in multi-tasking systems. *IBM Systems Journal* 7(2):74–84.
- Henderson, Peter. 1980. *Functional Programming: Application and Implementation*. Englewood Cliffs, N.J.: Prentice-Hall.
- Henderson, Peter. 1982. Functional Geometry. In *Conference Record of the 1982 ACM Symposium on Lisp and Functional Programming*, pp. 179–187.
- Hewitt, Carl E. 1969. PLANNER: A language for proving theorems in robots. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 295–301.
- Hewitt, Carl E. 1977. Viewing control structures as patterns of passing messages. *Journal of Artificial Intelligence* 8(3):323–364.
- Hoare, C. A. R. 1972. Proof of correctness of data representations. *Acta Informatica* 1(1):271–281.
- Hodges, Andrew. 1983. *Alan Turing: The Enigma*. New York: Simon and Schuster.
- Hofstadter, Douglas R. 1979. *Gödel, Escher, Bach: An Eternal Golden Braid*. New York: Basic Books.
- Hughes, R. J. M. 1990. Why functional programming matters. In *Research Topics in Functional Programming*, edited by David Turner. Reading, MA: Addison-Wesley, pp. 17–42.
- IEEE Std 1178-1990. 1990. *IEEE Standard for the Scheme Programming Language*.
- Ingerman, Peter, Edgar Irons, Kirk Sattley, and Wallace Feurzeig; assisted by M. Lind, Herbert Kanner, and Robert Floyd. 1960. THUNKS: A way of compiling procedure statements, with some comments on procedure declarations. Unpublished manuscript. (Also, private communication from Wallace Feurzeig.)
- Jaffar, Joxan, and Peter J. Stuckey. 1986. Semantics of infinite tree logic programming. *Theoretical Computer Science* 46:141–158.
- Kaldewaij, Anne. 1990. *Programming: The Derivation of Algorithms*. New York: Prentice-Hall.
- Knuth, Donald E. 1997a. *Fundamental Algorithms*. Volume 1 of *The Art of Computer Programming*. 3rd edition. Reading, MA: Addison-Wesley.
- Knuth, Donald E. 1997b. *Seminumerical Algorithms*. Volume 2 of *The Art of Computer Programming*. 3rd edition. Reading, MA: Addison-Wesley.

- Konopasek, Milos, and Sundaresan Jayaraman. 1984. *The TK!Solver Book: A Guide to Problem-Solving in Science, Engineering, Business, and Education*. Berkeley, CA: Osborne/McGraw-Hill.
- Kowalski, Robert. 1973. Predicate logic as a programming language. Technical report 70, Department of Computational Logic, School of Artificial Intelligence, University of Edinburgh.
- Kowalski, Robert. 1979. *Logic for Problem Solving*. New York: North-Holland.
- Lamport, Leslie. 1978. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM* 21(7):558–565.
- Lampson, Butler, J. J. Horning, R. London, J. G. Mitchell, and G. K. Popek. 1981. Report on the programming language Euclid. Technical report, Computer Systems Research Group, University of Toronto.
- Landin, Peter. 1965. A correspondence between Algol 60 and Church's lambda notation: Part I. *Communications of the ACM* 8(2):89–101.
- Lieberman, Henry, and Carl E. Hewitt. 1983. A real-time garbage collector based on the lifetimes of objects. *Communications of the ACM* 26(6):419–429.
- Liskov, Barbara H., and Stephen N. Zilles. 1975. Specification techniques for data abstractions. *IEEE Transactions on Software Engineering* 1(1):7–19.
- McAllester, David Allen. 1978. A three-valued truth-maintenance system. Memo 473, MIT Artificial Intelligence Laboratory.
- McAllester, David Allen. 1980. An outlook on truth maintenance. Memo 551, MIT Artificial Intelligence Laboratory.
- McCarthy, John. 1967. A basis for a mathematical theory of computation. In *Computer Programing and Formal Systems*, edited by P. Braffort and D. Hirschberg. North-Holland, pp. 33–70.
- McDermott, Drew, and Gerald Jay Sussman. 1972. Conniver reference manual. Memo 259, MIT Artificial Intelligence Laboratory.
- Miller, Gary L. 1976. Riemann's Hypothesis and tests for primality. *Journal of Computer and System Sciences* 13(3):300–317.
- Miller, James S., and Guillermo J. Rozas. 1994. Garbage collection is fast, but a stack is faster. Memo 1462, MIT Artificial Intelligence Laboratory.
- Moon, David. 1978. MacLisp reference manual, Version 0. Technical report, MIT Laboratory for Computer Science.

- Morris, J. H., Eric Schmidt, and Philip Wadler. 1980. Experience with an applicative string processing language. In *Proceedings of the 7th Annual ACM SIGACT/SIGPLAN Symposium on the Principles of Programming Languages*.
- O'Neill, Melissa E. 2009. The genuine sieve of Eratosthenes. *Journal of Functional Programming* 19(1):95–106.
- Phillips, Hubert. 1934. *The Sphinx Problem Book*. London: Faber and Faber.
- Phillips, Hubert. 1961. *My Best Puzzles in Logic and Reasoning*. New York: Dover Publications.
- Rabin, Michael O. 1980. Probabilistic algorithm for testing primality. *Journal of Number Theory* 12:128–138.
- Raymond, Eric. 1996. *The New Hacker's Dictionary*. 3rd edition. Cambridge, MA: MIT Press.
- Raynal, Michel. 1986. *Algorithms for Mutual Exclusion*. Cambridge, MA: MIT Press.
- Rees, Jonathan A., and Norman I. Adams IV. 1982. T: A dialect of Lisp or, lambda: The ultimate software tool. In *Conference Record of the 1982 ACM Symposium on Lisp and Functional Programming*, pp. 114–122.
- Rivest, Ronald L., Adi Shamir, and Leonard M. Adleman. 1978. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126.
- Robinson, J. A. 1965. A machine-oriented logic based on the resolution principle. *Journal of the ACM* 12(1):23.
- Robinson, J. A. 1983. Logic programming—Past, present, and future. *New Generation Computing* 1:107–124.
- Sagade, Y. 2015. SICP exercise 1.14
- Spafford, Eugene H. 1989. The Internet Worm: Crisis and aftermath. *Communications of the ACM* 32(6):678–688.
- Steele, Guy Lewis, Jr. 1977. Debunking the "expensive procedure call" myth. In *Proceedings of the National Conference of the ACM*, pp. 153–162.
- Steele, Guy Lewis, Jr., and Gerald Jay Sussman. 1975. Scheme: An interpreter for the extended lambda calculus. Memo 349, MIT Artificial Intelligence Laboratory.
- Steele, Guy Lewis, Jr., Donald R. Woods, Raphael A. Finkel, Mark R. Crispin, Richard M. Stallman, and Geoffrey S. Goodfellow. 1983. *The Hacker's Dictionary*. New York: Harper & Row.
- Stoy, Joseph E. 1977. *Denotational Semantics*. Cambridge, MA: MIT Press.

- Sussman, Gerald Jay, and Richard M. Stallman. 1975. Heuristic techniques in computer-aided circuit analysis. *IEEE Transactions on Circuits and Systems* CAS-22(11):857–865.
- Sussman, Gerald Jay, and Guy Lewis Steele Jr. 1980. Constraints—A language for expressing almost-hierarchical descriptions. *AI Journal* 14:1–39.
- Sussman, Gerald Jay, and Jack Wisdom. 1992. Chaotic evolution of the solar system. *Science* 257:256–262.
- Sussman, Gerald Jay, Terry Winograd, and Eugene Charniak. 1971. Microplanner reference manual. Memo 203A, MIT Artificial Intelligence Laboratory.
- Sutherland, Ivan E. 1963. SKETCHPAD: A man-machine graphical communication system. Technical report 296, MIT Lincoln Laboratory.
- Thatcher, James W., Eric G. Wagner, and Jesse B. Wright. 1978. Data type specification: Parameterization and the power of specification techniques. In *Conference Record of the Tenth Annual ACM Symposium on Theory of Computing*, pp. 119–132.
- Turner, David. 1981. The future of applicative languages. In *Proceedings of the 3rd European Conference on Informatics*, Lecture Notes in Computer Science, volume 123. New York: Springer-Verlag, pp. 334–348.
- Wand, Mitchell. 1980. Continuation-based program transformation strategies. *Journal of the ACM* 27(1):164–180.
- Waters, Richard C. 1979. A method for analyzing loop programs. *IEEE Transactions on Software Engineering* 5(3):237–247.
- Winston, Patrick. 1992. *Artificial Intelligence*. 3rd edition. Reading, MA: Addison-Wesley.
- Zabih, Ramin, David McAllester, and David Chapman. 1987. Non-deterministic Lisp with dependency-directed backtracking. *AAAI-87*, pp. 59–64.
- Zippel, Richard. 1979. Probabilistic algorithms for sparse polynomials. Ph.D. dissertation, Department of Electrical Engineering and Computer Science, MIT.
- Zippel, Richard. 1993. *Effective Polynomial Computation*. Boston, MA: Kluwer Academic Publishers.

About the SICPy Project

Background

The Python adaptation of SICP is an open-source community effort. The software and data required for making these web pages and the PDF edition are contained in the github repository `source-academy / sicp`, and improvements, extensions and discussions are handled in this repository using `git`.

Martin Henz started translating SICP to JavaScript in 2008. He obtained the original LaTeX sources of the second edition from Gerald Jay Sussman, and converted them to a custom-built XML format. The original sources are retained in the XML format, which allows for generating the comparison edition. A processing system written in XSLT resulted in the first version of the JavaScript adaptation around 2009, covering the first few sections of SICP. The content of SICP JS contained in the XML files are undergoing continuous improvement by the adapters Martin Henz and Tobias Wrigstad, and by the community of SICP JS readers, using the github repository.

Martin Henz started translating SICP to Python in 2026, based on the JavaScript edition. He and his colleagues Boyd Anderson and Sanka Rasnayaka at the School of Computing at the National University of Singapore are using SICPy in Semester 1 of the Academic Year 2026/27.

In the book, program fragments often require other program fragments. In order to collect and execute the necessary programs, the corresponding `SNIPPET` tags in the xml files include `REQUIRES` tags. The XML processors use these tags in order to assemble the executable programs. The project thus can be seen as a *literate programming system*, custom-made for authoring SICP JS.

Interactive SICPy

The software for Interactive SICPy was designed and implemented by Samuel Fang in 2021 for SICP JS. The XML textbook sources are translated to a JSON format, which are then read and rendered by a dedicated component of the Source Academy.

Comparison Edition

The Scheme/Python comparison edition provides a side-by-side comparison between the original Scheme version of SICP and SICPy. Earliest precursor of the comparison edition is the mobile-friendly web edition of SICP JS, designed and implemented by Liu Hang in 2017. Feng Piaopiao improved the system in 2018. He Xinyue and Wang Qian developed the software for the current comparison editions in 2020 for SICP JS. Formulas are retained in the resulting HTML files and are

typeset by the reader's browser on the fly, using the MathJax system. The comparison edition is maintained by Martin Henz.

PDF Edition

The early PDF editions (2010-2018) used XSLT for generating LaTeX from the XML sources. The first Node.js version of the PDF edition was designed and implemented by Chan Ger Hean in 2019 for SICP JS. Martin Henz is the main developer of this system.

Figures

Most figures are adapted from the HTML5/EPUB3 version of SICP by Andres Raba. The figures are licensed under Creative Commons Attribution-ShareAlike 4.0 International License (cc by-sa). JavaScript adaptations of figures were done by Tobias Wrigstad using Inkscape and gratuitous use of `sed`.